Notebook ICPC

algorithms_ICPC · C++ / Python · 5 de noviembre de 2026

0 Template **3**
0.1 simple.cpp 3

1 Estructuras **3**
1.1 Colas monótonas **3**
1.1.1 min-queue.cpp 3
1.2 Descomposición SQRT **3**
1.2.1 mo-xd.cpp 3
1.2.2 sqrt-descomposition.cpp 3
1.3 DSU **3**
1.3.1 dsu.cpp 3
1.3.2 union-find-partial-persistent.cpp 3
1.3.3 union-find-rollback.cpp 4
1.4 Fenwick **4**
1.4.1 fenwick-tree-2d.cpp 4
1.4.2 fenwick-tree.cpp 4
1.5 Hash maps **4**
1.5.1 custom-hash-pair.cpp 4
1.5.2 custom-hash.cpp 4
1.5.3 fast-hash-map.cpp 4
1.6 Heaps **4**
1.6.1 heap-sort.cpp 4
1.6.2 priority-queue.cpp 5
1.7 Intervalos y dominancia **5**
1.7.1 disjoint-intervals.cpp 5
1.7.2 gain-cost-set-vasito.cpp 5
1.8 Mo **5**
1.8.1 mo-s-hilbert-curve.cpp 5
1.8.2 mo-s.cpp 5
1.9 Ordered Set PBDS **5**
1.9.1 multiorderedset.cpp 5
1.9.2 orderedset.cpp 6
1.10 Rope **6**
1.10.1 rope.cpp 6
1.11 Segment Tree › Básico **6**
1.11.1 general-segtree.cpp 6
1.12 Segment Tree › Dinámico **6**
1.12.1 lazy-creation.cpp 6
1.13 Segment Tree › Dos dimensiones **6**
1.13.1 segment-tree-2d.cpp 6
1.14 Segment Tree › Lazy **7**
1.14.1 general-lazy-tree.cpp 7
1.14.2 iterative-lazytree.cpp 7
1.14.3 lazy-st1.cpp 8
1.14.4 polynomial-sum-lazy-segtree-problem.cpp 8
1.14.5 segment-tree-lazy-vasito.cpp 9
1.15 Segment Tree › Merge Sort Tree **9**
1.15.1 merge-sort-tree.cpp 9
1.16 Segment Tree › Persistente **9**
1.16.1 persistantsegmenttree.cpp 9
1.17 Sparse Table **10**
1.17.1 spare-table.cpp 10
1.17.2 sparse-table-2d.cpp 10
1.18 Treap **10**
1.18.1 persistent-treap.cpp 10
1.18.2 treap-implicit-father-vasito.cpp 10
1.18.3 treap-implicit-vasito.cpp 11
1.18.4 treap.cpp 11
1.19 Trie persistente **11**
1.19.1 persistent-trie.cpp 11
1.20 Wavelet Tree **11**
1.20.1 wavelet-tree-vasito.cpp 11

2 Grafos **11**
2.1 Árbol expansión mínima **11**
2.1.1 chu-liu-vasito.cpp 11
2.1.2 kruskal.cpp 12
2.1.3 prim-extra.cpp 12
2.2 Articulaciones puentes biconexas **12**
2.2.1 articulation-bridges-biconnected-extra.cpp 12
2.2.2 tarjan-y-blockcuttree.cpp 13
2.3 Caminos eulerianos **13**
2.3.1 eulerian-path.cpp 13
2.4 Caminos mínimos › Bellman Ford **13**
2.4.1 bellman-ford-extra.cpp 13
2.4.2 bellmanford-find-negative-cycle.cpp 13
2.5 Caminos mínimos › Dijkstra **14**
2.5.1 dijkstra-extra.cpp 14
2.5.2 dijkstra-k-shortest-path.cpp 14
2.6 Caminos mínimos › Floyd Warshall **14**
2.6.1 floyd-warshall-extra.cpp 14
2.6.2 floyd-warshall-negative-weights.cpp 14
2.7 Conectividad dinámica **14**
2.7.1 dynamic-connectivity.cpp 14
2.8 Dominadores **15**
2.8.1 dominator-tree.cpp 15
2.9 Orden topológico **15**
2.9.1 topological-sort.cpp 15
2.10 Recorridos y conectividad **15**
2.10.1 bfs-extra.cpp 15
2.10.2 bipartite-check-extra.cpp 15
2.10.3 cycle-detection-extra.cpp 16
2.10.4 dfs-extra.cpp 16
2.10.5 flood-fill-extra.cpp 16
2.10.6 parallel-dfs-vasito.cpp 16
2.10.7 transitive-closure.cpp 16
2.11 SCC y 2SAT **16**
2.11.1 or-statements-like-2-sat-problem.cpp 16
2.11.2 strongly-connected-components.cpp 16
2.11.3 two-sat.cpp 17
2.12 Secuencias de grados **17**
2.12.1 erdos-gallai.cpp 17
2.13 Teoría **17**
2.13.1 theorems.cpp 17

3 Árboles **17**
3.1 Descomposición centroide **17**
3.1.1 centroid-decomposition-extra.cpp 17
3.2 Descomposición SQRT **18**
3.2.1 block-tree.cpp 18
3.3 DSU on Tree **18**
3.3.1 dsu-on-tree-sack-extra.cpp 18
3.4 Euler Tour **18**
3.4.1 euler-tour-sum.cpp 18
3.5 Heavy Light **19**
3.5.1 hld-lca-con-heavy-paths.cpp 19
3.6 Isomorfismo y hashing **19**
3.6.1 tree-isomorphism-extra.cpp 19
3.7 LCA y binary lifting **19**
3.7.1 binary-lifting-o-n-memory.cpp 19
3.7.2 lca-multiplication-path.cpp 19
3.7.3 lowest-common-ancestor.cpp 20
3.7.4 max-weight-lca.cpp 20
3.8 Link Cut Tree **21**
3.8.1 link-cut.cpp 21
3.9 Mo en árboles **21**
3.9.1 mo-s-in-tree-s.cpp 21
3.10 Problemas **22**
3.10.1 nearest-selected-nodes-problem.cpp 22
3.10.2 two-pieces-on-tree-problem.cpp 22
3.11 Rerooting **22**
3.11.1 re-rooting.cpp 22

4 DP **22**
4.1 Cambio de monedas **22**
4.1.1 coin-change-large-n.cpp 22
4.1.2 coin-change.cpp 22
4.2 Contribuciones XOR **22**
4.2.1 sum-of-xor-subarrays-times-its-size-problem.cpp 22
4.3 Digit DP **23**
4.3.1 digit-dp.cpp 23
4.4 LCS y distancia edición **23**
4.4.1 edit-distance.cpp 23
4.4.2 lcs-bitset.cpp 23
4.4.3 lcs-lis.cpp 23
4.4.4 longest-common-subsequence-3.cpp 23
4.4.5 longest-common-subsequence.cpp 23
4.4.6 README.md 24
4.5 LIS **24**
4.5.1 longest-increasing-subsequence.cpp 24
4.6 Máscaras y SOS **24**
4.6.1 dp-mask-over-submasks.cpp 24
4.6.2 sos-dp-reverse.cpp 24
4.6.3 sos-dp.cpp 24
4.7 Máscaras y TSP **24**
4.7.1 elevator-problem.cpp 24
4.7.2 traveling-sales-man-cycle.cpp 25
4.7.3 traveling-sales-man-path.cpp 25
4.8 Notas **25**
4.8.1 dp.cpp 25
4.9 Optimizaciones › CHT y Li Chao **25**
4.9.1 convex-hull-trick-extra.cpp 25
4.9.2 convex-hull-trick.cpp 25
4.9.3 li-chao-tree-vasito.cpp 25
4.9.4 persistent-convex-hull-trick.cpp 26
4.10 Optimizaciones › Divide and Conquer **26**
4.10.1 divide-and-conquer.cpp 26

4.11 **Optimizaciones › Knuth** 26
4.11.1 knuths-optimization.cpp 26
4.12 **Optimizaciones › Mochila** 26
4.12.1 multiple-knacksack-optimizacion.cpp 26
4.13 **Optimizaciones › Separación** 26
4.13.1 separation-optimization.cpp 26
4.14 **Steiner Tree** 27
4.14.1 steiner-tree.cpp 27
4.15 **Subarreglos y submatrices** 27
4.15.1 max-sum-1d.cpp 27
4.15.2 max-sum-2d.cpp 27
4.15.3 max-sum-3d.cpp 27
4.15.4 min-max-range-extra.cpp 27

5 Flujos y matching 27
5.1 **Corte mínimo** 27
5.1.1 min-cut.cpp 27
5.1.2 stoer-wagner-extra.cpp 27
5.2 **Flujo costo mínimo** 28
5.2.1 min-cost-max-flow-vasito.cpp 28
5.3 **Flujo máximo** 28
5.3.1 max-flow.cpp 28
5.4 **Hopcroft Karp** 28
5.4.1 hopcroft-karp.cpp 28
5.5 **Húngaro** 29
5.5.1 hungarian.cpp 29
5.6 **Matching bipartito** 29
5.6.1 kuhn.cpp 29
5.7 **Matching general** 29
5.7.1 blossom-extra.cpp 29

6 Matemáticas 30
6.1 **Álgebra lineal › Base XOR** 30
6.1.1 xor-basis.cpp 30
6.2 **Álgebra lineal › Matrices y Gauss** 30
6.2.1 gauss.cpp 30
6.2.2 gaussian-function.cpp 30
6.2.3 matrix-determinant.cpp 30
6.2.4 matrix-fast-pow.cpp 31
6.2.5 matrix-multiplication-extra.cpp 31
6.3 **Aritmética modular › CRT** 31
6.3.1 chinese-remainder-theorem-extra.cpp 31
6.4 **Aritmética modular › Logaritmo discreto** 31
6.4.1 discrete-log.cpp 31
6.5 **Aritmética modular › Operaciones** 31
6.5.1 big-expontent-modular-exponentiation.cpp 31
6.5.2 fraction-modular.cpp 31
6.5.3 modular-exponentiation-extra.cpp 31
6.5.4 modular-inverse-extra.cpp 32
6.5.5 modular-multiplication-extra.cpp 32
6.6 **Aritmética modular › Raíces** 32
6.6.1 primitive-root-extra.cpp 32
6.6.2 tonelly-shanks.cpp 32
6.7 **Combinatoria** 32
6.7.1 catalan-convolution.cpp 32
6.7.2 catalan.cpp 32
6.7.3 combinatorics.cpp 32
6.7.4 modular-combinatorics.cpp 33
6.7.5 stirling-y-bell.cpp 33
6.7.6 subfactorial.cpp 33
6.8 **Fibonacci y Pisano** 33
6.8.1 fibonacci-mod-m-extra.cpp 33
6.8.2 pisano-period-extra.cpp 33
6.9 **Josephus** 33
6.9.1 josephus.cpp 33
6.10 **Métodos numéricos** 33
6.10.1 fraction-extra.cpp 33
6.10.2 simpson.cpp 34
6.10.3 stablesum.cpp 34
6.11 **Polinomios y transformadas** 34
6.11.1 bairstow-vasito.cpp 34
6.11.2 fast-hadamard-transform.cpp 34
6.11.3 fft-operations-vasito.cpp 34
6.11.4 fft-shifts-trick.cpp 35
6.11.5 fft.cpp 35
6.11.6 interpol-o-n-ex.cpp 36
6.11.7 karatsuba-vasito.cpp 36
6.11.8 ntt-vasito.cpp 36
6.11.9 poly-definitions.cpp 36
6.11.10 polynomial-vasito.cpp 36
6.12 **Programación lineal** 37
6.12.1 simplex-integer-extra.cpp 37
6.12.2 simplex.cpp 37
6.13 **Recurrencias lineales** 38
6.13.1 berlekamp-massey-vasito.cpp 38
6.13.2 linear-recurrence-cayley-halmiton.cpp 38
6.13.3 linear-recurrence-matrix-exponciation.cpp 38
6.14 **Sumas de pisos** 38
6.14.1 floor-division-vasito.cpp 38
6.14.2 floor-sums.cpp 38
6.14.3 harmonic-lemma.cpp 39
6.14.4 points-under-line-vasito.cpp 39
6.14.5 sqrt-mod.cpp 39
6.15 **Teoría de números › Diofánticas** 39
6.15.1 diophantine-ecuations-extra.cpp 39
6.16 **Teoría de números › Divisores** 39
6.16.1 divisors-extra.cpp 39
6.16.2 divisors-vasito.cpp 39
6.17 **Teoría de números › Euclides** 39
6.17.1 extended-euclides-extra.cpp 39
6.17.2 fast-gcd.cpp 39
6.17.3 greatest-common-divisor-extra.cpp 39
6.17.4 lowest-common-multiple-extra.cpp 39
6.18 **Teoría de números › Funciones multiplicativas** 40
6.18.1 euler-totient.cpp 40
6.18.2 gcd-a-i-a-j-k-text-queries.cpp 40
6.18.3 gcd-i-j-k.cpp 40
6.18.4 linear-sieve.cpp 40
6.18.5 mobius-inclusion-exclusion-example.tex 40
6.18.6 mobius.cpp 40
6.18.7 multiplicative-functions.tex 40
6.18.8 sigma-n-i-1-frac-n-gcd-i-n.cpp 41
6.19 **Teoría de números › Primos y factorización** 41
6.19.1 count-primes-with-pi-function.cpp 41
6.19.2 fermat.cpp 41
6.19.3 optimized-polard-rho.cpp 41
6.19.4 prime-factorization-extra.cpp 42
6.19.5 prime-numbers.cpp 42
6.19.6 sieve-of-eratosthenes-extra.cpp 42
6.19.7 sieve.cpp 42
6.20 **Teoría y fórmulas** 42
6.20.1 formulas-extra.tex 42
6.20.2 formulas.cpp 43
6.20.3 math-concepts.txt 43
6.20.4 pi.cpp 43
6.20.5 sequences-extra.tex 43
6.20.6 theo.cpp 44
6.20.7 theorems-vasito.txt 44
6.20.8 theorems.tex 44

7 Geometría 44
7.1 **Círculos y triángulos** 44
7.1.1 circle-extra.cpp 44
7.1.2 heron-formula.cpp 44
7.1.3 triangles-and-circles.cpp 44
7.2 **Convex Hull** 45
7.2.1 convexhull.cpp 45
7.3 **Delaunay y Voronoi** 45
7.3.1 stanford-delaunay.cpp 45
7.3.2 voronoy.cpp 45
7.4 **Geometría 3D** 46
7.4.1 a-geo3d-esencial.cpp 46
7.4.2 b-geo3d-solidos.cpp 49
7.4.3 c-geo3d-avanzado.cpp 53
7.4.4 cheat-sheet-3d.md 55
7.4.5 README.md 57
7.5 **Grafos planares** 58
7.5.1 planar-graph-dual.cpp 58
7.6 **Minkowski** 58
7.6.1 minkowski-sum-of-convex-polygons.cpp 58
7.7 **Polígonos** 58
7.7.1 centroide.cpp 58
7.7.2 lattice-points.cpp 58
7.7.3 point-in-convex-polygon.cpp 58
7.7.4 point-in-general-polygon.cpp 59
7.7.5 polygon-diameter.cpp 59
7.7.6 polygon-extra.cpp 60
7.8 **Proximidad y KD Tree** 60
7.8.1 closest-pair-of-points.cpp 60
7.8.2 kdtree.cpp 61
7.8.3 nearest-neighbour.cpp 61
7.9 **Puntos rectas y segmentos** 61
7.9.1 line-extra.cpp 61
7.9.2 point-extra.cpp 61
7.9.3 radial-order-vasito.cpp 62
7.9.4 segment-extra.cpp 62
7.9.5 segment-intersection.cpp 62
7.10 **Semiplanos** 63
7.10.1 halfplanes-extra.cpp 63
7.11 **Teoría y fórmulas** 63
7.11.1 some-formulas.tex 63
7.12 **Tres dimensiones** 63
7.12.1 latlong.cpp 63
7.12.2 plane-vasito.cpp 63

8 Búsqueda y ordenamiento 63
8.1 **Búsqueda binaria paralela** 63
8.1.1 parallel-binary-search.cpp 63
8.2 **Búsqueda ternaria** 63
8.2.1 ternary-search.cpp 63
8.3 **Divide and Conquer** 64
8.3.1 divide-and-conquer.cpp 64

8.4 Greedy y comparadores 64
8.4.1 exchange-argument.cpp 64
8.5 Merge sort e inversiones 64
8.5.1 merge-sort.cpp 64
9 Strings 64
9.1 Aho Corasick 64
9.1.1 ahocorasik.cpp 64
9.1.2 dinamic-aho-corasick.cpp 64
9.2 Hashing 65
9.2.1 fast-hashing.cpp 65
9.3 KMP 65
9.3.1 kmp-automaton.cpp 65
9.3.2 kmp-vasito.cpp 65
9.4 Manacher 65
9.4.1 manacher.cpp 65
9.5 Palindromic Tree 66
9.5.1 palindromic-tree.cpp 66
9.6 Rotación mínima 66
9.6.1 minimum-expression-extra.cpp 66
9.7 Suffix Array y LCP 66
9.7.1 suffix-array-extra.cpp 66
9.8 Suffix Automaton 66
9.8.1 suffix-automaton-extra.cpp 66
9.9 Suffix Tree 67
9.9.1 suffix-tree-vasito.cpp 67
9.10 Trie 67
9.10.1 trie.cpp 67
9.11 Z Function 67
9.11.1 z-function-vasito.cpp 67
10 Utilidades 67
10.1 Aleatoriedad 67
10.1.1 randoms.cpp 67
10.2 Bits 68
10.2.1 bit-tricks.cpp 68
10.2.2 bits-manipulation-extra.cpp 68
10.2.3 optimizing-pragmas-for-bitset.cpp 68
10.3 CPP 68
10.3.1 bultin-functions-out-dated.cpp 68
10.3.2 cpp-stuff.cpp 68
10.3.3 exponential-notation.cpp 68
10.4 Entrada salida 68
10.4.1 io-int128.cpp 68
10.4.2 python-fast-in.py 68
10.4.3 scanf-and-printf-extra.cpp 68
10.4.4 split-string-extra.cpp 68
10.4.5 string-streams.cpp 68
10.5 Fechas 68
10.5.1 dates.cpp 68
10.6 Interactivos 69
10.6.1 main.cpp 69
10.7 Interactivos › pruebas 69
10.7.1 interactive-tester-vasito.py 69
10.8 Python 69
10.8.1 decimal-precisition-python.py 69
10.8.2 python.py 69
10.9 Rendimiento 69
10.9.1 pragmas.cpp 69
11 Python 69
11.1 enteros_y_bits.py 69
11.2 estructuras.py 70
11.3 problemas_python_vs_cpp.py 70
11.4 template_completo.py 71
11.5 template.py 71

§0 Template

§0.1 simple.cpp

00_Template

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define fast ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0)
#define pb push_back
#define F first
#define S second
#define all(x) (x).begin(), (x).end()
#define sortt(x) sort(all(x))
#define sortn(x, n) sort((x), (x) + (n));
#define pi pair<int, int>
#define vi vector<int>
#define vb vector<bool>
#define vpi vector<pair<int, int>>
#define vs vector<string>
#define endl '\n'
void solve(){
}
signed main(){
    fast;
    // int t; cin >> t;
    // while(t--)
    solve();
}
```

§1 Estructuras

§1.1 Colas monótonas

§1.1.1 min-queue.cpp

01_Estructuras/Colas_monotonas

```cpp
// > Funciona como una queue<> que además permite hallar el elemento mínimo en O(1). Para mantener el máximo: cambiar > por <.
template<typename T>
struct mnque {
    deque<T> dq, mn;
    void push(T x) {
        dq.pb(x);
        while (mn.size() && mn.back() > x) mn.pop_back(); // o < para max
        mn.pb(x);
    }
    void pop() {
        if (dq.front() == mn.front()) mn.pop_front();
        dq.pop_front();
    }
    T min() { return mn.front(); }
};
mnque<int> q;
```

§1.2 Descomposición SQRT

§1.2.1 mo-xd.cpp

01_Estructuras/Descomposicion_SQRT

```
square decomposition.
dividir array en sqrt(n) bloques => por lo tanto si tenemos n = 1e5 -> sqrt(n) = 300. Por lo tanto es posible tener mas o menos 3e7 -> (es pesado).
Operaciones => actualizaciones O(1), no entrara log(...) -> es muy caro
(suele ser trampa: brute force mas optimizado - inteligente)
algoritmo MO: avanzada de square decomposition.
1. agrupar consultas adyacentes maximo sqrt(q)
2. ordenar por end
3. tenemos 3 operaciones que se deben hacer: add, remove, query. *** la mayoria o es dificil ponerle un update a la estructura -> debe ser bien pensado.
```

§1.2.2 sqrt-descomposition.cpp

01_Estructuras/Descomposicion_SQRT

```cpp
// Suma en rango y asignación puntual en O(sqrt(n)) por operación.
// Uso: leer n, q y los n valores en v; llamar sqrt_(). Consultas (1-indexadas):
//   "1 l r" -> imprime la suma de v[l..r]      "2 i x" -> v[i] = x
// Tested (query + bloques):
https://judge.yosupo.jp/problem/point_add_range_sum
vi v, p;
int block, n, q;
int query(int i) { // suma de v[0..i] (0-indexado); i < 0 -> 0
    if (i<0) return 0;
    int res = 0;
    for (int j = 0; j<i/block; j++) res+=p[j];
    for (int j = (i/block)*block; j<=i; j++) res+=v[j];
    return res;
}
void sqrt_() {
    block = sqrt(n)+2;
    v.resize(n, 0), p.assign(block, 0);
    for (int i =0 ; i<n;i++) p[i/block]+=v[i];
    for (; q--; ) {
        int type; cin>>type; // antes "if (query)": evaluaba el puntero a la función (siempre true)
        if (type == 1) {
            int l, r; cin>>l>>r; l--, r--;
            cout<<query(r)-query(l-1)<<endl; // sum in range
        } else {// point update
            int i, x; cin>>i>>x; i--;
            p[i/block]-=v[i];
            v[i] = x;
            p[i/block]+=v[i];
        }
    }
}
```

§1.3 DSU

§1.3.1 dsu.cpp

01_Estructuras/DSU

```cpp
struct DSU {
    vi p;
    DSU(int ni) {p.resize(ni+1, -1);}
    int find(int x) {return p[x] < 0? x: p[x] = find(p[x]);}
    void merge(int a, int b) {
        a = find(a), b = find(b);
        if (a==b) return;
        if (p[a] > p[b]) swap(a, b);
        p[a]+=p[b];
        p[b] = a;
    }
    int siz(int x) {return -p[find(x)];}
};
```

§1.3.2 union-find-partial-persistent.cpp

01_Estructuras/DSU

```cpp
// DSU parcialmente persistente: unite y find(a, tiempo) O(log n).
// his[x] = tiempo en que x recibió padre; t = cantidad de uniones efectivas.
int t, par[N], sz[N], his[N];
```

```
int find(int a, int t){
    if(par[a] == a) return a;
    if(his[a] > t) return a;
    return find(par[a], t);
}
void unite(int a, int b){
    if(find(a, t) == find(b, t)) return;
    a = find(a, t), b = find(b, t), t++;
    if(sz[a] < sz[b]) swap(a, b);
    sz[a] += sz[b], par[b] = a, his[b] = t;
}
// Llamar al inicio del main (antes estaba suelto fuera de una
función y no compilaba).
// Requiere definir N antes de incluir este archivo.
void initDSU(){
    t = 0;
    for(int i = 0; i < N; i++) par[i] = i, sz[i] = 1, his[i] = 0;
}
```

§1.3.3 union-find-rollback.cpp

01_Estructuras/DSU

```
using pii = pair<int, int>;
// DSU con rollback: unite y find O(log n) (sin compresión de
caminos), rollback O(1).
// par = padre, sz[find(x)] = tamaño; sp/ss = pilas (nodo, valor
previo) para deshacer.
int par[N], sz[N];
stack <pii> sp, ss;
int find (int a) { return par[a] == a ? a : find(par[a]); }
void unite (int a, int b) {
    if ((a = find(a)) == (b = find(b))) return;
    if (sz[a] < sz[b]) swap(a, b);
    ss.push({a, sz[a]});
    sp.push({b, par[b]});
    sz[a] += sz[b];
    par[b] = a;
}
void rollback() {
    par[sp.top().st] = sp.top().nd; sp.pop();
    sz[ss.top().st]  = ss.top().nd; ss.pop();
}
signed main(){
    for (int i = 0; i < N; i++) par[i] = i, sz[i] = 1;
    return 0;
}
```

§1.4 Fenwick

§1.4.1 fenwick-tree-2d.cpp

01_Estructuras/Fenwick

```
using ll = long long;
using vl = vector<long long>;
struct BIT2D { // 1-indexed
    vector<vl> bit;
    ll n, m;
    BIT2D(ll n, ll m) : bit(n+1, vl(m+1)), n(n), m(m) {}
    ll lsb(ll i) { return i & -i; }
    void add(int row, int col, ll x) {
        for (int i = row;i<=n;i+=lsb(i))
            for (int j = col;j<=m;j+=lsb(j))
                bit[i][j] += x;
    }
    ll sum(int row, int col) {
        ll res = 0;
        for (int i = row;i>0;i-=lsb(i))
            for (int j = col;j>0;j-=lsb(j))
                res += bit[i][j];
        return res;
    }
    ll sum(int x1, int y1, int x2, int y2) {
        return sum(x2,y2) - sum(x1-1,y2) - sum(x2,y1-1) + sum(x1-
1,y1-1);
    }
    void set(int x, int y, ll val) {
        add(x,y,val-sum(x,y,x,y));
    }
};
```

§1.4.2 fenwick-tree.cpp

01_Estructuras/Fenwick

```
struct FenwickTree {
    vector<int> bit;
    int n;
    FenwickTree(int n) {
        this->n = n;
        bit.assign(n, 0);
    }
    FenwickTree(vector<int> a) : FenwickTree(a.size()) {
        for (size_t i = 0; i < a.size(); i++)
            add(i, a[i]);
    }
    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret += bit[r];
        return ret;
    }
    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }
    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] += delta;
    }
    // TODO: Lower Bound
};
```

§1.5 Hash maps

§1.5.1 custom-hash-pair.cpp

01_Estructuras/Hash_maps

```
using ll = long long;
// Example: unordered_set<pair<ll,ll>, HASH> exists;
// It's better to convine with other custom hash
struct chash {
  size_t operator()(const pair<ll,ll>&x)const{
    return hash<ll>()(((ll)x.first)^(((ll)x.second)<<32));
  }
};
```

§1.5.2 custom-hash.cpp

01_Estructuras/Hash_maps

```
// Avoid hashing hacks and improve performance of hash structures
// e.g. unordered_map<ll,ll,custom_hash>
struct custom_hash_splitmix64 {
    static uint64_t splitmix64(uint64_t x) {
        // http://xorshift.di.unimi.it/splitmix64.c
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

§1.5.3 fast-hash-map.cpp

01_Estructuras/Hash_maps

```
#include <ext/pb_ds/assoc_container.hpp>
using ll = long long;
using namespace __gnu_pbds;
// gp_hash_table usa los bits BAJOS del hash como índice. Con "x ^
RANDOM" las claves que
// comparten bits bajos (ej. múltiplos de 2^k) caen todas en el mismo
grupo: TLE en
// Library Checker associative_array (caso 2_powers). splitmix64
mezcla todos los bits.
struct chash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(const ll &x) const {
        static const uint64_t RANDOM =
chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + RANDOM);
    }
};
// There is no mp.count(x), so use mp.find(x) != mp.end()
gp_hash_table<ll,ll,chash> mp;
```

§1.6 Heaps

§1.6.1 heap-sort.cpp

01_Estructuras/Heaps

```
// Max-heap en un vector: v.pb(x); insert(v) | del(v, i) borra la
posición i | find(v, x, 0) índice de x o -1.
// Corregido: find comprobaba i>n (leía v[n]); del dejaba el último
elemento sin subir (rompía
// el heap) y usaba -1 como centinela (fallaba con valores
negativos).
int find(vi &v, int x, int i) {
    int n = v.size();
    //cout<<i<<endl;
    if (i>=n || v[i]<x) return -1;
    if (v[i]==x) return i;
    int a = i*2+1, b = i*2+2;
    return max(find(v,x,a),find(v,x,b));
}
void insert(vi &v) {
    int i = v.size()-1;
    while (i>0) {
        int p = i;
        if (!(p&1)) p--;
        p>>=1;
            if (v[p]>v[i]) break;
        swap(v[p], v[i]);
        i = p;
    }
}
void del(vi &v, int i) {
    v[i] = LLONG_MIN; // centinela menor que cualquier valor
    int a = i*2+1,b=i*2+2, n=v.size();
    int max = -1;
    if(a<n) max = a;
    if(b<n)
           if(v[max]<v[b])
        max = b;
    if(max!=-1)
        swap(v[i], v[max]), del(v,max);
    else {
        swap(v[i],v[n-1]);
        v.pop_back();
        // el elemento que llegó desde el final puede ser mayor que
su nuevo padre: subirlo
        for (; i > 0 && i < (int)v.size() && v[(i-1)/2] < v[i]; i =
(i-1)/2) swap(v[(i-1)/2], v[i]);
    }
```

```
}
void solve() {
    int n;
    vi v; cin>>n;
    for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i) {
        int x; cin>>x;
        v.pb(x);
        insert(v);
    }
    del(v,3);
    n = v.size();
    for (int i=(0), nb_fin_3=(n); i<nb_fin_3; ++i)
        cout<<v[i]<<" \n"[i==n-1];
    cout<<find(v,2,0)<<endl;
}
```

§1.6.2 priority-queue.cpp

01_Estructuras/Heaps

```
template<class T> using pql =
priority_queue<T,vector<T>,greater<T>>;// less first
template<class T> using pqg = priority_queue<T>; // greater first
```

§1.7 Intervalos y dominancia

§1.7.1 disjoint-intervals.cpp

01_Estructuras/Intervalos_y_dominancia

```
// stores disjoint intervals as [first, second)
struct disjoint_intervals {
    set<pair<int,int> > s;
    void insert(pair<int,int> v){
        if(v.first>=v.second) return;
        auto at=s.lower_bound(v);auto it=at;
        if(at!=s.begin()&&(--at)->second>=v.first)v.first=at->first,--it;
        for(;it!=s.end()&&it->first<=v.second;s.erase(it++))
            v.second=max(v.second,it->second);
        s.insert(v);
    }
};
```

§1.7.2 gain-cost-set-vasito.cpp

01_Estructuras/Intervalos_y_dominancia

```
// stores pairs (benefit,cost) (erases non-optimal pairs)
struct GCS {
    set<pair<int,int> > s;
    void add(int g, int c){
        pair<int,int> x={g,c};
        auto p=s.lower_bound(x);
        if(p!=s.end()&&p->second<=x.second)return;
        if(p!=s.begin()){ // erase pairs with less benefit
            --p;            // and more cost
            while(p->second>=x.second){
                if(p==s.begin()){s.erase(p);break;}
                s.erase(p--);
            }
        }
        s.insert(x);
    }
    int get(int gain){ // min cost for some benefit
        auto p=s.lower_bound((pair<int,int>){gain,-INF});
        int r=p==s.end()?INF:p->second;
        return r;
    }
};
```

§1.8 Mo

§1.8.1 mo-s-hilbert-curve.cpp

01_Estructuras/Mo

```
/*
Hilbert Curve for Mo's,
This is a better ordering for Mo (review Mo's algorithm),
sometimes it take the half time, other times it takes
more time. So it needs a revision to identify when it's better!!!
https://codeforces.com/blog/entry/61203
*/
inline int64_t gilbertOrder(int x, int y, int pow, int rotate) {
    if (pow == 0) {
        return 0;
    }
    int hpow = 1 << (pow-1);
    int seg = (x < hpow) ? (
        (y < hpow) ? 0 : 3
    ) : (
        (y < hpow) ? 1 : 2
    );
    seg = (seg + rotate) & 3;
    const int rotateDelta[4] = {3, 0, 0, 1};
    int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
    int nrot = (rotate + rotateDelta[seg]) & 3;
    int64_t subSquareSize = int64_t(1) << (2*pow - 2);
    int64_t ans = seg * subSquareSize;
    int64_t add = gilbertOrder(nx, ny, pow-1, nrot);
    ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
    return ans;
}
struct Query {
    int l, r, idx;
    int64_t ord;
    inline void calcOrder() {
        ord = gilbertOrder(l, r, 21, 0); // n,q <= 1e5
    }
};
inline bool operator<(const Query &a, const Query &b) {
    return a.ord < b.ord;
}
```

§1.8.2 mo-s.cpp

01_Estructuras/Mo

```
using ll = long long;
using vl = vector<long long>;
/*
Mo's algorithm O((N + Queries)*SQRT(N))
It answers queries offline using SQRT Descomposition,
sometimes get TLE when you have Set, Map, various iteration when adding,
So you need to implement efficiently (with arrays) in O(1). You can answer
questions like in a range, the number of distincts values.
*/
/*
Uso: las consultas [l, r] (0-indexadas, inclusivas, l <= r) e idx = posición de la respuesta.
add(i) / remove(i) actualizan el estado al meter / sacar el índice i; getAnswer() lo lee.
Se pasan como lambdas (antes estaban definidas vacías en este archivo y no se podían completar).
Ejemplo: cantidad de valores distintos (valores comprimidos a [0, n)):
    vi cnt(n); ll distintos = 0;
    auto add = [&](int i) { if (cnt[a[i]]++ == 0) distintos++; };
    auto rem = [&](int i) { if (--cnt[a[i]] == 0) distintos--; };
    vector<ll> res = mo(queries, n, add, rem, [&]() { return distintos; });
Tested https://judge.yosupo.jp/problem/static_range_count_distinct (N = Q = 5e5)
*/
int BLOCK_SIZE = 430; // mo() lo recalcula: ~ n / sqrt(q) (antes fijo: 1e5 -> 310, 2e5 -> 430)
struct query {
    int l, r, idx;
    bool operator <(const query &other) const {
        if (l / BLOCK_SIZE != other.l / BLOCK_SIZE) return l < other.l;
        // orden par/impar: en bloques impares r decrece, así r no vuelve al inicio en cada bloque
        return (l / BLOCK_SIZE) & 1 ? r > other.r : r < other.r;
    }
};
template<class Add, class Remove, class Get>
vector<ll> mo(vector<query> queries, int n, Add add, Remove remove, Get getAnswer) {
    vector<ll> answers(queries.size());
    if (queries.empty()) return answers;
    BLOCK_SIZE = max<ll>(1, n / max<ll>(1, sqrtl(queries.size())));
    int l = 0;
    int r = -1;
    sort(all(queries));
    for (auto &q : queries) {
        while (q.l < l) add(--l);
        while (r < q.r) add(++r);
        while (l < q.l) remove(l++);
        while (q.r < r) remove(r--);
        answers[q.idx] = getAnswer();
    }
    return answers;
}
```

§1.9 Ordered Set PBDS

§1.9.1 multiorderedset.cpp

01_Estructuras/Ordered_Set_PBDS

```
// Requisito: el comparador less_equal no es un orden estricto: usar solo los métodos del struct (probados en _pruebas/estres);
oset.find/erase(valor) directos no funcionan.
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
using ll = long long;
using namespace __gnu_pbds;
struct multiordered_set {
    tree<ll,
        null_type,
        less_equal<ll>, // this is the trick
        rb_tree_tag,
        tree_order_statistics_node_update> oset;
    void insert(ll x) {
        oset.insert(x);
    }
    bool exists(ll x) {
        auto it = oset.upper_bound(x);
        if (it == oset.end()) {
            return false;
        }
        return *it == x;
    }
    void erase(ll x) {
        if (exists(x)) {
            oset.erase(oset.upper_bound(x));
        }
    }
    ll find_by_order(ll pos) {
        return *(oset.find_by_order(pos));
    }
    int first_index(ll x) {
        if (!exists(x)) {
            return -1;
        }
        return (oset.order_of_key(x));
    }
    int last_index(ll x) {
        if (!exists(x)) {
            return -1;
```

```
        }
        if (find_by_order(size() -1) == x) {
            return size() - 1;
        }
        return first_index(*oset.lower_bound(x)) -1;
    }
    int count(ll x) {
        if (!exists(x)) {
            return -1;
        }
        return last_index(x) - first_index(x) + 1;
    }
    // cantidad de valores en [l, r]; first_index/last_index/count(x)
devuelven -1 si x no está
    int count(ll l, ll r) {
        auto left = oset.upper_bound(l);
        if (left == oset.end() || *left>r) {
            return 0;
        }
        auto right = oset.upper_bound(r);
        if (right != oset.end()) {
            right = oset.find_by_order(oset.order_of_key(*right));
        }
        if (right == oset.end() || *right >r) {
            if (right == oset.begin()) return 0;
            right--;
        }
        return last_index(*right)-first_index(*left)+1;
    }
    void clear() {
        oset.clear();
    }
    ll size() {
        return (ll)oset.size();
    }
};
```

§1.9.2 orderedset.cpp

01_Estructuras/Ordered_Set_PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using ll = long long;
// Same as set, but you can get the order of an element or the
// element in given sorted position in O(log n)
using namespace __gnu_pbds;
#define oset tree<ll, null_type,less<ll>,
rb_tree_tag,tree_order_statistics_node_update>
//find_by_order(k) order_of_key(k)
```

§1.10 Rope

§1.10.1 rope.cpp

01_Estructuras/Rope

```
#include <ext/rope>
// rope (GNU): insertar en la posición i, cortar y reinsertar
subcadenas en O(log n).
// En la práctica ~2 veces más lento que un treap propio (TLE con 2e5
en CSES 1749).
using namespace __gnu_cxx;
void ropeExample() {
    rope<int> v(5,0); // initialize with 5 zeroes
    for (int i=0; i<(int)v.size(); ++i) v.mutable_reference_at(i) =
i+1;
    for (int i=0; i<5; ++i) v.pb(i+1); // constant time pb
    rope<int> cur = v.substr(1,2);
    v.erase(1,3); // erase 3 elements starting from 1st element
    for (rope<int>::iterator it = v.mutable_begin();
        it != v.mutable_end(); ++it) cout << (int)*it << ' ';
    cout << '\n'; // 1 5 1 2 3 4 5
    v.insert(v.mutable_begin()+2,cur); // or just 2
    v += cur; for (int i=0; i<(int)v.size(); ++i) cout << v[i] << '
';
    cout << '\n'; // 1 5 2 3 1 2 3 4 5 2 3
}
// push_back/pop_back/substr/replace O(log n); insert O(log n), peor
caso O(n).
```

§1.11 Segment Tree › Básico

§1.11.1 general-segtree.cpp

01_Estructuras/Segment_Tree/Basico

```
using ll = long long;
// >>>>>>>>> Implement
struct Node { ll x = 0; };
Node e() { return Node(); } // null element
Node op(Node &a, Node &b) { // operation
    Node c;
    c.x = a.x + b.x;
    return c;
}
// <<<<<<<
struct segtree {
    vector<Node> t;
    ll n;
    void init(int n) {
        t.assign(n * 2, e());
        this->n = n;
    }
    void init(vector<Node>& s) {
        n = s.size();
        t.assign(n * 2, e());
        for (int i = 0; i < n; i++) {
            t[i+n] = s[i];
        }
        build();
    }
    void build() {  // build the tree
        for (int i = n - 1; i > 0; --i) t[i] = op(t[i<<1],
t[i<<1|1]);
    }
    // set value at position p
    void update(int p, const Node& value) {
        for (t[p += n] = value; p >>= 1; ) t[p] = op(t[p<<1],
t[p<<1|1]);
    }
    // sum on interval [l, r]
    Node query(int l, int r) {
        r++; // make this inclusive
        Node resl=e(), resr=e(); // null element
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l&1) resl = op(resl, t[l++]);
            if (r&1) resr = op(t[--r], resr);
        }
        return op(resl, resr);
    }
    Node get(int i) {
        return query(i, i); // improve to o(1)
    }
    // TODO: implement left, right binary search
};
```

§1.12 Segment Tree › Dinámico

§1.12.1 lazy-creation.cpp

01_Estructuras/Segment_Tree/Dinamico

```
struct st
{
    int val;
    st* r;
    st* l;
    st() {r = l = NULL; val = 0;}
    st(int v) {r = l = NULL; val = v;}
    st(st* L, st* R) {l = L; r = R, val = l->val + r->val;}
};
typedef st* pst;
void update(pst &t, int b, int e, int pos, int val)
{
    if(!t) t = new st();
    if(b == e)
    {
        t->val += val;
        return;
    }
    int mid = (b + e) / 2;
    if(mid >= pos)
        update(t->l, b, mid, pos, val);
    else
        update(t->r, mid + 1, e, pos, val);
    t->val = (t->l?t->l->val:0) + (t->r?t->r->val:0);
}
int query(pst t, int b, int e, int i, int j)
{
    if(!t)
        return 0;
    if(b >= i && e <= j)
        return t->val;
    int mid = (b + e) / 2;
    if(mid >= j)
        return query(t->l, b, mid, i, j);
    if(mid < i)
        return query(t->r, mid +1, e, i, j);
    return query(t->l, b, mid, i, j) + query(t->r, mid +1, e, i, j);
}
```

§1.13 Segment Tree › Dos dimensiones

§1.13.1 segment-tree-2d.cpp

01_Estructuras/Segment_Tree/Dos_dimensiones

```
using ll = long long;
// Tested: https://www.spoj.com/problems/MATSUM/
struct Node { ll x = 0; };
Node e() { return Node(); } // Identity element, op(x,e()) = x
// Binary Operation, Associative. (a+b)+c = a+(b+c)
Node op(const Node &a, const Node &b) {
    Node c;
    c.x = a.x + b.x;
    return c;
}
struct segtree2d {
    int n, m;
    vector<vector<Node>> t;
    void init(int n, int m) {
        this->n = n;
        this->m = m;
        t.assign(n*2, vector<Node>(m*2, e()));
    }
    void init(int n, int m, vector<vector<Node>> &nums) {
        init(n, m);
        for (int i = 0;i<n;i++) { // build leaf segtrees
            for (int j=0;j<m;j++) t[i+n][j+m] = nums[i][j];
            for (int j=m-1;j>0;j--) t[i+n][j] = op(t[i+n][j*2],
t[i+n][j*2+1]);
        }
        for (int i = n-1;i>0;i--) { // build non leaf segtrees
            for (int j=0;j<2*m;j++) t[i][j] = op(t[i*2][j], t[i*2+1]
[j]);
        }
    }
    Node query_y(int x, int l, int r) {
        l += m, r += m+1;
        Node resl = e(), resr = e();
```

```
        for (; l < r; l>>=1, r>>=1) {
            if (l&1) resl = op(resl, t[x][l++]);
            if (r&1) resr = op(t[x][--r], resr);
        }
        return op(resl, resr);
    }
    Node query(int x1, int y1, int x2, int y2) {
        ll l = x1+n, r = x2+n+1;
        Node resl = e(), resr = e();
        for (; l < r; l>>=1, r>>=1) {
            if (l&1) resl = op(resl, query_y(l++, y1, y2));
            if (r&1) resr = op(query_y(--r,y1,y2), resr);
        }
        return op(resl,resr);
    }
    void update(int x, int y, const Node &v) {
        x += n, y += m;
        t[x][y] = v; // fix leaf segtrees
        for (int j = y; j>>=1; ) {
            t[x][j] = op(t[x][j*2], t[x][j*2+1]);
        }
        for (; x >>= 1; ) { // fix non leaf segtree
            t[x][y] = op(t[x*2][y], t[x*2+1][y]);
            for (int j = y; j>>=1; ) {
                t[x][j] = op(t[x*2][j], t[x*2+1][j]);
            }
        }
    };
    Node get(int x, int y) { return t[x+n][y+m]; }
};
```

§1.14 Segment Tree › Lazy

§1.14.1 general-lazy-tree.cpp

01_Estructuras/Segment_Tree/Lazy

```
// Requisito de selección: Inicializar todos los valores con update antes de sumar en rangos: el neutro INT64_MAX no admite incrementos positivos.
using ll = long long;
// TODO: Make this iterative
struct Node { // Structure
    ll mn;
    ll size = 1;
    Node(ll mn):mn(mn) {
    }
};
struct Func { // Applied function
    ll a = 0;
};
Node e() { // op(x, e()) = x
    Node a(INT64_MAX); // neutral element
    return a;
};
Func id() { // mapping(x, id()) = x
    Func l = {0}; // identify func
    return l;
}
Node op(Node &a, Node &b) { // associative property
    Node c = e();           // binary operation
    c.size = a.size + b.size;
    c.mn = min(a.mn, b.mn);
    return c;
}
Node mapping(Node node, Func &lazy) {
    node.mn += lazy.a; // appling function
    return node;
}
Func composicion(Func &prev, Func &actual) {
    prev.a = prev.a + actual.a; // composing funcs
    return prev;
}
struct lazytree {
    int n;
    vector<Node> nodes;
    vector<Func> lazy;
    void init(int nn) {
        n = nn;
        int size = 1;
        while (size < n) {
            size *= 2;
        }
        ll m = size *2;
        nodes.assign(m, e());
        lazy.assign(m, id());
    }
    void push(int i, int sl, int sr) {
        nodes[i] = mapping(nodes[i], lazy[i]);
        if (sl != sr) {
            lazy[i * 2 + 1] = composicion(lazy[i*2+1],lazy[i]);
            lazy[i * 2 + 2] = composicion(lazy[i*2+2],lazy[i]);
        }
        lazy[i] = id();
    }
    void apply(int i, int sl, int sr, int l, int r, Func f) {
        push(i, sl, sr);
        if (l <= sl && sr <= r) {
            lazy[i] = f;
            push(i,sl,sr);
        } else if (sr < l || r < sl) {
        } else {
            int mid = (sl + sr) >> 1;
            apply(i * 2 + 1, sl, mid, l, r, f);
            apply(i * 2 + 2, mid + 1, sr, l, r, f);
            nodes[i] = op(nodes[i*2+1],nodes[i*2+2]);
        }
    }
    void apply(int l, int r, Func f) {
        apply(0, 0, n - 1, l, r, f);
    }
    void update(int i, Node node) {
        update(0, 0, n-1, i, node);
    }
    void update(int i, int sl, int sr, int pos, Node node) {
        // push SIEMPRE (también en el hijo que no contiene pos): si no, el padre se recalcula
        // con el valor viejo de ese hijo y se pierde su lazy pendiente.
        push(i,sl,sr);
        if (sl <= pos && pos <= sr) {
            if (sl == sr) {
                nodes[i] = node;
            } else {
                int mid = (sl + sr) >> 1;
                update(i * 2 + 1, sl, mid, pos, node);
                update(i * 2 + 2, mid + 1, sr, pos, node);
                nodes[i] = op(nodes[i*2+1], nodes[i*2+2]);
            }
        }
    }
    Node query(int i, int sl, int sr, int l, int r) {
        push(i,sl,sr);
        if (l <= sl && sr <= r) {
            return nodes[i];
        } else if (sr < l || r < sl) {
            return e();
        } else {
            int mid = (sl + sr) >> 1;
            auto a = query(i * 2 + 1, sl, mid, l, r);
            auto b = query(i * 2 + 2, mid + 1, sr, l, r);
            return  op(a,b);
        }
    }
    Node query(int l, int r) {
        return query(0, 0, n - 1, l, r);
    }
};
```

§1.14.2 iterative-lazytree.cpp

01_Estructuras/Segment_Tree/Lazy

```
// Requisito de selección: Para max_right/min_left usar tamaño potencia de dos y predicado monótono que cumpla g(e()).
using ll = long long;
/*
Lazy Iterative Seg Tree.
This example is for https://cses.fi/problemset/task/1735/
*/
struct Node { ll x; };
const ll SET = 1;
const ll ADD = 2;
struct Func { ll type, val; }; // applied Function
Node e() { return { 0 }; }; // op(x, e()) = x
Func id() { return {ADD, 0}; } // mapping(x, id()) = x
Node op(Node &a, Node &b) { // associative property
    return {a.x + b.x};
}
Node mapping(Node node, ll sz, const Func &lazy) {
    if (lazy.type == SET) node.x = lazy.val * sz;
    else node.x += lazy.val * sz;
    return node;
}
Func comp(Func prev, const Func &actual) {
    if (actual.type==SET) return actual;
    if (prev.type == SET) return {SET, prev.val + actual.val};
    return {ADD, prev.val + actual.val };
}
struct lazyseg {
    ll n, h;
    vector<Node> t;
    vector<Func> lz;
    vector<int> sz; // if sz of node needed
    lazyseg(ll n) : n(n), t(2*n,e()), lz(n, id()), sz(2*n,1) {
        h = 32-__builtin_clz(n);
        for (int i = n-1;i>0;i--) sz[i] = sz[i*2]+sz[i*2+1];
    }
    void all_apply(ll x, const Func & f) {
        t[x] = mapping(t[x], sz[x], f);
        if (x<n) lz[x] = comp(lz[x], f);
    }
    void push(ll x) {
        all_apply(x*2, lz[x]);
        all_apply(x*2+1, lz[x]);
        lz[x] =id();
    }
    void build(ll x) {
        t[x] = op(t[x*2], t[x*2+1]);
    }
    bool ok(ll x, ll i) {
        return ((x>>i)<<i)!=x;
    }
    void update(ll p, Node v) {
        p += n;
        for (int i =h;i>0;i--) push(p>>i);
        t[p] = v;
        for (int i =1;i<=h;i++) build(p>>i);
    }
    Node query(ll l, ll r) {
        if (l>r) return e();
        l += n, r += n+1;
        for (int i=h;i>0;i--) {
```

```
        if (ok(l,i)) push(l>>i);
        if (ok(r,i)) push((r-1)>>i);
    }
    Node ansl = e(), ansr = e();
    for (;l<r;l>>=1,r>>=1) {
        if (l&1) ansl = op(ansl, t[l++]);
        if (r&1) ansr = op(t[--r], ansr);
    }
    return op(ansl, ansr);
}
void apply(ll l, ll r, const Func &f) {
    if (l>r) return;
    l += n, r += n+1;
    for (int i =h;i>0;i--) {
        if (ok(l,i)) push(l>>i);
        if (ok(r,i)) push((r-1)>>i);
    }
    {
        int l2 = l, r2 = r;
        for (;l<r;l>>=1,r>>=1) {
            if (l&1) all_apply(l++, f);
            if (r&1) all_apply(--r,f);
        }
        l = l2, r= r2;
    }
    for (int i =1;i<=h;i++) {
        if (ok(l,i)) build(l>>i);
        if (ok(r,i)) build((r-1)>>i);
    }
};
// Maximum r that satisfy g(op(t[l], t[l+1], ..., t[r-1])) = True
// if there's no g() that returns true, then r = l
template <class G> int max_right(int l, G g) {
    if (l == n) return n;
    l += n;
    for (int i = h; i >= 1; i--) push(l >> i);
    Node sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!g(op(sm, t[l]))) {
            while (l < n) {
                push(l);
                l = (2 * l);
                if (g(op(sm, t[l]))) {
                    sm = op(sm, t[l]);
                    l++;
                }
            }
            return l - n;
        }
        sm = op(sm, t[l]);
        l++;
    } while ((l & -l) != l);
    return n;
}
// If g is monotone, this is the minimum l that satisfies
// g(op(a[l], a[l + 1], ..., a[r - 1])) = true.
template <class G> int min_left(int r, G g) {
    if (r == 0) return 0;
    ll size = n;
    r += size;
    for (int i = h; i >= 1; i--) push((r - 1) >> i);
    Node sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!g(op(t[r], sm))) {
            while (r < size) {
                push(r);
                r = (2 * r + 1);
                if (g(op(t[r], sm))) {
                    sm = op(t[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(t[r], sm);
    } while ((r & -r) != r);
    return 0;
}
};
```

§1.14.3 lazy-st1.cpp

01_Estructuras/Segment_Tree/Lazy

```
struct ST {
    struct st {
        int val, lazy;
        st* l;
        st* r;
        st() : val(0), lazy(0), l(nullptr), r(nullptr) {}
        st(int v): val(v), lazy(0), l(nullptr), r(nullptr) {}
    };
    typedef st* pst;
    pst root;
    ST() {root = new st();}
    int get(pst &t) {return t ? t->val : 0;}
    void apply(pst &t, int b, int e, int v) {
        if (!t) return;
        t->val += (e - b) * v;
        t->lazy += v;
    }
    void push(pst &t, int b, int e) {
        if (!t || t->lazy == 0) return;
        int mid = (b + e) >> 1;
        if (!t->l) t->l = new st();
        if (!t->r) t->r = new st();
        apply(t->l, b, mid, t->lazy);
        apply(t->r, mid, e, t->lazy);
        t->lazy = 0;
    }
    void pull(pst &t) {t->val = get(t->l) + get(t->r);}
    void upd(pst &t, int l, int r, int i, int j, int val) {
        if (j <= l || r <= i) return;
        if (!t) t = new st();
        if (i <= l && r <= j) {
            apply(t, l, r, val);
            return;
        }
        push(t, l, r);
        int mid = (l+r)>>1;
        upd(t->l, l, mid, i, j, val);
        upd(t->r, mid, r, i, j, val);
        pull(t);
    }
    int query(pst &t, int l, int r, int i, int j) {
        if (!t || j <= l || r <= i) return 0;
        if (i <= l && r <= j) return t->val;
        push(t, l, r);
        int mid = (l+r)>>1;
        return query(t->l, l, mid, i, j) + query(t->r, mid, r, i, j);
    }
    void upd(int l, int r, int i, int j, int val) {
        upd(root, l, r, i, j, val);
    }
    int query(int l, int r, int i, int j) {
        return query(root, l, r, i, j);
    }
};
```

§1.14.4 polynomial-sum-lazy-segtree-problem.cpp

...as/Segment_Tree/Lazy

```
using ll = long long;
/* Polynomial Queries, queries
1. Increase [a,b] by 1, second by 2, third by 3, and so on
2. Sum of [a,b]
Use:
cin >> nums[i],tree.update(i, { nums[i] })
For 1: tree.apply(l,r,{0,1});
For 2: tree.query(l,r).sum
*/
struct Node { ll sum = 0; };
struct Func { ll add, ops; };
Node e() { return {0}; };
Func id() { return {0, 0}; }
Node op(Node a, Node b) { return {a.sum + b.sum }; }
ll f(ll x) { return x * (x+1)/2; }
Node mapping(Node node, Func lazy, ll sz) {
    return { node.sum + sz*lazy.add + lazy.ops*f(sz) };
}
Func composicion(Func prev, Func actual) {
    Func ans = { prev.add + actual.add, prev.ops + actual.ops };
    return ans;
}
Func sumF(Func f, ll x) { return {f.add + x*f.ops, f.ops }; }
struct lazytree {
    int n;
    vector<Node> nodes;
    vector<Func> lazy;
    void init(int nn) {
        n = nn;
        int size = 1;
        while (size < n) { size *= 2; }
        ll m = size * 2;
        nodes.assign(m, e());
        lazy.assign(m, id());
    }
    void push(int i, int sl, int sr) {
        nodes[i] = mapping(nodes[i], lazy[i], sr-sl+1);
        if (sl != sr) {
            ll cnt = (sr+sl)/2-sl+1; // changed
            lazy[i * 2 + 1] = composicion(lazy[i*2+1],lazy[i]);
            lazy[i * 2 + 2] =
composicion(lazy[i*2+2],sumF(lazy[i],cnt));
        }
        lazy[i] = id();
    }
    void apply(int i, int sl, int sr, int l, int r, Func f) {
        push(i, sl, sr);
        if (l <= sl && sr <= r) {
            lazy[i] = sumF(f, abs(sl-l)); //Changed
            push(i,sl,sr);
        } else if (sr < l || r < sl) {
        } else {
            int mid = (sl + sr) >> 1;
            apply(i * 2 + 1, sl, mid, l, r, f);
            apply(i * 2 + 2, mid + 1, sr, l, r, f);
            nodes[i] = op(nodes[i*2+1],nodes[i*2+2]);
        }
    }
    void apply(int l, int r, Func f) {
        apply(0, 0, n - 1, l, r, f);
    }
    void update(int i, Node node) {
        update(0, 0, n-1, i, node);
    }
    void update(int i, int sl, int sr, int pos, Node node) {
        // push SIEMPRE (también en el hijo que no contiene pos): si
no, el padre se recalcula
```

```
        // con el valor viejo de ese hijo y se pierde su lazy
pendiente.
        push(i,sl,sr);
        if (sl <= pos && pos <= sr) {
            if (sl == sr) {
                nodes[i] = node;
            } else {
                int mid = (sl + sr) >> 1;
                update(i * 2 + 1, sl, mid, pos, node);
                update(i * 2 + 2, mid + 1, sr, pos, node);
                nodes[i] = op(nodes[i*2+1], nodes[i*2+2]);
            }
        }
    }
    Node query(int i, int sl, int sr, int l, int r) {
        push(i,sl,sr);
        if (l <= sl && sr <= r) {
            return nodes[i];
        } else if (sr < l || r < sl) {
            return e();
        } else {
            int mid = (sl + sr) >> 1;
            auto a = query(i * 2 + 1, sl, mid, l, r);
            auto b = query(i * 2 + 2, mid + 1, sr, l, r);
            return  op(a,b);
        }
    }
    Node query(int l, int r) {
        return query(0, 0, n - 1, l, r);
    }
};
```

§1.14.5 segment-tree-lazy-vasito.cpp

01_Estructuras/Segment_Tree/Lazy

```
struct STree { // example: range sum with range addition
    vector<int> st,lazy;int n;
    STree(int n): st(4*n+5,0), lazy(4*n+5,0), n(n) {}
    void init(int k, int s, int e, int *a){
        lazy[k]=0;  // lazy neutral element
        if(s+1==e){st[k]=a[s];return;}
        int m=(s+e)/2;
        init(2*k,s,m,a);init(2*k+1,m,e,a);
        st[k]=st[2*k]+st[2*k+1]; // operation
    }
    void push(int k, int s, int e){
        if(!lazy[k])return; // if neutral, nothing to do
        st[k]+=(e-s)*lazy[k]; // update st according to lazy
        if(s+1<e){ // propagate to children
            lazy[2*k]+=lazy[k];
            lazy[2*k+1]+=lazy[k];
        }
        lazy[k]=0; // clear node lazy
    }
    void upd(int k, int s, int e, int a, int b, int v){
        push(k,s,e);
        if(s>=b||e<=a)return;
        if(s>=a&&e<=b){
            lazy[k]+=v; // accumulate lazy
            push(k,s,e);return;
        }
        int m=(s+e)/2;
        upd(2*k,s,m,a,b,v);upd(2*k+1,m,e,a,b,v);
        st[k]=st[2*k]+st[2*k+1]; // operation
    }
    int query(int k, int s, int e, int a, int b){
        if(s>=b||e<=a)return 0; // operation neutral
        push(k,s,e);
        if(s>=a&&e<=b)return st[k];
        int m=(s+e)/2;
        return query(2*k,s,m,a,b)+query(2*k+1,m,e,a,b); // operation
    }
    void init(int *a){init(1,0,n,a);}
    void upd(int a, int b, int v){upd(1,0,n,a,b,v);}
    int query(int a, int b){return query(1,0,n,a,b);}
}; // usage: STree rmq(n);rmq.init(x);rmq.upd(s,e,v);rmq.query(s,e);
```

§1.15 Segment Tree › Merge Sort Tree

§1.15.1 merge-sort-tree.cpp

01_Estructuras/Segment_Tree/Merge_Sort_Tree

```
using ll = long long;
using vl = vector<long long>;
// usage
// vector<node*> nodes;
// tree.query(l, r, nodes);
// returns log(n) sorted segments in a range (l, r)
struct node {
    ll l, r;
    vl nums;
    vl prefix;
};
struct segtree {
    int n;
    vector<node> tree;
    void init(int nn, vl& nodes) {
        tree.clear();
        n = nn;
        if (n == 0) return; // build(0, 0, -1) no terminaría
        int size = 1;
        while (size < n) {
            size *= 2;
        }
        tree.resize(size * 2);
        build(0, 0, n - 1, nodes);
    }
    void query(ll i, ll sl, ll sr, ll l, ll r, vector<node*> &ans) {
        if (l <= sl && sr <= r) {
            ans.pb(&tree[i]);
        } else if (sr < l || r < sl) {
        } else {
            int mid = (sl + sr) >> 1;
            query(i * 2 + 1, sl, mid, l, r, ans);
            query(i * 2 + 2, mid + 1, sr, l, r, ans);
        }
    }
    void query(ll l, ll r, vector<node*> &ans) { // [l, r]; rango
vacío o n == 0 -> nada
        if (n == 0 || l > r) return;
        return query(0, 0, n - 1, l, r, ans);
    }
    void build(int nodei, int l, int r, vl &nums) {
        if (l == r) {
            tree[nodei].nums = { nums[l] };
            tree[nodei].prefix = {nums[l]};
            tree[nodei].l = l;
            tree[nodei].r = r;
        } else {
            ll mid = (l + r) >> 1;
            build(nodei * 2 + 1, l, mid, nums);
            build(nodei * 2 + 2, mid + 1, r, nums);
            ll a = tree[nodei*2+1].nums.size();
            ll b = tree[nodei*2+2].nums.size();
            tree[nodei].nums.reserve(a + b);
            tree[nodei].prefix.resize(a+b);
            ll i = 0;
            ll j = 0;
            while (i < a && j < b) {
                ll simon = tree[nodei*2+1].nums[i];
                ll simon2 = tree[nodei*2+2].nums[j];
                if (simon <= simon2) {
                    tree[nodei].nums.pb(simon);
                    i++;
                } else {
                    tree[nodei].nums.pb(simon2);
                    j++;
                }
            }
            while (i < a) {
                tree[nodei].nums.pb(tree[nodei*2+1].nums[i]);
                i++;
            }
            while (j < b) {
                tree[nodei].nums.pb(tree[nodei*2+2].nums[j]);
                j++;
            }
            tree[nodei].prefix[0] = tree[nodei].nums[0];
            for (int i = 1; i < a + b; i++) {
                tree[nodei].prefix[i] = tree[nodei].prefix[i - 1] +
tree[nodei].nums[i];
            }
            tree[nodei].l = l;
            tree[nodei].r = r;
        }
    }
};
```

§1.16 Segment Tree › Persistente

§1.16.1 persistantsegmenttree.cpp

01_Estructuras/Segment_Tree/Persistente

```
using ll = long long;
struct Vertex {
    Vertex *l, *r;
    ll sum;
    Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
    Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
        if (l) sum += l->sum;
        if (r) sum += r->sum;
    }
};
Vertex* build(vector<ll>& a, int tl, int tr) {
    if (tl == tr)
        return new Vertex(a[tl]);
    int tm = (tl + tr) / 2;
    return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
}
ll get_sum(Vertex* v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && tr == r)
        return v->sum;
    int tm = (tl + tr) / 2;
    return get_sum(v->l, tl, tm, l, min(r, tm))
         + get_sum(v->r, tm+1, tr, max(l, tm+1), r);
}
Vertex* update(Vertex* v, int tl, int tr, int pos, int new_val) {
    if (tl == tr)
        return new Vertex(new_val);
    int tm = (tl + tr) / 2;
    if (pos <= tm)
        return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
    else
        return new Vertex(v->l, update(v->r, tm+1, tr, pos,
new_val));
}
Vertex* build(vector<ll> &a) {
    return build(a,0,a.size()-1);
}
ll get_sum(Vertex *v,ll n,int l, int r) {
    return get_sum(v,0,n-1,l,r);
```

```
}
Vertex* update(Vertex* v,ll n, int pos, int newV) {
    return update(v,0,n-1,pos,newV);
}
Vertex* copy(Vertex* v) {
    return new Vertex(*v); // antes new Vertex(v->l, v->r): en una hoja (n = 1) perdía su valor
}
```

§1.17 Sparse Table

§1.17.1 spare-table.cpp

01_Estructuras/Sparse_Table

```
struct rmq { // query complexity O(1)
    vector<vi> v;
    vi log2;
    int n;
    rmq() {}
    rmq(vi a): n(a.size()+10) {
        int L = 32 - __builtin_clz(n);
        log2.resize(n+1);
        log2[1] = 0;
        for (int i = 2; i <= n; i++) log2[i] = log2[i>>1] + 1;
        v.resize(L, vi(n));
        v[0] = a;
        v[0].resize(n, LLONG_MAX); // n = a.size()+10: sin este relleno se leía v[0] fuera de rango
        for (int k = 1; k<L; k++)
        for (int i = 0; i+(1<<k)<=n; i++)
            v[k][i] = min(v[k-1][i], v[k-1][i+(1<<(k-1))]);
    }
    int get(int l, int r) { // [l, r]
        int k = log2[r - l + 1];
        return min(v[k][l], v[k][r - (1 << k) + 1]);
    }
};
```

§1.17.2 sparse-table-2d.cpp

01_Estructuras/Sparse_Table

```
const int MAX_N = 100;
const int MAX_M = 100;
const int KN = log2(MAX_N)+1;
const int KM = log2(MAX_M)+1;
int table[KN][MAX_N][KM][MAX_M];
int _log2N[MAX_N+1];
int _log2M[MAX_M+1];
int MAT[MAX_N][MAX_M];
int n, m, ic, ir, jc, jr;
void calc_log2() {
  _log2N[1] = 0;
  _log2M[1] = 0;
  for (int i = 2; i <= MAX_N; i++)
    _log2N[i] = _log2N[i/2] + 1;
  for (int i = 2; i <= MAX_M; i++)
    _log2M[i] = _log2M[i/2] + 1;
}
void build() {
  for (ir = 0; ir < n; ir++) {
    for (ic = 0; ic < m; ic++)
      table[0][ir][0][ic] = MAT[ir][ic];
    for (jc = 1; jc < KM; jc++)
      for (ic = 0; ic + (1 <<(jc-1)) < m; ic++)
        table[0][ir][jc][ic] = min(
          table[0][ir][jc-1][ic],
          table[0][ir][jc-1][ic + (1 << (jc-1))]
        );
  }
  for (jr = 1; jr < KN; jr++)
  for (ir = 0; ir < n; ir++)
  for (jc = 0; jc < KM; jc++)
  for (ic = 0; ic < m; ic++)
    table[jr][ir][jc][ic] = min(
      table[jr-1][ir][jc][ic],
      table[jr-1][ir+(1<<(jr-1))][jc][ic]
    );
}
int rmq(int x1, int y1, int x2, int y2) {
  int lenx = x2-x1+1;
  int kx = _log2N[lenx];
  int leny = y2-y1+1;
  int ky = _log2M[leny];
  int min_R1 = min(
    table[kx][x1][ky][y1],
    table[kx][x1][ky][y2 + 1 - (1<<ky)]
  );
  int min_R2 = min(
    table[kx][x2+1-(1<<kx)][ky][y1],
    table[kx][x2+1- (1<<kx)][ky][y2 + 1 - (1<<ky)]
  );
  return min(min_R1, min_R2);
}
```

§1.18 Treap

§1.18.1 persistent-treap.cpp

01_Estructuras/Treap

```
// Treap implícito persistente (cada split/merge copia los nodos que toca).
// Uso: splitat(new item(), t, l, r, k) -> l = posiciones [0..k], r = el resto.
//      pitem res = new item(); merge(res, l, r);   (res queda NULL si ambos son NULL)
//      query(t, 0, cnt(t)-1, L, R) = suma de las claves en las posiciones [L, R].
// Antes: "#define pitem item*" hacía que "pitem L, R;" declarara R como item (no puntero), y
// "#define f first / s second" rompían cualquier variable f o s del código posterior.
using namespace std;
typedef long long ll;
struct item;
typedef item* pitem;
struct item
{
    ll tot;
    int key,cnt;
    pitem l;
    pitem r;
    item(){l=r=NULL;}
    item(int key):key(key),l(0),r(0){
        tot=key;
        cnt=1;
    }
};
int cnt(pitem t){return t?t->cnt:0;}
ll gtot(pitem t){return t?t->tot:0;}
void upd_cnt(pitem t)
{
    if (!t)return;
    t->cnt=cnt(t->l)+cnt(t->r)+1;
    t->tot=t->key+gtot(t->l)+gtot(t->r);
}
void splitat(pitem newT,pitem t,pitem &l,pitem &r,int key)
{
    if (!t)
        return void(l=r=NULL);
    (*newT)=(*t);//.-.
    int cur_at=cnt(t->l);
    if (cur_at>key)
        newT->l=new item(),splitat(newT->l,t->l,l,newT->l,key),r=newT;
    else
        newT->r=new item(),splitat(newT->r,t->r,newT->r,r,key-cur_at-1),l=newT;
    upd_cnt(newT);
}
void merge(pitem &newT,pitem l,pitem r)//.-.
{
    if (!l && !r)   return void(newT=NULL);
    if (!l||!r)
        return void((*newT)=l?(*l):(*r));
    // mt19937_64 en lugar de rand(): en MinGW RAND_MAX = 32767 y el módulo quedaba sesgado
    // (prior < cnt(l) con probabilidad incorrecta) cuando cnt(l)+cnt(r) > 32767.
    static mt19937_64 rngPersistentTreap(chrono::steady_clock::now().time_since_epoch().count());
    int prior=rngPersistentTreap()%(cnt(l)+cnt(r));//andrew_part
    if (prior<cnt(l))
        (*newT)=(*l),newT->r=new item(),merge(newT->r,l->r,r);
    else
        (*newT)=(*r),newT->l=new item(),merge(newT->l,l,r->l);
    upd_cnt(newT);
}
ll query(pitem t,int iz,int der,int &riz,int &rder)
{
    if (!t)return 0;
    if (iz>rder|| der<riz)return 0;
    if (iz>=riz&& der<=rder)
        {
            return t->tot;
        }
    return query(t->l,iz,cnt(t->l)+iz,riz,rder)+
           query(t->r,cnt(t->l)+iz+1,der,riz,rder)+(((cnt(t->l)+iz>=riz&& cnt(t->l)+iz<=rder)?t->key:0));
}
```

§1.18.2 treap-implicit-father-vasito.cpp

01_Estructuras/Treap

```
// node father is useful to keep track of the chain of each node
// alternative: splay tree
// IMPORTANT: add pointer f in struct item
void merge(pitem& t, pitem l, pitem r){
    push(l);push(r);
    if(!l||!r)t=l?l:r;
    else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=l;
    else merge(r->l,l,r->l),r->l->f=t=r;
    upd_cnt(t);
}
void split(pitem t, pitem& l, pitem& r, int sz){
    if(!t){l=r=0;return;}
    push(t);
    if(sz<=cnt(t->l)){
        split(t->l,l,t->l,sz);r=t;
        if(l)l->f=0;
        if(t->l)t->l->f=t;
    }
    else {
        split(t->r,t->r,r,sz-1-cnt(t->l));l=t;
        if(r)r->f=0;
        if(t->r)t->r->f=t;
    }
    upd_cnt(t);
}
void push_all(pitem t){
    if(t->f)push_all(t->f);
    push(t);
}
pitem root(pitem t, int& pos){ // get root and position for node t
    push_all(t);
```

```
    pos=cnt(t->l);
    while(t->f){
        pitem f=t->f;
        if(t==f->r)pos+=cnt(f->l)+1;
        t=f;
    }
    return t;
}
```

§1.18.3 treap-implicit-vasito.cpp

01_Estructuras/Treap

```
// example that supports range reverse and addition updates, and
range sum query
// (commented parts are specific to this  problem)
// Prioridades con mt19937: rand() en MinGW (Windows) solo llega a
RAND_MAX = 32767 y repite.
mt19937
rngTreapImplicit(chrono::steady_clock::now().time_since_epoch().count
());
typedef struct item *pitem;
struct item {
    int pr,cnt,val;
//  int sum; // (paramters for range query)
//  bool rev;int add; // (parameters for lazy prop)
    pitem l,r;
    item(int val):
pr(rngTreapImplicit()),cnt(1),val(val),l(0),r(0)/*,sum(val),rev(0),ad
d(0)*/ {}
};
void push(pitem it){
    if(it){
        /*if(it->rev){
            swap(it->l,it->r);
            if(it->l)it->l->rev^=true;
            if(it->r)it->r->rev^=true;
            it->rev=false;
        }
        it->val+=it->add;it->sum+=it->cnt*it->add;
        if(it->l)it->l->add+=it->add;
        if(it->r)it->r->add+=it->add;
        it->add=0;*/
    }
}
int cnt(pitem t){return t?t->cnt:0;}
// int sum(pitem t){return t?push(t),t->sum:0;}
void upd_cnt(pitem t){
    if(t){
        t->cnt=cnt(t->l)+cnt(t->r)+1;
        // t->sum=t->val+sum(t->l)+sum(t->r);
    }
}
void merge(pitem& t, pitem l, pitem r){
    push(l);push(r);
    if(!l||!r)t=l?l:r;
    else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
    else merge(r->l,l,r->l),t=r;
    upd_cnt(t);
}
void split(pitem t, pitem& l, pitem& r, int sz){ // sz:desired size
of l
    if(!t){l=r=0;return;}
    push(t);
    if(sz<=cnt(t->l))split(t->l,l,t->l,sz),r=t;
    else split(t->r,t->r,r,sz-1-cnt(t->l)),l=t;
    upd_cnt(t);
}
void output(pitem t){ // useful for debugging
    if(!t)return;
    push(t);
    output(t->l);printf(" %lld",t->val);output(t->r);
}
```

§1.18.4 treap.cpp

01_Estructuras/Treap

```
// Prioridades con mt19937: rand() en MinGW (Windows) solo llega a
RAND_MAX = 32767 y repite.
// Tested https://judge.yosupo.jp/problem/ordered_set
mt19937
rngTreap(chrono::steady_clock::now().time_since_epoch().count());
struct item {
    int key, pri, siz;
    item *l, *r;
    item() {}
    item(int key) : key(key), siz(1), pri(rngTreap()), l(0), r(0) {}
};
typedef item* pitem;
int sz(pitem t) {
    return (t?t->siz:0);
}
void up_sz(pitem t) {
    if(t) t->siz = sz(t->l) + 1 + sz(t->r);
}
void split(pitem t, pitem &l, pitem &r, int val) {
    if(!t) r = l = NULL;
    else if(t->key < val) split(t->r, t->r, r, val), l = t;
    else split(t->l, l, t->l, val), r = t;
    up_sz(t);
}
void merge(pitem &t, pitem l, pitem r) {
    if(!l || !r) t=(l?l:r);
    else if(l->pri >= r->pri) merge(l->r, l->r, r), t = l;
    else merge(r->l, l, r->l),t=r;
    up_sz(t);
}
```

§1.19 Trie persistente

§1.19.1 persistent-trie.cpp

01_Estructuras/Trie_persistente

```
struct PersistentTrie{
    vector<int> trie[2];
    vector<int> root;
    vector<int> frec;
    vector<bool> term;
    int tao;
    PersistentTrie(int tao) : tao(tao){
        root.pb(0);
        add_node();
    }
    void add_node(){
        trie[0].pb(0);
        trie[1].pb(0);
        frec.pb(0);
        term.pb(false);
    }
    void add_node(int from) {
        trie[0].pb(0);
        trie[1].pb(0);
        term.pb(term[from]);
        frec.pb(frec[from]);
    }
    void insert(int x, int last_root, int new_root) {
        for(int i = tao - 1; i >= 0; i--) {
            int nxt = (x >> i) & 1;
            trie[nxt ^ 1][new_root] = ~last_root ? trie[nxt ^ 1]
[last_root] : 0;
            if(~last_root && trie[nxt][last_root]) {
                trie[nxt][new_root] = trie[0].size();
                add_node(trie[nxt][last_root]);
            }
            else{
                trie[nxt][new_root] = trie[0].size();
                add_node();
            }
            last_root = ~last_root && trie[nxt][last_root] ?
trie[nxt][last_root] : -1;
            new_root = trie[nxt][new_root];
            frec[new_root] += 1;
        }
        term[new_root] = true;
    }
    void insert(int x) {
        root.pb(trie[0].size());
        add_node();
        insert(x, root[root.size() - 2], root.back());
    }
};
```

§1.20 Wavelet Tree

§1.20.1 wavelet-tree-vasito.cpp

01_Estructuras/Wavelet_Tree

```
struct WT {
    vector<int> wt[1<<20];int n;
    void init(int k, int s, int e){
        if(s+1==e)return;
        wt[k].clear();wt[k].pb(0);
        int m=(s+e)/2;
        init(2*k,s,m);init(2*k+1,m,e);
    }
    void add(int k, int s, int e, int v){
        if(s+1==e)return;
        int m=(s+e)/2;
        if(v<m)wt[k].pb(wt[k].back()),add(2*k,s,m,v);
        else wt[k].pb(wt[k].back()+1),add(2*k+1,m,e,v);
    }
    int query0(int k, int s, int e, int a, int b, int i){
        if(s+1==e)return s;
        int m=(s+e)/2;
        int q=(b-a)-(wt[k][b]-wt[k][a]);
        if(i<q)return query0(2*k,s,m,a-wt[k][a],b-wt[k][b],i);
        else return query0(2*k+1,m,e,wt[k][a],wt[k][b],i-q);
    }
    void upd(int k, int s, int e, int i){
        if(s+1==e)return;
        int m=(s+e)/2;
        int v0=wt[k][i+1]-wt[k][i],v1=wt[k][i+2]-wt[k][i+1];
        if(!v0&&!v1)upd(2*k,s,m,i-wt[k][i]);
        else if(v0&&v1)upd(2*k+1,m,e,wt[k][i]);
        else if(v0)wt[k][i+1]--;
        else wt[k][i+1]++;
    }
    void init(int _n){n=_n;init(1,0,n);} // (values in range [0,n))
    void add(int v){add(1,0,n,v);}
    int query0(int a, int b, int i){ // ith element in range [a,b)
        return query0(1,0,n,a,b,i);    // (if it was sorted)
    }
    void upd(int i){ // swap positions i,i+1
        upd(1,0,n,i);
    }
};
```

§2 Grafos

§2.1 Árbol expansión mínima

§2.1.1 chu-liu-vasito.cpp

02_Grafos/Arbol_expansion_minima

```
//O(n*m) minimum spanning tree in directed graph
//returns -1 if not possible
```

```
//included i-th edge if take[i]!=0
typedef int tw; tw INF=1ll<<30;
struct edge{int u,v,id;tw len;};
struct ChuLiu{
    int n; vector<edge> e;
    vector<int> inc,dec,take,pre,num,id,vis;
    vector<tw> inw;
    void add_edge(int x, int y, tw w){
        inc.pb(0); dec.pb(0); take.pb(0);
        e.pb({x,y,static_cast<long long>((e).size()),w});
    }
    ChuLiu(int n):n(n),pre(n),num(n),id(n),vis(n),inw(n){}
    tw doit(int root){
        auto e2=e;
        tw ans=0; int eg=static_cast<long long>
((e).size())-1,pos=static_cast<long long>((e).size())-1;
        while(1){
            for (int i=(0), nb_fin_4=(n); i<nb_fin_4; ++i)
inw[i]=INF,id[i]=vis[i]=-1;
            for(auto ed:e2) if(ed.len<inw[ed.v]){
                inw[ed.v]=ed.len; pre[ed.v]=ed.u;
                num[ed.v]=ed.id;
            }
            inw[root]=0;
            for (int i=(0), nb_fin_6=(n); i<nb_fin_6; ++i)
if(inw[i]==INF) return -1;
            int tot=-1;
            for (int i=(0), nb_fin_8=(n); i<nb_fin_8; ++i){
                ans+=inw[i];
                if(i!=root)take[num[i]]++;
                int j=i;
                while(vis[j]!=i&&j!=root&&id[j]<0)vis[j]=i,j=pre[j];
                if(j!=root&&id[j]<0){
                    id[j]=++tot;
                    for(int k=pre[j];k!=j;k=pre[k]) id[k]=tot;
                }
            }
            if(tot<0)break;
            for (int i=(0), nb_fin_10=(n); i<nb_fin_10; ++i) if(id[i]
<0)id[i]=++tot;
            n=tot+1; int j=0;
            for (int i=(0), nb_fin_12=(static_cast<long long>
((e2).size())); i<nb_fin_12; ++i){
                int v=e2[i].v;
                e2[j].v=id[e2[i].v];
                e2[j].u=id[e2[i].u];
                if(e2[j].v!=e2[j].u){
                    e2[j].len=e2[i].len-inw[v];
                    inc.pb(e2[i].id);
                    dec.pb(num[v]);
                    take.pb(0);
                    e2[j++].id=++pos;
                }
            }
            e2.resize(j);
            root=id[root];
        }
        while(pos>eg){
            if(take[pos]>0) take[inc[pos]]++, take[dec[pos]]--;
            pos--;
        }
        return ans;
    }
};
```

§2.1.2 kruskal.cpp

02_Grafos/Arbol_expansion_minima

```
using ll = long long;
using vl = vector<long long>;
/* Kruskal O(|edges|*Log|edges|)
   Finds the max/min spanning tree of an undirected graph
   provide the undirected edges with its costs vector<{cost(a, b), a,
b}>
   and the size
*/
struct union_find {
    vl p;
    union_find(int n) : p(n,-1) {}
    ll find(ll x) {
        if (p[x] == -1) return x;
        return p[x] = find(p[x]);
    }
    bool group(ll a, ll b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        p[a] = b;
        return true;
    }
};
ll kruskal(vector<tuple<ll,ll,ll>> &edges, ll nodes) {
    union_find uf(nodes+1);
    sort(all(edges));
    reverse(all(edges)); // for max
    ll answer = 0;
    for (auto edge : edges)  {
        ll cost, a, b;
        tie(cost, a, b) = edge;
        if (uf.group(a, b))
            answer += cost;
    }
    return answer;
}
```

§2.1.3 prim-extra.cpp

02_Grafos/Arbol_expansion_minima

```
using ll = long long;
// Dado un grafo halla el costo total de su arbol cobertor minimo.
struct edge {
    int v; ll w;
    bool operator < (const edge &o) const {
        return o.w < w; // invertidos para que la pq ordene de < a >
    }
};
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<edge> g[MX]; // Lista de adyacencia
vector<bool> vis; // Marca los nodos ya visitados
ll ans; // Costo total del arbol cobertor minimo
int n, m; // Cantidad de nodos y aristas
void prim() {
    priority_queue<edge> pq;
    vis[0] = true;
    for (auto &ed : g[0]) {
        int v = ed.v;
        if (!vis[v]) pq.push({v, ed.w});
    }
    while (pq.size()) {
        int u = pq.top().v;
        ll w = pq.top().w;
        pq.pop();
        if (!vis[u]) {
            ans += w;
            vis[u] = true;
            for (auto &ed : g[u]) {
                int v = ed.v;
                if (!vis[v]) pq.push({v, ed.w});
            }
        }
    }
}
```

```
void init() {
    ans = 0;
    vis.assign(n, false);
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.2 Articulaciones puentes biconexas

§2.2.1 articulation-bridges-biconnected-extra.cpp

...nes_puentes_biconexas

```
// Dado un grafo no dirigido halla los puntos de articulación,
puentes y componentes biconexas. Para construir el block cut tree
quitar los comentarios.
struct edge {
    int u, v, comp; // A que componente biconexa pertenece
    bool bridge; // Si la arista es un puente
};
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<int> g[MX]; // Lista de adyacencia
vector<edge> e; // Lista de aristas
stack<int> st;
int low[MX], num[MX], cont;
bool art[MX]; // Si el nodo es un punto de articulacion
// vector<set<int>> comps; // Componentes biconexas
// vector<vector<int>> tree; // Block cut tree
// vector<int> id; // Id del nodo en el block cut tree
int BCC; // Cantidad de componentes biconexas
int n, m; // Cantidad de nodos y aristas
void add_edge(int u, int v) {
    g[u].pb(e.size());
    g[v].pb(e.size());
    e.pb({u, v, -1, false});
}
void dfs(int u, int p = -1) {
    low[u] = num[u] = cont++;
    for (auto &i : g[u]) {
        edge &ed = e[i];
        int v = ed.u^ed.v^u;
        if (num[v] == -1) {
            st.push(i);
            dfs(v, i);
            if (low[v] > num[u])
                ed.bridge = true; // bridge
            if (low[v] >= num[u]) {
                art[u] = (num[u] > 0 || num[v] > 1); // articulation
                int last; // start biconnected
                // comps.pb({});
                do {
                    last = st.top(); st.pop();
                    e[last].comp = BCC;
                    // comps.back().insert(e[last].u);
                    // comps.back().insert(e[last].v);
                } while (last != i);
                BCC++; // end biconnected
            }
            low[u] = min(low[u], low[v]);
        } else if (i != p && num[v] < num[u]) {
            st.push(i);
            low[u] = min(low[u], num[v]);
        }
    }
}
void build_tree() {
    tree.clear(); id.resize(n);
    for (int u = 0; u < n; u++) {
        if (art[u]) {
            id[u] = tree.size();
            tree.pb({});
```

```
        }
    }
    for (auto &comp : comps) {
        int node = tree.size();
        tree.pb({});
        for (auto &u : comp) {
            if (art[u]) {
                tree[id[u]].pb(node);
                tree[node].pb(id[u]);
            } else id[u] = node;
        }
    }
}
void init() {
    cont = BCC = 0;
    // comps.clear();
    e.clear();
    for (int i = 0; i <= n; i++) {
        g[i].clear();
        num[i] = -1; // no visitado
        art[i] = false;
    }
}
```

§2.2.2 tarjan-y-blockcuttree.cpp

02_Grafos/Articulaciones_puentes_biconexas

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
int timer;
vector<vector<int>> comps; // componentes biconexos
stack<int> stk;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    stk.push(v);
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v])
            {
             if (p != -1) IS_CUTPOINT(v);
             comps.push_back({v});
             while (comps.back().back() != to)
             {
                 comps.back().push_back(stk.top());
                 stk.pop();
             }
            }
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs (i);
    }
}
vector<int> id;
int curBCTNode;
vector<vector<int> > tree;
vector<bool> isAP;
void buildTree() {
    curBCTNode = 0;
    id.assign(n, -1);
    tree.clear();
    isAP.clear();
    for (int v=(0), nb_fin_1=(n); v<nb_fin_1; ++v) {
        if (cutpoint[v]) {
            id[v] = tree.size();
            tree.pb({});
            isAP.pb(true);
        }
    }
    for (auto comp : comps) {
        int v = tree.size();
        tree.pb({});
        isAP.pb(false);
        for (int x : comp) {
            if (cutpoint[x]) {
                tree[v].pb(id[x]);
                tree[id[x]].pb(v);
            }
            else {
                id[x] = v;
            }
        }
    }
}
```

§2.3 Caminos eulerianos

§2.3.1 eulerian-path.cpp

02_Grafos/Caminos_eulerianos

```
// Directed version (uncomment commented code for undirected)
struct edge {
  int y;
//  list<edge>::iterator rev;
  edge(int y):y(y){}  };
list<edge> g[MAXN];
void add_edge(int a, int b){
  g[a].push_front(edge(b));//auto ia=g[a].begin();
//  g[b].push_front(edge(a));auto ib=g[b].begin();
//  ia->rev=ib;ib->rev=ia;
}
vector<int> p;
void go(int x){
  while(g[x].size()) {
    int y=g[x].front().y;
    //g[y].erase(g[x].front().rev);
    g[x].pop_front();
    go(y);  }
  p.push_back(x);}
vector<int> get_path(int x){ // get a path that begins in x
// check that a path exists from x before calling to get_path!
  p.clear();go(x);reverse(p.begin(),p.end());
  return p;
}
```

§2.4 Caminos mínimos › Bellman Ford

§2.4.1 bellman-ford-extra.cpp

02_Grafos/Caminos_minimos/Bellman_Ford

```
using ll = long long;
using pii = pair<int, int>;
// Dado un grafo con pesos, positivos o negativos, halla la ruta de
costo minimo entre un nodo inicial u y todos los demas nodos.
// Tambien halla ciclos negativos.
const ll inf = 1e18;
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<pii> g[MX]; // Lista de adyacencia, u->[(v, cost)]
vector<ll> dist; // Almacena la distancia a cada nodo
// vector<int> cycle; // Para construir el ciclo negativo
int n, m; // Cantidad de nodos y aristas
// O(n*m)
void bellmanFord(int src) {
    dist.assign(n, inf);
    dist[src] = 0;
    for (int i = 0; i < n-1; i++)
        for (int u = 0; u < n; u++)
            if (dist[u] != inf)
                for (auto &v : g[u]) {
                    dist[v.first] = min(dist[v.first], dist[u] +
v.second);
                }
    // Encontrar ciclos negativos
    // cycle.clear();
    for (int u = 0; u < n; u++)
        if (dist[u] != inf)
            for (auto &v : g[u])
                if (dist[v.first] > dist[u] + v.second) { // Ciclo
negativo
                    dist[v.first] = -inf;
                    // cycle.pb(v.F); // Para reconstruir
                }
}
void init() {
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.4.2 bellmanford-find-negative-cycle.cpp

...Caminos_minimos/Bellman_Ford

```
using ll = long long;
// This uses Bellmanford algorithm to find a negative cycle
// O(n*m) m=edges, n=nodes
void test_case() {
    ll n, m;
    cin >> n >> m;
    vector<ll> dist(n+1);
    vector<ll> p(n+1);
    vector<tuple<ll,ll,ll>> edges(m);
    for (int i =0; i < m; i ++) {
        ll x, y, z;
        cin >> x >> y >> z;
        edges[i] = {x, y, z};
    }
    ll efe = -1;
    for (int i = 0; i < n; i++) {
        efe = -1;
        for (auto pp : edges) {
            ll x,y,z;
            tie(x,y,z) = pp;
            if (dist[x] + z < dist[y]) {
                dist[y] = dist[x] + z;
                p[y] = x;
                efe = y;
            }
        }
    }
    if (efe == -1) {
        cout << "NO\n";
    } else {
        cout << "YES\n";
```

```
        ll x = efe;
        for (int i = 0; i < n; i++) {
            x = p[x];
        }
        vector<ll> cycle;
        ll y = x;
        while (cycle.size() == 0 || y != x) {
            cycle.pb(y);
            y = p[y];
        }
        cycle.pb(x);
        reverse(all(cycle));
        for (int i =0; i < cycle.size(); i++) {
            cout << cycle[i] << " \n" [i == cycle.size() -1];
        }
    }
}
```

§2.5 Caminos mínimos › Dijkstra

§2.5.1 dijkstra-extra.cpp

02_Grafos/Caminos_minimos/Dijkstra

```
using ll = long long;
// Dado un grafo con pesos no negativos halla la ruta de costo minimo
entre un nodo inicial u y todos los demas nodos.
struct edge {
    int v; ll w;
    bool operator < (const edge &o) const {
        return o.w < w; // invertidos para que la pq ordene de < a >
    }
};
const ll inf = 1e18;
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<edge> g[MX]; // Lista de adyacencia
vector<bool> vis; // Marca los nodos ya visitados
vector<ll> dist; // Almacena la distancia a cada nodo
int pre[MX]; // Almacena el nodo anterior para construir las rutas
int n, m; // Cantidad de nodos y aristas
void dijkstra(int u) {
    priority_queue<edge> pq;
    pq.push({u, 0});
    dist[u] = 0;
    while (pq.size()) {
        u = pq.top().v; pq.pop();
        if (!vis[u]) {
            vis[u] = true;
            for (auto &ed : g[u]) {
                int v = ed.v;
                if (!vis[v] && dist[v] > dist[u] + ed.w) {
                    dist[v] = dist[u] + ed.w;
                    pre[v] = u;
                    pq.push({v, dist[v]});
                }
            }
        }
    }
}
void init() {
    vis.assign(n, false);
    dist.assign(n, inf);
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.5.2 dijkstra-k-shortest-path.cpp

02_Grafos/Caminos_minimos/Dijkstra

```
using ll = long long;
// Using djisktra, finds the k shortesth paths from 1 to n
// 2≤n≤10^5, 1≤m≤2·10^5, 1≤weight≤10^9, 1≤k≤10
// complexity seems O(k*m)
#define P pair<ll,ll>
void test_case() {
    ll n, m, k;
    cin >> n >> m >> k;
    vector<ll> visited(n+1, 0);
    vector<vector<pair<ll,ll>>> adj(n+1);
    for (int i = 0; i < m; i++) {
        ll a, b, c;
        cin >> a >> b >> c;
        adj[a].pb({b, c});
    }
    vector<ll> ans;
    priority_queue<P,vector<P>, greater<P>> q;
    q.push({0, 1});
    ll kk = k;
    while (q.size()) {
        ll x = q.top().second;
        ll z = q.top().first;
        q.pop();
        if (visited[x] >= kk) {
            continue;
        }
        visited[x]++;
        if (x == n) {
            ans.pb(z);
            k--;
            if (k == 0) break;
        }
        for (auto yy : adj[x]) {
            q.push({yy.second + z, yy.first});
        }
    }
    for (int i = 0; i < ans.size(); i++) {
        cout << ans[i] << " \n" [i == ans.size() - 1];
    }
}
```

§2.6 Caminos mínimos › Floyd Warshall

§2.6.1 floyd-warshall-extra.cpp

02_Grafos/Caminos_minimos/Floyd_Warshall

```
// Dado un grafo halla la distancia minima entre cualquier par de
nodos. g[i][j] guardara la distancia minima entre el nodo i y el j.
const int inf = 1e9;
const int MX = 505; // Cantidad maxima de nodos
int g[MX][MX]; // Matriz de adyacencia
int n, m; // Cantidad de nodos y aristas
void floydWarshall() {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
}
void init() {
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= n; j++) {
            g[i][j] = (i == j ? 0 : inf);
        }
    }
}
```

§2.6.2 floyd-warshall-negative-weights.cpp

...aminos_minimos/Floyd_Warshall

```
// Requisito de selección: El ejemplo original no lee/inicializa n ni
m antes de reservar dist; completar la entrada antes de ejecutarlo.
using ll = long long;
// Distancia mínima de todo i a todo j con pesos negativos. dist = -
inf: hay ciclo negativo en
// algún camino i->j; inf: no alcanzable. Todo camino real debe ser <
inf.
const ll inf = INT32_MAX;
void test_case() {
    ll n, m; // nodes, edges
    vector<vector<ll>> dist(n, vector<ll>(n, inf));
    for (int i = 0; i < n; i++) dist[i][i] = 0;
    for (int i = 0; i < m; i++) {
        ll a, b, w;
        cin >> a >> b >> w; // negative weights
        dist[a][b] = min(dist[a][b], w);
    }
    // floid warshall
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] == inf || dist[k][j] == inf) continue;
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);
            }
        }
    }
    // find negative cycles for a node
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < 0) dist[i][i] = -inf;
    }
    // find negative cycles betweens a routes from i to j
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n ; j++) {
            for (int k = 0; k < n; k++) {
                if (dist[k][k] < 0 && dist[i][k] != inf && dist[k][j]
!= inf) {
                    dist[i][j] = -inf;
                }
            }
        }
    }
}
```

§2.7 Conectividad dinámica

§2.7.1 dynamic-connectivity.cpp

02_Grafos/Conectividad_dinamica

```
using ll = long long;
struct union_find {
    struct uf_save { ll a, rnka, b, rnkb; };
    vector<uf_save> saves;
    vector<ll> p, rnk;
    ll n, comps;
    union_find(ll _n) : n(_n), p(_n), rnk(_n) {
        for (int i = 0;i<n;i++) p[i] = i;
        comps = n;
    };
    ll find(ll x) { return p[x] == x ? x : find(p[x]); }
    int merge(ll a, ll b) {
        a = find(a);
        b = find(b);
        if (a == b) return 0;
        if (rnk[b] > rnk[a]) swap(a,b);
        comps--;
        saves.pb({a, rnk[a], b, rnk[b]});
        p[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return 1;
    };
    void rollback() {
        if (saves.empty()) return;
        uf_save lst = saves.back();
        saves.pop_back();
        comps++;
        p[lst.a] = lst.a, rnk[lst.a] = lst.rnka;
        p[lst.b] = lst.b, rnk[lst.b] = lst.rnkb;
    }
```

```
};
struct query { ll a, b; };
struct segtree {
    int n; // size of queries!!!
    vector<vector<query>> t; // node si a vector<query>
    segtree(int queries_n) : n(queries_n), t(2*n) {}
    void addQuery(int l, int r, const query &q) {
        l += n, r += n+1;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l&1) t[l++].pb(q);
            if (r&1) t[--r].pb(q);
        }
    }
    void dfs(int id, vector<ll> &ans, union_find &uf) {
        ll cnt = 0;
        for (auto & q : t[id]) cnt += uf.merge(q.a, q.b);
        if (id < n) dfs(id*2, ans, uf), dfs(id*2+1,ans,uf);
        else ans[id-n] = uf.comps;
        for (int i =0;i<cnt;i++) uf.rollback();
    }
    vector<ll> solve(union_find &uf) {
        vector<ll> ans(n);
        dfs(1, ans, uf);
        return ans;
    }
};
void test_case() {
    ll n, m, q; cin >> n >> m >> q;
    segtree tree(q+1);
    vector<map<int,int>> lst(n); // to manage living time!
    auto addEdge = [&](int x, int y, int time) {
        if (y<x) swap(x,y);
        lst[x][y] = time;
    };
    auto erEdge = [&](int x, int y, int time) {
        if (y<x) swap(x,y);
        tree.addQuery(lst[x][y], time-1, {x,y});
        lst[x].erase(y);
    };
    for (int i = 0;i<m;i++) {
        ll x, y; cin >> x >> y; x--,y--;
        addEdge(x,y,0);
    }
    for (int time =1;time<=q;time++) {
        ll t, x, y; cin >> t >> x >> y;
        x--,y--;
        if (t==1) addEdge(x,y,time);
        else erEdge(x,y,time);
    }
    for (int i = 0;i < n;i++)
        for (auto &[j,v] : lst[i]) // be carefull with it and erasing
            tree.addQuery(v,q,{i,j});
    union_find uf(n);
    auto ans = tree.solve(uf);
    for (int i = 0;i < ans.size(); i++) {
        cout << ans[i] << " \n" [i+1 == ans.size()];
    }
} // https://cses.fi/problemset/task/2133/
```

§2.8 Dominadores

§2.8.1 dominator-tree.cpp

02_Grafos/Dominadores

```
//idom[i]=parent of i in dominator tree with root=rt, or -1 if not
exists
int n,rnk[tam],pre[tam],anc[tam],idom[tam],semi[tam],low[tam];
vi g[tam],rev[tam],dom[tam],ord;
void dfspre(int pos){
    rnk[pos]=sz(ord); ord.pb(pos);
    for(auto x:g[pos]){
        rev[x].pb(pos);
        if(rnk[x]==n) pre[x]=pos,dfspre(x);
    }
}
int eval(int v){
    if(anc[v]<n&&anc[anc[v]]<n){
        int x=eval(anc[v]);
        if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
        anc[v]=anc[anc[v]];
    }
    return low[v];
}
void dominators(int rt){
    for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i){
        dom[i].clear(); rev[i].clear();
        rnk[i]=pre[i]=anc[i]=idom[i]=n;
        semi[i]=low[i]=i;
    }
    ord.clear(); dfspre(rt);
    for(int i=sz(ord)-1;i;i--){
        int w=ord[i];
        for(int v:rev[w]){
            int u=eval(v);
            if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
        }
        dom[semi[w]].pb(w); anc[w]=pre[w];
        for(int v:dom[pre[w]]){
            int u=eval(v);
            idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
        }
        dom[pre[w]].clear();
    }
    for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w]=idom[idom[w]];
    for (int i=(0), nb_fin_3=(n); i<nb_fin_3; ++i)
if(idom[i]==n)idom[i]=-1; else dom[idom[i]].pb(i);
}
```

§2.9 Orden topológico

§2.9.1 topological-sort.cpp

02_Grafos/Orden_topologico

```
using ll = long long;
// Find the topological order of a graph in O(n)
const int N = 1e5;
vector<vector<ll>> adj(N + 10);
vector<ll> visited(N +10);
bool cycle = false; // reports if doesn't exists a topological sort
vector<ll> topo;
void dfs(ll x) {
    if (visited[x] == 2) {
        return;
    } else if (visited[x] == 1) {
        cycle = true;
        return;
    }
    visited[x] = 1;
    for (auto y : adj[x]) dfs(y);
    visited[x] = 2;
    topo.pb(x);
}
void test_case() {
    ll n, m; cin >> n >> m;
    for (int i =0; i < m; i++) {
        ll x, y; cin >> x >> y;
        adj[x].pb(y);
    }
    for (int i = 1; i <= n; i++) dfs(i);
    reverse(topo.begin(), topo.end());
    if (cycle) {
        cout << "IMPOSSIBLE\n";
    } else {
        for (int i =0; i < n; i++) {
            cout << topo[i] << " \n" [i == n - 1];
        }
    }
}
```

§2.10 Recorridos y conectividad

§2.10.1 bfs-extra.cpp

02_Grafos/Recorridos_y_conectividad

```
using ll = long long;
// Busqueda en anchura sobre grafos. Recibe un nodo inicial u y
visita todos los nodos alcanzables desde u.
// BFS tambien halla la distancia mas corta entre el nodo inicial u y
los demas nodos si todas las aristas tienen peso 1.
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<int> g[MX]; // Lista de adyacencia
vector<ll> dist; // Almacena la distancia a cada nodo
int n, m; // Cantidad de nodos y aristas
void bfs(int u) {
    queue<int> q;
    q.push(u);
    dist[u] = 0;
    while (q.size()) {
        u = q.front();
        q.pop();
        for (auto &v : g[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
void init() {
    dist.assign(n, -1);
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.10.2 bipartite-check-extra.cpp

02_Grafos/Recorridos_y_conectividad

```
// Modificacion del BFS para detectar si un grafo es bipartito.
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<int> g[MX]; // Lista de adyacencia
vector<int> color; // Almacena el color de cada nodo
bool bipartite; // true si el grafo es bipartito
int n, m; // Cantidad de nodos y aristas
void bfs(int u) {
    queue<int> q;
    q.push(u);
    color[u] = 0;
    while (q.size()) {
        u = q.front();
        q.pop();
        for (auto &v : g[u]) {
            if (color[v] == -1) {
                color[v] = 1-color[u];
                q.push(v);
            } else if (color[v] == color[u]) {
                bipartite = false;
                return;
            }
        }
    }
}
void init() {
    bipartite = true;
    color.assign(n, -1);
```

```
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.10.3 cycle-detection-extra.cpp

02_Grafos/Recorridos_y_conectividad

```
// Determina si un grafo dirigido tiene o no ciclos (si es un DAG o
no).
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<int> g[MX]; // Lista de adyacencia
vector<int> vis; // Marca el estado de los nodos ya visitados
bool cycle; // true si el grafo tiene ciclos
int n, m; // Cantidad de nodos y aristas
void dfs(int u) {
    if (cycle) return;
    vis[u] = 1;
    for (auto &v : g[u]) {
        if (!vis[v]) dfs(v);
        else if (vis[v] == 1) {
            cycle = true;
            break;
        }
    }
    vis[u] = 2;
}
void init() {
    vis.assign(n, 0);
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.10.4 dfs-extra.cpp

02_Grafos/Recorridos_y_conectividad

```
// Busqueda en profundidad sobre grafos. Recibe un nodo inicial u y
visita todos los nodos alcanzables desde u.
// DFS puede ser usado para contar la cantidad de componentes conexas
en un grafo y puede ser modificado para que retorne informacion de
los nodos dependiendo del problema.
const int MX = 1e5+5; // Cantidad maxima de nodos
vector<int> g[MX]; // Lista de adyacencia
vector<bool> vis; // Marca los nodos ya visitados
int n, m; // Cantidad de nodos y aristas
void dfs(int u) {
    vis[u] = true;
    for (auto &v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}
void init() {
    vis.assign(n, false);
    for (int i = 0; i <= n; i++) {
        g[i].clear();
    }
}
```

§2.10.5 flood-fill-extra.cpp

02_Grafos/Recorridos_y_conectividad

```
// Dado un grafo implicito como matriz, "colorea" y cuenta el tamaño
de las componentes conexas.
// Esta funcion debe ser llamada con las coordenadas (i, j) donde se
inicia el recorrido, busca cada caracter c1 de la componente, los
remplaza por el caracter c2 y retorna el tamaño.
const int MX = 1e3; // Tamanio maximo de la matriz
int dy[] = {1, 1, 0,-1,-1,-1, 0, 1}; // Posibles movimientos:
int dx[] = {0, 1, 1, 1, 0,-1,-1,-1}; // (8 direcciones)
char grid[MX][MX]; // Matriz de caracteres
int n, m; // Tamanio de la matriz
int floodfill(int y, int x, char c1, char c2) {
    if (y < 0 || y >= n || x < 0 || x >= m) return 0;
    if (grid[y][x] != c1) return 0;
    grid[y][x] = c2;
    int ans = 1;
    for (int i = 0; i < 8; i++) {
        ans += floodfill(y + dy[i], x + dx[i], c1, c2);
    }
    return ans;
}
```

§2.10.6 parallel-dfs-vasito.cpp

02_Grafos/Recorridos_y_conectividad

```
struct Tree {
    int n,z[2];
    vector<vector<int>> g;
    vector<int> ex,ey,p,w,f,v[2];
    Tree(int n):g(n),w(n),f(n){}
    void add_edge(int x, int y){
        p.pb(g[x].size());g[x].pb(ex.size());ex.pb(x);ey.pb(y);
        p.pb(g[y].size());g[y].pb(ex.size());ex.pb(y);ey.pb(x);
    }
    bool go(int k){ // returns true if it finds new node
        int& x=z[k];
        while(x>=0&&
            (w[x]==g[x].size()||w[x]==g[x].size()-1&&
(g[x].back()^1)==f[x]))
            x=f[x]>=0?ex[f[x]]:-1;
        if(x<0)return false;
        if((g[x][w[x]]^1)==f[x])w[x]++;
        int e=g[x][w[x]],y=ey[e];
        f[y]=e;w[x]++;w[y]=0;x=y;
        v[k].pb(x);
        return true;
    }
    vector<int> erase_edge(int e){
        e*=2; // erases eth edge, returns smaller component
        int x=ex[e],y=ey[e];
        p[g[x].back()]=p[e];
        g[x][p[e]]=g[x].back();g[x].pop_back();
        p[g[y].back()]=p[e^1];
        g[y][p[e^1]]=g[y].back();g[y].pop_back();
        f[x]=f[y]=-1;
        w[x]=w[y]=0;
        z[0]=x;z[1]=y;
        v[0]={x};v[1]={y};
        bool d0=true,d1=true;
        while(d0&&d1)d0=go(0),d1=go(1);
        if(d1)return v[0];
        return v[1];
    }
};
```

§2.10.7 transitive-closure.cpp

02_Grafos/Recorridos_y_conectividad

```
using ll = long long;
/* Transitive Closure O(n^3)
Check if from A can reach B in an undirected graph.
*/
const int inf = 1e9;
vector<int> adj[n];
ll distance[n][n];
void floydWarshall() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            distance[i][j] = false;
        }
    }
    for (int i = 0; i < n; i++) {
        for (int b : adj[i]) {
            distance[i][b] = true;
        }
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                distance[i][j] |= distance[i][k] & distance[k][j];
            }
        }
    }
}
```

§2.11 SCC y 2SAT

§2.11.1 or-statements-like-2-sat-problem.cpp

02_Grafos/SCC_y_2SAT

```
using ll = long long;
// Return the smaller lexicographic array of size n that satities a_i
| a_j = z
// a_i | a_i = z is allowed.
// there must exists a solution.
vector<ll> f(ll n, vector<tuple<ll,ll,ll>> &statements) {
    ll m = statements.size();
    vector<vector<pair<ll,ll>>> adj(n + 1);
    const ll bits = 30;
    vector<ll> taken(n+1, (1 << bits) - 1), answer(n+1, (1 << bits) -
1);
    for (int i = 0; i < m; i++) {
        ll x, y, z;
        tie(x, y, z) = statements[i];
        answer[x] &= z;
        answer[y] &= z;
        if (x == y) {
            taken[x] = 0;
            continue;
        }
        taken[x] &= z;
        taken[y] &= z;
        adj[x].pb({y, z});
        adj[y].pb({x, z});
    }
    for (int x = 1; x <= n; x++) {
        for (int i = 0; i < bits; i++) {
            if (!((taken[x] >> i) & 1)) continue;
            ll allHave = true;
            for (auto y : adj[x]) {
                if ((y.second >> i) & 1) {
                    allHave &= ((taken[y.first] >> i) & 1) ||
((answer[y.first] >> i) & 1);
                }
            }
            taken[x] -= 1 << i;
            if (allHave) {
                answer[x] -= 1 << i;
                for (auto y : adj[x]) {
                    if ((y.second >> i) & 1) {
                        taken[y.first] |= 1 << i;
                        taken[y.first] ^= 1 << i;
                    }
                }
            }
        }
    }
    answer.erase(answer.begin());
    return answer;
}
```

§2.11.2 strongly-connected-components.cpp

02_Grafos/SCC_y_2SAT

```
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
/*
 Tarjan t(graph); provides you the SCC of that graph
 passing the adjancency list of the graph (as vector<vl>)
```

```
 This is 0-indexed, (but you can have node 0 as dummy-node)
 Use t.comp[x] to get the component of node x
 SCC is the total number of components
 adjComp() gives you the adjacency list of strongly components
*/
struct Tarjan {
    vl low, pre, comp;
    ll cnt, SCC, n;
    vvl g;
    const int inf = 1e9;
    Tarjan(vvl &adj) {
        n = adj.size();
        g = adj;
        low = vl(n);
        pre = vl(n,-1);
        cnt = SCC = 0;
        comp = vl(n,-1);
        for (int i = 0;i<n;i++)
            if (pre[i] == -1) tarjan(i);
    }
    stack<int> st;
    void tarjan(int u) {
        low[u] = pre[u] = cnt++;
        st.push(u);
        for (auto &v : g[u]) {
            if (pre[v] == -1) tarjan(v);
            low[u] = min(low[u],low[v]);
        }
        if (low[u] == pre[u]) {
            while (true) {
                int v = st.top();st.pop();
                low[v] = inf;
                comp[v] = SCC;
                if (u == v) break;
            }
            SCC++;
        }
    }
    vvl adjComp() {
        vvl adj(SCC);
        for (int i = 0;i<n;i++) {
            for (auto j : g[i]) {
                if (comp[i] == comp[j]) continue;
                adj[comp[i]].pb(comp[j]);
            }
        }
        for (int i = 0;i<SCC;i++) {
            sort(all(adj[i]));
            adj[i].erase(
                unique(all(adj[i])),
                adj[i].end());
        }
        return adj;
    }
};
/* Another way is with with Kosaraju:
    1. Find topological order of G
    2. Run dfs in topological order in reverse Graph
       to find o connected component
*/
```

§2.11.3 two-sat.cpp

02_Grafos/SCC_y_2SAT

```
/*
2-Sat (Boolean satisfiability problem with 2-clause literals)
Complexity: O(n)
Tested: https://cses.fi/problemset/task/1684
To find a solution that makes this true with N boolean vars as form:
  (x or y) and (~x or y) and (z or ~x) and d and (x => y)
Call s.satisfiable() to see if solution, and sat2.value to see
values of variables
*/
struct sat2 {
  int n;
  vector<vector<vector<int>>> g;
  vector<int> tag;
  vector<bool> seen, value;
  stack<int> st;
  sat2(int n) : n(n), g(2, vector<vector<int>>(2*n)), tag(2*n),
seen(2*n), value(2*n) { }
  int neg(int x) { return 2*n-x-1; }
  void add_or(int u, int v) { implication(neg(u), v); }
  void make_true(int u) { add_edge(neg(u), u); }
  void make_false(int u) { make_true(neg(u)); }
  void eq(int u, int v) {
    implication(u, v);
    implication(v, u);
  }
  void diff(int u, int v) { eq(u, neg(v)); }
  void implication(int u, int v) {
    add_edge(u, v);
    add_edge(neg(v), neg(u));
  }
  void add_edge(int u, int v) {
    g[0][u].push_back(v);
    g[1][v].push_back(u);
  }
  void dfs(int id, int u, int t = 0) {
    seen[u] = true;
    for(auto& v : g[id][u])
      if(!seen[v])
        dfs(id, v, t);
    if(id == 0) st.push(u);
    else tag[u] = t;
  }
  void kosaraju() {
    for(int u = 0; u < n; u++) {
      if(!seen[u]) dfs(0, u);
      if(!seen[neg(u)]) dfs(0, neg(u));
    }
    fill(seen.begin(), seen.end(), false);
    int t = 0;
    while(!st.empty()) {
      int u = st.top(); st.pop();
      if(!seen[u]) dfs(1, u, t++);
    }
  }
  bool satisfiable() {
    kosaraju();
    for(int i = 0; i < n; i++) {
      if(tag[i] == tag[neg(i)]) return false;
      value[i] = tag[i] > tag[neg(i)];
    }
    return true;
  }
};
```

§2.12 Secuencias de grados

§2.12.1 erdos-gallai.cpp

02_Grafos/Secuencias_de_grados

```
using ll = long long;
// Erdos-Gallai - O(nlogn)
// check if it's possible to create a simple graph (undirected edges)
from
// a sequence of vertice's degrees
bool gallai(vector<int> v) {
  vector<ll> sum;
  sum.resize(v.size());
  sort(v.begin(), v.end(), greater<int>());
  sum[0] = v[0];
  for (int i = 1; i < v.size(); i++) sum[i] = sum[i-1] + v[i];
  if (sum.back() % 2) return 0;
  for (int k = 1; k < v.size(); k++) {
    int p = lower_bound(v.begin(), v.end(), k, greater<int>()) -
v.begin();
    if (p < k) p = k;
    if (sum[k-1] > 1ll*k*(p-1) + sum.back() - sum[p-1]) return 0;
  }
  return 1;
}
```

§2.13 Teoría

§2.13.1 theorems.cpp

02_Grafos/Teoria

```
apply bfs 0/1 when your states moves in 1 as much as possible
otherwise (the worst case: dijkstra)
```

§3 Árboles

§3.1 Descomposición centroide

§3.1.1 centroid-decomposition-extra.cpp

.._Arboles/Descomposicion_centroide

```
// Requisito de selección: Antes de build, dimensionar c_ans a N con
N como infinito; inicializar dep y dist. Las consultas esperan nodos
previamente marcados con upd.
// > !! Usar build() para inicializar. Problema de ejemplo: dado un
árbol, las actualizaciones son marcar nodos, y las consultas son
hallar la distancia mínima entre un nodo u y un nodo marcado.
const int N = 1e5+5;
vector<int> g[N];
int par[N], dep[N], sz[N];
map<int, int> dist[N]; // Distancia entre dos nodos en el árbol
original
vector<int> c_ans; // Respuesta optima de cada centroide hacía su
subárbol
int pre(int u, int p) {
    sz[u] = 1;
    for (auto &v : g[u])
        if (!dep[v] && v != p)
            sz[u] += pre(v, u);
    return sz[u];
}
int centroid(int u, int p, int k) {
    for (auto &v : g[u])
        if (!dep[v] && v != p && sz[v] > k)
            return centroid(v, u, k);
    return u;
}
// Calcular información de cada centroide hacía los nodos de su
subárbol
void dfs(int u, int p, int c, int d = 0) {
    dist[c][u] = d; // Depende de lo que pida el problema
    for (auto &v : g[u]) {
        if (!dep[v] && v != p) {
            dfs(v, u, c, d+1);
        }
    }
}
// O(nlogn * k) En este ejemplo k = logn por el mapa dist en el dfs
void build(int u, int p = -1, int d = 1) {
    int k = pre(u, p);
    int c = centroid(u, p, k/2);
    par[c] = p; dep[c] = d;
    dfs(c, p, c);
    for (auto &v : g[c])
        if (!dep[v]) build(v, c, d+1);
}
```

```cpp
// O(logn) lca de u y v en el árbol de centroides
// La ruta de u a v en el árbol original pasa por este lca
int lca(int u, int v) {
    for (; u != v; u = par[u])
        if (dep[v] > dep[u]) swap(u, v);
    return u;
}
// O((logn)^2) Actualiza la respuesta de los ancestros de u
void upd(int u) {
    for (int c = u; c != -1; c = par[c]) {
        c_ans[c] = min(c_ans[c], dist[c][u]);
    }
}
// O((logn)^2) Combina la respuesta del centroide c + el costo de c a u
int query(int u) {
    int mn = N;
    for (int c = u; c != -1; c = par[c]) {
        mn = min(mn, c_ans[c] + dist[c][u]);
    }
    return mn;
}
```

§3.2 Descomposición SQRT

§3.2.1 block-tree.cpp

03_Arboles/Descomposicion_SQRT

```cpp
using ll = long long;
/*
SQRT on Trees.
For having queries and updates online to subtrees or sub forests.
See: https://codeforces.com/blog/entry/46843
Solution to: https://codeforces.com/gym/100589/problem/A
Properties:
  - Each node will have atmost SQRT children nodes in Block Tree
  - There will be atmost SQRT Blocks
  - There will be atmost SQRT nodes belongs to a block
  - Not completely sure, but for merging answers of childs it should be fast
*/
struct Block { // Block storing info depending of problem!!!
    // O(n) memory here is possible, but better O(sqrt n) to BlockTree creation be O(n)
    vector<int> depthCnt;
    ll sum = 0;
    void update(int lvl, ll v) { // !!! depends of problem
        if (lvl >= depthCnt.size()) return;
        sum += v * depthCnt[lvl];
    }
    ll query() { return sum; }
};
struct BlockTree {
    const int SQRT = 320; // sqrt of max N
    ll n;
    vector<Block> blocks;
    vector<vector<int>> child; // if node < n,then normal node, otherwise block node
    vector<int> depth;
    vector<int> leader; // for the sqrt group of nodes the first ancestor that is not in the group
    vector<int> blockId;
    // ===== depends of the problem!!!
    vector<ll> sum;
    BlockTree(ll n, vector<vector<ll>> &adj) : n(n), child(2*n+5), depth(2*n+5),
                                               leader(2*n+5), blockId(2*n+5) {
        sum = vector<ll>(n);
        build(adj); // adjacency should be x -> y, and y -> x
    }
    bool is_block(ll x) { return x >= n; }
    void build(vector<vector<ll>> &adj, ll root = 0) {
        auto dfs = [&](ll x, ll p, auto &&dfs) -> void {
            for (auto y : adj[x]) if (y != p) {
                depth[y] = depth[x]+1;
                dfs(y,x,dfs);
            }
        };
        dfs(root, -1, dfs); // !!! Don't forget
        ll idNewBlock = n;
        auto createBlock = [&](int parent, const vector<ll> &nodes) {
            ll id = idNewBlock++;
            if (parent != -1) {
                child[parent].push_back(id);
                depth[id] = depth[parent]+1;
            }
            Block block; // Modify the creation of your block here!!!
            leader[id] = parent;
            blockId[id] = id;
            block.depthCnt = vector<int>(n);
            auto dfsOverBlock = [&](ll x, auto &&dfsOverBlock)->void {
                leader[x] = parent;
                blockId[x] = id;
                block.depthCnt[depth[x]]++;
                for (auto y : child[x]) {
                    if (!is_block(y)) dfsOverBlock(y, dfsOverBlock);
                    else child[id].push_back(y); // depending of the problem, you can add
                    // relationship between block node -> node or block node -> block node, like this
                }
            };
            for (auto & x : nodes) dfsOverBlock(x, dfsOverBlock);
            blocks.push_back(block);
        };
        vector<vector<ll>> byDepth(n); // counting sort
        for (int i =0;i<n;i++) byDepth[n-1-depth[i]].pb(i);
        vector<ll> sz(n);
        for (auto & nodes : byDepth) {
            for (int x : nodes) {
                ll blockSz = 0;
                vector<ll> curBlock;
                sz[x] = 1;
                for (int y : adj[x]) if (depth[x] < depth[y]) {
                    sz[x] += sz[y];
                    blockSz += sz[y];
                    curBlock.pb(y);
                    if (blockSz > SQRT) {
                        createBlock(x, curBlock);
                        sz[x] -= blockSz;
                        blockSz = 0;
                        curBlock.clear();
                    }
                }
                child[x].insert(child[x].end(), curBlock.begin(), curBlock.end());
            }
        }
        createBlock(-1, { root });
    }
    void update(int lvl, ll v) {
        sum[lvl] += v;
        for (auto & block : blocks) {
            block.update(lvl, v);
        }
    }
    ll query(ll x) { // dfs over node blocks and non node blocks
        ll ans = is_block(x) ? blocks[x-n].query() : sum[depth[x]];
        for (auto & y : child[x])
            ans += query(y);
        return ans;
    }
};
```

§3.3 DSU on Tree

§3.3.1 dsu-on-tree-sack-extra.cpp

03_Arboles/DSU_on_Tree

```cpp
// > Util para responder queries del tipo "cuantos vertices en el subárbol de u cumplen con x propiedad" en O(nlogn * k) para todas las queries.
// > Esta implementación permite mantener una única estructura global agregando y quitando información. k es el costo de actualizar esa estructura global.
// > Problema de ejemplo: dado un árbol con los nodos coloreados, las consultas son: cuántos nodos del subárbol de u tienen color igual a c.
const int N = 1e5+5;
vector<int> g[N];
int sz[N], big[N]; // big[u] = el hijo de u con mayor tamaño
int col[N]; // Del problema de ejemplo
int cnt[N]; // Estructura global del ejemplo
int pre(int u, int p) {
    sz[u] = 1; big[u] = -1;
    int mx = 0;
    for (auto &v : g[u]) {
        if (v != p) {
            sz[u] += pre(v, u);
            if (sz[v] > mx) mx = sz[v], big[u] = v;
        }
    }
    return sz[u];
}
void upd(int u, int p, int x) {
    cnt[col[u]] += x; // En el problema de ejemplo k = O(1)
    for (auto &v : g[u])
        if (v != p && v != big[u])
            upd(v, u, x);
}
void dfs(int u, int p, bool keep) {
    for (auto &v : g[u])
        if (v != p && v != big[u])
            dfs(v, u, false);
    if (big[u] != -1) dfs(big[u], u, true);
    // Agregar información del subárbol de los hijos de u != a big[u]
    upd(u, p, 1);
    // Aqui se responderían las queries
    // cnt[c] = cantidad de nodos del subárbol de u con color c
    big[u] = -1;
    if (!keep) upd(u, p, -1);  // Quitar información del subárbol de u
}
```

§3.4 Euler Tour

§3.4.1 euler-tour-sum.cpp

03_Arboles/Euler_Tour

```cpp
#include "../../01_Estructuras/Fenwick/fenwick-tree.cpp"
using ll = long long;
using vl = vector<long long>;
/* Euler Tour Sum Path O(n log n)
Given a Tree, and values in each node, process this queries:
 - Calculate the sum of values in the Path 1 to a 'given node'.
 - Update value of a node
Also you can extend this to sum path from A to B with updates.
Just add LCA and sum(0,A) + sum(0,B) - 2*(sum(0,lca)-values[lca])
Tested: https://cses.fi/problemset/task/1138/
*/
```

```
void test_case() {
    ll n, m; cin >> n >> m;
    vector<vl> adj(n+1); // 1-indexed
    vl nums(n+1), tin(n+1), tout(n+1);
    FenwickTree tree(2*n+2); // Add Fenwick Tree 0-indexed
    for (int i = 1; i <= n; i++) cin >> nums[i];
    for (int i = 0; i < n-1; i++) {
        ll x, y; cin >> x >> y;
        adj[x].pb(y); adj[y].pb(x);
    }
    ll time = 0;
    function<void(int,int)> dfs =[&](int x, int p) {
        tin[x] = time++;
        for (auto y : adj[x]) if (y != p) dfs(y, x);
        tout[x] = time++;
        tree.add(tin[x], nums[x]);
        tree.add(tout[x], -nums[x]);
    };
    dfs(1, 0);
    for (int i =0; i < m; i++) {
        ll t, x;
        cin >> t >> x;
        if (t == 1) { // update
            ll y; cin >> y;
            ll diff = y - nums[x];
            nums[x] = y;
            tree.add(tin[x], diff);
            tree.add(tout[x], -diff);
        } else { // query
            cout << tree.sum(0, tin[x]) << "\n";
        }
    }
}
```

§3.5 Heavy Light

§3.5.1 hld-lca-con-heavy-paths.cpp

03_Arboles/Heavy_Light

```
/*
Nota:
- el segment tree no esta implementado, solo HLD
Uso:
- poner grafo en G
- correr init(N)
- para cada nodo v -> segBase[pos[v]] = val[v] (segBase es un nuevo
arreglo)
- inicializar segment tree para [0, N-1]
- implementar la funcion segQuery(int l, int r)
*/
vector<vi> G;
vi parent, depth, heavy, head, pos;
int curPos;
int dfs(int v) {
    int size = 1;
    int maxChildrenSize = 0;
    for (int c : G[v]) {
        if (c == parent[v]) continue;
        parent[c] = v;
        depth[c] = depth[v] + 1;
        int cSize = dfs(c);
        size += cSize;
        if (cSize > maxChildrenSize) {
            maxChildrenSize = cSize;
            heavy[v] = c;
        }
    }
    return size;
}
void decompose(int v, int h) {
    head[v] = h;
    pos[v] = curPos++;
    if (heavy[v] != -1) {
        decompose(heavy[v], h);
    }
    for (int c : G[v]) {
        if (c != parent[v] && c != heavy[v]) {
            decompose(c, c);
        }
    }
}
void init(int nodes) {
    parent.assign(nodes, -1);
    depth.resize(nodes);
    heavy.assign(nodes, -1);
    head.resize(nodes);
    pos.resize(nodes);
    curPos = 0;
    // Raiz 0
    dfs(0);
    decompose(0, 0);
}
int segQuery(int l, int r);
int hldQuery(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]]) {
            swap(a, b);
        }
        int pathMax = segQuery(pos[head[b]], pos[b]);
        res = max(pathMax, res);
    }
    if (depth[a] > depth[b]) swap(a, b);
    int lastPath = segQuery(pos[a], pos[b]);
    res = max(res, lastPath);
    return res;
}
```

§3.6 Isomorfismo y hashing

§3.6.1 tree-isomorphism-extra.cpp

03_Arboles/Isomorfismo_y_hashing

```
using ll = long long;
using ull = unsigned long long;
// Hash de árboles enraizados: hash(u) = R[suma de hash(hijos)], R
aleatoria de 64 bits.
// Subárboles isomorfos <=> mismo hash (colisión ~n^2/2^64).
Iterativo; O(n log n).
// Tested
https://judge.yosupo.jp/problem/rooted_tree_isomorphism_classificatio
n
mt19937_64
rngIso(chrono::steady_clock::now().time_since_epoch().count());
map<ull, ull> table;
ull get(ull x) {
    auto it = table.find(x);
    if (it != table.end()) return it->second;
    return table[x] = rngIso();
}
vector<ull> hashes, sum;
// g: lista de adyacencia (dirigida padre->hijos o no dirigida).
Calcula hashes[u] del
// subárbol de u con raíz root. O(n log n) por el map.
void dfs(const vector<vector<int>> &g, int root) {
    int n = g.size();
    hashes.assign(n, 0); sum.assign(n, 0);
    vector<int> order = {root}, par(n, -1);
    for (size_t k = 0; k < order.size(); k++) {
        int u = order[k];
        for (int v : g[u]) if (v != par[u]) par[v] = u, order.pb(v);
    }
    for (int k = (int)order.size() - 1; k >= 0; k--) {   // hijos
antes que padres
        int u = order[k];
        hashes[u] = get(sum[u]);
        if (par[u] >= 0) sum[par[u]] += hashes[u];
    }
}
```

§3.7 LCA y binary lifting

§3.7.1 binary-lifting-o-n-memory.cpp

03_Arboles/LCA_y_binary_lifting

```
struct Node{ int de; Node *par, *go; };
typedef Node* pNode;
pNode find(pNode nod, int de)
{
    while(nod->de > de)
        if(nod->go->de < de)
            nod = nod->par;
        else
            nod = nod->go;
    return nod;
}
pNode add(pNode nod, int ind)
{
    pNode nuevo = new Node();
    nuevo->par = nod;
    nuevo->de = nod->de + 1;
    if(nod->de - nod->go->de == nod->go->de - nod->go->go->de)
        nuevo->go = nod->go->go;
    else
        nuevo->go = nod;
    return nuevo;
}
pNode lca(pNode a, pNode b)
{
    if(a->de < b->de) swap(a, b);
    a = find(a, b->de);
    while(a != b)
        if(a->go == b->go)
            a = a->par, b = b->par;
        else
            a = a->go, b = b->go;
    return a;
}
```

§3.7.2 lca-multiplication-path.cpp

03_Arboles/LCA_y_binary_lifting

```
using namespace std;
#define double long double
const int N = 1e5+10;
const int INF = 1e17+1;
const int mod = 1e9+7;
//const int mod = 998244353;
const double eps = 1e-6;
typedef pair<int,int> pii;
int n, q;
vi grafo[N+1], tin, tout;
vector<vi> dp;
int timer = 0, l;
struct node {
    vi v;
    void insert(int x) {
        auto it = lower_bound(v.begin(), v.end(), x);
        if (it != v.end() && *it == x ) return;
        v.insert(it, x);
        if (v.size() > 10) v.pop_back();
    }
};
node merge(node &a, node &b) {
    node res;
```

```
    for (int i = 0, j = 0; res.v.size() < 10; )
    if (i < a.v.size() && j < b.v.size()) {
        if (a.v[i] < b.v[j])
            res.insert(a.v[i++]);
        else
            res.insert(b.v[j++]);
    } else if (i<a.v.size()) {
        res.insert(a.v[i++]);
    } else if (j<b.v.size()) {
        res.insert(b.v[j++]);
    } else break;
    return res;
}
vector<node> a;
vector<vector<node>> dpa;
void dfs(int i, int p = 1) {
    tin[i] = timer++;
    dp[i][0] = p;
    dpa[i][0] = merge(a[i], a[p]);
    for (int j = 1; j<=l; j++)
        dp[i][j] = dp[dp[i][j-1]][j-1],
        dpa[i][j] = merge(dpa[i][j-1], dpa[dp[i][j-1]][j-1]);
    for (auto &to: grafo[i])
        if (to!=p) dfs(to, i);
    tout[i] = timer++;
}
int is_ancestor(int i, int j) {
    return tin[i]<tin[j] && tout[i]>tout[j];
}
int lca(int i, int j) {
    // to append functions take in mind: fdp[i][k] = [i, p_k-1(i)) U
[p_k-1(i), p_k(i)]
    if (i==j) return i;
    if (is_ancestor(i, j)) return i;
    if (is_ancestor(j, i)) return j;
    for (int k = l; k>-1; k--)
        if (!is_ancestor(dp[i][k], j)) i = dp[i][k];
    return dp[i][0];
}
node fa(int i, int p) {
    node res;
    for (int k = l; k>-1; k--)
    if (!is_ancestor(dp[i][k], p))
        res = merge(res, dpa[i][k]), i = dp[i][k];
    if (i==p) res = merge(res, a[i]);
    else res = merge(res, dpa[i][0]);
    return res;
}
void init() {
    for (int i = 1; i<=n; i++) grafo[i].clear();
    tin.resize(n+1), tout.resize(n+1);
    l = 0;
    while ((1<<l) < n) l++;
    dp.resize(n+1, vi(l+1, -1));
    dpa.resize(n+1, vector<node>(l+1));
    a.resize(n+1);
}
void solve() {
    int m; cin>>n>>m>>q;
    init();
    for (int i = 1; i<n; i++) {
        int a, b; cin>>a>>b;
        grafo[a].pb(b);
        grafo[b].pb(a);
    }
    for (int i = 1; i<=m; i++) {
        int x; cin>>x;
        a[x].insert(i);
    }
    dfs(1);
    while (q--) {
        int ax, bx, x; cin>>ax>>bx>>x;
        int p = lca(ax, bx);
        auto fia = fa(ax, p); auto fib = fa(bx, p);
        node res = merge(fia, fib);
        vi aux = res.v;
        cout<<min(x, (int)aux.size())<<' ';
        for (int i =0; i<min(x, (int)aux.size()); i++) cout<<aux[i]
<<' ';
            cout<<endl;
    }
}
signed main() {
    fast;
    int T = 1; //cin>>T;
    while (T--) solve();
    //while (!cin.eof()) solve();
    return 0;
}
```

§3.7.3 lowest-common-ancestor.cpp

03_Arboles/LCA_y_binary_lifting

```
// Requisito de selección: Árbol dirigido de padre a hijos, raíz 0.
LOG se calcula en init según n; kParent requiere k<=depth[x].
using ll = long long;
using vvl = vector<vector<long long>>;
// Give a rooted tree, find the Lowest Common Ancestor node
// of A and B.
// Tested https://cses.fi/problemset/task/1688/
// Tested https://judge.yosupo.jp/problem/lca (N = 5e5, árboles en
camino)
vector<vector<ll>> up;
vector<ll> depth;
int LOG = 1; // se calcula en init: 2^LOG > n (antes fijo en 18:
fallaba con profundidad >= 2^18)
void init(const vector<vector<ll>> &children, ll n) {
    LOG = 1;
    while ((1LL << LOG) <= n) LOG++;
    up.assign(LOG,vector<ll>(n,0));
    depth.assign(n,0);
    // Recorrido iterativo (BFS): una recursión de profundidad ~n
puede desbordar la pila.
    vector<ll> order = {0}; // raíz 0
    for (size_t k = 0; k < order.size(); k++) {
        ll x = order[k];
        for (auto y : children[x]) {
            up[0][y] = x;
            depth[y] = depth[x] +1;
            order.pb(y);
        }
    }
    for (int i = 1; i < LOG; i++)
        for (int j = 0; j < n; j++)
            up[i][j] = up[i-1][up[i-1][j]];
}
ll kParent(ll x, ll k) {
    ll i = 0;
    while (k) {
        if (k & 1) x = up[i][x];
        k >>= 1;
        i++;
    }
    return x;
}
ll query(ll x, ll y) {
    if (depth[x] < depth[y]) swap(x, y);
    x = kParent(x, depth[x] - depth[y]);
    if (x == y) {
        return x;
    }
    for (int i = LOG - 1; i >= 0; i--) {
        if (up[i][x] != up[i][y]) {
            x = up[i][x];
            y = up[i][y];
        }
    }
    return up[0][x];
}
void test_case() {
    ll n, q; cin >> n >> q;
    vvl children(n);
    for (int i = 1; i < n; i++) {
        ll p; cin >> p; p--;
        children[p].pb(i);
    }
    init(children, n);
    for (int i = 0; i < q; i++) {
        ll x, y;cin >> x >> y;
        x--,y--;
        cout << query(x, y) + 1 << "\n";
    }
}
```

§3.7.4 max-weight-lca.cpp

03_Arboles/LCA_y_binary_lifting

```
using pii = pair<int, int>;
// Using LCA to find max edge weight between (u, v)
const int N = 1e5+5;  // Max number of vertices
const int K = 20;     // Each 1e3 requires ~ 10 K
const int M = K+5;
int n;                // Number of vertices
vector <pii> adj[N];
int vis[N], h[N], anc[N][M], mx[N][M];
void dfs (int u) {
    vis[u] = 1;
    for (auto p : adj[u]) {
        int v = p.first;
        int w = p.second;
        if (!vis[v]) {
            h[v] = h[u]+1;
            anc[v][0] = u;
            mx[v][0] = w;
            dfs(v);
        }
    }
}
void build () {
    // cl(mn, 63) -- Don't forget to initialize with INF if min edge!
    anc[1][0] = 1;
    dfs(1);
    for (int j = 1; j <= K; j++) for (int i = 1; i <= n; i++) {
        anc[i][j] = anc[anc[i][j-1]][j-1];
        mx[i][j] = max(mx[i][j-1], mx[anc[i][j-1]][j-1]);
    }
}
int mxedge (int u, int v) {
    int ans = 0;
    if (h[u] < h[v]) swap(u, v);
    for (int j = K; j >= 0; j--) if (h[anc[u][j]] >= h[v]) {
        ans = max(ans, mx[u][j]);
        u = anc[u][j];
    }
    if (u == v) return ans;
    for (int j = K; j >= 0; j--) if (anc[u][j] != anc[v][j]) {
        ans = max(ans, mx[u][j]);
        ans = max(ans, mx[v][j]);
        u = anc[u][j];
        v = anc[v][j];
    }
```

```
    return max({ans, mx[u][0], mx[v][0]});
}
```

§3.8 Link Cut Tree

§3.8.1 link-cut.cpp

03_Arboles/Link_Cut_Tree

```
const int N_DEL = 0, N_VAL = 0; //delta, value
inline int mOp(int x, int y){return x+y;}//modify
inline int qOp(int lval, int rval){return lval + rval;}//query
inline int dOnSeg(int d, int len){return d==N_DEL ? N_DEL : d*len;}
//mostly generic
inline int joinD(int d1, int d2){
  if(d1==N_DEL)return d2;if(d2==N_DEL)return d1;return mOp(d1, d2);}
inline int joinVD(int v, int d){return d==N_DEL ? v : mOp(v, d);}
struct Node_t{
  int sz, nVal, tVal, d;
  bool rev;
  Node_t *c[2], *p;
  Node_t(int v) : sz(1), nVal(v), tVal(v), d(N_DEL), rev(0), p(0){
  c[0]=c[1]=0;
  }
  bool isRoot(){return !p || (p->c[0] != this && p->c[1] != this);}
  void push(){
  if(rev){
    rev=0; swap(c[0], c[1]);
    for (int x=(0), nb_fin_1=(2); x<nb_fin_1; ++x)if(c[x])c[x]-
>rev^=1;
  }
  nVal=joinVD(nVal, d); tVal=joinVD(tVal, dOnSeg(d, sz));
  for (int x=(0), nb_fin_3=(2); x<nb_fin_3; ++x)if(c[x])c[x]-
>d=joinD(c[x]->d, d);
  d=N_DEL;
  }
  void upd();
};
typedef Node_t* Node;
int getSize(Node r){return r ? r->sz : 0;}
int getPV(Node r){
  return r ? joinVD(r->tVal, dOnSeg(r->d,r->sz)) : N_VAL;}
void Node_t::upd(){
  tVal = qOp(qOp(getPV(c[0]), joinVD(nVal, d)), getPV(c[1]));
  sz = 1 + getSize(c[0]) + getSize(c[1]);
}
void conn(Node c, Node p, int il){if(c)c->p=p;if(il>=0)p->c[!il]=c;}
void rotate(Node x){
  Node p = x->p, g = p->p;
  bool gCh=p->isRoot(), isl = x==p->c[0];
  conn(x->c[isl],p,isl); conn(p,x,!isl);
  conn(x,g,gCh?-1:(p==g->c[0])); p->upd();
}
void spa(Node x){//splay
  while(!x->isRoot()){
  Node p = x->p, g = p->p;
  if(!p->isRoot())g->push();
  p->push(); x->push();
  if(!p->isRoot())rotate((x==p->c[0])==(p==g->c[0])? p : x);
  rotate(x);
  }
  x->push(); x->upd();
}
Node exv(Node x){//expose
  Node last=0;
  for(Node y=x; y; y=y->p)spa(y),y->c[0]=last,y->upd(),last=y;
  spa(x);
  return last;
}
void mkR(Node x){exv(x);x->rev^=1;}//makeRoot
// Al bajar por el splay hay que hacer push: mkR deja inversiones
(rev) pendientes y sin push
// se baja por el hijo equivocado (getR, father y lift daban nodos
incorrectos tras mkR).
Node getR(Node x){exv(x);for(x->push();x->c[1];x->push())x=x-
>c[1];spa(x);return x;}
Node lca(Node x, Node y){exv(x); return exv(y);}
bool connected(Node x, Node y){exv(x);exv(y); return x==y?1:x->p!=0;}
void link(Node x, Node y){mkR(x); x->p=y;}
void cut(Node x, Node y){mkR(x); exv(y); y->c[1]->p=0; y->c[1]=0;}
Node father(Node x){
  exv(x);
  Node r=x->c[1];
  if(!r)return 0;
  for(r->push();r->c[0];r->push())r=r->c[0];
  spa(r); // splay del nodo hallado: mantiene la complejidad
amortizada
  return r;
}
void cut(Node x){ // cuts x from father keeping tree root. Requiere
que x NO sea la raíz.
  exv(father(x));x->p=0;}
int query(Node x, Node y){mkR(x); exv(y); return getPV(y);}
void modify(Node x, Node y, int d){mkR(x);exv(y);y->d=joinD(y->d,d);}
Node lift_rec(Node x, int t){
  if(!x)return 0;
  x->push();
  if(t==getSize(x->c[0])){spa(x);return x;}
  if(t<getSize(x->c[0]))return lift_rec(x->c[0],t);
  return lift_rec(x->c[1],t-getSize(x->c[0])-1);
}
Node lift(Node x, int t){ // t-th ancestor of x (lift(x,1) is x's
father)
  exv(x);return lift_rec(x,t);}
int depth(Node x){ // distance from x to its tree root
  exv(x);return getSize(x)-1;}
```

§3.9 Mo en árboles

§3.9.1 mo-s-in-tree-s.cpp

03_Arboles/Mo_en_arboles

```
using namespace std;
typedef vector<vi> vvi;
map<int, int> getID;
map<int, int>::iterator it;
const int LOGN = 20;
int id, bs, N;
int counter[50050];
int A[50050], P[100050];
int res[100050];
int st[50050], ed[50050];
int DP[20][50050], level[50050];
bool flag[50050];
bool seen[50050];
vvi edges;
struct Q {
   int l, r, p, id;
   bool operator < (const Q& other) const {
      return (l / bs < other.l / bs || (l / bs == other.l / bs && r <
other.r));
   }//operator <
} q[100050];
void DFS0(const int u) {
   seen[u] = 1;
   P[id] = u;
   st[u] = id++;
   for (auto& e : edges[u]) {
      if (!seen[e]) {
         DP[0][e] = u;
         level[e] = level[u] + 1;
         DFS0(e);
      }//if
   }//for
   P[id] = u;
   ed[u] = id++;
}//DFS0
void prep(const int r) {
   level[r] = 0;
   for (int i = 0; i < LOGN; i++)
      DP[i][r] = r;
   id = 0;
   DFS0(r);
   for (int i = 1; i < LOGN; i++)
      for (int j = 1; j <= N; j++)
         DP[i][j] = DP[i - 1][DP[i - 1][j]];
}//prep
int LCA(int a, int b) {
   if (level[a] > level[b])
      swap(a, b);
   int diff = level[b] - level[a];
   for (int i = 0; i < LOGN; i++)
      if (diff & (1 << i))
         b = DP[i][b]; //move 2^i parents upwards
   if (a == b)
      return a;
   for (int i = LOGN - 1; i >= 0; i--)
      if (DP[i][a] != DP[i][b])
         a = DP[i][a], b = DP[i][b];
   return DP[0][a];
}//LCA
signed main() {
   int Q, n1, n2, L, R, a, v = 1, tot;
   scanf("%lld %lld", &N, &Q);
   edges.assign(N + 5, vi());
   bs = sqrt(N);
   for (int i = 1; i <= N; i++) {
      scanf("%lld", &a);
      A[i] = ((it = getID.find(a)) != getID.end()) ? it->second :
(getID[a] = v++);
   }//for
   for (int i = 0; i < N - 1; i++) {
      scanf("%lld %lld", &n1, &n2);
      edges[n1].push_back(n2);
      edges[n2].push_back(n1);
   }//for
   prep(1);
   for (int i = 0; i < Q; i++) {
      scanf("%lld %lld", &n1, &n2);
      if (st[n1] > st[n2])
         swap(n1, n2);
      q[i].p = LCA(n1, n2);
      if (q[i].p == n1)
         q[i].l = st[n1], q[i].r = st[n2];
      else
         q[i].l = ed[n1], q[i].r = st[n2];
      q[i].id = i;
   }//for
   sort(q, q + Q);
   L = 0; R = -1; tot = 0;
   for (int i = 0; i < Q; i++) {
      while (R < q[i].r) {
         if (!flag[P[++R]])
            tot += (++counter[A[P[R]]] == 1);
         else
            tot -= (--counter[A[P[R]]] == 0);
         flag[P[R]] = !flag[P[R]];
      }//while
      while (R > q[i].r) {
         if (!flag[P[R]])
            tot += (++counter[A[P[R]]] == 1);
         else
```

```
            tot -= (--counter[A[P[R]]] == 0);
         flag[P[R]] = !flag[P[R]];
         R--;
      }//while
      while (L < q[i].l) {
         if (!flag[P[L]])
            tot += (++counter[A[P[L]]] == 1);
         else
            tot -= (--counter[A[P[L]]] == 0);
         flag[P[L]] = !flag[P[L]];
         L++;
      }//while
      while (L > q[i].l) {
         if (!flag[P[--L]])
            tot += (++counter[A[P[L]]] == 1);
         else
            tot -= (--counter[A[P[L]]] == 0);
         flag[P[L]] = !flag[P[L]];
      }//while
      res[q[i].id] = tot + (q[i].p != P[q[i].l] &&
!counter[A[q[i].p]]);
   }//for
   for (int i = 0; i < Q; i++)
      printf("%lld\n", res[i]);
   return 0;
}//main
```

§3.10 Problemas

§3.10.1 nearest-selected-nodes-problem.cpp

03_Arboles/Problemas

```
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
// Given an order of selected nodes in a tree, you should print the
miminum distance between two selected nodes after each operation.
// O(nlogn or n*sqrt(n)); n <= 2*10^5, 2.7 seconds.
// adj is the adjacency list, order is the selected nodes in order
// n is the numeber of nodes, returns the minimum after each
operation
// note that operation 0 answer is 1e9
vl f(vvl &adj, vl &order, ll n) {
    vl answer;
    vl dist(n + 1, 1e9);
    ll best = 1e9;
    vl q(n + 1);
    ll sz = 0;
    for (int i = 0; i < n; i++) {
        best = min(best, dist[order[i]]);
        sz = 0;
        dist[order[i]] = 0;
        q[sz++] = order[i];
        ll idx = 0;
        while (idx < sz) {
            ll x = q[idx++];
            if (dist[x] + 1 >= best) break;
            for (auto &y : adj[x]) {
                if (dist[x] + 1 < dist[y]) {
                    dist[y] = dist[x] + 1;
                    q[sz++] = y;
                }
            }
        }
        answer.pb(best);
    }
    return answer;
}
```

§3.10.2 two-pieces-on-tree-problem.cpp

03_Arboles/Problemas

```
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
// In a tree with 'n' nodes where 2 pieces starting from root 1
// must go to certain nodes each one and must not exceed 'd' between
// the two pieces, after they have to return to root 1
// two_pieces_on_tree() find the minimum quantity of moves
// My submittion in CF:
https://codeforces.com/contest/1774/submission/189071201
ll n, d; // quantity of nodes, maximum distance between pieces
vvl children; // tree
vector<int> a, b; // nodes that must visit first and second piecesS
void dfs(ll x, vl &route) {
    route.pb(x);
    ll kParent = 1; //route
    if (route.size() - 1 >= d) {
        kParent = route[route.size() - 1 - d];
    }
    b[kParent] |= a[x];
    a[kParent] |= b[x];
    for (auto &y : children[x]) {
        dfs(y, route);
        a[x] |= a[y];
        b[x] |= b[y];
    }
    route.pop_back();
}
ll two_pieces_on_tree() {
    ll root = 1;
    vl emptyRoute = vl();
    dfs(root, emptyRoute);
    ll total = 0;
    for (int i = 1; i <= n; i++) {
        total += a[i] + b[i];
    }
    return total * 2 - 4;
}
```

§3.11 Rerooting

§3.11.1 re-rooting.cpp

03_Arboles/Rerooting

```
/*
 * simple idea: just do dfs first and then do
 * dfs reversed (dfs pass data for child node.
 * Example: maximum path for each node in O(n):
 * */
void dfs(int i, int p = -1) {
    // pasa: camino maximo hasta i.
    priority_queue<int> pq;
    for (auto &to: grafo[i])
        if (to!=p)
            dfs(to, i), pq.push(dp[to]);
    if (pq.size()) {
        dp[i] = pq.top()+1; pq.pop();
        if (pq.size()) maxi[i] = pq.top()+1;
    }
}
void dfss(int i, int p  = -1, int k = 0) {
    priority_queue<int> pq;
    pq.push(dp[i]); pq.push(maxi[i]), pq.push(k);
    dp[i] = pq.top(); pq.pop();
    maxi[i] = pq.top(); pq.pop();
    for (auto &to: grafo[i])
        if (to!=p) {
            int r = dp[i]==dp[to]+1? maxi[i]: dp[i];
            dfss(to, i, r+1);
        }
}
```

```
    }
}
```

§4 DP

§4.1 Cambio de monedas

§4.1.1 coin-change-large-n.cpp

04_DP/Cambio_de_monedas

```
using namespace std;
signed main() {
    // coin change optimization over n -> 1e9 xor max_value_coin<1e4
    int n, piv; cin>>n;
    vi coins, dp;
    // you have m distinct coins:
    sort(coins.begin(), coins.end());
    piv = lcm(coins[m-1], coins[max(0ll, m-2)]); // upper_bound +10
    dp.resize(m+100, INF);
    dp[0] =0 ;
    for (int i= 1 ;i<m; i++)
    for (auto x: coins)
        if (i-x>-1) dp[i] = min(dp[i], dp[i-x]+1);
    int res = INF;
    if (n<m) res= dp[n];
    else {
        for (int i = 0; i<m; i++) {
            int r = n-i;
            if (r%coins[m-1]==0) res = min(res, dp[i]+r/coins[m-1]);
        }
    }
    return res;
}
```

§4.1.2 coin-change.cpp

04_DP/Cambio_de_monedas

```
using ll = long long;
using vl = vector<long long>;
// infinite number of coins
// Get the minimum number of coins that sum a value.
void test_case() {
    ll n, x;
    cin >> n >> x;
    vl dp(x + 1, inf - 1);
    vl coin(n);
    for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i) cin >> coin[i];
    dp[0] = 0;
    for (int i=(0), nb_fin_3=(x); i<nb_fin_3; ++i) {
        for (auto &c : coin) {
            if (c + i > x) continue;
            dp[i + c] = min(dp[i + c], dp[i] + 1);
        }
    }
    if (dp[x] + 1== inf) {
        cout << "-1\n";
    } else {
        cout << dp[x] << "\n";
    }
}
```

§4.2 Contribuciones XOR

§4.2.1 sum-of-xor-subarrays-times-its-size-problem.cpp

...ibuciones_XOR

```
using ll = long long;
using vl = vector<long long>;
// Given an array A
// calculate:
// (∑l=0 to (n-1) ∑r=l to (n-1) f(l,r)·(r−l+1)) % mod
// f(l,r) = a[l] ^ a[l+1] ^ ... ^ a[r]
// In other words, it calculate the sum
```

```
// of xor-subarrays multiplied by its size
// A or nums is 0-indexed
// sum(nums,n,998244353)
const int mod = 998244353;
ll sum(vl &nums, ll n) {
    vector<ll> pref(n+1);
    for (int i = 1; i <= n; i++) {
        pref[i] = pref[i-1] ^ nums[i-1];
    }
    ll ans =0;
    for (ll bit = 0; bit <= 60; bit++) {
        vl cnt(2);
        vl dist(2);
        ll sum = 0;
        for (int i = 0; i <= n; i++) {
            ll actual = (pref[i] >> bit) & 1;
            sum = (sum + dist[!actual]) % mod;
            cnt[actual]++;
            dist[actual] = (dist[actual] + cnt[actual]) % mod;
            dist[!actual] = (dist[!actual] + cnt[!actual]) % mod;
        }
        ans += (((1ll << bit) % mod) * sum) % mod;
        ans %= mod;
    }
    return ans;
}
```

§4.3 Digit DP

§4.3.1 digit-dp.cpp

04_DP/Digit_DP

```
// Problem:
/*
Your task is to count the number of integers between [0, n]
where no two adjacent digits are the same.
*/
// ** all states where ith > 0 are intial -1 and ith = 1 by default
int dp[N][10][10][2]; // ith, prev. digit, digit, trailing zeros
int fi(int i, int antd, int d, int tz = 0) {
    if (antd == d && !(d==0 && tz)) return 0;
    if (dp[i][antd][d][tz]!=-1) return dp[i][antd][d][tz];
    dp[i][antd][d][tz] = 0;
    for (int di = 0; di<10; di++)
        dp[i][antd][d][tz] += fi(i-1, d, di, tz & (!di));
    return dp[i][antd][d][tz];
}
int f(string &s, int i, int tz = 0) {
    if (i<0) return 1;
    int res = 0;
    if (s[i] != s[i+1] || (tz && s[i]=='0'))
        res += f(s, i-1, tz & (s[i]=='0'));
    for (int d = s[i]-'0'-1; d >= 0; d--) {
        int ttz = tz & (d==0);
        res += fi(i, s[i+1]-'0', d, ttz);
    }
    return res;
}
int f(int n) { // principal function
    if (n<0) return 0;
    string s = to_string(n);
    reverse(s.begin(), s.end()); s.pb('0');
    return f(s, s.size()-2, 1);
}
```

§4.4 LCS y distancia edición

§4.4.1 edit-distance.cpp

04_DP/LCS_y_distancia_edicion

```
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
// My own
ll editDistance(string &s, string &t) {
    ll n = s.size();
    ll m = t.size();
    vvl dp(n+1, vl(m+1, 0));
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (min(i, j) == 0) dp[i][j] = max(i, j);
            else if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = min(dp[i-1][j], min(dp[i][j-1], dp[i-1]
[j-1])) + 1;
        }
    }
    return dp[n][m];
}
```

§4.4.2 lcs-bitset.cpp

04_DP/LCS_y_distancia_edicion

```
// Implementación añadida al notebook; base: 00_Template/simple.cpp.
// Longitud de LCS para strings, incluidos caracteres repetidos y
bytes 0..255.
// Referencia: https://allisons.org/ll/Publications/1986IPL/
// n = mayor longitud, m = menor longitud, W = ceil(m/64).
// Tiempo O(m + (n+256)*W); memoria O(256*W) palabras de 64 bits.
// No es O(N log N) general. No reconstruye la subsecuencia.
// Usa bytes: para Unicode, string representa bytes UTF-8, no puntos
de código.
// uint64_t es intencional: NO sustituir por int (Long long signed en
la plantilla).
// Ejemplo: cout << lcs_bitset("abcde", "ace"); // 3
#ifndef NOTEBOOK_LCS_BITSET
#define NOTEBOOK_LCS_BITSET
int lcs_bitset(string_view a, string_view b) {
    if (a.size() < b.size()) swap(a, b);
    if (b.empty()) return 0;
    const int palabras = ((int)b.size() + 63) / 64;
    array<vector<uint64_t>, 256> mascara;
    for (auto& bits : mascara) bits.assign(palabras, 0);
    for (int j = 0; j < (int)b.size(); ++j)
        mascara[static_cast<unsigned char>(b[j])][j / 64] |=
uint64_t(1) << (j % 64);
    vector<uint64_t> fila(palabras, 0);
    for (unsigned char c : a) {
        uint64_t acarreo = 1, prestamo = 0;
        for (int j = 0; j < palabras; ++j) {
            const uint64_t anterior = fila[j];
            const uint64_t x = anterior | mascara[c][j];
            const uint64_t y = (anterior << 1) | acarreo;
            acarreo = anterior >> 63;
            // Resta multipalabra x - y - prestamo, sin overflow
signed.
            const uint64_t parcial = x - y;
            const uint64_t resta = parcial - prestamo;
            prestamo = (x < y) || (parcial < prestamo);
            fila[j] = x & ~resta;
        }
    }
    // Los bits fuera de b siguen en cero: fila siempre es un
subconjunto de x.
    int respuesta = 0;
    for (uint64_t palabra : fila) respuesta +=
__builtin_popcountll(palabra);
    return respuesta;
}
#endif
```

§4.4.3 lcs-lis.cpp

04_DP/LCS_y_distancia_edicion

```
// Implementación añadida al notebook; base: 00_Template/simple.cpp.
// Longitud de LCS mediante posiciones + LIS (Hunt-Szymanski).
// Referencia: https://doi.org/10.1145/359581.359603
// n = a.size(), m = b.size(), r = número de pares (i,j) con a[i] ==
b[j].
// Tiempo O((n+m+r) * log(m+2)), memoria O(m). No materializa los r
pares.
// Si b no tiene repetidos: r <= n, tiempo O((n+m) * log(m+2)).
// En particular, dos permutaciones de tamaño N: O(N log N).
// Con muchos repetidos puede ser O(n*m*log(m+2)): usar lcs_bitset
para strings.
// Admite string y vector<int>; devuelve la longitud, no reconstruye
la LCS.
// Ejemplo: vi a={3,1,2}, b={1,3,2}; cout << lcs_lis(a,b); // 2
#ifndef NOTEBOOK_LCS_LIS
#define NOTEBOOK_LCS_LIS
template<class Secuencia>
int lcs_lis(const Secuencia& a, const Secuencia& b) {
    using Valor = typename Secuencia::value_type;
    map<Valor, vi> posiciones;
    for (int j = 0; j < (int)b.size(); ++j) posiciones[b[j]].pb(j);
    vi mejores;
    mejores.reserve(min(a.size(), b.size()));
    for (const auto& valor : a) {
        auto encontrado = posiciones.find(valor);
        if (encontrado == posiciones.end()) continue;
        const vi& indices = encontrado->second;
        // Descendente: no usar dos coincidencias de un mismo a[i] en
la LIS.
        for (auto it = indices.rbegin(); it != indices.rend(); ++it)
{
            auto lugar = lower_bound(all(mejores), *it);
            if (lugar == mejores.end()) mejores.pb(*it);
            else *lugar = *it;
        }
    }
    return (int)mejores.size();
}
#endif
```

§4.4.4 longest-common-subsequence-3.cpp

04_DP/LCS_y_distancia_edicion

```
string X = "AGGT12";
string Y = "12TXAYB";
string Z = "12XBA";
bool calc[100][100][100];
int dp[100][100][100];
//lcsOf3(X.size() - 1, Y.size() - 1, Z.size() - 1);
int lcsOf3(int i, int j,int k) {
    if(i==-1||j==-1||k==-1) // outbounds
        return 0;
    if(calc[i][j][k]) //memo
        return dp[i][j][k];
    calc[i][j][k] = true;
    if(X[i]==Y[j] && Y[j]==Z[k]) // same
        return dp[i][j][k] = 1+lcsOf3(i-1,j-1,k-1);
    else // best of reducine any
        return dp[i][j][k] = max(max(lcsOf3(i-1,j,k),
                                 lcsOf3(i,j-1,k)),lcsOf3(i,j,k-1));
}
```

§4.4.5 longest-common-subsequence.cpp

04_DP/LCS_y_distancia_edicion

```
const int M_MAX = 20;
const int N_MAX = 20;
int m, n;
string X;
string Y;
int memo[M_MAX + 1][N_MAX + 1];
// Encuetra el Longest Common Subsequence de string X e Y. m y n son
sus tamaños
// lcs de abfgh aeeeeiiiiigh =  agh
int lcs (int m, int n) {
```

```
  for (int i = 0; i <= m; i++) {
    for (int j = 0; j <= n; j++) {
      if (i == 0 || j == 0) memo[i][j] = 0;
      else if (X[i - 1] == Y[j - 1]) memo[i][j] = memo[i - 1][j - 1]
+ 1;
      else memo[i][j] = max(memo[i - 1][j], memo[i][j - 1]);
    }
  }
  return memo[m][n];
}
```

§4.4.6 README.md

04_DP/LCS_y_distancia_edicion

LIS y LCS: versión a elegir

Todas las funciones devuelven la **longitud** de la subsecuencia; no la reconstruyen.

Caso	Función / archivo	Tiempo
LIS estrictamente creciente	`lis(a)` en LIS existente	O(N log N), memoria O(N)
LCS de permutaciones o con valores distintos en la segunda secuencia	`lcs_lis(a,b)` en lcs-lis.cpp	O((n+m) log(m+2)); O(N log N) para tamaños N
LCS con pocas coincidencias, incluso repetidos	La misma `lcs_lis(a,b)`	O((n+m+r) log(m+2)), donde r cuenta pares de posiciones con valores iguales
LCS de strings generales, incluidos muchos repetidos	`lcs_bitset(a,b)` en lcs-bitset.cpp	O(m + (n+256)·ceil(m/64)), con n mayor longitud y m menor
DP clásica de referencia	`lcs(m,n)` en LCS existente	O(n·m) tiempo y memoria; tabla original limitada a 20 por cadena

La reducción LCS → LIS no proporciona O(N log N) para cadenas arbitrarias: con repeticiones puede haber n·m coincidencias. El recorrido descendente de posiciones evita contar varias veces un mismo elemento. La versión con bloques de 64 bits acelera la DP general, pero tampoco ofrece O(N log N).

`lcs_lis` acepta dos `vi` o dos `string`. `lcs_bitset` recibe strings/string_view y compara bytes. Ambas soportan entradas vacías, no tienen el límite fijo de 20 y usan el preámbulo de `00_Template/simple.cpp`. Los tipos `uint64_t` de la versión por bits deben mantenerse unsigned aunque la plantilla defina `int` como `long long`.

```
vi a = {3, 1, 2}, b = {1, 3, 2};
cout << lcs_lis(a, b) << endl; // 2
cout << lcs_bitset("banana", "atana") << endl; // 4
```

Referencias: Hunt–Szymanski, Allison–Dix.

Pruebas reproducibles: _pruebas/lis-lcs.cpp, con oráculos de DP, entradas vacías, repetidos, permutaciones, enteros de 64 bits y límites entre palabras de 64 bits. Ver el resultado.

§4.5 LIS

§4.5.1 longest-increasing-subsequence.cpp

04_DP/LIS

```
using vl = vector<long long>;
// Find the Longest Increasing Subsequence of an array in O(n log n)
// 1 2 3 5 10 2 -1 100 500 -> input
// 1 2 3 5 10 100 500 -> Lis
int lis(vl &nums) {
    vl best;
    int n = nums.size();
    for (int i = 0; i < n; i++) {
        // For non-decreasing
        // int idx = upper_bound(all(best), nums[i]) - best.begin();
        // For increasing
        int idx = lower_bound(all(best), nums[i]) - best.begin();
        if (idx == best.size()) {
            best.pb(nums[i]);
        } else {
            best[idx] = min(best[idx], nums[i]);
        }
    }
    return best.size();
}
// Also you can do this with Segment Tree in O(n Log n)
```

§4.6 Máscaras y SOS

§4.6.1 dp-mask-over-submasks.cpp

04_DP/Mascaras_y_SOS

```
using ll = long long;
using vl = vector<long long>;
// DP of submask over submasks
// O(3^n)
// j&(-j); get a '1' bit of j
// for (int j=i;j;j = (j-1)&i){...} j is submask of 'i' the mask
ll dp[1<<18]; // answer of mask
ll cst[1<<18]; // cost of use a submask
ll a[18][18]; // elements
ll pos[1<<18]; // trick to get fast the pos
void test_case() {
    ll n;
    cin >> n;
    for (int i =0;i<n;i++) {
        for (int j =0;j<n;j++) {
            cin >> a[i][j];
        }
    }
    for (int i =0;i<n;i++) {
        pos[1<<i] = i;
    }
    for (int i = 0;i<(1<<n);i++) {
        ll j = i;
        vl idxs;
        while (j) {
            ll k = j&(-j);
            idxs.pb(pos[k]);
            j^=k;
        }
        for (int j = 0;j<idxs.size();j++) {
            for (int k = j+1;k<idxs.size();k++) {
                cst[i] += a[idxs[j]][idxs[k]];
            }
        }
        dp[i] = cst[i];
        for (int j=i;j;j = (j-1)&i) {
            dp[i] = max(dp[i],cst[j]+dp[i^j]);
        }
    }
    cout << dp[(1<<n)-1] << "\n";
}
```

§4.6.2 sos-dp-reverse.cpp

04_DP/Mascaras_y_SOS

```
// Requisito: n, M y vectores; pegar el bloque dentro de solve().
using ll = long long;
// Useful to calculate reverse sos DP.
// DP[x] = op(DP[y_1], DP[y_2], ...), where x is submask of y_i
    vector<ll> nums(n);
    for (int i =0;i<n;i++) {
        nums[i] = rd(); // read submask, and set init state
        dp[nums[i]] = min(dp[nums[i]], {0, i});
    }
    for (int mask = (1<<m)-1; mask >= 0; mask--) { // You can switch
fors depends of the problem
        if (dp[mask].first > 0) continue; // Don't propagate
        for (int i =0;i<m;i++) {
            if ((mask>>i)&1) { // Propagate to all submasks
                auto nxt = dp[mask]; nxt.first--;
                dp[mask^(1ll<<i)] = min(dp[mask^(1ll<<i)], nxt);
            }
        }
    }
```

§4.6.3 sos-dp.cpp

04_DP/Mascaras_y_SOS

```
// Requisito: M y dp; pegar el bucle dentro de solve() o una función.
// F(mask) = op(F(x_1), F(x_2), ...) where x is submask of mask
// init sos[z]
for (int i = 0; i < M; i++) {
  for (int mask = 0; mask < (1 << M); mask++) {
    if (mask & (1 << i)) {
      sos[mask] += sos[mask ^ (1 << i)];
    }       }
}
```

§4.7 Máscaras y TSP

§4.7.1 elevator-problem.cpp

04_DP/Mascaras_y_TSP

```
using ll = long long;
using vl = vector<long long>;
// Given n <= 20 persons, print the minimum number of travels
// to move everyone in a elevator with capacity k.
ll n, k;
vl nums;
vector<pair<ll,ll>> dp;
// minimum travels, last travel with minimum weight.
// use  f((1 << n) - 1).F
pair<ll,ll> f(ll mask) {
    if (dp[mask] != make_pair(-1ll, -1ll)) {
        return dp[mask];
    }
    if (mask == 0) {
        return dp[mask] = {0, k};
    }
    dp[mask] = {n + 1, 0}; // one person in a travel, or use
popcount.
    for (int i = 0; i < n; i++) {
        // person i is the last to enter to elevator.
        if ((mask >> i) & 1) {
            auto actual = f(mask ^ (1 << i)); // best option without
this last person.
            if (actual.second + nums[i] <= k) {
                actual.second += nums[i];
                // what happened if there are a better minimum.
                // well in that case the last person should be other
one.
                // so we are trying all options that last person will
be better.
            } else {
                actual.second = nums[i];
                actual.first++;
            }
            dp[mask] = min(dp[mask], actual);
        }
    }
    return dp[mask];
}
// Iterative
void test_case() {
    ll n, k;
    cin >> n >> k;
    vl nums(n);
    vector<pair<ll,ll>> dp(1 << n, {n+1, 0});
    for (int i =0; i < n; i++) cin >> nums[i];
    dp[0] = {0, k};
    for (int i = 1; i < (1 << n); i++) {
        for (int j = 0; j < n; j++) {
            if (i& (1 << j)) {
```

```
                auto actual = dp[i ^ (1 << j)];
                if (actual.second + nums[j] <= k) {
                    actual.second += nums[j];
                } else {
                    actual.first++;
                    actual.second = nums[j];
                }
                dp[i] = min(dp[i], actual);
            }
        }
    }
    cout << dp[(1 << n) -1].first << "\n";
}
```

§4.7.2 traveling-sales-man-cycle.cpp

04_DP/Mascaras_y_TSP

```
using vpl = vector<pair<long long, long long>>;
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
// Given directed weighted graph, gets the minimun halmilton cycle.
// Use dfs(0, 1<<(initial)), if 1e9 then it impossible, otherwise get
the min.
const int MAX_SIZE = 15;
const ll IMPOSSIBLE = 1e9;
ll INITIAL = 0; // initial node
vpl adj[MAX_SIZE];
vvl dp(MAX_SIZE, vl(1 << MAX_SIZE, -1));
ll n, m;
ll target; // init as (1 << n) - 1, full visited
ll dfs(ll x, ll mask) {
    if (dp[x][mask] != -1) {
        return dp[x][mask];
    }
    if (mask == target) {
        for (auto &yy : adj[x]) {
            if (yy.first == INITIAL) {
                return yy.second;
            }
        }
        return dp[x][mask] = IMPOSSIBLE;
    }
    ll ans = IMPOSSIBLE;
    for (auto &yy : adj[x]) {
        ll y, d;
        tie(y, d) = yy;
        if ((mask >> y) & 1) continue;
        ll actual = dfs(y, mask | (1 << y)) + d;
        ans = min(ans, actual);
    }
    return dp[x][mask] = ans;
}
```

§4.7.3 traveling-sales-man-path.cpp

04_DP/Mascaras_y_TSP

```
using ll = long long;
using vl = vector<long long>;
using vvl = vector<vector<long long>>;
// Given directed weighted graph, gets the minimun halmilton path.
ll n;
vvl adj;
int dp[1<<20][20];
ll f(ll mask, ll x) {
    if (dp[mask][x] != -1) {
        return dp[mask][x];
    }
    if (x == n-1) {
        return (mask == 0);
    }
    ll ways = 0;
    for (auto y : adj[x]) {
        if ((mask>>y)&1) {
            ways = (ways + f(mask^(1<<y), y)) % MOD;
        }
    }
    return dp[mask][x] = ways;
}
void test_case() {
    ll m;
    cin >> n >> m;
    adj.assign(n,vl());
    memset(dp,-1,sizeof dp);
    for (int i=0;i<m;i++) {
        ll x, y;
        cin >>x >> y;
        x--,y--;
        adj[x].pb(y);
    }
    cout << f((1<<n)-1-1,0) << "\n";
}
```

§4.8 Notas

§4.8.1 dp.cpp

04_DP/Notas

```
sub array problems where the function is like this:
f(l, r) - k*(r-l+1)/m; there is an algorithm that works in O (n*m)
using:
dp[i] = max/min over all len <= m: f(i-len+1, i) - k + dp[i-len]
because dp[i] we assume that puts a complete cycle interval
(division) to apply -k * 1 (new cycle created.
check all submask from a mask efficiently: O(3^n):
for (submask = 0; submask; submask = (submask&mask)-1)
SOS dp (cuenta bien pero no se como)
for (i = 0; i<n; i++) for (int mask = 0; mask < (1<<n); mask)
if (mask & (1<<i)) dp[mask] += dp[mask ^ (1<<i)];
```

§4.9 Optimizaciones › CHT y Li Chao

§4.9.1 convex-hull-trick-extra.cpp

04_DP/Optimizaciones/CHT_y_Li_Chao

```
using ll = long long;
// > Agregar líneas en orden no-creciente por pendiente m. Permite
consultar el mínimo f(x). Para máximo, cambiar el signo de las
líneas: -m, -h.
typedef ll T;
struct line { T m, h; };
struct cht {
    vector<line> c;
    int pos = 0;
    T inter(line a, line b) {
        T x = b.h-a.h, y = a.m-b.m;
        return x/y + (x%y ? (x>0)==(y>0) : 0); // == ceil(x/y)
    }
    void add(T m, T h) {
        line l = {m, h};
        if (c.size() && m == c.back().m) {
            l.h = min(h, c.back().h);
            c.pop_back(); if (pos) pos--;
        }
        while (c.size() > 1 && inter(c.back(), l) <=
inter(c[c.size()-2], c.back())) {
            c.pop_back(); if (pos) pos--;
        }
        c.pb(l);
    }
    inline bool fbin(T x, int m) { return inter(c[m], c[m+1]) > x; }
    T query(T x) {
        // O(log n) query:
        int l = 0, r = c.size();
        while (r-l > 1) {
            int m = (l+r)/2;
            if (fbin(x, m-1)) r = m;
            else l = m;
        }
        return c[l].m*x + c[l].h;
        // O(1) query (para x's ordenadas):
        while (pos > 0 && fbin(x, pos-1)) pos--;
        while (pos < (int)c.size()-1 && !fbin(x, pos)) pos++;
        return c[pos].m*x + c[pos].h;
    }
};
```

§4.9.2 convex-hull-trick.cpp

04_DP/Optimizaciones/CHT_y_Li_Chao

```
using ll = long long;
// Agregar rectas k*x + m y consultar el MÁXIMO en x, O(log n). Para
mínimo: negar k, m y la respuesta.
// Tested https://atcoder.jp/contests/dp/submissions/55836691
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

§4.9.3 li-chao-tree-vasito.cpp

04_DP/Optimizaciones/CHT_y_Li_Chao

```
// LiChaoTree lct(sz) -> allows queries for all x in range [0,sz)
// lct.add(L)         -> add line L to range [0,sz) in O(log(sz))
// lct.add(L,st,en)   -> add line L to range [st,en) in O(log^2(sz))
// lct.query(x)       -> query maximum L(x) across all current lines
in O(log(sz))
typedef long long T;
T INF=1e18;
struct Line{
    T a, b;
    Line():a(0),b(-INF){}
    Line(T a, T b):a(a),b(b){}
    T get(T x){return a*x+b;}
};
struct LiChaoTree{
    struct Node{
        Line line;
        Node *lc,*rc=0;
    }*root;
    int sz;
    LiChaoTree(int sz): root(new Node()), sz(sz){}
    void addRec(Node* &n, int tl, int tr, Line x){
```

```
        if(!n) n=new Node();
        if(n->line.get(tl)<x.get(tl)) swap(n->line,x);
        if(n->line.get(tr-1)>=x.get(tr-1)||tl+1==tr) return;
        int mid=(tl+tr)/2;
        if(n->line.get(mid)>x.get(mid)){
            addRec(n->rc,mid,tr,x);
        }
        else{
            swap(n->line,x);
            addRec(n->lc,tl,mid,x);
        }
    }
    void add(Node* &n, int tl, int tr, int l, int r, Line x){
        if(tr<=l||r<=tl) return;
        if(!n) n=new Node();
        if(l<=tl&&tr<=r) return addRec(n,tl,tr,x);
        int mid=(tl+tr)/2;
        add(n->lc,tl,mid,l,r,x);
        add(n->rc,mid,tr,l,r,x);
    }
    T query(Node* &n, int tl, int tr, int x){
        if(!n) return -INF;
        if(tl+1==tr) return n->line.get(x);
        T res=n->line.get(x);
        int mid=(tl+tr)/2;
        if(x<mid) res=max(res, query(n->lc,tl,mid,x));
        else res=max(res, query(n->rc,mid,tr,x));
        return res;
    }
    void add(Line x){add(root,0,sz,0,sz,x);}
    void add(Line x, int l, int r){add(root,0,sz,l,r,x);}
    T query(int x){return query(root,0,sz,x);}
};
```

§4.9.4 persistent-convex-hull-trick.cpp

04_DP/Optimizaciones/CHT_y_Li_Chao

```
//  add lines of the form kx+m, and query maximum values at points x.
struct per_hull{
    const int logtam = 18;
    vector<int> k, m;
    vector<vector<int>> pars;
    per_hull(){pars.push_back(vi(logtam)), k.push_back(0),
m.push_back(0);}
    int query(int x, int head)
    {
        auto f = [&](int x, int head) {
            return x * k[head] + m[head];
        };
        for(int i = logtam - 1; i > -1; i--)
            if(f(x, pars[head][i]) < f(x, pars[pars[head][i]][1]))
                head = pars[pars[head][i]][1];
        return f(x, head);
    }
    int add(int k1, int m1, int head)
    {
        for(int i = logtam - 1; i > -1; i--)
        {
            if(pars[head][i] == 0) continue;
            int b = pars[head][i];
            int a = pars[pars[head][i]][1];
            if((m[b] - m[a]) * (k[a] - k1) > (m1 - m[a]) * (k[a] -
k[b]))
                head = pars[pars[head][i]][1];
        }
        int node = k.size();
        k.push_back(k1), m.push_back(m1);
        pars.push_back(vector<int>(logtam));
        pars[node][0] = node;
        pars[node][1] = head;
        for(int i = 2; i < logtam; i++)
            pars[node][i] = pars[pars[node][i - 1]][i - 1];
        return node;
    }
};
```

§4.10 Optimizaciones › Divide and Conquer

§4.10.1 divide-and-conquer.cpp

04_DP/Optimizaciones/Divide_and_Conquer

```
// Requisito de selección: El comentario anuncia mínimo, pero el
cuerpo usa max y centinela -1. Ajustar objetivo, centinela, tam y
función de coste juntos.
using ll = long long;
/*
DP[i][j] = min( DP[i-1][k] + C[k][j] )
K[i][j] <= K[i][j+1]
*/
ll lastDP[tam], DP[tam];
int C[tam][tam]; // Cambiar a una funcion de costo si pre-procesar
ocupa mucha memoria
void DC(int b, int e, int KL, int KR) {
  int mid = (b + e) / 2;
  pair<ll, int> best = make_pair(-1, KL);
  for (int k = KL; k < min(mid, KR+1); k++)
    best = max( best, make_pair(lastDP[k] + C[k+1][mid], k) );
  DP[mid] = best.first;
  int K = best.second;
  if (b <= mid-1)
    DC(b, mid-1, KL, K);
  if (mid+1 <= e)
    DC(mid+1, e, K, KR);
}
```

§4.11 Optimizaciones › Knuth

§4.11.1 knuths-optimization.cpp

04_DP/Optimizaciones/Knuth

```
using ll = long long;
/*
Knuth's Optimization
For dp[i][j] = min_{i<=k<j} dp[i][k] + dp[k+1][j] + cost[i][j]
Optimizing that from O(n^3) to O(n^2), it's required to hold the
following:
opt[i][j] = optimal splitting point of dp[i][j], the 'k' the
minimizes the above definition
opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
You can demostrate that by the following:
a<=b<=c<=d
cost(b,c) <= cost(a,d), // an contained interval is <= of the
interval
cost(a,c)+cost(b,d) <= cost(a,d)+cost(b,c) // partial intersection <=
total intersection
Complexity: O(n^2)
*/
void test_case() {
    cin >> n;nums.assign(n,0);pf.assign(n+1,0); // READ input!!!
    for (int i =0;i<n;i++) cin>>nums[i], pf[i+1] = pf[i] + nums[i];
    for (int i = 0;i<n;i++) { // base case
        dp[i][i] = 0; // depends of the dp!!!!
        opt[i][i] = i;
    }
    for (int i =n-2;i>=0;i--) {
        for (int j = i+1;j<n;j++) {
            dp[i][j] = inf; // set to inf, any option is better, or
use -1 or a flag!!
 //note about dp[i][j] is that I set inf instead of cost(i,j) and it
got accepted, not sure why
            ll cost = sum(i,j); // depends of problem
            for (int k = opt[i][j-1]; k<= min(j-1ll,opt[i+1][j]);k++)
{
                ll actual = dp[i][k] + dp[k+1][j] + cost;
                if (actual < dp[i][j]) { // if flag '-1' used, change
here!!
                    dp[i][j] = actual;
                    opt[i][j] = k;
                }
            }
        }
    }
    cout << dp[0][n-1] << "\n";
}
// https://cses.fi/problemset/task/2088
// https://www.spoj.com/problems/BRKSTRNG/
```

§4.12 Optimizaciones › Mochila

§4.12.1 multiple-knacksack-optimizacion.cpp

04_DP/Optimizaciones/Mochila

```
// Requisito de selección: El original intercambia valor y peso al
descomponer objetos; revisar/corregir antes de usar.
using ll = long long;
using vl = vector<long long>;
/*
Multiple Knacksack
You have a knacksack of a capacity, and 'n' objects with
value, weight, and a number of copies that you can buy of that
object.
Maximize the value without exceding the capacity of the knacksack.
Time complexity is O(W*N*sum)
W = capacity, N = number of objects,
sum is: for (int i =0, sum = 0;i<n;i++) sum += log2(copies[i])
Tested in https://cses.fi/problemset/task/1159/
n<=100, capacity<=10^5, copies[i]<=1000
*/
ll multipleKnacksack(vl &value, vl& weight, vl&copies, ll capacity) {
    vl nb_vs,ws;
    ll n = value.size();
    for (int i = 0;i<n;i++) {
        ll h=value[i],s=weight[i],k=copies[i];
        ll p = 1;
        while (k>p) {
            k-=p; // Binary Grouping Optimization
            nb_vs.pb(s*p);
            ws.pb(h*p);
            p*=2;
        }
        if (k) {
            nb_vs.pb(s*k);
            ws.pb(h*k);
        }
    }
    vl dp(capacity+1);
    // 0-1 knacksack
    for (int i =0;i<ws.size();i++) {
        for (int j = capacity;j>=ws[i];j--) {
            dp[j] = max(dp[j],dp[j-ws[i]] + nb_vs[i]);
        }
    }
    return dp[capacity];
}
```

§4.13 Optimizaciones › Separación

§4.13.1 separation-optimization.cpp

04_DP/Optimizaciones/Separacion

No se si tiene un nombre, pero en varios problemas vi este truco:
dp[x] = f(x,y) donde se necesita un optimo 'y'.
Truco: si puedes separarlo en f(x,y) en f(x) (+) f(y), puedes guardar en una estructa optima f(y) como un segment tree.

§4.14 Steiner Tree

§4.14.1 steiner-tree.cpp

04_DP/Steiner_Tree

```cpp
// Steiner-Tree O(2^t*n^2 + n*3^t + APSP)
// N - number of nodes
// T - number of terminals
// dist[N][N] - Adjacency matrix
// steiner_tree() = min cost to connect first t nodes, 1-indexed
// dp[i][bit_mask] = min cost to connect nodes active in bitmask rooting in i
// min{dp[i][bit_mask]}, i <= n if root doesn't matter
int n, t, dp[N][(1 << T)], dist[N][N];
int steiner_tree() {
  for (int k = 1; k <= n; ++k)
    for (int i = 1; i <= n; ++i)
      for (int j = 1; j <= n; ++j)
        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
  for(int i = 1; i <= n; i++)
    for(int j = 0; j < (1 << t); j++)
      dp[i][j] = INF;
  for(int i = 1; i <= t; i++) dp[i][1 << (i-1)] = 0;
  for(int msk = 0; msk < (1 << t); msk++) {
    for(int i = 1; i <= n; i++) {
      for(int ss = msk; ss > 0; ss = (ss - 1) & msk)
        dp[i][msk] = min(dp[i][msk], dp[i][ss] + dp[i][msk - ss]);
      if(dp[i][msk] != INF)
        for(int j = 1; j <= n; j++)
          dp[j][msk] = min(dp[j][msk], dp[i][msk] + dist[i][j]);
    }
  }
  int mn = INF;
  for(int i = 1; i <= n; i++) mn = min(mn, dp[i][(1 << t) - 1]);
  return mn;
}
```

§4.15 Subarreglos y submatrices

§4.15.1 max-sum-1d.cpp

04_DP/Subarreglos_y_submatrices

```cpp
int maxRangeSum(vector<int> a){
    int sum = 0, ans = 0;
    for (int i = 0; i < a.size(); i++){
        if (sum + a[i] >= 0) {
            sum += a[i];
          ans = max(ans, sum);
      } else sum = 0;
    }
    return ans;
}
```

§4.15.2 max-sum-2d.cpp

04_DP/Subarreglos_y_submatrices

```cpp
int INF = -100000007; // minimo valor
int n, m; //filas y columnas
const int MAX_N = 105, MAX_M = 105;
int values[MAX_N][MAX_M];
int max_range_sum2D(){
  for(int i=0; i<n;i++){
    for(int j=0; j<m; j++){
      if(i>0) values[i][j] += values[i-1][j];
      if(j>0) values[i][j] += values[i][j-1];
      if(i>0 && j>0) values[i][j] -= values[i-1][j-1];
    }
  }
  int max_mat = INF;
  for(int i=0; i<n;i++){
    for(int j=0; j<m; j++){
      for(int h = i; h<n; h++){
        for(int k  = j; k<m; k++){
          int sub_mat = values[h][k];
          if(i>0) sub_mat -= values[i-1][k];
          if(j>0) sub_mat -= values[h][j-1];
          if(i>0 && j>0) sub_mat += values[i-1][j-1];
          max_mat = max(sub_mat, max_mat);
        }
      }
    }
  }
  return max_mat;
}
```

§4.15.3 max-sum-3d.cpp

04_DP/Subarreglos_y_submatrices

```cpp
const long long a=20, b=20, c=20;
long long acum[a][b][c];
long long INF = -100000000007;
long long max_range_3D(){
  for(int x=0; x<a; x++){
    for(int y = 0; y<b; y++){
      for(int z = 0; z<c; z++){
        if(x>0) acum[x][y][z] += acum[x-1][y][z];
        if(y>0) acum[x][y][z] += acum[x][y-1][z];
        if(z>0) acum[x][y][z] += acum[x][y][z-1];
        if(x>0 && y>0) acum[x][y][z] -= acum[x-1][y-1][z];
        if(x>0 && z>0) acum[x][y][z] -= acum[x-1][y][z-1];
        if(y>0 && z>0) acum[x][y][z] -= acum[x][y-1][z-1];
        if(x>0 && y>0 && z>0) acum[x][y][z] += acum[x-1][y-1][z-1];
      }
    }
  }
  long long max_value = INF;
  for(int x=0; x<a; x++){
    for(int y = 0; y<b; y++){
      for(int z = 0; z<c; z++){
        for(int h = x; h<a; h++){
          for(int k = y; k<b; k++){
            for(int l = z; l<c; l++){
              long long aux = acum[h][k][l];
              if(x>0) aux -= acum[x-1][k][l];
              if(y>0) aux -= acum[h][y-1][l];
              if(z>0) aux -= acum[x][k][z-1];
              if(x>0 && y>0) aux += acum[x-1][y-1][l];
              if(x>0 && z>0) aux += acum[x-1][k][z-1];
              if(z>0 && y>0) aux += acum[h][y-1][z-1];
              if(x>0 && y>0 && z>0) aux -= acum[x-1][y-1][z-1];
              max_value = max(max_value, aux);
            }
          }
        }
      }
    }
  }
  return max_value;
}
```

§4.15.4 min-max-range-extra.cpp

04_DP/Subarreglos_y_submatrices

```cpp
// Requisito: El original carece de return ans, inicializa ans a -1 para minimizar y recurre con i+1 aunque el corte termina en j: revisar transición e inicialización antes de usar.
using ll = long long;
// Devuelve el el minimo de los maximos entre pares de rangos consecutivos haciendo cortes en el Array.
const int MAX = 1005;
ll dp[MAX][MAX];
ll sum_ran[MAX][MAX];
int N;
ll f(int i, int cuts) {
    if (cuts == 0) return sum_ran[i][N-1];
    if (i == N) return 0;
    ll &ans = dp[i][cuts];
    if (ans != - 1) return ans;
    for (int j = i; j < N; j++) {
        ans = min(ans, max(sum_ran[i][j], f(i + 1, cuts - 1)));
    }
}
```

§5 Flujos y matching

§5.1 Corte mínimo

§5.1.1 min-cut.cpp

05_Flujos_y_matching/Corte_minimo

```cpp
#include "../Flujo_maximo/max-flow.cpp"
using ll = long long;
using vvl = vector<vector<long long>>;
/*
Minimum Cut, need revision for general cases!!!
Tested in https://cses.fi/problemset/task/1695/
  with undirected graph with capacity 1
1. Run Max Flow Algorithm
2. Get reachable vertices from source in the residual graph!
   using BFS or DFS, call this reachable vertices = S
3. Iterate S, iterate edges of S in normal graph that not belongs to S,
   thats the mincut edges of the answer.
*/
void test_case() {
    ll n, m; cin >> n >> m;
    vvl g(n);
    Dinitz f(n,0,n-1); // add Dinitz for maxFlow!!
    for (int i =0;i<m;i++) {
        ll x, y;cin >>x >> y;x--,y--;
        g[x].pb(y); g[y].pb(x); //this example the graph is bidirectional
        f.addEdge(x,y,1); f.addEdge(y,x,1);
    }
    f.maxFlow(); // step 1
    vvl residual(n); // Step 2
    for (int i =0;i<n;i++) for (auto j:f.G[i])
        if ((j.f<0 && j.cap==0) || (j.f==0 && j.cap > 0))
            residual[i].pb(j.to); // residual graph
    set<ll> vis;
    function<void(int)> dfs = [&](int x) {
        vis.insert(x); // dfs on residual from source
        for (auto y : residual[x]) if (!vis.count(y)) dfs(y);
    };
    dfs(0);
    vector<pair<ll,ll>> ans; // step 3
    for (auto x : vis)
        for (auto y : g[x])
            if (!vis.count(y))
                ans.pb({x,y});
    cout << ans.size() << "\n";
    for (auto e : ans) {
        cout << e.first + 1 << " " << e.second + 1 << "\n";
    }
}
```

§5.1.2 stoer-wagner-extra.cpp

05_Flujos_y_matching/Corte_minimo

```cpp
// Halla el corte mínimo en un grafo no dirigido y con pesos O(V ^ 3)
struct stoer_wagner {
    int n;
    vector<vector<int>> g;
    stoer_wagner(int n) : n(n), g(n, vector<int>(n)) {}
    void add_edge(int a, int b, int w) {
        g[a][b] = g[b][a] = w;
    }
    pair<int, vector<int>> min_cut() {
        vector<int> used(n);
```

```
        vector<int> cut, best_cut;
        int best_weight = -1;
        for (int p = n-1; p >= 0; --p) {
            vector<int> w = g[0];
            vector<int> added = used;
            int prv, lst = 0;
            for (int i = 0; i < p; ++i) {
                prv = lst; lst = -1;
                for (int j = 1; j < n; ++j) {
                    if (!added[j] && (lst == -1 || w[j] > w[lst]))
lst = j;
                }
                if (i == p-1) {
                    for (int j = 0; j < n; j++)
                        g[prv][j] += g[lst][j];
                    for (int j = 0; j < n; j++)
                        g[j][prv] = g[prv][j];
                    used[lst] = true;
                    cut.pb(lst);
                    if (best_weight == -1 || w[lst] < best_weight) {
                        best_cut = cut;
                        best_weight = w[lst];
                    }
                } else {
                    for (int j = 0; j < n; j++)
                        w[j] += g[lst][j];
                    added[lst] = true;
                }
            }
        }
        return {best_weight, best_cut}; // best_cut contains all
nodes in the same set
    }
};
```

§5.2 Flujo costo mínimo

§5.2.1 min-cost-max-flow-vasito.cpp

05_Flujos_y_matching/Flujo_costo_minimo

```
using ll = long long;
typedef ll tf;
typedef ll tc;
const tf INFFLOW=1e9;
const tc INFCOST=1e9;
struct MCF{
    int n;
    vector<tc> prio, pot; vector<tf> curflow; vector<int>
prevedge,prevnode;
    priority_queue<pair<tc, int>, vector<pair<tc, int>>,
greater<pair<tc, int>>> q;
    struct edge{int to, rev; tf f, cap; tc cost;};
    vector<vector<edge>> g;
    MCF(int
n):n(n),prio(n),curflow(n),prevedge(n),prevnode(n),pot(n),g(n){}
    void add_edge(int s, int t, tf cap, tc cost) {
        g[s].pb((edge){t,static_cast<long long>
((g[t]).size()),0,cap,cost});
        g[t].pb((edge){s,static_cast<long long>
((g[s]).size())-1,0,0,-cost});
    }
    pair<tf,tc> get_flow(int s, int t) {
        tf flow=0; tc flowcost=0;
        while(1){
            q.push({0, s});
            fill(all(prio),INFCOST);
            prio[s]=0; curflow[s]=INFFLOW;
            while(!q.empty()) {
                auto cur=q.top();
                tc d=cur.first;
                int u=cur.second;
                q.pop();
                if(d!=prio[u]) continue;
                for(int i=0; i<static_cast<long long>
((g[u]).size()); ++i) {
                    edge &e=g[u][i];
                    int v=e.to;
                    if(e.cap<=e.f) continue;
                    tc nprio=prio[u]+e.cost+pot[u]-pot[v];
                    if(prio[v]>nprio) {
                        prio[v]=nprio;
                        q.push({nprio, v});
                        prevnode[v]=u; prevedge[v]=i;
                        curflow[v]=min(curflow[u], e.cap-e.f);
                    }
                }
            }
            if(prio[t]==INFCOST) break;
            for (int i=(0), nb_fin_5=(n); i<nb_fin_5; ++i)
pot[i]+=prio[i];
            tf df=min(curflow[t], INFFLOW-flow);
            flow+=df;
            for(int v=t; v!=s; v=prevnode[v]) {
                edge &e=g[prevnode[v]][prevedge[v]];
                e.f+=df; g[v][e.rev].f-=df;
                flowcost+=df*e.cost;
            }
        }
        return {flow,flowcost};
    }
};
```

§5.3 Flujo máximo

§5.3.1 max-flow.cpp

05_Flujos_y_matching/Flujo_maximo

```
using ll = long long;
//#define int long long // take care int overflow with this
//#define vi vector<long long>
struct Dinitz{
    const int INF = 1e9 + 7;
    Dinitz(){}
    Dinitz(int n, int s, int t) {init(n, s, t);}
    void init(int n, int s, int t)
    {
        nb_S = s, T = t;
        nodes = n;
        G.clear(), G.resize(n);
        Q.resize(n);
    }
    struct flowEdge
    {
        int to, rev, f, cap;
    };
    vector<vector<flowEdge> > G;
    // Añade arista (st -> en) con su capacidad
    void addEdge(int st, int en, int cap) {
        flowEdge A = {en, (int)G[en].size(), 0, cap};
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].pb(A);
        G[en].pb(B);
    }
    int nodes, nb_S, T; // asignar estos valores al armar el grafo G
                        // nodes = nodos en red de flujo. Hacer
G.clear(); G.resize(nodes);
    vi work, lvl;
    vi Q;
    bool bfs() {
        int qt = 0;
        Q[qt++] = nb_S;
        lvl.assign(nodes, -1);
        lvl[nb_S] = 0;
        for (int qh = 0; qh < qt; qh++) {
            int v = Q[qh];
            for (flowEdge &e : G[v]) {
                int u = e.to;
                if (e.cap <= e.f || lvl[u] != -1) continue;
                lvl[u] = lvl[v] + 1;
                Q[qt++] = u;
            }
        }
        return lvl[T] != -1;
    }
    int dfs(int v, int f) {
        if (v == T || f == 0) return f;
        for (int &i = work[v]; i < G[v].size(); i++) {
            flowEdge &e = G[v][i];
            int u = e.to;
            if (e.cap <= e.f || lvl[u] != lvl[v] + 1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if (df) {
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }
    int maxFlow() {
        int flow = 0;
        while (bfs()) {
            work.assign(nodes, 0);
            while (true) {
                int df = dfs(nb_S, INF);
                if (df == 0) break;
                flow += df;
            }
        }
        return flow;
    }
};
void ejemplo_max_flow() {
    ll n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinitz flow;
    flow.init(n, s, t);
    for (int i =0; i < m; i++) {
        ll a, b, c;
        cin >> a >> b >> c;
        flow.addEdge(a, b, c);
    }
    ll f = flow.maxFlow(); // max flow
    vector<tuple<ll,ll,ll>> edges; // edges used with flow
    for (int i = 0; i < n; i++) {
        for (auto edge : flow.G[i]) {
            if (edge.f > 0) {
                edges.pb({i, edge.to, edge.f});
            }
        }
    }
}
```

§5.4 Hopcroft Karp

§5.4.1 hopcroft-karp.cpp

05_Flujos_y_matching/Hopcroft_Karp

```
struct HopcroftKarp{
    const int INF = 1e9 + 7;
    HopcroftKarp(){}
    HopcroftKarp(int nLeft, int nRight){ init(nLeft, nRight); }
    void init(int nLeft, int nRight){
```

```
        L = nLeft, R = nRight;
        G.clear(), G.resize(L);
        pairU.assign(L, -1), pairV.assign(R, -1);
        dist.resize(L), Q.resize(L);
    }
    int L, R;
    vector<vi> G;
    vi pairU, pairV, dist, Q;
    void addEdge(int u, int v){ G[u].pb(v); }
    bool bfs(){
        int qh = 0, qt = 0;
        for(int u = 0; u < L; u++){
            if(pairU[u] == -1) dist[u] = 0, Q[qt++] = u;
            else dist[u] = INF;
        }
        bool ok = 0;
        while(qh < qt){
            int u = Q[qh++];
            for(int v : G[u]){
                int u2 = pairV[v];
                if(u2 == -1) ok = 1;
                else if(dist[u2] == INF) dist[u2] = dist[u] + 1,
Q[qt++] = u2;
            }
        }
        return ok;
    }
    bool dfs(int u){
        for(int v : G[u]){
            int u2 = pairV[v];
            if(u2 == -1 || (dist[u2] == dist[u] + 1 && dfs(u2))){
                pairU[u] = v, pairV[v] = u;
                return 1;
            }
        }
        dist[u] = INF;
        return 0;
    }
    int maxMatching(){
        pairU.assign(L, -1), pairV.assign(R, -1);
        int match = 0;
        while(bfs()) for(int u = 0; u < L; u++)
            if(pairU[u] == -1 && dfs(u)) match++;
        return match;
    }
};
```

§5.5 Húngaro

§5.5.1 hungarian.cpp

05_Flujos_y_matching/Hungaro

```
using ll = long long;
// Halla el maximo match en un grafo bipartito con pesos (min cost)
O(V ^ 3)
typedef ll T;
const T inf = 1e18;
struct hung {
    int n, m;
    vector<T> u, v; vector<int> p, way;
    vector<vector<T>> g;
    hung(int n, int m):
        n(n), m(m), g(n+1, vector<T>(m+1, inf-1)),
        u(n+1), v(m+1), p(m+1), way(m+1) {}
    void set(int u, int v, T w) { g[u+1][v+1] = w; }
    T assign() {
        for (int i = 1; i <= n; ++i) {
            int j0 = 0; p[0] = i;
            vector<T> minv(m+1, inf);
            vector<char> used(m+1, false);
            do {
                used[j0] = true;
                int i0 = p[j0], j1; T delta = inf;
                for (int j = 1; j <= m; ++j) if (!used[j]) {
                    T cur = g[i0][j] - u[i0] - v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta) delta = minv[j], j1 = j;
                }
                for (int j = 0; j <= m; ++j)
                    if (used[j]) u[p[j]] += delta, v[j] -= delta;
                    else minv[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do {
                int j1 = way[j0]; p[j0] = p[j1]; j0 = j1;
            } while (j0);
        }
        return -v[0];
    }
};
```

§5.6 Matching bipartito

§5.6.1 kuhn.cpp

05_Flujos_y_matching/Matching_bipartito

```
struct Kuhn{
    Kuhn(){}
    Kuhn(int nLeft, int nRight){ init(nLeft, nRight); }
    void init(int nLeft, int nRight){
        L = nLeft, R = nRight;
        G.clear(), G.resize(L);
        mt.assign(R, -1);
        used.assign(L, 0);
        cc = 1;
    }
    int L, R, cc;
    vector<vi> G;
    vi mt, used; // mt[to] = nodo izquierdo emparejado con to
    void addEdge(int u, int v){ G[u].pb(v); }
    bool dfs(int v){
        if(used[v] == cc) return 0;
        used[v] = cc;
        for(int to : G[v]) if(mt[to] == -1 || dfs(mt[to])) return
mt[to] = v, 1;
        return 0;
    }
    int maxMatching(){
        mt.assign(R, -1);
        int match = 0;
        for(int v = 0; v < L; v++){
            ++cc;
            if(dfs(v)) match++;
        }
        return match;
    }
};
```

§5.7 Matching general

§5.7.1 blossom-extra.cpp

05_Flujos_y_matching/Matching_general

```
// Halla el máximo match en un grafo general O(E * v ^2)
struct network {
    struct struct_edge {
        int v; struct_edge * n;
    };
    typedef struct_edge* edge;
    int n;
    struct_edge pool[MAXE]; // 2*n*n;
    edge top;
    vector<edge> adj;
    queue<int> q;
    vector<int> f, base, inq, inb, inp, match;
    vector<vector<int>> ed;
    network(int n) : n(n), match(n, -1), adj(n), top(pool), f(n),
base(n),
        inq(n), inb(n), inp(n), ed(n, vector<int>(n)) {}
    void add_edge(int u, int v) {
        if (ed[u][v]) return;
        ed[u][v] = 1;
        top->v = v, top->n = adj[u], adj[u] = top++;
        top->v = u, top->n = adj[v], adj[v] = top++;
    }
    int get_lca(int root, int u, int v) {
        fill(inp.begin(), inp.end(), 0);
        while (1) {
            inp[u = base[u]] = 1;
            if (u == root) break;
            u = f[match[u]];
        }
        while (1) {
            if (inp[v = base[v]]) return v;
            else v = f[match[v]];
        }
    }
    void mark(int lca, int u) {
        while (base[u] != lca) {
            int v = match[u];
            inb[base[u]] = 1;
            inb[base[v]] = 1;
            u = f[v];
            if (base[u] != lca) f[u] = v;
        }
    }
    void blossom_contraction(int s, int u, int v) {
        int lca = get_lca(s, u, v);
        fill(inb.begin(), inb.end(), 0);
        mark(lca, u); mark(lca, v);
        if (base[u] != lca) f[u] = v;
        if (base[v] != lca) f[v] = u;
        for (int u = 0; u < n; u++) {
            if (inb[base[u]]) {
                base[u] = lca;
                if (!inq[u]) {
                    inq[u] = 1;
                    q.push(u);
                }
            }
        }
    }
    int bfs(int s) {
        fill(inq.begin(), inq.end(), 0);
        fill(f.begin(), f.end(), -1);
        for (int i = 0; i < n; i++) base[i] = i;
        q = queue<int>();
        q.push(s);
        inq[s] = 1;
        while (q.size()) {
            int u = q.front(); q.pop();
            for (edge e = adj[u]; e; e = e->n) {
                int v = e->v;
                if (base[u] != base[v] && match[u] != v) {
                    if ((v == s) || (match[v] != -1 && f[match[v]] !=
-1)) {
                        blossom_contraction(s, u, v);
                    } else if (f[v] == -1) {
                        f[v] = u;
                        if (match[v] == -1) return v;
                        else if (!inq[match[v]]) {
                            inq[match[v]] = 1;
```

```
                        q.push(match[v]);
                    }
                }
            }
        }
        return -1;
    }
    int doit(int u) {
        if (u == -1) return 0;
        int v = f[u];
        doit(match[v]);
        match[v] = u; match[u] = v;
        return u != -1;
    }
    // (i < net.match[i]) => means match
    int maximum_matching() {
        int ans = 0;
        for (int u = 0; u < n; u++)
            ans += (match[u] == -1) && doit(bfs(u));
        return ans;
    }
};
```

§6 Matemáticas

§6.1 Álgebra lineal › Base XOR

§6.1.1 xor-basis.cpp

06_Matematicas/Algebra_lineal/Base_XOR

```
using ll = long long;
#define i23 long long
vector<i23> basis;
void add(i23 x) {
  for (int i = 0; i < (int)basis.size(); i++) {
    // reduce x using the current basis vectors
    x = min(x, x ^ basis[i]);
  }
  if (x != 0) { basis.push_back(x); }
}
// Other implement
// To count number of xor subsets
void test_case() {
    ll n;cin >> n;
    vector<ll> basis(32,0);
    ll cnt = 0;
    auto add = [&](ll x) {
        for (ll i = 31;i>=0;i--) {
            if ((x>>i)&1) {
                if (basis[i]==0) {
                    basis[i]=x;
                    cnt++;
                    return;
                }
                x ^= basis[i];
            }
        }
    };
    for (int i =0;i<n;i++) {
        ll x;
        cin >> x;
        add(x);
    }
    ll ans = 1ll << (cnt);
    cout << ans << "\n";
}
```

§6.2 Álgebra lineal › Matrices y Gauss

§6.2.1 gauss.cpp

06_Matematicas/Algebra_lineal/Matrices_y_Gauss

```
// resuelve Ax = b, dada la matriz a de n * (m + 1), n ecuaciones y m variables, siendo la ultima columna el vector b
// The function returns the number of solutions of the system (0,1,or ∞). if there's at least a solution, it's in ans
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a big number
int gauss (vector < vector<double> > a, vector<double> & ans) {
  int n = (int) a.size();
  int m = (int) a[0].size() - 1;
  vector<int> where (m, -1);
  for (int col=0, row=0; col<m && row<n; ++col) {
    int sel = row;
    for (int i=row; i<n; ++i)
      if (abs (a[i][col]) > abs (a[sel][col]))
        sel = i;
    if (abs (a[sel][col]) < EPS)
      continue;
    for (int i=col; i<=m; ++i)
      swap (a[sel][i], a[row][i]);
    where[col] = row;
    for (int i=0; i<n; ++i)
      if (i != row) {
        double c = a[i][col] / a[row][col];
        for (int j=col; j<=m; ++j)
          a[i][j] -= a[row][j] * c;
      }
    ++row;
  }
  ans.assign (m, 0);
  for (int i=0; i<m; ++i)
    if (where[i] != -1)
      ans[i] = a[where[i]][m] / a[where[i]][i];
  for (int i=0; i<n; ++i) {
    double sum = 0;
    for (int j=0; j<m; ++j)
      sum += ans[j] * a[i][j];
    if (abs (sum - a[i][m]) > EPS)
      return 0;
  }
  for (int i=0; i<m; ++i)
    if (where[i] == -1)
      return INF;
  return 1;
}
```

§6.2.2 gaussian-function.cpp

06_Matematicas/Algebra_lineal/Matrices_y_Gauss

```
// Requisito de selección: Gauss modular requiere divide/sub/mul/add; Gauss XOR requiere N. La comprobación final XOR original usa ans[i] para i<n y debe revisar consistencia de las filas reducidas.
// Algoritmo de eliminación Gauss-Jordan O(N ^ 3)
// Soluciona un sistema de ecuaciones de la forma:
// a11 x1 + a12 x2 + ... + a1m xm = b1
// a21 x1 + a22 x2 + ... + a2m xm = b2
// an x1 + an2 x2 + ... + anm xm = bn
//El vector a contiene los valores de la matriz,
//cada fila es una ecuación,
// la última columna contiene los valores b.
const int EPS = 1;
int gauss(vector<vector<int>>& a, vector<int> &ans) {
  int n = a.size(), m = a[0].size()-1;
  vector<int> where(m, -1);
  for (int col = 0, row = 0; col < m && row < n;++col){
    int sel = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < EPS) continue;
    swap(a[sel], a[row]);
    where[col] = row;
    for (int i = 0; i < n; ++i)
      if (i != row) {
        int c = divide(a[i][col], a[row][col]);
        for (int j = col; j <= m; ++j)
          a[i][j] = sub(a[i][j], mul(a[row][j], c));
      }
    ++row;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1)
      ans[i] = divide(a[where[i]][m], a[where[i]][i]);
  for (int i = 0; i < n; ++i) {
    int sum = 0;
    for (int j = 0; j < m; ++j)
        sum = add(sum, mul(ans[j], a[i][j]));
    if (sum != a[i][m]) return 0;
  }
  for (int i = 0; i < m; ++i)
    if (where[i] == -1) return -1;
  return 1;
}
// Gauss jordan para operaciones de xor
int gauss(
  vector<bitset<N>> &a,
  vector<bitset<N>> &b, int n, int m,
  vector<bitset<N>> &ans
) {
  vector<int> where(m, -1);
  for (int col = 0, row = 0; col < m && row < n;++col){
    for (int i = row; i < n; ++i) {
      if (a[i][col]) {
        swap(a[i], a[row]);
        swap(b[i], b[row]);
        break;
      }
    }
    if (!a[row][col])continue;
    where[col] = row;
    for (int i = 0; i < n; ++i)
      if (i != row && a[i][col]) {
        a[i] ^= a[row];
        b[i] ^= b[row];
      }
    ++row;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1) ans[i] = b[where[i]];
  for (int i = 0; i < n; ++i) {
    if (ans[i] == 0) return 0;
  }
  for (int i = 0; i < m; ++i)
    if (where[i] == -1)return -1; //inf solutions
  return 1;
}
```

§6.2.3 matrix-determinant.cpp

06_Matematicas/Algebra_lineal/Matrices_y_Gauss

```
const double EPS=1e-4;
double reduce(vector<vector<double> >& x){ // returns determinant
  int n=x.size(),m=x[0].size();
  int i=0,j=0;double r=1.;
  while(i<n&&j<m){
    int l=i;
    for (int k=(i+1), nb_fin_1=(n); k<nb_fin_1; ++k)if(abs(x[k]
```

```
[j])>abs(x[l][j]))l=k;
    if(abs(x[l][j])<EPS){j++;r=0.;continue;}
    if(l!=i){r=-r;swap(x[i],x[l]);}
    r*=x[i][j];
    for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
    for (int k=(0), nb_fin_3=(n); k<nb_fin_3; ++k){
      if(k==i)continue;
      for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][l];
    }
    i++;j++;
  }
  return r;
}
```

§6.2.4 matrix-fast-pow.cpp

06_Matematicas/Algebra_lineal/Matrices_y_Gauss

```
using ll = long long;
// kth term of linear recurrence
// of size m a_i = sum(a_(i-j)*p_j)
// f(x) = x^m - sum(x^(m-j)*p_j)
// g(x^k) = g(x^k MOD f)
typedef vector<vector<ll> > Matrix;
Matrix ones(int n) {
  Matrix r(n,vector<ll>(n));
  for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i)r[i][i]=1;
  return r;
}
Matrix operator*(Matrix &a, Matrix &b) {
  int n=sz(a),m=sz(b[0]),z=sz(a[0]);
  Matrix r(n,vector<ll>(m));
  for (int i=(0), nb_fin_3=(n); i<nb_fin_3; ++i)for (int j=(0),
nb_fin_5=(m); j<nb_fin_5; ++j)for (int k=(0), nb_fin_7=(z);
k<nb_fin_7; ++k)
    r[i][j]+=a[i][k]*b[k][j],r[i][j]%=MOD;
  return r;
}
Matrix be(Matrix b, ll e) {
  Matrix r=ones(sz(b));
  while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
  return r;
}
```

§6.2.5 matrix-multiplication-extra.cpp

...aticas/Algebra_lineal/Matrices_y_Gauss

```
using ll = long long;
// Estructura para realizar operaciones de multiplicacion y
exponenciacion modular sobre matrices.
const int mod = 1e9+7;
struct matrix {
    vector<vector<int>> v;
    int n, m;
    matrix(int n, int m, bool o = false) : n(n), m(m), v(n,
vector<int>(m)) {
        if (o) while (n--) v[n][n] = 1;
    }
    matrix operator * (const matrix &o) {
        matrix ans(n, o.m);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++) if (v[i][k])
                for (int j = 0; j < o.m; j++)
                    ans[i][j] = (1ll*v[i][k]*o.v[k][j] + ans[i][j]) %
mod;
        return ans;
    }
    vector<int>& operator [] (int i) { return v[i]; }
};
matrix pow(matrix b, ll e) {
    matrix ans(b.n, b.m, true);
    while (e) {
        if (e&1) ans = ans*b;
        b = b*b;
        e /= 2;
    }
    return ans;
}
```

§6.3 Aritmética modular › CRT

§6.3.1 chinese-remainder-theorem-extra.cpp

...aticas/Aritmetica_modular/CRT

```
#include "../../Teoria_de_numeros/Euclides/extended-euclides-
extra.cpp"
using pii = pair<int, int>;
// Encuentra un x tal que para cada i : x es congruente con A_i mod
M_i
// Devuelve {x, lcm}, donde x es la solucion con modulo lcm (lcm =
LCM(M_0, M_1, ...)). Dado un k : x + k*lcm es solucion tambien.
// Si la solucion no existe o la entrada no es valida devuelve {-1,
-1}
// Agregar Extended Euclides.
pii crt(vector<int> A, vector<int> M) {
    int n = A.size(), ans = A[0], lcm = M[0];
    for (int i = 1; i < n; i++) {
        int d = euclid(lcm, M[i]);
        if ((A[i] - ans) % d) return {-1, -1};
        int mod = lcm / d * M[i];
        ans = (ans + x * (A[i] - ans) / d % (M[i] / d) * lcm) % mod;
        if (ans < 0) ans += mod;
        lcm = mod;
    }
    return {ans, lcm};
}
```

§6.4 Aritmética modular › Logaritmo discreto

§6.4.1 discrete-log.cpp

06_Matematicas/Aritmetica_modular/Logaritmo_discreto

```
using ll = long long;
// Devuelve un entero x tal que a^x = b (mod m) or -1 si no existe
tal x.
int expmod(int b, int e, int m) { // Always check if change int to ll
!!!
    int ans = 1;
    while (e) {
        if (e&1) ans = (1ll*ans*b) % m;
        b = (1ll*b*b) % m;
        e /= 2;
    }
    return ans;
}
ll discrete_log(ll a, ll b, ll m) {
    a %= m, b %= m;
    if (b == 1) return 0;
    int cnt = 0;
    ll tmp = 1;
    for (int g = __gcd(a, m); g != 1; g = __gcd(a, m)) {
        if (b%g) return -1;
        m /= g, b /= g;
        tmp = tmp*a / g % m;
        ++cnt;
        if (b == tmp) return cnt;
    }
    map<ll, int> w;
    int s = ceil(sqrt(m));
    ll base = b;
    for (int i = 0; i < s; i++) {
        w[base] = i;
        base = base*a % m;
    }
    base = expmod(a, s, m);
    ll key = tmp;
    for (int i = 1; i <= s+1; i++) {
        key = key*base % m;
        if (w.count(key)) return i*s - w[key] + cnt;
    }
    return -1;
}
```

§6.5 Aritmética modular › Operaciones

§6.5.1 big-expontent-modular-exponentiation.cpp

...modular/Operaciones

```
using ll = long long;
// Calc a^b^c % MOD
// MOD is prime
ll pou(ll a, ll b, ll m) {
    ll ans = 1;
    while (b) {
        if (b&1) ans *= a, ans%=m;
        a*=a;
        a%=m;
        b/=2;
    }
    return ans;
}
void test_case() {
    ll a, b, c;
    cin >> a >> b >> c;
    // fermat theorem
    // a^(p-1) = 1 (mod p)
    b = pou(b, c, MOD - 1);
    a = pou(a, b, MOD);
    cout << a << "\n";
}
```

§6.5.2 fraction-modular.cpp

06_Matematicas/Aritmetica_modular/Operaciones

```
using ll = long long;
const ll MOD = 998244353;
struct frac : public pair<ll, ll> {
    using pair<ll, ll>::pair;
    frac simplify() {
        if (first == 0) {
            return frac(0, 1);
        }
        ll gcd_val = __gcd(first, second);
        return frac(first / gcd_val, second / gcd_val);
    }
    frac operator*(const frac &other) {
        // a * invmod(b) = a / b
        // a * invmod(b) * a2 * invmod(b2) = (a * a2) / (b * b2)
        return frac((first * other.first) % MOD , (second *
other.second) % MOD ).simplify();
    }
    frac operator+(const frac &other) {
        // opertaor + with module
        // a * invmod(b) + a2 * invmod(b2) = (a * b2 + a2 * b) / (b *
b2)
        ll up = (first * other.second) % MOD + (other.first * second)
% MOD;
        ll down = (second * other.second) % MOD;
        // TODO: check if simplify should be here
        return frac(up, down).simplify();
    }
};
// Expected Value is the sum of all the possible values multiplied by
their probability
// Geometrica is reversed.
```

§6.5.3 modular-exponentiation-extra.cpp

...as/Aritmetica_modular/Operaciones

```
// * Calcula (b^e) % m (e puede ser ll). b debe estar ya con modulo
m.
```

```
// Si m es ll se debe cambiar todo a ll, agregar Modular
Multiplication y calcular las multiplicaciones con mulmod().
// O(log(e))
int expmod(int b, int e, int m) {
    int ans = 1;
    while (e) {
        if (e&1) ans = (1ll*ans*b) % m;
        b = (1ll*b*b) % m;
        e /= 2;
    }
    return ans;
}
```

§6.5.4 modular-inverse-extra.cpp ..._Matematicas/Aritmetica_modular/Operaciones

```
#include "../../Teoria_de_numeros/Euclides/extended-euclides-
extra.cpp"
#include "modular-exponentiation-extra.cpp"
using ll = long long;
// Inverso b de a mod m: a*b % m = 1; existe si gcd(a, m) = 1. (n/a)
% m = n*b % m.
// Con Euclides extendido, O(log(max(a, m))); -1 si no existe.
int invmod(int a, int m) {
    int d = euclid(a, m);
    if (d > 1) return -1;
    return (x % m + m) % m;
}
// m primo: Fermat, a^(m-2), O(log m).
int invmod_primo(int a, int m) {
    return expmod(a, m-2, m);
}
// Inversos de 1..MX módulo m (primo) en O(MX).
const int MX = 1e6;
int inv[MX+1];
void invmod(int m) {
    inv[1] = 1;
    for (int i = 2; i <= MX; i++)
        inv[i] = m - ll(m/i) * inv[m%i] % m;
}
```

§6.5.5 modular-multiplication-extra.cpp ...icas/Aritmetica_modular/Operaciones

```
// Requisito de selección: La fórmula original usa overflow signed en
a*b; requiere reemplazo por __int128 para aritmética definida en todo
el rango positivo de int64.
using ll = long long;
// * Calcula (a*b) % m sin overflow cuando m es ll.
// O(1)
ll mulmod(ll a, ll b, ll m) {
    ll r = a*b-(ll)((long double)a*b/m+.5)*m;
    return r < 0 ? r+m : r;
}
```

§6.6 Aritmética modular › Raíces

§6.6.1 primitive-root-extra.cpp 06_Matematicas/Aritmetica_modular/Raices

```
#include "../Operaciones/modular-exponentiation-extra.cpp"
// g es una raíz primitiva módulo n si y solo si para cualquier
entero a tal que gcd(a, n) = 1, existe un entero k tal que: g^k = a
mod n.
// La raíz primitiva módulo n existe si y solo si: n = 1, 2, 4, o n =
p^k, o n = 2 * p^k, donde p es un número primo impar.
// Agregar Modular Exponentiation.
// O(ans * log(phi(p)) * log(p))
int primitive_root(int p) { // Función para cuando p es primo
    vector<int> fact;
    int phi = p-1, n = phi; // Si p no es primo, calcular Euler
totient de p.
    for (int i = 2; i*i <= n; ++i)
        if (n%i == 0) {
            fact.push_back(i);
            while (n%i == 0) n /= i;
        }
    if (n > 1) fact.push_back(n);
    for (int ans = 2; ans <= p; ++ans) {
        bool ok = true;
        for (int i = 0; i < fact.size() && ok; ++i)
            ok &= expmod(ans, phi / fact[i], p) != 1;
        if (ok) return ans;
    }
    return -1;
}
```

§6.6.2 tonelly-shanks.cpp 06_Matematicas/Aritmetica_modular/Raices

```
using ll = long long;
ll legendre(ll a, ll p){
  if(a%p==0)return 0; if(p==2)return 1;
  return fpow(a,(p-1)/2,p);
}
ll tonelli_shanks(ll n, ll p){    // sqrt(n) mod p (p must be a prime)
  assert(legendre(n,p)==1); if(p==2)return 1;
  ll s=__builtin_ctzll(p-1), q=(p-1LL)>>s, z=rnd(1,p-1);
  if(s==1)return fpow(n,(p+1)/4LL,p);
  while(legendre(z,p)!=p-1)z=rnd(1,p-1);
  ll c=fpow(z,q,p), r=fpow(n,(q+1)/2,p), t=fpow(n,q,p), m=s;
  while(t!=1){
    ll i=1, ts=(t*t)%p;
    while(ts!=1)i++,ts=(ts*ts)%p;
    ll b=c;
    for (int _=(0), nb_fin_1=(m-i-1); _<nb_fin_1; ++_)b=(b*b)%p;
    r=r*b%p;c=b*b%p;t=t*c%p;m=i;
  }
  return r;
}
```

§6.7 Combinatoria

§6.7.1 catalan-convolution.cpp 06_Matematicas/Combinatoria

```
using ll = long long;
using vl = vector<long long>;
/*
Return Catalan Convolution.
Convolution for k=3
((( A ) B ) C ) D
Where A + B + C + D = N, for N + 1
*/
const int MOD = 1e9 + 7;
ll mul(ll x, ll y) { return (x*y)%MOD; }
ll pot(ll x, ll y) {
    if(y==0) return 1;
    ll ans = pot(x,y/2);
    ans = mul(ans,ans);
    if (y&1)ans=mul(ans,x);
    return ans;
}
ll inv(ll x) { return pot(x, MOD-2); }
// mxN it the double of the max input N, plus max K
const int mxN = 2e6 + 1e6 + 10;
vl fact(mxN,1);
ll cnk(ll n, ll k) {
    if (k < 0 || n < k) return 0;
    ll nOverK = mul(fact[n],inv(fact[k]));
    return mul(nOverK,inv(fact[n-k]));
}
void init() {
    for (int i =1;i<=mxN;i++) {
        fact[i] = mul(fact[i-1],i);
    }
}
// for parethesis example
// number of n+k pairs having k open parethesis at beginning
// (cnk(2n+k,n)*(k+1))/(n+k+1)
ll catalanCov(ll n, ll k) {
    ll up = mul(cnk(2*n+k,n),(k+1)%MOD);
    ll down = (n+k+1)%MOD;
    return mul(up,inv(down));
}
/*
6
(()
ans: 2
*/
// size, and prefix
ll countParenthesisWithPrefix(ll n, string &p) {
    if (n&1) return 0;
    ll k = 0;
    for (auto c : p) {
        if (c=='(') k++;
        else k--;
        if (k<0) return 0;
    }
    n=(n-(ll)p.size()-k)/2;
    return catalanCov(n,k);
}
```

§6.7.2 catalan.cpp 06_Matematicas/Combinatoria

```
using ll = long long;
using vl = vector<long long>;
/*Catalan, counts the number of ways of:
( A ) B, where |A|+|B| = N, for N+1
*/
const int MOD = 1e9 + 7;
ll mul(ll x, ll y) { return (x*y)%MOD; }
ll pot(ll x, ll y) {
    if(y==0) return 1;
    ll ans = pot(x,y/2);
    ans = mul(ans,ans);
    if (y&1)ans=mul(ans,x);
    return ans;
}
ll inv(ll x) { return pot(x, MOD-2); }
// mxN it the double of the max input 'n'
const int mxN = 2e6 + 10;
vl fact(mxN,1);
void init() {
    for (int i =1;i<=mxN;i++) {
        fact[i] = mul(fact[i-1],i);
    }
}
ll catalan(ll n) {
    if (n<0) return 0;
    ll up = fact[2*n];
    ll down = mul(fact[n],fact[n+1]);
    return mul(up,inv(down));
}
```

§6.7.3 combinatorics.cpp 06_Matematicas/Combinatoria

```
using ll = long long;
// if k == 0 then 1
// if k negative or no enough choices then 0
// O(min(n, n-k)) lineal
ll nck(ll n, ll k) {
    if (k < 0 || n < k) return 0;
    k = min(k, n-k);
    ll ans = 1;
    for (int i = 1; i <= k; i++) {
        ans = ans * (n-i+1) / i;
    }
```

```
    return ans;
}
```

§6.7.4 modular-combinatorics.cpp

06_Matematicas/Combinatoria

```
using ll = long long;
using vl = vector<long long>;
/* Combinatorics with a Prime Module
nCk(n,k) = How many ways you can choose 'k' items from an array of
'n' items with a given prime module.
Use:
 - NCK nck(max N, prime module);
 - nck.nCk(n,k)
*/
struct NCK {
    ll MAX_N;
    ll MOD;
    vl fact;
    explicit NCK(ll maxN, ll mod) : MAX_N(maxN), MOD(mod) {
        fact.resize(MAX_N + 1, 1);
        fact[0] = 1;
        for (int i=(1), nb_fin_1=(MAX_N); i<=nb_fin_1; ++i) {
            fact[i] = fact[i - 1] * (i % MOD);
            fact[i] %= MOD;
        }
    }
    ll inv(ll a){
        return powmod(a, MOD-2); // MOD is prime, otherwise use
powmod(a, eulerPhi(mod) - 1)
    }
    ll powmod(ll a, ll b){
        if (b == 0) return 1;
        ll mid = powmod(a, b / 2);
        ll ans = (mid * mid) % MOD;
        if (b & 1) {
            ans *= a;
            ans %= MOD;
        }
        return ans;
    }
    ll nCk(ll n, ll k){ // TODO: add if's when case is zero
        ll nOverK = (fact[n] * inv(fact[k])) % MOD;
        return (nOverK * inv(fact[n-k])) % MOD;
    }
};
```

§6.7.5 stirling-y-bell.cpp

06_Matematicas/Combinatoria

```
using ll = long long;
// stir[n][k] cantidad de formas de paritcionar un conjunto de n
elementos en k conjuntos
// bell[n] cantidad de formas de particionar un conjunto
ll stir[tam][tam];
ll bell[tam];
void stirBell() {
  for (int i=(1), nb_fin_1=(tam); i<nb_fin_1; ++i) {
  stir[i][1] = 1;
  for (int j=(2), nb_fin_3=(1010); j<nb_fin_3; ++j)
    stir[i][j] = (j * stir[i - 1][j] % MOD + stir[i - 1][j - 1]) %
MOD;
  }
  for (int i=(1), nb_fin_5=(tam); i<nb_fin_5; ++i)
  for (int j=(1), nb_fin_7=(i + 1); j<nb_fin_7; ++j)
    bell[i] = (bell[i] + stir[i][j]) % MOD;
}
```

§6.7.6 subfactorial.cpp

06_Matematicas/Combinatoria

```
using ll = long long;
using vl = vector<long long>;
/* Denote as !n or derangement numbers
  Count the number of permutations where no element is in
  the originial position, formally p[i] != i
  it can be seen as f(n) = n!-sum_i=1_to_n { cnk(n,i)*f(n-i) }
  f(0)=1, f(1) = 1
  1 0 1  2 9 44 265 1,854 14,833 133,496
  n! = sumi=0,i<=n,{cnk(n,i)!i}
  d[i] = (d[i-1]+d[i-2])*(i-1)
*/
const int mxN = 2e6 + 10; // max number
ll add(ll x, ll y) { return (x+y)%MOD; }
ll mul(ll x, ll y) { return (x*y)%MOD; }
vl subFact(mxN);
void init() {
    subFact[0] = 1;
    subFact[1] = 0;
    for (int i = 2;i<mxN;i++) {
        subFact[i] = mul(add(subFact[i-1],subFact[i-2]),i-1);
    }
}
```

§6.8 Fibonacci y Pisano

§6.8.1 fibonacci-mod-m-extra.cpp

06_Matematicas/Fibonacci_y_Pisano

```
using ll = long long;
// Calcula fibonacci(n) % m.
// O(log(n))
int fibmod(ll n, int m) {
    int a = 0, b = 1, c;
    for (int i = 63-__builtin_clzll(n); i >= 0; i--) {
        c = a;
        a = (1ll*c*(2ll*b-c+m)) % m;
        b = (1ll*c*c + 1ll*b*b) % m;
        if ((n>>i) & 1) {
            c = (a+b) % m;
            a = b; b = c;
        }
    }
    return a;
}
```

§6.8.2 pisano-period-extra.cpp

06_Matematicas/Fibonacci_y_Pisano

```
using ll = long long;
// Calcula el Periodo de Pisano de m, que es el periodo con el cual
se repite la Sucesion de Fibonacci modulo m.
// Si m es primo el algoritmo funciona (considerable) para m < 10^6.
Aregar Modular Exponentiation (sin el modulo) y Lowest Common
Multiple (para ll).
ll period(ll m) {
    ll a = 0, b = 1, c, pp = 0;
    do {
        c = (a+b) % m;
        a = b; b = c; pp++;
    } while (a != 0 || b != 1);
    return pp;
}
ll pisanoPrime(ll p, int e) {
    return expmod(p, e-1) * period(p);
}
ll pisanoPeriod(ll m) {
    ll pp = 1;
    for (ll p = 2; p <= m/p; p++) {
        if (m%p == 0) {
            int e = 0;
            while (m%p == 0) e++, m /= p;
            pp = lcm(pp, pisanoPrime(p, e));
        }
    }
    if (m > 1) pp = lcm(pp, period(m));
    return pp;
}
```

§6.9 Josephus

§6.9.1 josephus.cpp

06_Matematicas/Josephus

```
using ll = long long;
// UFMG
/* Josephus Problem - It returns the position to be, in order to not
die. O(n)*/
/* With k=2, for instance, the game begins with 2 being killed and
then n+2, n+4, ... */
ll josephus(ll n, ll k) {
  if(n==1) return 1;
  else return (josephus(n-1, k)+k-1)%n+1;
}
```

§6.10 Métodos numéricos

§6.10.1 fraction-extra.cpp

06_Matematicas/Metodos_numericos

```
using ll = long long;
// Estructura para trabajar con fracciones como un tipo de dato
typedef ll T;
struct Fraction {
    T num, den;
    Fraction() : num(0), den(1) {}
    Fraction(T n) : num(n), den(1) {}
    Fraction(T n, T d) : num(n), den(d) { reduce(); }
    void reduce() {
        assert(den != 0);
        T gcd = __gcd(num, den);
        num /= gcd, den /= gcd;
        if (den < 0) num = -num, den = -den;
    }
    Fraction fractional_part() const { // x - floor(x)
        Fraction fp = Fraction(num % den, den);
        if (fp < Fraction(0)) fp += Fraction(1);
        return fp;
    }
    T compare(Fraction f) const {
        return num * f.den - den * f.num;
    }
    Fraction operator + (const Fraction& f) {
        return Fraction(num * f.den + den * f.num, den * f.den);
    }
    Fraction operator - (const Fraction& f) {
        return Fraction(num * f.den - den * f.num, den * f.den);
    }
    Fraction operator * (const Fraction& f) {
        Fraction a = Fraction(num, f.den);
        Fraction b = Fraction(f.num, den);
        return Fraction(a.num * b.num, a.den * b.den);
    }
    Fraction operator / (const Fraction& f) {
        return *this * Fraction(f.den, f.num);
    }
    Fraction operator += (const Fraction& f) { return *this = *this +
f; }
    Fraction operator -= (const Fraction& f) { return *this = *this -
f; }
    Fraction operator *= (const Fraction& f) { return *this = *this *
f; }
    Fraction operator /= (const Fraction& f) { return *this = *this /
f; }
    bool operator == (const Fraction& f) const { return compare(f) ==
0; }
    bool operator != (const Fraction& f) const { return compare(f) !=
0; }
    bool operator >= (const Fraction& f) const { return compare(f) >=
0; }
    bool operator <= (const Fraction& f) const { return compare(f) <=
```

```
0; }
    bool operator > (const Fraction& f) const { return compare(f) >
0; }
    bool operator < (const Fraction& f) const { return compare(f) <
0; }
};
Fraction operator - (const Fraction& f) { return Fraction(-f.num,
f.den); }
ostream& operator << (ostream& os, const Fraction& f) {
    return os << "(" << (ll)f.num << "/" << (ll)f.den << ")";
}
```

§6.10.2 simpson.cpp

06_Matematicas/Metodos_numericos

```
double simpson(double f(double), double a, double b) {
    int n = 100000;
    double s = f(a) + f(b);
    double h = (b - a) / n;
    for (int i=(1), nb_fin_1=(n); i<nb_fin_1; ++i)
        s += ((i & 1) ? 4  : 2) * f(a + h * i);
    return s * (h / 3);
}
```

§6.10.3 stablesum.cpp

06_Matematicas/Metodos_numericos

```
struct stableSum {
  int cnt = 0;
  vector<double> v, prefs{0};
  void operator += (double a) {
    assert(a >= 0);
    int s = ++cnt;
    while(s % 2 == 0) {
      a += v.back();
      v.pop_back(), pref.pop_back();
      s /= 2;
    }
    v.pb(a);
    pref.pb(pref.back() + a);
  }
  double val() {return pref.back();}
};
```

§6.11 Polinomios y transformadas

§6.11.1 bairstow-vasito.cpp

06_Matematicas/Polinomios_y_transformadas

```
double pget(poly<>& p, int k){return k<p.c.size()?p[k]:0;}
poly<> bairstow(poly<> p){ // returns polynomial of degree 2 that
    int n=p.c.size()-1;    // divides p
    assert(n>=3&&abs(p.c.back())>EPS);
    double u=p[n-1]/p[n],v=p[n-2]/p[n];
    for (int _=(0), nb_fin_1=(ITER); _<nb_fin_1; ++_){
        auto w=polydiv(p,{v,u,1});
        poly<> q=w.first,r0=w.second;
        poly<> r1=polydiv(q,{v,u,1}).second;
        double c=pget(r0,1),d=pget(r0,0),g=pget(r1,1),h=pget(r1,0);
        double det=1/(v*g*g+h*(h-u*g)),uu=u;
        u-=det*(-h*c+g*d);v-=det*(-g*v*c+(g*uu-h)*d);
    }
    return {v,u,1};
}
void addr(vector<double>& r, poly<>& p){
    assert(p.c.size()<=3);
    if(p.c.size()<=1)return;
    if(p.c.size()==2)r.pb(-p[0]/p[1]);
    if(p.c.size()==3){
        double a=p[2],b=p[1],c=p[0];
        double d=b*b-4*a*c;
        if(d<-0.1)return; // huge epsilon because of bad precision
        d=d>0?sqrt(d):0;r.pb((-b-d)/2/a);r.pb((-b+d)/2/a);
    }
}
vector<double> roots(poly<> p){
    while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();
    for (int i=(0), nb_fin_3=(p.c.size()); i<nb_fin_3;
++i)p[i]/=p.c.back();
    vector<double> r;int n;
    while((n=p.c.size()-1)>=3){
        poly<> q=bairstow(p);addr(r,q);
        p=polydiv(p,q).first;
        while(p.c.size()>n-1)p.c.pop_back();
    }
    addr(r,p);
    return r;
}
```

§6.11.2 fast-hadamard-transform.cpp

...atematicas/Polinomios_y_transformadas

```
using ll = long long;
using vl = vector<long long>;
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
const int MAXN=1<<18;
ll c1[MAXN+9],c2[MAXN+9];//MAXN must be power of 2!
void fht(ll* p, int n, bool inv){
    for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)for(int
j=static_cast<long long>(0);j<static_cast<long long>(l);++j){
        ll u=p[i+j],v=p[i+l+j];
        // if(!inv)p[i+j]=u+v,p[i+L+j]=u-v; // XOR
        // else p[i+j]=(u+v)/2,p[i+L+j]=(u-v)/2;
        //if(!inv)p[i+j]=v,p[i+L+j]=u+v; // AND
        //else p[i+j]=-u+v,p[i+L+j]=u;
        if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
        else p[i+j]=v,p[i+l+j]=u-v;
    }
}
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
    int n=1<<(32-__builtin_clz(max(((int)(p1).size()),((int)
(p2).size()))-1));
    for(int i=static_cast<long long>(0);i<static_cast<long long>
(n);++i)c1[i]=0,c2[i]=0;
    for(int i=static_cast<long long>(0);i<static_cast<long long>
(((int)(p1).size()));++i)c1[i]=p1[i];
    for(int i=static_cast<long long>(0);i<static_cast<long long>
(((int)(p2).size()));++i)c2[i]=p2[i];
    fht(c1,n,false);fht(c2,n,false);
    for(int i=static_cast<long long>(0);i<static_cast<long long>
(n);++i)c1[i]*=c2[i];
    fht(c1,n,true);
    return vector<ll>(c1,c1+n);
}
// maxime the OR of a pair of given nums and count
// how many pairs can get that maximum OR
// tested: https://csacademy.com/contest/archive/task/maxor
void test_case() {
    ll n; cin >> n;
    vl a(MAXN),b(MAXN);
    for (int i =0;i<n;i++) {
        ll x;
        cin >> x;
        a[x]++;
        b[x]++;
    }
    vl c = multiply(a,b);
    pair<ll,ll> best = {0, c[0]};
    for (int i = 0;i<MAXN;i++) {
        if (c[i]) best = {i,(c[i]-a[i])/2};
    }
    cout <<best.first << " " << best.second << endl;
}
```

§6.11.3 fft-operations-vasito.cpp

06_Matematicas/Polinomios_y_transformadas

```
// Requisito de selección: Necesita proveedor con API
poly/multiply/fft del mismo diseño; no basta pegar la FFT de Alex por
nombre.
// Polynomial division: O(n*log(n))
// Multi-point polynomial evaluation for arbitrarily large points:
O(n+k*log^2(k))
// Multi-point polynomial evaluation for all points in [0, MOD):
O((n+MOD)*log(n+MOD))
// Polynomial interpolation: O(n*log^2(n))
// Inverse: O(n*log(n))
// Log: O(n*log(n))
// Exp: O(n*log(n))
//Works with NTT. For FFT, just replace addmod,submod,mulmod,inv
poly add(const poly &a, const poly &b){
    int n=static_cast<long long>((a).size()),m=static_cast<long long>
((b).size());
    poly ans(max(n,m));
    for (int i=(0), nb_fin_3=(max(n,m)); i<nb_fin_3; ++i)
ans[i]=addmod(i<static_cast<long long>((a).size())?a[i]:0,
i<static_cast<long long>((b).size())?b[i]:0);
    while(static_cast<long long>
((ans).size())>1&&!ans.back())ans.pop_back();
    return ans;
}
// derivative of p
poly derivative(const poly &p){
    poly ans(max(1, static_cast<long long>((p).size())-1));
    for (int i=(1), nb_fin_9=(static_cast<long long>((p).size())));
i<nb_fin_9; ++i) ans[i-1]=mulmod(p[i],i);
    return ans;
}
// integral of p
poly integrate(const poly &p){
    int n=static_cast<long long>((p).size());
    poly ans(n+1);
    if(!n) return ans;
    poly pre(n+1); pre[0]=1;
    for (int i=(1), nb_fin_12=(n+1); i<nb_fin_12; ++i)
pre[i]=mulmod(pre[i-1],i);
    tf iv=inv(pre[n]);
    for(int i=n;i>=1;i--){
        ans[i]=mulmod(p[i-1], mulmod(iv,pre[i-1]));
        iv=mulmod(iv,i);
    }
    return ans;
}
// p % (x^n)
poly takemod(const poly &p, int n){
    poly res=p;
    res.resize(min(static_cast<long long>((res).size()),n));
    while(static_cast<long long>((res).size())>1&&res.back()==0)
res.pop_back();
    return res;
}
// first d terms of 1/p
poly invert(const poly &p, int d){
    assert(p[0]);
    poly res={inv(p[0])};
    int sz=1;
    while(sz<d){
        sz*=2;
        poly pre(p.begin(), p.begin()+min(static_cast<long long>
((p).size()),sz));
        poly cur=multiply(res,pre);
        for (int i=(0), nb_fin_17=(static_cast<long long>
((cur).size())); i<nb_fin_17; ++i) cur[i]=submod(0,cur[i]);
        cur[0]=addmod(cur[0],2);
```

```
        cur=takemod(cur,sz);
        res=multiply(res,cur);
        res=takemod(res,sz);
    }
    res.resize(d);
    return res;
}
// first d terms of log(p)
poly log(const poly &p, int d){
    assert(p[0]==1);
    poly cur=takemod(p,d);
    poly a=invert(cur,d), b=derivative(cur);
    auto res=multiply(a,b);
    res=takemod(res,d-1);
    res=integrate(res);
    return res;
}
// first d terms of exp(p)
poly exp(const poly &p, int d){
    assert(!p[0]);
    poly res={1};
    int sz=1;
    while(sz<d){
        sz*=2;
        poly lg=log(res, sz), cur(sz);
        for (int i=(0), nb_fin_19=(sz); i<nb_fin_19; ++i)
cur[i]=submod(i<static_cast<long long>((p).size())?p[i]:0,
i<static_cast<long long>((lg).size())?lg[i]:0);
        cur[0]=addmod(cur[0],1);
        res=multiply(res,cur);
        res=takemod(res, sz);
    }
    res.resize(d);
    return res;
}
pair<poly,poly> divslow(const poly &a, const poly &b){
    poly q,r=a;
    tf ib=inv(b.back());
    while(static_cast<long long>((r).size())>=static_cast<long long>
((b).size())){
        q.pb(mulmod(r.back(),ib));
        if(q.back()) for (int i=(0), nb_fin_25=(static_cast<long
long>((b).size())); i<nb_fin_25; ++i){
            r[static_cast<long long>((r).size())-i-
1]=submod(r[static_cast<long long>((r).size())-i-
1],mulmod(q.back(),b[static_cast<long long>((b).size())-i-1]));
        }
        r.pop_back();
    }
    reverse(all(q));
    return {q,r};
}
pair<poly,poly> divide(const poly &a, const poly &b){  //returns
{quotient,remainder}
    int m=static_cast<long long>((a).size()),n=static_cast<long long>
((b).size()),MAGIC=128;
    if(m<n) return {{0},a};
    if(min(m-n,n)<MAGIC)return divslow(a,b);
    poly ap=a; reverse(all(ap));
    poly bp=b; reverse(all(bp));
    bp=invert(bp,m-n+1);
    poly q=multiply(ap,bp);
    q.resize(m-n+1,0);
    reverse(all(q));
    poly bq=multiply(b,q);
    for (int i=(0), nb_fin_36=(static_cast<long long>((bq).size()));
i<nb_fin_36; ++i) bq[i]=submod(0,bq[i]);
    poly r=add(a,bq);
    return {q,r};
}
struct EvalTree {
    int k;
    vector<poly> tree;
    EvalTree(const vector<tf> &x): k(static_cast<long long>
((x).size())), tree(2*max(static_cast<long long>((x).size()),1)) {
        for (int i=(k), nb_fin_40=(2*k); i<nb_fin_40; ++i) tree[i]=
{submod(0,x[i-k]),1};
        for(int i=k-1;i;i--) tree[i]=multiply(tree[2*i],tree[2*i+1]);
    }
};
poly mulT(const poly &M, const poly &a, int n){
    poly c=multiply(M,a);
    int m=static_cast<long long>((M).size());
    poly res(n);
    for (int j=(0), nb_fin_43=(n); j<nb_fin_43; ++j) res[j]=(j+m-
1<static_cast<long long>((c).size()))?c[j+m-1]:0;
    return res;
}
void downsweep(const EvalTree &et, int v, poly q, vector<tf> &out){
    if(v>=et.k){ out[v-et.k]=q[0]; return; }
    poly ql=mulT(et.tree[2*v+1],q,static_cast<long long>
((et.tree[2*v]).size())-1);
    poly qr=mulT(et.tree[2*v],  q,static_cast<long long>
((et.tree[2*v+1]).size())-1);
    poly().swap(q);
    downsweep(et,2*v,move(ql),out);
    downsweep(et,2*v+1,move(qr),out);
}
vector<tf> evaluate(const poly &a, const EvalTree &et){
    int k=et.k;
    if(!k) return {};
    poly f=static_cast<long long>((a).size())>=static_cast<long long>
((et.tree[1]).size())?divide(a,et.tree[1]).second:a;
    poly rm=et.tree[1]; reverse(all(rm));
    poly h=invert(rm,static_cast<long long>((f).size()));
    poly().swap(rm);
    reverse(all(h));
    poly q1=mulT(h,f,static_cast<long long>((et.tree[1]).size())-1);
    poly().swap(h); poly().swap(f);
    vector<tf> out(k);
    downsweep(et,1,move(q1),out);
    return out;
}
// Multi-point polynomial evaluation for arbitrarily large points
vector<tf> evaluate(const poly &a, const vector<tf> &x){
    if(x.empty()) return {};
    EvalTree et(x);
    return evaluate(a,et);
}
poly upsweep(EvalTree &et, int v, const vector<tf> &c){
    if(v>=et.k) return {c[v-et.k]};
    poly L=upsweep(et,2*v,c), R=upsweep(et,2*v+1,c);
    poly p1=multiply(et.tree[2*v],R); poly().swap(R);
    poly p2=multiply(et.tree[2*v+1],L); poly().swap(L);
    poly().swap(et.tree[2*v]); poly().swap(et.tree[2*v+1]);
    poly res=add(p1,p2);
    return res;
}
poly interpolate(const vector<tf> &x, const vector<tf> &y){
    int k=static_cast<long long>((x).size());
    if(!k) return {0};
    EvalTree et(x);
    vector<tf> d=evaluate(derivative(et.tree[1]),et);
    vector<tf> pre(k+1); pre[0]=1;
    for (int i=(0), nb_fin_55=(k); i<nb_fin_55; ++i)
pre[i+1]=mulmod(pre[i],d[i]);
    tf iv=inv(pre[k]);
    vector<tf> c(k);
    for(int i=k-1;i>=0;i--){
        c[i]=mulmod(y[i],mulmod(iv,pre[i]));
        iv=mulmod(iv,d[i]);
    }
    return upsweep(et,1,c);
}
// Given any number g, perform multi-point polynomial evaluation
// for all points of the form g^i, for each i in [0, k)
// in O((n+k) * log(n+k))
vector<int> chirpTransform(const poly &p, int g, int k){
    int inv_g=inv(g), n=static_cast<long long>((p).size())-1,
sz=min(k,MOD-1);
    poly ap(n+1), bp(n+sz);
    for (int i=(0), nb_fin_58=(n+sz); i<nb_fin_58; ++i){
        int exp=1ll*i*(i-1)/2%(MOD-1);
        if(i<=n) ap[n-i]=mulmod(p[i],fpow(inv_g, exp));
        bp[i]=fpow(g, exp);
    }
    poly cp=multiply(ap, bp);
    vector<int> ans(k);
    for (int i=(0), nb_fin_60=(sz); i<nb_fin_60; ++i){
        int exp=1ll*i*(i-1)/2%(MOD-1), val=0;
        if(n+i<static_cast<long long>((cp).size())) val=cp[n+i];
        val=mulmod(val, fpow(inv_g,exp));
        ans[i]=val;
    }
    for (int i=(sz), nb_fin_63=(k); i<nb_fin_63; ++i) ans[i]=ans[i-
MOD+1];
    return ans;
}
// Multi-point polynomial evaluation for all points in [0, MOD)
// MOD needs to be a prime number
vector<int> evaluate_all_points(const poly &p){
    int g=MOD_PRIMITIVE_ROOT, gk=1;
    assert(g!=-1);
    vector<int> ch=chirpTransform(p,g,MOD-1), ans(MOD, p[0]);
    for (int i=(0), nb_fin_65=(MOD-1); i<nb_fin_65; ++i){
        ans[gk]=ch[i];
        gk=mulmod(gk,g);
    }
    return ans;
}
```

§6.11.4 fft-shifts-trick.cpp

06_Matematicas/Polinomios_y_transformadas

```
// Requisito: actual, answer y multiply; pegar el bloque dentro de
solve().
using ll = long long;
//FFT Trick, it very useful for shifts in the following:
// Sum j_0_to_n-1 a[j]*a[j+i]
// where i is the number of shifts, and 'a' is some array.
    auto copy = actual;
    reverse(all(copy));
    // be careful with doubles precision, so maybe NTT could be
useful here.
    // mulitply is the method of FTT or NTT
    auto polinomy = multiply(actual, copy);
    ll m = actual.size();
    answer[0] = polinomy[m-1]; // 0 with m-1, 1 with m-2 =m-1
    for (int i = 1; i <= m-1; i++) { // 1 step no m-1 steps
        // 0 with m-2 is 1 step, 1 with m-3 is one then m-1-1, also
the last one m-1 is with m-1
        // 0 with m-3 is 2 step, m-1 with m-1-1
        answer[i] = polinomy[m-1-i] + polinomy[2*(m-1)-i+1];
    }
```

§6.11.5 fft.cpp

06_Matematicas/Polinomios_y_transformadas

```
using cd = complex<double>;
const double PI = acos(-1);
/*only works for n <= 2e5 aprox -> 1(s) and n<=1e6 takes 1.5(s) */
```

```cpp
void fft(vector<cd> & a, int inv = 0) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (inv ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
    if (inv)
        for (cd & x : a) x /= n;
}
vi multiply(vi &a, vi &b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n<a.size()+b.size()) n<<=1;
    fa.resize(n), fb.resize(n);
    fft(fa), fft(fb);
    for (int i =0; i<n;i ++) fa[i]*=fb[i];
    fft(fa, 1);
    vi res(n);
    for (int i= 0; i<n;i ++) res[i] = round(fa[i].real());
    return res;
}
```

§6.11.6 interpol-o-n-ex.cpp

06_Matematicas/Polinomios_y_transformadas

```cpp
using ll = long long;
using vll = vector<long long>;
// evaluar un "polinomio interpolado" en o(nlogMOD)
// debe cumplir xi+1 - xi = xj+1 - xj for all i, j < n
// recibe vector de ys tal que f(i) = y[i]
vi fac(tam), sig(tam), facin(tam);
ll eval(vll &ys, ll x) {
    int n = ys.size();
    if(x < n) return ys[x];
    ll upi, dowi, res = 0;
    vi pre(n), suf(n);
    pre[1] = 1, suf[n - 1] = 1;
    for (int i=(1), nb_fin_1=(n); i<nb_fin_1; ++i)
    {
        if(i > 1)
        pre[i] = pre[i - 1] * (x - i + 1) % MOD;
        suf[n - i - 1] = suf[n - i] * (x - n + i) % MOD;
    }
    for (int i=(1), nb_fin_3=(n); i<nb_fin_3; ++i) {
        upi = pre[i] * suf[i] % MOD;
        dowi = facin[n - i - 1] * facin[i - 1] % MOD * sig[n - i - 1]
% MOD;
        res = (res + ys[i] * upi % MOD * dowi % MOD) % MOD;
    }
    return res;
}
```

§6.11.7 karatsuba-vasito.cpp

06_Matematicas/Polinomios_y_transformadas

```cpp
using ll = long long;
typedef ll tp;
#define add(n,s,d,k) for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i)
(d)[i]+=(s)[i]*k
tp* ini(int n){tp *r=new tp[n];fill(r,r+n,0);return r;}
void karatsura(int n, tp* p, tp* q, tp* r){
    if(n<=0)return;
    if(n<35)for (int i=(0), nb_fin_3=(n); i<nb_fin_3; ++i)for (int j=
(0), nb_fin_5=(n); j<nb_fin_5; ++j)r[i+j]+=p[i]*q[j];
    else {
        int nac=n/2,nbd=n-n/2;
        tp *a=p,*b=p+nac,*c=q,*d=q+nac;
        tp
*ab=ini(nbd+1),*cd=ini(nbd+1),*ac=ini(nac*2),*bd=ini(nbd*2);
        add(nac,a,ab,1);add(nbd,b,ab,1);
        add(nac,c,cd,1);add(nbd,d,cd,1);
        karatsura(nac,a,c,ac);karatsura(nbd,b,d,bd);
        add(nac*2,ac,r+nac,-1);add(nbd*2,bd,r+nac,-1);
        add(nac*2,ac,r,1);add(nbd*2,bd,r+nac*2,1);
        karatsura(nbd+1,ab,cd,r+nac);
        free(ab);free(cd);free(ac);free(bd);
    }
}
vector<tp> multiply(vector<tp> p0, vector<tp> p1){
    int n=max(p0.size(),p1.size());
    tp *p=ini(n),*q=ini(n),*r=ini(2*n);
    for (int i=(0), nb_fin_7=(p0.size()); i<nb_fin_7; ++i)p[i]=p0[i];
    for (int i=(0), nb_fin_9=(p1.size()); i<nb_fin_9; ++i)q[i]=p1[i];
    karatsura(n,p,q,r);
    vector<tp> rr(r,r+p0.size()+p1.size()-1);
    free(p);free(q);free(r);
    return rr;
}
```

§6.11.8 ntt-vasito.cpp

06_Matematicas/Polinomios_y_transformadas

```cpp
// Requisito: Requiere compilador GNU con enteros de 128 bits.
using ll = long long;
// The maximum length of the resulting convolution vector is 2^LG
const int LG = 20;
typedef vector<ll> poly;
template<class u, class uu, u p, u root>
struct FFT {
    u r[1+(2<<LG)];
    constexpr u m(u a, u b) {
        uu k = uu(a)*b;
        #define op(g) g*(g*p+2)
        k += u(k) * (op(op(op(op(op(-p))))) * uu(p);
        #undef op
        return u(k>>(8*sizeof(u)));
    }
    constexpr u red(u k, u a){return a-k*(a>=k);}
    FFT() {
        u k=r[2<<LG]=-p%p, b=root, e=p>>LG;
        for(;e;e/=2,b=m(b,b)) if(e&1) k=m(k,b);
        for(int i=(2<<LG)-1;i>=0;i--) r[i]=red(p, m(r[i+1], k)), i&
(i-1)?0:k=m(k,k);
    }
    poly cv(const poly &as, const poly &bs, u *v) {
        int c=max<long long>(static_cast<long long>
((as).size())+static_cast<long long>((bs).size())-1, 0), n=1;
        assert(c <= (1<<LG));
        u h=u(uu(-p)*-p%p), a=m(h, p/2+1), x, y;
        while(n<c) n*=2, h=red(p, m(h, a));
        for (int i=(0), nb_fin_3=(n); i<nb_fin_3; ++i){
            v[i]=i<static_cast<long long>((as).size())?u(as[i]):0,
            v[i+n]=i<static_cast<long long>((bs).size())?u(bs[i]):0;
        }
        for(auto s:{v,v+n}) for(int j=n;j>=2;j--) for(int k=j&-j;
k/=2;) for (int i=(j-k), nb_fin_7=(j); i<nb_fin_7; ++i){
            x=s[i], y=s[i-k];
            s[i-k] = red(2*p, x+y);
            s[i] = m(2*p+y-x, r[3*k-j+i]);
        }
        for (int i=(0), nb_fin_9=(n); i<nb_fin_9; ++i) v[i]=m(v[i],
v[i+n]);
        for (int j=(2), nb_fin_11=(n+1); j<nb_fin_11; ++j) for(int
k=1; !(k&j); k*=2) for (int i=(j-k), nb_fin_13=(j); i<nb_fin_13; ++i)
{
            x = m(v[i], r[3*k+j-i]);
            y = red(2*p, v[i-k]);
            v[i-k]=x+y, v[i]=2*p+y-x;
        }
        for (int i=(0), nb_fin_15=(c); i<nb_fin_15; ++i) v[i]=red(p,
m(v[i], h));
        return poly(v, v+c);
    }
};
// For modular convolutions modulo 998244353.
// Replace with any NTT-friendly mod by doing:
// FFT<uint32_t, uint64_t, MOD, primitiveRoot(MOD)>
poly conv_small(const poly &as, const poly &bs) {
    static uint32_t v[2<<LG];
    static FFT<uint32_t, uint64_t, 998244353, 3> fft;
    return fft.cv(as, bs, v);
}
// For modular convolutions modulo a 62 bit prime.
poly conv_big(const poly &as, const poly &bs) {
    static uint64_t v[2<<LG];
    static FFT<uint64_t, __uint128_t, (1ull<<62)-(18ull<<32)+1, 3>
fft;
    return fft.cv(as, bs, v);
}
// For modular convolutions modulo an arbitrary 32-bit modulus.
// Slightly slower than above versions, because it performs two calls
to fft
poly conv_sunzi(const poly &v1, const poly &v2, ll m) {
    const uint64_t inv = 2703402103339935109ull,
        mod1 = (1ull<<62)-(18ull<<32)+1,
        mod2 = (1ull<<62)-(76ull<<32)+1;
    static uint64_t v[2<<LG];
    static FFT<uint64_t, __uint128_t, mod1, 3> fft1;
    static FFT<uint64_t, __uint128_t, mod2, 17> fft2;
    auto as=fft1.cv(v1, v2, v), bs=fft2.cv(v1, v2, v);
    for (int i=(0), nb_fin_17=(static_cast<long long>((as).size()));
i<nb_fin_17; ++i){
        auto d = fft1.m(mod1+as[i]-bs[i], inv);
        d -= mod1*(d >= mod1); d %= m;
        as[i] = (bs[i] + mod2%m*d)%m;
    }
    return as;
}
```

§6.11.9 poly-definitions.cpp

06_Matematicas/Polinomios_y_transformadas

```cpp
A(x) = Sum i=0 to n ( a_i * x^i )  y B(x) Sum i=0 to m ( b_i * x^i)
A(x)*B(x) Sum i=0 to (n+m) Sum j=0 to (n+m)  (a_j)*(b_i-j))x^i
```

§6.11.10 polynomial-vasito.cpp

06_Matematicas/Polinomios_y_transformadas

```cpp
typedef int tp; // type of polynomial
template<class T=tp>
struct poly {  // poly<> : 1 variable, poly<poly<>>: 2 variables,
etc.
    vector<T> c;
    T& operator[](int k){return c[k];}
    poly(vector<T>& c):c(c){}
    poly(initializer_list<T> c):c(c){}
    poly(int k):c(k){}
    poly(){}
    poly operator+(poly<T> o){
        int m=c.size(),n=o.c.size();
        poly res(max(m,n));
```

```
        for (int i=(0), nb_fin_1=(m); i<nb_fin_1;
++i)res[i]=res[i]+c[i];
        for (int i=(0), nb_fin_3=(n); i<nb_fin_3;
++i)res[i]=res[i]+o.c[i];
        return res;
    }
    poly operator*(tp k){
        poly res(c.size());
        for (int i=(0), nb_fin_5=(c.size()); i<nb_fin_5;
++i)res[i]=c[i]*k;
        return res;
    }
    poly operator*(poly o){
        int m=c.size(),n=o.c.size();
        poly res(m+n-1);
        for (int i=(0), nb_fin_7=(m); i<nb_fin_7; ++i)for (int j=(0),
nb_fin_9=(n); j<nb_fin_9; ++j)res[i+j]=res[i+j]+c[i]*o.c[j];
        return res;
    }
    poly operator-(poly<T> o){return *this+(o*-1);}
    T operator()(tp v){
        T sum(0);
        for(int i=c.size()-1;i>=0;--i)sum=sum*v+c[i];
        return sum;
    }
};
// example: p(x,y)=2*x^2+3*x*y-y+4
// poly<poly<>> p={{4,-1},{0,3},{2}}
// printf("%d\n",p(2)(3)) // 27 (p(2,3))
set<tp> roots(poly<> p){ // only for integer polynomials
    set<tp> r;
    while(!p.c.empty()&&!p.c.back())p.c.pop_back();
    if(!p(0))r.insert(0);
    if(p.c.empty())return r;
    tp a0=0,an=abs(p[p.c.size()-1]);
    for(int k=0;!a0;a0=abs(p[k++]));
    vector<tp> ps,qs;
    for (int i=(1), nb_fin_11=(sqrt(a0)+1); i<nb_fin_11;
++i)if(a0%i==0)ps.pb(i),ps.pb(a0/i);
    for (int i=(1), nb_fin_13=(sqrt(an)+1); i<nb_fin_13;
++i)if(an%i==0)qs.pb(i),qs.pb(an/i);
    for(auto pt:ps)for(auto qt:qs)if(pt%qt==0){
        tp x=pt/qt;
        if(!p(x))r.insert(x);
        if(!p(-x))r.insert(-x);
    }
    return r;
}
pair<poly<>,tp> ruffini(poly<> p, tp r){ // returns pair (result,rem)
    int n=p.c.size()-1;
    vector<tp> b(n);
    b[n-1]=p[n];
    for(int k=n-2;k>=0;--k)b[k]=p[k+1]+r*b[k+1];
    return {poly<>(b),p[0]+r*b[0]};
}
// only for double polynomials
pair<poly<>,poly<> > polydiv(poly<> p, poly<> q){ // returns pair
(result,rem)
    int n=p.c.size()-q.c.size()+1;
    vector<tp> b(n);
    for(int k=n-1;k>=0;--k){
        b[k]=p.c.back()/q.c.back();
        for (int i=(0), nb_fin_15=(q.c.size()); i<nb_fin_15;
++i)p[i+k]-=b[k]*q[i];
        p.c.pop_back();
    }
    while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();
    return {poly<>(b),p};
}
// only for double polynomials
poly<> interpolate(vector<tp> x, vector<tp> y){ //TODO TEST
    poly<> q={1},nb_S={0};
    for(tp a:x)q=poly<>({-a,1})*q;
    for (int i=(0), nb_fin_17=(x.size()); i<nb_fin_17; ++i){
        poly<> Li=ruffini(q,x[i]).first;
        Li=Li*(1.0/Li(x[i])); // change for int polynomials
        nb_S=nb_S+Li*y[i];
    }
    return nb_S;
}
```

§6.12 Programación lineal

§6.12.1 simplex-integer-extra.cpp

06_Matematicas/Programacion_lineal

```
// Resuelve el problema de Linear Integer Programming (LPI).
// Agregar Fraction con tipo de dato __int128 y hacer los siguientes
ajustes:
// Cambiar el tipo de dato del Simplex a Fraction
typedef Fraction lf;
// Agregar constantes
const Fraction ZERO, INF(1e18);
// Cambiar las siguientes líneas en el Simplex
z = ZERO; // Línea 19
a[x][y] = Fraction(1) / a[x][y]; // Línea 30
if (i != x && a[i][y] != ZERO) { // Línea 32
lf mn = ZERO; // Línea 50
if (a[x][i] < ZERO) { // Línea 56
lf mx = ZERO; // Línea 65
lf mn = INF; // Línea 71
if (a[i][y] > ZERO && b[i] / a[i][y] < mn)  // Línea 73
// Agregar el siguiente método en el Simplex
pair<Fraction, vector<Fraction>> maximize_int() {
    while (true) {
        auto sol = maximize();
        bool all_int = true;
        for (auto &x : sol.S) all_int &= x.fractional_part() == ZERO;
        if (all_int) return sol;
        Fraction nw_b = ZERO;
        int id = -1;
        for (int i = 0; i < n; i++) {
            Fraction fp = b[i].fractional_part();
            if (fp >= nw_b) nw_b = fp, id = i;
        }
        vector<Fraction> nw_a;
        for (auto &x : a[id]) nw_a.pb(-x.fractional_part());
        a.pb(nw_a);
        b.pb(-nw_b);
        Y.pb(n+m); n++;
    }
}
```

§6.12.2 simplex.cpp

06_Matematicas/Programacion_lineal

```
// Maximizar la ecuación x1 + x2 + x3 ...
// sujeta a restricciones x1 + x2 <= 2, x2 + x3 <= 5...
// A: matriz de ecuaciones,
// contiene los coeficientes de cada variable
// B: vector con los coeficientes de las restricciones
// C: costos de las variables
const double EPS = 1e-6;
typedef double lf;
struct simplex {
  vector<int> X, Y;
  vector<vector<lf>> a;
  vector<lf> b, c;
  lf z;
  int n, m;
  simplex(
    vector<vector<lf>> &A,
    vector<lf> &B,
    vector<lf> &C)
{
    a = A; b = B; c = C;
    n = b.size(); m = c.size(); z = 0.0;
    X.resize(m); iota(X.begin(), X.end(), 0);
    Y.resize(n); iota(Y.begin(), Y.end(), m);
}
void pivot(int x, int y) {
  swap(X[y], Y[x]);
  b[x] /= a[x][y];
  for (int i = 0; i < m; i++) {
    if (i != y) a[x][i] /= a[x][y];
  }
  a[x][y] = 1 / a[x][y];
  for (int i = 0; i < n; i++) {
    if (i != x && abs(a[i][y]) > EPS) {
      b[i] -= a[i][y] * b[x];
      for (int j = 0; j < m; j++) {
        if (j != y)
          a[i][j] -= a[i][y] * a[x][j];
      }
      a[i][y] =- a[i][y] * a[x][y];
    }
  }
  z += c[y] * b[x];
  for (int i = 0; i < m; i++) {
    if (i != y) c[i] -= c[y] * a[x][i];
  }
  c[y] =- c[y] * a[x][y];
}
pair<lf, vector<lf>> maximize() {
  while (true) {
    int x = -1, y = -1;
    lf mn = -EPS;
    for (int i = 0; i < n; i++) {
      if (b[i] < mn) mn = b[i], x = i;
    }
    if (x < 0) break;
    for (int i = 0; i < m; i++) {
      if (a[x][i] < -EPS) {
        y = i;
        break;
      }
    }
    // no hay solución para Ax <= b
    assert(y >= 0);
    pivot(x, y);
  }
  while (true) {
    lf mx = EPS;
    int x = -1, y = -1;
    for (int i = 0; i < m; i++) {
      if (c[i] > mx) mx = c[i], y = i;
    }
    if (y < 0) break;
    lf mn = 1e200;
    for (int i = 0; i < n; i++) {
      if (a[i][y]>EPS && b[i] / a[i][y] < mn)
      mn = b[i] / a[i][y], x = i;
    }
    assert(x >= 0); // unbounded
    pivot(x, y);
  }
  vector<lf> r(m);
  for (int i = 0; i < n; i++) {
    if (Y[i] < m) r[Y[i]] = b[i];
  }
  return {z, r};
```

```
  }
};
```

§6.13 Recurrencias lineales

§6.13.1 berlekamp-massey-vasito.cpp

06_Matematicas/Recurrencias_lineales

```
using ll = long long;
using ld = long double;
vi BM(vi x){
    vi ls,cur;int lf,ld;
  for (int i=(0), nb_fin_1=(static_cast<long long>((x).size())); i<nb_fin_1; ++i){
        ll t=0;
    for (int j=(0), nb_fin_3=(static_cast<long long>((cur).size())); j<nb_fin_3; ++j)t=(t+x[i-j-1]*(ll)cur[j])%MOD;
        if((t-x[i])%MOD==0)continue;
    if(!static_cast<long long>((cur).size()))
{cur.resize(i+1);lf=i;ld=(t-x[i])%MOD;continue;}
        ll k=-(x[i]-t)*fast_pow(ld,MOD-2)%MOD;
        vi c(i-lf-1);c.pb(k);
    for (int j=(0), nb_fin_6=(static_cast<long long>((ls).size())); j<nb_fin_6; ++j)c.pb(-ls[j]*k%MOD);
    if(static_cast<long long>((c).size())<static_cast<long long>
((cur).size()))c.resize(static_cast<long long>((cur).size()));
    for (int j=(0), nb_fin_11=(static_cast<long long>((cur).size())); j<nb_fin_11; ++j)c[j]=(c[j]+cur[j])%MOD;
    if(i-lf+static_cast<long long>((ls).size())>=static_cast<long
long>((cur).size()))ls=cur,lf=i,ld=(t-x[i])%MOD;
        cur=c;
    }
  for (int i=(0), nb_fin_15=(static_cast<long long>((cur).size())); i<nb_fin_15; ++i)cur[i]=(cur[i]%MOD+MOD)%MOD;
    return cur;
}
```

§6.13.2 linear-recurrence-cayley-halmiton.cpp

...aticas/Recurrencias_lineales

```
using ll = long long;
// O(N^2 log K)
const int mod = 1e9 + 7;
template <int32_t MOD>
struct modint {
  int32_t value;
  modint() = default;
  modint(int32_t value_) : value(value_) {}
  inline modint<MOD> operator + (modint<MOD> other) const { int32_t c = this->value + other.value; return modint<MOD>(c >= MOD ? c - MOD : c); }
  inline modint<MOD> operator * (modint<MOD> other) const { int32_t c = (int64_t)this->value * other.value % MOD; return modint<MOD>(c < 0 ? c + MOD : c); }
  inline modint<MOD> & operator += (modint<MOD> other) { this->value += other.value; if (this->value >= MOD) this->value -= MOD; return *this; }
  modint<MOD> pow(uint64_t k) const {
    modint<MOD> x = *this, y = 1;
    for (; k; k >>= 1) {
      if (k & 1) y *= x;
      x *= x;
    }
    return y;
  }
  modint<MOD> inv() const { return pow(MOD - 2); }  // MOD must be a prime
};
using mint = modint<mod>;
vector<mint> combine (int n, vector<mint> &a, vector<mint> &b, vector<mint> &tr) {
  vector<mint> res(n * 2 + 1, 0);
  for (int i = 0; i < n + 1; i++) {
    for (int j = 0; j < n + 1; j++) res[i + j] += a[i] * b[j];
  }
  for (int i = 2 * n; i > n; --i) {
    for (int j = 0; j < n; j++) res[i - 1 - j] += res[i] * tr[j];
  }
  res.resize(n + 1);
  return res;
};
// transition -> for(i = 0; i < x; i++) f[n] += tr[i] * f[n-i-1]
// S contains initial values, k is 0 indexed
mint LinearRecurrence(vector<mint> &nb_S, vector<mint> &tr, long long k) {
  int n = nb_S.size(); assert(n == (int)tr.size());
  if (n == 0) return 0;
  if (k < n) return nb_S[k];
  vector<mint> pol(n + 1), e(pol);
  pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(n, pol, e, tr);
    e = combine(n, e, e, tr);
  }
  mint res = 0;
  for (int i = 0; i < n; i++) res += pol[i + 1] * nb_S[i];
  return res;
}
void test_case() {
    ll n;
    cin >> n; // Fibonacci
    vector<mint> initial = {0, 1}; // F0, F1
    vector<mint> tr = {1, 1};
    cout << LinearRecurrence(initial, tr, n).value << "\n";
}
```

§6.13.3 linear-recurrence-matrix-exponciation.cpp

...s/Recurrencias_lineales

```
using ll = long long;
using vl = vector<long long>;
// Solves F_n = C_n-1 * F_n-1 + ... + C_0*F_0 + p + q*n + r*n^2
// O((n+3)^3*log(k))
// Also solves (k steps)-min path of a matrix in same complexity
// Tested and for more details see:
https://codeforces.com/blog/entry/80195
const int MOD = 1e9 + 7;
const int N = 10 + 3; // 10 is MAX N, 3 is for p,q,r
inline ll add(ll x, ll y) { return (x+y)%MOD; }
inline ll mul(ll x, ll y) { return (x*y)%MOD; }
// const ll inf = ll(1e18) + 5; // for k-min path
struct Mat {
    array<array<ll,N>,N> mt;
    Mat(bool id=false) {
        for (auto &x : mt) fill(all(x),0);
        if (id) for (int i=0;i<N;i++) mt[i][i]=1;
    //for (auto &x : mt) fill(all(x),inf); // For k-min path
        //if (id) for (int i =0;i<N;i++) mt[i][i]=0;
    }
    inline Mat operator * (const Mat &b) {
        Mat ans;
        for (int k=0;k<N;k++)for(int i=0;i<N;i++)for(int j=0;j<N;j++)
            ans.mt[i][j]=add(ans.mt[i][j],mul(mt[i][k],b.mt[k][j]));
            //ans.mt[i][j] = min(ans.mt[i][j],mt[i][k]+b.mt[k][j]);
// For K-min Path
        return ans;
    }
    inline Mat pow(ll k) {
        Mat ans(true),p=*this; // Note '*'!!
        while (k) {
            if (k&1) ans = ans*p;
            p=p*p;
            k>>=1;
        }
        return ans;
    }
    string db() { // Optional for debugging
        string ans;
        for (int i =0;i<mt.size();i++)for (int j=0;j<mt.size();j++)
                ans +=to_string(mt[i][j]),ans+=" \n"[j==N-1];
        return "\n"+ans;
    }
};
// Important!!! Remember to set N = MAX_N + 3
// Solves F_n = C_n-1 * F_n-1 + ... + C_0*F_0 + p + q*n + r*n^2
// f = {f_0,f_1,f_2,f_3,...,f_n}
// c = {c_0,c_1,c_2,c_3,...,c_n}
ll fun(vl f, vl c, ll p, ll q, ll r, ll k) {
    ll n = c.size();
    if (k < n) return f[k];
    reverse(all(c)),reverse(all(f));
    Mat mt,st;
    for (int i = 0;i<n;i++) mt.mt[0][i]=c[i];
    for (int i = 1;i<n;i++) mt.mt[i][i-1]=1;
    for (int i = 0;i<n;i++) st.mt[i][0]=f[i];
    vl extra = {p,q,r}; // To extend here with 1*p,i*q,i*i*r,etc
    for (int i=0;i<extra.size();i++) {
        st.mt[n+i][0]=1; //1,i,i*i,i*i*i
        mt.mt[0][n+i]=extra[i];//p,q,r
        mt.mt[n+i][n]=1; //pascal
        for (int j=1;j<=i;j++) { //pascal
            st.mt[n+i][0]*=n;//1,i*i,i*i*i
            mt.mt[n+i][n+j]=mt.mt[n+i-1][n+j]+mt.mt[n+i-1][n+j-1];
        }
    }
    return (mt.pow(k-(n-1))*st).mt[0][0];
}
```

§6.14 Sumas de pisos

§6.14.1 floor-division-vasito.cpp

06_Matematicas/Sumas_de_pisos

```
using ll = long long;
void floordiv(ll x, ll y, ll& q, ll& r) { // (for negative x)
    q=x/y;r=x%y;
    if((r!=0)&&((r<0)!=(y<0)))q--,r+=y;
}
```

§6.14.2 floor-sums.cpp

06_Matematicas/Sumas_de_pisos

```
// from atcoder
// floor_sum(n,m,a,b) = sum{0}to{n-1} [(a*i+b)/m]
// O(log m), mod 2^64, n<2^32, m<2^32
constexpr long long safe_mod(long long x, long long m) {
    x %= m;
    if (x < 0) x += m;
    return x;
}
unsigned long long floor_sum_unsigned(unsigned long long n,
                                      unsigned long long m,
                                      unsigned long long a,
                                      unsigned long long b) {
    unsigned long long ans = 0;
    while (true) {
        if (a >= m) {
            ans += n * (n - 1) / 2 * (a / m);
            a %= m;
        }
        if (b >= m) {
            ans += n * (b / m);
            b %= m;
        }
        unsigned long long y_max = a * n + b;
        if (y_max < m) break;
        // y_max < m * (n + 1)
```

```
        // floor(y_max / m) <= n
        n = (unsigned long long)(y_max / m);
        b = (unsigned long long)(y_max % m);
        swap(m, a);
    }
    return ans;
}
long long floor_sum(long long n, long long m, long long a, long long
b) {
    assert(0 <= n && n < (1LL << 32));
    assert(1 <= m && m < (1LL << 32));
    unsigned long long ans = 0;
    if (a < 0) {
        unsigned long long a2 = safe_mod(a, m);
        ans -= 1ULL * n * (n - 1) / 2 * ((a2 - a) / m);
        a = a2;
    }
    if (b < 0) {
        unsigned long long b2 = safe_mod(b, m);
        ans -= 1ULL * n * ((b2 - b) / m);
        b = b2;
    }
    return ans + floor_sum_unsigned(n, m, a, b);
}
```

§6.14.3 harmonic-lemma.cpp

06_Matematicas/Sumas_de_pisos

```
for (int i = 1, la; i <= n; i = la + 1) {
    la = n / (n / i);
    //n / x yields the same value for i <= x <= la.
}
```

§6.14.4 points-under-line-vasito.cpp

06_Matematicas/Sumas_de_pisos

```
using ll = long long;
// # of lattice points s.t. ax+by<=c, x,y>0 (a,b is positive integer)
ll f(ll a, ll b, ll c){
    if(c<=0) return 0;
    if(a<b) swap(a, b);
    ll m=c/a;
    if(a==b) return m*(m-1)/2;
    ll k=(a-1)/b, h=(c-a*m)/b;
    return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
}
// # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=Y (a,b is positive
integer)
ll g(ll a, ll b, ll c, ll X, ll Y){
    if(a*X+b*Y<=c) return X*Y;
    return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c-a*X-b*Y);
}
```

§6.14.5 sqrt-mod.cpp

06_Matematicas/Sumas_de_pisos

```
// Requisito: n y nums; el llenado de ch va dentro de solve().
/*
Trick using SQRT for problems using de diferent floor values o a num.
A num has only around SQRT(n) possible values when it is floor
divided, for 2e5 is around 890
e.g. floor(n/i)=j. j has around sqrt(n) values.
e.g. floor(10/i)=j. j only can be 1, 2, 3, 5, 10 for any i
The following function calculates when a n/i changes if you iterate
over i increasing.
You can solve https://codeforces.com/gym/105009/problem/L using this.
*/
const int MXN = 2e5 + 10;
int nums[MXN];
vector<int> ch[MXN];
for (int i =0;i<n;i++) {
  for (int d = 1; d <= nums[i]; d = nums[i]/(nums[i]/d)+1) {
    ch[d].pb(i);
  }
  ch[nums[i]+1].pb(i);
}
```

§6.15 Teoría de números › Diofánticas

§6.15.1 diophantine-ecuations-extra.cpp

...ticas/Teoria_de_numeros/Diofanticas

```
#include "../Euclides/extended-euclides-extra.cpp"
using ll = long long;
// Encuentra x, y en la ecuación de la forma ax + by = c
// Agregar Extended Euclides.
ll g;
bool diophantine(ll a, ll b, ll c) {
    x = y = 0;
    if (!a && !b) return (!c); // sólo hay solución con c = 0
    g = euclid(abs(a), abs(b));
    if (c % g) return false;
    a /= g; b /= g; c /= g;
    if (a < 0) x *= -1;
    x = (x % b) * (c % b) % b;
    if (x < 0) x += b;
    y = (c - a*x) / b;
    return true;
}
```

§6.16 Teoría de números › Divisores

§6.16.1 divisors-extra.cpp

06_Matematicas/Teoria_de_numeros/Divisores

```
using ll = long long;
// * Calcula la suma de los divisores de n. Agregar Prime
Factorization y Modular Exponentiation (sin el modulo).
ll sumDiv(ll n) {
    map<ll, int> f;
    fact(n, f);
    ll ans = 1;
    for (auto p : f) {
        ans *= (exp(p.first, p.second+1)-1)/(p.first-1);
    }
    return ans;
}
// * Calcula la cantidad de divisores de n. Agregar Prime
Factorization.
ll cantDiv(ll n) {
    map<ll, int> f;
    fact(n, f);
    ll ans = 1;
    for (auto p : f) ans *= (p.second + 1);
    return ans;
}
// * Calcular la cantidad de divisores para todos los numeros menores
o iguales a MX con Sieve of Eratosthenes.
const int MX = 1e6;
bool marked[MX+1];
vector<int> cnt;
void sieve() {
    cnt.assign(MX+1, 1);
    marked[0] = marked[1] = true;
    for (int i = 2; i <= MX; i++) {
        if (marked[i]) continue;
        cnt[i]++;
        for (int j = i*2; j <= MX ; j += i) {
            int n = j, c = 1;
            while (n%i == 0) n /= i, c++;
            cnt[j] *= c;
            marked[j] = true;
        }
    }
}
```

§6.16.2 divisors-vasito.cpp

06_Matematicas/Teoria_de_numeros/Divisores

```
using ll = long long;
void div_rec(vector<ll>& r, vector<pair<ll,int> >& f, int k, ll c){
    if(k==f.size()){r.pb(c);return;}
    for (int i=(0), nb_fin_1=(f[k].second+1); i<nb_fin_1;
++i)div_rec(r,f,k+1,c),c*=f[k].first;
}
vector<ll> divisors(vector<pair<ll,int> > f){
    vector<ll> r; // returns divisors given factorization
    div_rec(r,f,0,1);
    return r;
}
```

§6.17 Teoría de números › Euclides

§6.17.1 extended-euclides-extra.cpp

..._Matematicas/Teoria_de_numeros/Euclides

```
// El algoritmo de Euclides extendido retorna el gcd(a, b) y calcula
los coeficientes enteros X y Y que satisfacen la ecuacion: a*X + b*Y
= gcd(a, b).
int x, y;
// O(log(max(a, b)))
int euclid(int a, int b) {
    if (b == 0) { x = 1; y = 0; return a; }
    int d = euclid(b, a%b);
    int aux = x;
    x = y;
    y = aux - a/b*y;
    return d;
}
```

§6.17.2 fast-gcd.cpp

06_Matematicas/Teoria_de_numeros/Euclides

```
// Requisito de selección: Adaptación de 64 bits: __builtin_ctz debe
ser __builtin_ctzll. Usar entradas no negativas y tratar el cero
antes del builtin.
// Requisito: Revisar casos cero, productos y desplazamientos antes
de usar en el rango completo de 64 bits.
int gcd(int a, int b) {
  if (!a || !b)
    return a | b;
  unsigned shift = __builtin_ctz(a | b);
  a >>= __builtin_ctz(a);
  do {
    b >>= __builtin_ctz(b);
    if (a > b)
      swap(a, b);
    b -= a;
  } while (b);
  return a << shift;
}
```

§6.17.3 greatest-common-divisor-extra.cpp

...icas/Teoria_de_numeros/Euclides

```
// Calcula el maximo comun divisor entre a y b mediante el algoritmo
de Euclides. Tambien se puede usar __gcd(a, b).
// O(log(max(a, b)))
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a%b);
}
```

§6.17.4 lowest-common-multiple-extra.cpp

...icas/Teoria_de_numeros/Euclides

```
// Calculo del minimo comun multiplo usando el maximo comun divisor.
// O(log(max(a, b)))
int lcm(int a, int b) {
    return a / __gcd(a, b) * b;
}
```

§6.18.1 euler-totient.cpp ..._Matematicas/Teoria_de_numeros/Funciones_multiplicativas

```cpp
//La funcion phi de Euler devuelve la cantidad de
//enteros positivos menores o iguales a n que son
//coprimos con n (gcd(n, i) = 1)
// O(sqrt(n))
int phi(int n) {
  int ans = n;
  for (int p = 2; p <= n/p; ++p) {
    if (n % p == 0) ans -= ans / p;
    while (n % p == 0) n /= p;
  }
  if (n > 1) ans -= ans / n;
  return ans;
}
// Calcular el Euler totient para todos los numeros
//menores o iguales a MX con Sieve of Eratosthenes.
const int MX = 1e6;
bool marked[MX+1];
int phi_tabla[MX+1];
// O(MX log(log(MX)))
void sieve() {
  iota(phi_tabla, phi_tabla+MX+1, 0);
  marked[0] = marked[1] = true;
  for (int i = 2; i <= MX; i++) {
    if (marked[i]) continue;
    for (int j = i; j <= MX ; j += i) {
      phi_tabla[j] -= phi_tabla[j] / i;
      marked[j] = true;
    }
    marked[i] = false;
  }
}
```

§6.18.2 gcd-a-i-a-j-k-text-queries.cpp ...ia_de_numeros/Funciones_multiplicativas

```cpp
using ll = long long;
using vl = vector<long long>;
// Given an array and q queries of count pairs gcd(a_i,a_j)==k
// i < j, a_i < 1e5, q < 1e5, n < 1e5
// Complexity O(n * log n + q)
// Tested: https://www.hackerrank.com/contests/ab-yeh-kar-ke-dikhao-returns/challenges/gcd-pairs
const int mxN = 1e5 + 10;
vl mo(mxN);
void init() { // Call init() first !!!
    mo[1] = 1;
    for (int i = 1;i<mxN;i++)
        for (int j = i+i;j<mxN;j+=i) mo[j]-=mo[i];
}
vl cnt(mxN), dcnt(mxN), ans(mxN);
void test_case() {
    ll n,q; cin >> n >> q;
    for (int i=0;i<n;i++) {
        ll x;cin >> x;
        cnt[x]++; // cnt[x] = quantity of X's in array
    }
    for (int i = 1;i<mxN;i++)
        for (int j = i;j<mxN;j+=i)
            dcnt[i] += cnt[j];
    // dcnt[x] = quantity of a_i divisible by x
    for (int k = 1;k<mxN;k++) {
        for (int d = 1; d <= mxN/k; d++) {
            ll totalCnt = d*k < mxN ? dcnt[d*k] : 0;
            ans[k] += mo[d] * totalCnt * totalCnt;
        }
        ans[k] += cnt[k]; // substracting j>=i
        ans[k] /= 2;
    }
    while (q--) {
        ll k; cin >> k;
        if (k < mxN) cout << ans[k] << "\n";
        else cout << 0 << "\n";
    }
}
```

§6.18.3 gcd-i-j-k.cpp 06_Matematicas/Teoria_de_numeros/Funciones_multiplicativas

```cpp
using ll = long long;
// Counts pairs gcd(i,j) == k
// 1 <= i <= a, 1 <= j <= b, where (1,2) equals to (2,1)
// Call Mobius First !!!
// O(min(a,b)/K)
// Tested: https://vjudge.net/problem/HDU-1695
ll solve(ll a, ll b, ll k) {
    if (k==0) return 0;
    a/=k;
    b/=k;
    if (a > b) swap(a,b);
    if (a == 0) return 0;
    ll ans = 0;
    for (ll d = 1; d <= a; d++) {
        ans += (a/d) * (b/d) * mo[d];
    }
    ll sub = 0; // Substracting equals, e.g. (1,2) to (2,1)
    for (ll d = 1;d <= a; d++) {
        sub += (a/d)*(a/d) * mo[d];
    }
    ans-=(sub-1)/2;
    return ans;
}
```

§6.18.4 linear-sieve.cpp 06_Matematicas/Teoria_de_numeros/Funciones_multiplicativas

```cpp
using ll = long long;
using vl = vector<long long>;
/* For getting the primes less than mxN in O(mxN)*/
const int mxN = 1e6 + 10;
vl sv(mxN); // if prime sv[i]==i, it stores the lowest prime of 'i'
vl primes;
void init() { // O(n)
    for (int i = 2;i<mxN;i++) {
        if (sv[i]==0) {
            sv[i]=i;
            primes.pb(i);
        }
        for (int j = 0;j<primes.size() && primes[j]*i<mxN;j++) {
            sv[primes[j]*i] = primes[j];
            if (primes[j] == sv[i]) break;
        }
    }
}
// factorization using linear sieve, O(prime_count(n))
// Very fast factorization but only for (n < mxN) !!!
void fact(map<ll,int> &f, ll num) {
    while (num>1) {
        ll p = sv[num];
        while (num % p == 0) num/=p, f[p]++;
    }
}
```

§6.18.5 mobius-inclusion-exclusion-example.tex ...Funciones_multiplicativas

How many numbers are there less than or equal to n that are free of squares?. Contrainsts: $1 <= n <= 10^{12}$ Change the statement to count the reverse and then sustract: How many numbers are ... that can be divided by a square of a prime. So the answer will be $n-$ Summatory with Inclusion-Exclusion of $f(prime)$

$$f(prime) = floor(\frac{n}{p*p})$$

So in those cases of summatory with primes, you can use Mobius to adding or substracting. Final answer is

$$n - \Sigma_{i=1}^{\sqrt{n}} \mu(i) \left\lfloor \frac{n}{i^2} \right\rfloor$$

Or in programming terms:

```cpp
long long ans = n;
for (int i =1;i<=sqrt(n);i++) {
    ans -= mo[i] * n/(i*i);
}
```

§6.18.6 mobius.cpp 06_Matematicas/Teoria_de_numeros/Funciones_multiplicativas

```cpp
// Llamar variante_lineal::init() una vez antes de consultar mo[n].
using vl = vector<long long>;
/*
Mobius Function.
 Multiplicative function that is useful to inclusion/exclusion with
 prime numbers (it gives the coeficient). Also you can reduce some
 sumatories using its equality with unit(n) function (see the key
below)
 unit(n) = [ n == 1 ]
 unit(1) = 1, unit(2) = 0, unit(0) = 0
 (This is the key!!)
 unit(n) = sum_{ d divides n } mobius(d)
 mobius(1) = 1
 mobius(quantity of primes is odd) = -1
 mobius(quantity of primes is even) = 1
 mobius(n is divisible by a square prime) = 0
 Check https://codeforces.com/blog/entry/53925 for more information.
 Check mobius examples
*/
namespace variante_lineal {
// This is O(n), but not too much difference of speed with the other
const int mxN = 1e6 + 10;
vl sv(mxN); // prime if sv[i]==i, it stores the lowest prime of i
vl primes; // Primes less than mxN
vl mo(mxN); // Mobius
void init() {
    // sv[1] = 1; // Check if needed
    for (int i = 2;i<mxN;i++) { // Linear Sieve
        if (sv[i]==0) { sv[i]=i; primes.pb(i); }
        for (int j = 0;j<primes.size() && primes[j]*i<mxN;j++) {
            sv[primes[j]*i] = primes[j];
            if (primes[j] == sv[i]) break;
        }
    }
    mo[1] = 1; // Mobius
    for (int i=2;i<mxN;i++) {
       if (sv[i/sv[i]] == sv[i]) mo[i] = 0;
       else mo[i] = -1*mo[i/sv[i]];
    }
}
} // namespace variante_lineal
```

§6.18.7 multiplicative-functions.tex ...eoria_de_numeros/Funciones_multiplicativas

The following functions are all multiplicative functions, where p is a prime number and k is a positive integer:

- The constant function: $I(p^k) = 1$.
- The identity function: $\mathrm{Id}(p^k) = p^k$.
- The power function: $\mathrm{Id}_a(p^k) = p^{ak}$, where a is a constant.
- The unit function: $\chi(p^k) = [p^k = 1]$.
- The divisor function: $\sigma_a(p^k) = \sum_{i=0}^{k} p^{ai}$, denoting the sum of the a-th powers of all the positive divisors of the number.
- The Möbius function: $\mu(p^k) = [k = 0] - [k = 1]$.

- Euler's totient function: $\varphi(p^k) = p^k - p^{k-1}$.

Note: $[P]$ refers to the boolean expression, i.e., $[P] = 1$ when P is true, and 0 otherwise.

§6.18.8 sigma-n-i-1-frac-n-gcd-i-n.cpp ..._de_numeros/Funciones_multiplicativas

```cpp
using ll = long long;
using vl = vector<long long>;
// Multiplicative Function
// Calc f(n), f(n) = sum_{1 to n} n / gcd(n,i)
// f(p) = (p-1)*p + 1
// f(p^k) = f(p^(k-1)) + p^k*p^(k-1)*(p-1)
const int mxN = ll(1e7) + 10;
vl lp(mxN); // least prime factor
vl pw(mxN); // power of least prime factor
vl fn(mxN); // answer of f(n)
vl primes;
void init() { // O(n), Call init first !!!!
    fn[1] = pw[1] = lp[1] = 1;
    for (ll i = 2;i<mxN;i++) {
        if (lp[i] == 0) {
            lp[i] = pw[i] = i;
            fn[i] = (i-1)*i + 1;
            primes.pb(i);
        }
        for (auto p : primes) {
            ll j = i*p;
            if (j >= mxN) break;
            if (lp[i] != p) {
                lp[j] = pw[j] = p;
                fn[j] = fn[i] * fn[p];
            } else {
                lp[j] = p;
                pw[j] = pw[i] * p;
                ll fk = fn[pw[i]] + pw[j] * pw[i] * (p-1);
                fn[j] = fn[i/pw[i]] * fk;
                break;
            }
        }
    }
}
```

§6.19 Teoría de números › Primos y factorización

§6.19.1 count-primes-with-pi-function.cpp ..._numeros/Primos_y_factorizacion

```cpp
using ll = long long;
// sprime.count_primes(n);
// O(n^(2/3))
// PI(n) = Count prime numbers until n inclusive
struct count_primers_struct {
    vector<int> primes;
    vector<int> mnprimes;
    ll ans;
    ll y;
    vector<pair<pair<ll, int>, char>> queries;
    ll count_primes(ll n) {
        // this y is actually n/y
        // also no logarithms, welcome to reality, this y is the best for n=10^12 or n=10^13
        y = pow(n, 0.64);
        if (n < 100) y = n;
        // linear sieve
        primes.clear();
        mnprimes.assign(y + 1, -1);
        ans = 0;
        for (int i = 2; i <= y; ++i) {
            if (mnprimes[i] == -1) {
                mnprimes[i] = primes.size();
                primes.push_back(i);
            }
            for (int k = 0; k < primes.size(); ++k) {
                int j = primes[k];
                if (i * j > y) break;
                mnprimes[i * j] = k;
                if (i % j == 0) break;
            }
        }
        if (n < 100) return primes.size();
        ll s = n / y;
        for (int p : primes) {
            if (p > s) break;
            ans++;
        }
        // pi(n / y)
        int ssz = ans;
        // F with two pointers
        int ptr = primes.size() - 1;
        for (int i = ssz; i < primes.size(); ++i) {
            while (ptr >= i && (ll)primes[i] * primes[ptr] > n)
                --ptr;
            if (ptr < i) break;
            ans -= ptr - i + 1;
        }
        // phi, store all queries
        phi(n, ssz - 1);
        sort(queries.begin(), queries.end());
        int ind = 2;
        int sz = primes.size();
        // the order in fenwick will be reversed, because prefix sum in a fenwick is just one query
        fenwick fw(sz);
        for (auto qq : queries) {
            auto na = qq.first;
            auto sign = qq.second;
            auto n = na.first;
            auto a = na.second;
            while (ind <= n)
                fw.add(sz - 1 - mnprimes[ind++], 1);
            ans += (fw.ask(sz - a - 2) + 1) * sign;
        }
        queries.clear();
        return ans - 1;
    }
    void phi(ll n, int a, int sign = 1) {
        if (n == 0) return;
        if (a == -1) {
            ans += n * sign;
            return;
        }
        if (n <= y) {
            queries.emplace_back(make_pair(n, a), sign);
            return;
        }
        phi(n, a - 1, sign);
        phi(n / primes[a], a - 1, -sign);
    }
    struct fenwick {
        vector<int> tree;
        int n;
        fenwick(int n = 0) : n(n) {
            tree.assign(n, 0);
        }
        void add(int i, int k) {
            for (; i < n; i = (i | (i + 1)))
                tree[i] += k;
        }
        int ask(int r) {
            int res = 0;
            for (; r >= 0; r = (r & (r + 1)) - 1)
                res += tree[r];
            return res;
        }
    };
} ;
count_primers_struct sprime;
```

§6.19.2 fermat.cpp 06_Matematicas/Teoria_de_numeros/Primos_y_factorizacion

```cpp
// ll fermatFactors(ll n) {
//     ll a = ceil(sqrt(n)) ;
//     if(a * a == n){
//         return a;
//     }
//     ll b;
//     while(true) {
//         ll b1 = a * a - n ;
//         b = (ll)sqrt(b1) ;
//         if(b * b == b1)
//             break;
//         else
//             a += 1;
//     }
//     return min(a - b, a + b);
// }
```

§6.19.3 optimized-polard-rho.cpp ...cas/Teoria_de_numeros/Primos_y_factorizacion

```cpp
// Requisito de selección: Llamar init_sv(). La multiplicación modular aproximada original tiene overflow signed; reemplazar por __int128 antes de usar valores grandes. rabin no es determinista para todo el rango de 64 bits.
using ll = long long;
// Fast factorization with big numbers, use fact method
// Seems to be O(log^3(n)) !!! Need revision
ll gcd(ll a, ll b){return a?gcd(b%a,a):b;}
ll mulmod(ll a, ll b, ll m) {
  ll r=a*b-(ll)((long double)a*b/m+.5)*m;
  return r<0?r+m:r;
}
ll expmod(ll b, ll e, ll m){
  if(!e)return 1;
  ll q=expmod(b,e/2,m);q=mulmod(q,q,m);
  return e&1?mulmod(b,q,m):q;
}
bool is_prime_prob(ll n, int a){
  if(n==a)return true;
  ll s=0,d=n-1;
  while(d%2==0)s++,d/=2;
  ll x=expmod(a,d,n);
  if((x==1)||(x+1==n))return true;
  for(int _ = 0; _ < s-1; _++){
    x=mulmod(x,x,n);
    if(x==1)return false;
    if(x+1==n)return true;
  }
  return false;
}
bool rabin(ll n){ // true iff n is prime
  if(n==1)return false;
  int ar[]={2,3,5,7,11,13,17,19,23};
  for(int i = 0; i < 9; i++)if(!is_prime_prob(n,ar[i]))return false;
  return true;
}
// optimized version: replace rho and fact with the following:
const int MAXP=1e6+1; // sieve size
int sv[MAXP]; // sieve
ll add(ll a, ll b, ll m){return (a+=b)<m?a:a-m;}
ll rho(ll n){
  static ll s[MAXP];
```

```
  while(1){
    ll x=rand()%n,y=x,c=rand()%n;
    ll *px=s,*py=s,v=0,p=1;
    while(1){
      *py++=y=add(mulmod(y,y,n),c,n);
      *py++=y=add(mulmod(y,y,n),c,n);
      if((x=*px++)==y)break;
      ll t=p;
      p=mulmod(p,abs(y-x),n);
      if(!p)return gcd(t,n);
      if(++v==26){
        if((p=gcd(p,n))>1&&p<n)return p;
        v=0;
      }
    }
    if(v&&(p=gcd(p,n))>1&&p<n)return p;
  }
}
void init_sv(){
  for(int i = 2; i < MAXP; i++)if(!sv[i])for(ll
j=i;j<MAXP;j+=i)sv[j]=i;
}
void fact(ll n, map<ll,int>& f){ // call init_sv first!!!
  for(auto&& p:f){
    while(n%p.first==0){
      p.second++; n/=p.first;
    }
  }
  if(n<MAXP)while(n>1)f[sv[n]]++,n/=sv[n];
  else if(rabin(n))f[n]++;
  else {ll q=rho(n);fact(q,f);fact(n/q,f);}
}
```

§6.19.4 prime-factorization-extra.cpp

...oria_de_numeros/Primos_y_factorizacion

```
using ll = long long;
// Tres funciones diferentes que guardan en el map f los pares
<primo, exponente> de la descomposicion en factores primos de n.
// 1.1) Iterando hasta sqrt(n)
// O(sqrt(n))
void fact(ll n, map<ll, int> &f) {
    for (int p = 2; 1ll*p*p <= n; p++)
        while (n%p == 0) f[p]++, n /= p;
    if (n > 1) f[n]++;
}
```

§6.19.5 prime-numbers.cpp

...atematicas/Teoria_de_numeros/Primos_y_factorizacion

```
2    3    5    7   11   13   17   19   23   29
  31   37   41   43   47   53   59   61   67   71
  73   79   83   89   97  101  103  107  109  113
 127  131  137  139  149  151  157  163  167  173
 179  181  191  193  197  199  211  223  227  229
 233  239  241  251  257  263  269  271  277  281
 283  293  307  311  313  317  331  337  347  349
 353  359  367  373  379  383  389  397  401  409
 419  421  431  433  439  443  449  457  461  463
 467  479  487  491  499  503  509  521  523  541
 547  557  563  569  571  577  587  593  599  601
 607  613  617  619  631  641  643  647  653  659
 661  673  677  683  691  701  709  719  727  733
 739  743  751  757  761  769  773  787  797  809
 811  821  823  827  829  839  853  857  859  863
 877  881  883  887  907  911  919  929  937  941
 947  953  967  971  977  983  991  997 1009 1013
1019 1021 1031 1033 1039 1049 1051 1061 1063 1069
1087 1091 1093 1097 1103 1109 1117 1123 1129 1151
1153 1163 1171 1181 1187 1193 1201 1213 1217 1223
1229 1231 1237 1249 1259 1277 1279 1283 1289 1291
1297 1301 1303 1307 1319 1321 1327 1361 1367 1373
1381 1399 1409 1423 1427 1429 1433 1439 1447 1451
1453 1459 1471 1481 1483 1487 1489 1493 1499 1511
1523 1531 1543 1549 1553 1559 1567 1571 1579 1583
1597 1601 1607 1609 1613 1619 1621 1627 1637 1657
1663 1667 1669 1693 1697 1699 1709 1721 1723 1733
1741 1747 1753 1759 1777 1783 1787 1789 1801 1811
1823 1831 1847 1861 1867 1871 1873 1877 1879 1889
1901 1907 1913 1931 1933 1949 1951 1973 1979 1987
      970'997       971'483   921'281'269   999'279'733
1'000'000'009 1'000'000'021 1'000'000'409 1'005'012'527
10005512850571600911
```

§6.19.6 sieve-of-eratosthenes-extra.cpp

...ia_de_numeros/Primos_y_factorizacion

```
using ll = long long;
// Guarda en primes los numeros primos menores o iguales a MX. Para
saber si p es un número primo, hacer: if (!marked[p]).
const int MX = 1e6;
bool marked[MX+1];
vector<int> primes;
// O(MX log(log(MX)))
void sieve() {
    marked[0] = marked[1] = true;
    for (int i = 2; i <= MX; i++) {
        if (marked[i]) continue;
        primes.pb(i);
        for (ll j = 1ll*i*i; j <= MX; j += i) marked[j] = true;
    }
}
```

§6.19.7 sieve.cpp

06_Matematicas/Teoria_de_numeros/Primos_y_factorizacion

```
// Requisito: Requiere compilador GNU con enteros de 128 bits.
using u64 = uint64_t;
using u128 = __uint128_t;
u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}
bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};
bool MillerRabin(u64 n, int iter=5) { // returns true if n is
probably prime, else returns false.
    if (n < 4)
        return n == 2 || n == 3;
    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }
    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

§6.20 Teoría y fórmulas

§6.20.1 formulas-extra.tex

06_Matematicas/Teoria_y_formulas

PERMUTACIÓN Y COMBINACIÓN	
Combinación (Coeficiente Binomial)	Número de subconjuntos de k elementos escogidos de un conjunto con n elementos. $\binom{n}{k} = \binom{n}{n-k} = \frac{n!}{k!(n-k)!}$
Combinación con repetición	Número de grupos formados por n elementos, partiendo de m tipos de elementos. $CR_m^n = \binom{m+n-1}{n} = \frac{(m+n-1)!}{n!(m-1)!}$
Permutación	Número de formas de agrupar n elementos, donde importa el orden y sin repetir elementos $P_n = n!$
Permutación múltiple	Elegir r elementos de n posibles con repetición n^r
Permutación con repetición	Se tienen n elementos donde el primer elemento se repite a veces , el segundo b veces , el tercero c veces, ... $PR_n^{a,b,c...} = \frac{P_n}{a!b!c!...}$
Permutaciones sin repetición	Núumero de formas de agrupar r elementos de n disponibles, sin repetir elementos $\frac{n!}{(n-r)!}$

DISTANCIAS	
Distancia Euclideana	$d_E(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
Distancia Manhattan	$d_M(P_1, P_2) = \lvert x_2 - x_1 \rvert + \lvert y_2 - y_1 \rvert$

CIRCUNFERENCIA Y CÍRCULO	
Considerando r como el radio, α como el ángulo del arco o sector, y (R, r) como radio mayor y menor respectivamente.	
Área	$A = \pi * r^2$
Longitud	$L = 2 * \pi * r$
Longitud de un arco	$L = \frac{2 * \pi * r * \alpha}{360}$
Área sector circular	$A = \frac{\pi * r^2 * \alpha}{360}$
Área corona circular	$A = \pi(R^2 - r^2)$

TRIÁNGULO	
Considerando b como la longitud de la base, h como la altura, letras minúsculas como la longitud de los lados, letras mayúsculas como los ángulos, y r como el radio de círcunferencias asociadas.	
Área conociendo base y altura	$A = \frac{1}{2}b * h$
Área conociendo 2 lados y el ángulo que forman	$A = \frac{1}{2}b * a * sin(C)$
Área conociendo los 3 lados	$A = \sqrt{p(p-a)(p-b)(p-c)}$ con $p = \frac{a+b+c}{2}$
Área de un triángulo circunscrito a una circunferencia	$A = \frac{abc}{4r}$
Área de un triángulo inscrito a una	$A = r(\frac{a+b+c}{2})$

circunferencia	
Área de un triangulo equilátero	$A = \frac{\sqrt{3}}{4}a^2$

RAZONES TRIGONOMÉTRICAS
Considerando un triangulo rectángulo de lados a, b y c, con vértices A, B y C (cada vértice opuesto al lado cuya letra minuscula coincide con el) y un ángulo α con centro en el vertice A. a y b son catetos, c es la hipotenusa:
$$sin(\alpha) = \frac{cateto\ opuesto}{hipotenusa} = \frac{a}{c}$$
$$cos(\alpha) = \frac{cateto\ adyacente}{hipotenusa} = \frac{b}{c}$$
$$tan(\alpha) = \frac{cateto\ opuesto}{cateto\ adyacente} = \frac{a}{b}$$
$$sec(\alpha) = \frac{1}{cos(\alpha)} = \frac{c}{b}$$
$$csc(\alpha) = \frac{1}{sin(\alpha)} = \frac{c}{a}$$
$$cot(\alpha) = \frac{1}{tan(\alpha)} = \frac{b}{a}$$

PROPIEDADES DEL MÓDULO (RESIDUO)	
Propiedad neutro	(a % b) % b = a % b
Propiedad asociativa en multiplicación	(ab) % c = ((a % c)(b % c)) % c
Propiedad asociativa en suma	(a + b) % c = ((a % c) + (b % c)) % c

CONSTANTES	
Pi	$\pi = acos(-1) \approx 3.14159$
e	$e \approx 2.71828$
Número áureo	$\phi = \frac{1+\sqrt{5}}{2} \approx 1.61803$

§6.20.2 formulas.cpp

06_Matematicas/Teoria_y_formulas

```
//Stirling number of the second kind is the number of ways to
partition a set of n objects into k non-empty subsets.
S(n,k)=k·s(n-1,k)+S(n-1,k-1),where S(0,0)=1,s(n,0)=s(0,n)=0
S(n,2)=2^(n-1)-1
S(n,k)·k!=number of ways to color n nodes using colors from 1 to k
such that each color is used at least once.
An r-associated Stirling number of the second kind is the number of
ways to partition a set of n objects into k subsets,
with each subset containing at least r elements.It is denoted by
Sr(n,k) and obeys the recurrence relation.
Sr(n+1,k)=kSr(n,k)+C(n,r-1)Sr(n-r+1,k-1)
//The Stirling numbers of the first kind count permutations according
to their number of cycles (counting fixed points as cycles of length
one).
S(n,k) counts the number of permutations of n elements with k
disjoint cycles.
S(n,k) = (n-1)·S(n-1,k)+S(n-1,k-1),where,s(0,0) =1,S(n,0)=s(0,n)=0
Sum(k,0,n) S(n,k) = n!
The unsigned Stirling numbers may also be defined algebraically, as
the coefficient of the rising factorial: x^(~n)=x(x+1)···
(x+n−1)=Sum(k,0,n)s(n,k)x^k
//Bell number count the number of partition of a set
Bn+1 = Sum(k,0,n){C(n,k)*Bk}
Bn = S Sum(k,0,n)Sr(n,k), where Sr is Stirling number of 2kind
//Formally, for a sequence of numbers {ai}, we define the ordinary
generating function (OGF) of a to be A(x)=Sum(i,0,inf)aix^i.
1/(1−x) = 1+ x + x^2+...= Sum(n,0,inf)x^n
−ln(1−x)=x + x^2/2 + x^3/3+...=Sum(n,0,inf)x^n/n
e^x=1+x + x^2/2! + x^3/3!+...=Sum(n,0,inf)x^n/n!
(1−x)^−k=C(k−1,0)x^0+C(k,1)x^1+C(k+1,2)x^2+...=Sum(n,0,inf)C(n+k-
1,n)x^n
For OGF, C(x)=A(x)^k     generates the sequence
cn=Sum(i1...ik,i1+i2+...+ik=n)(ai1*ai2...*aik)
For EGF, C(x)=A(x)^k     generates the sequence
cn=Sum(i1...ik,i1+i2+...+ik=n)(ai1*ai2...*aik)*n!/(i1!*...ik!)
Suppose want to generate the sequence cn=a0+a1+...+an. Then, we can
take C(x)=1/(1−x)*A(x).
```

§6.20.3 math-concepts.txt

06_Matematicas/Teoria_y_formulas

```
Binary exponentiation:
An algorithm to do the exponentiation of:
a^b
where b -> is a large number, in a O(log(b)) instead of O(b).
algorithm: file binpow.cpp
Homogeneous coordinates:
We have a point in the space in the syntax:
P(x_1, x_2, ..., x_n)
in the cartesian space, but if we want to explain it more, for
project the geometry of any point, and work with matrix, instead
of vectors.
So for do that, we have something called homogeneous coordinates,
with represent a point in a n+1 variables in the space to add a
scalar time for any point, where the scalar must be diferent of 0.

So the result would be:
P'(x_1*z, x_2*z, ..., x_n*z, x_n+1=z)
where z is the scalar factor of the point:
P(x_1, x_2, ..., x_n)


Fibonacci Numbers:
Properties:
1. Cassini's identity:
    F_n-1*F_n+1 - F_n^2 = (-1)^n
2. Addition rule:
    F_n+k = =F_k*F_n+1 + F_k-1*F_n
3. F_nk is multiple of F_n
4. GCD identity:
    GCD(F_m, F_n) = F_gcd(m,n)
5.

You have a grid to fill with tiles of 1x2 and 2x1 properties:
always possible if n%2 = 0 and think of a chess board (a tile
always occupies 1 black and 1 white)
```

§6.20.4 pi.cpp

06_Matematicas/Teoria_y_formulas

```
using ld = long double;
const ld PI = acos(-1);
```

§6.20.5 sequences-extra.tex

06_Matematicas/Teoria_y_formulas

Listado de secuencias mas comunes y como hallarlas.

Estrellas octangulares	0, 1, 14, 51, 124, 245, 426, 679, 1016, 1449, 1990, 2651, ...
	$f(n) = n * (2 * n^2 - 1)$.
22cm Euler totient	1, 1, 2, 2, 4, 2, 6, 4, 6, 4, 10, 4, 12, 6,...
	$f(n) =$ Cantidad de números naturales $\leq n$ coprimos con n.
Números de Bell	1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, ...
	Se inicia una matriz triangular con f[0][0] = f[1][0] = 1. La suma de estos dos se guarda en f[1][1] y se traslada a f[2][0]. Ahora se suman f[1][0] con f[2][0] y se guarda en f[2][1]. Luego se suman f[1][1] con f[2][1] y se guarda en f[2][2] trasladandose a f[3][0] y así sucesivamente. Los valores de la primera columna contienen la respuesta.
22cm Números de Catalán	1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, ...
	$f(n) = \frac{(2n)!}{(n+1)!n!}$
Números de Fermat	3, 5, 17, 257, 65537, 4294967297, 18446744073709551617, ...
	$f(n) = 2^{(2^n)} + 1$
22cm Números de Fibonacci	0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...
	$f(0) = 0; f(1) = 1; f(n) = f(n-1) + f(n-2)$ para $n > 1$
22cm Números de Lucas	2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, ...
	$f(0) = 2; f(1) = 1; f(n) = f(n-1) + f(n-2)$ para $n > 1$
Números de Pell	0, 1, 2, 5, 12, 29, 70, 169, 408, 985, 2378, 5741, 13860, ...
	$f(0) = 0; f(1) = 1; f(n) = 2f(n-1) + f(n-2)$ para $n > 1$
22cm Números de Tribonacci	0, 0, 1, 1, 2, 4, 7, 13, 24, 44, 81, 149, 274, 504, ...
	$f(0) = f(1) = 0; f(2) = 1; f(n) = f(n-1) + f(n-2) + f(n-3)$ para $n > 2$
Números factoriales	1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, ...
	$f(0) = 1; f(n) = \prod_{k=1}^{n} k$ para $n > 0$.
Números piramidales cuadrados	0, 1, 5, 14, 30, 55, 91, 140, 204, 285, 385, 506, 650, ...
	$f(n) = \frac{n * (n+1) * (2 * n + 1)}{6}$
Números primos de Mersenne	3, 7, 31, 127, 8191, 131071, 524287, 2147483647, ...
	$f(n) = 2^{p(n)} - 1$ donde p representa valores primos iniciando en $p(0) = 2$.
Números tetraedrales	1, 4, 10, 20, 35, 56, 84, 120, 165, 220, 286, 364, 455, ...
	$f(n) = \frac{n * (n+1) * (n+2)}{6}$
Números triangulares	0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66, 78, 91, 105, ...
	$f(n) = \frac{n(n+1)}{2}$
OEIS A000127	1, 2, 4, 8, 16, 31, 57, 99, 163, 256, 386, 562, ...
	$f(n) = \frac{(n^4 - 6n^3 + 23n^2 - 18n + 24)}{24}$.

Secuencia de Narayana	1, 1, 1, 2, 3, 4, 6, 9, 13, 19, 28, 41, 60, 88, 129, ...
	$f(0) = f(1) = f(2) = 1; f(n) = f(n-1) + f(n-3)$ para todo $n > 2$.
22cm Secuencia de Silvestre	2, 3, 7, 43, 1807, 3263443, 10650056950807, ...
	$f(0) = 2; f(n+1) = f(n)^2 - f(n) + 1$
Secuencia de vendedor perezoso	1, 2, 4, 7, 11, 16, 22, 29, 37, 46, 56, 67, 79, 92, 106, ...
	Equivale al triangular(n) + 1. Máxima número de piezas que se pueden formar al hacer n cortes a un disco. $f(n) = \frac{n(n+1)}{2} + 1$
Suma de los divisores de un número	1, 3, 4, 7, 6, 12, 8, 15, 13, 18, 12, 28, 14, 24, ...
	Para todo $n > 1$ cuya descomposición en factores primos es $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ se tiene que: $f(n) = \frac{p_1^{a_1+1} - 1}{p_1 - 1} * \frac{p_2^{a_2+1} - 1}{p_2 - 1} * \ldots * \frac{p_k^{a_k+1} - 1}{p_k - 1}$

§6.20.6 theo.cpp

06_Matematicas/Teoria_y_formulas

```
Erdős–Gallai theorem
A sequence of non-negative integers d1>=...>=d_n can be represented
as the degree sequence of a finite simple graph on n vertices if and
only if
d1+...+dn is even and sum(i=1->k, di) <= k*(k - 1) + sum(i=k+1->n,
min(di, k))
holds for every k in 1<=k<=n
teorema de Erdős-Szekeres
Para r, s dados, cualquier sucesión de longitud al menos (r-1)(s-1)+1
contiene
una subsucesión monótonamente creciente de longitud r o una
subsucesión
monótonamente decreciente de longitud s
The harmonic lemma
(n/1, n/2, ..., n/n) has at most 2*sqrt(n) different elements
greatest x that satisfy d(x) = d(i) is n / (n / i)
(Tutte) A graph, G = (V, E), has a perfect matching if and only if
for every subset U of V, the subgraph induced by V - U has at most
|U| connected components with an odd number of vertices.
Petersens Theorem. Every cubic, bridgeless graph contains a perfect
matching.
(Dilworth) In any finite partially ordered set, the maximum number of
elements in any antichain equals the minimum number of chains in any
partition of the set into chains
Pick: A=I+B/2-1  (area of polygon, points inside, points on border)
(de Brujin) (k!)^(k(n-1))/k^n, # of de Brujin sequences, remove
fractional part to consider rotated strings
```

§6.20.7 theorems-vasito.txt

06_Matematicas/Teoria_y_formulas

```
(Tutte) A graph, G = (V, E), has a perfect matching if and only if
for every subset U of V, the subgraph induced by V - U has at most
|U| connected components with an odd number of vertices.
Petersens Theorem. Every cubic, bridgeless graph contains a
perfect matching.
(Dilworth) In any finite partially ordered set, the maximum number
of elements in any antichain equals the minimum number of chains
in any partition of the set into chains
Pick: A=I+B/2-1  (area of polygon, points inside, points on
border)
```

§6.20.8 theorems.tex

06_Matematicas/Teoria_y_formulas

Erdős–Szekeres Theorem

This theorem is related to increasing and decreasing sequences.

Suppose $a, b \in \mathbb{N}$, $n = ab + 1$, and $x_1, x_2, \ldots, x_n$ is a sequence of n real numbers. Then this sequence contains a monotonic increasing (decreasing) subsequence of $a + 1$ terms or a monotonic decreasing (increasing) subsequence of $b + 1$ terms. Dilworth's lemma is a generalization of this theorem.

Grundy Numbers in Game Theory

Grundy numbers are used in game theory to analyze games that can be represented as directed state graphs. In these graphs, if a player loses in a state, its Grundy number is zero; otherwise, it is a positive number. The Grundy number for each vertex is defined as:

$$\text{Grundy}(\text{losing state with no moves}) = 0$$
$$\text{Grundy}(\text{vertex}) = \text{MEX}(\text{adjacent_nodes}[\text{vertex}])$$

where MEX stands for the "minimum excludant," which is the smallest non-negative integer not present in the set of Grundy numbers of adjacent nodes.

If you have multiple independent games, the final Grundy number is calculated as:

$$\text{Grundy}(\text{game}_1) \oplus \text{Grundy}(\text{game}_2) \oplus \text{Grundy}(\text{game}_3) \oplus \cdots \oplus \text{Grundy}(\text{game}_n)$$

where $\oplus$ denotes the bitwise XOR operation.

§7 Geometría

§7.1 Círculos y triángulos

§7.1.1 circle-extra.cpp

07_Geometria/Circulos_y_triangulos

```
#include "../Puntos_rectas_y_segmentos/point-extra.cpp"
#include "../Puntos_rectas_y_segmentos/line-extra.cpp"
struct circle {
    pt c; T r;
};
// (x-xo)^2 + (y-yo)^2 = r^2
// circle that passes through abc
circle center(pt a, pt b, pt c) {
    b = b-a, c = c-a;
    assert(cross(b, c) != 0); // no circumcircle if A, B, C aligned
    pt cen = a + rot90ccw(b*norm(c) - c*norm(b))/cross(b, c)/2;
    return {cen, abs(a-cen)};
}
// centers of the circles that pass through ab and have radius r
vector<pt> centers(pt a, pt b, T r) {
    if (abs(a-b) > 2*r + eps) return {};
    pt m = (a+b)/2;
    double f = sqrt(r*r/norm(a-m) - 1);
    pt c = rot90ccw(a-m)*f;
    return {m-c, m+c};
}
int inter_cl(circle c, line l, pair<pt, pt> &out) {
    lf h2 = c.r*c.r - l.sq_dist(c.c);
    if (h2 >= 0) { // line touches circle
        pt p = l.proj(c.c);
        pt h = l.v*sqrt(h2)/abs(l.v); // vector of len h parallel to
line
        out = {p-h, p+h};
    }
    return 1 + sign(h2); // if 1 -> out.F == out.S
}
int inter_cc(circle c1, circle c2, pair<pt, pt> &out) {
    pt d = c2.c-c1.c;
    double d2 = norm(d);
    if (d2 == 0) { assert(c1.r != c2.r); return 0; } // concentric
circles (identical)
    double pd = (d2 + c1.r*c1.r - c2.r*c2.r)/2; // = |O_1P| * d
    double h2 = c1.r*c1.r - pd*pd/d2; // = h^2
    if (h2 >= 0) {
        pt p = c1.c + d*pd/d2, h = rot90ccw(d)*sqrt(h2/d2);
        out = {p-h, p+h};
    }
    return 1 + sign(h2);
}
// circle-line inter = 1
int tangents(circle c1, circle c2, bool inner, vector<pair<pt, pt>>
&out) {
    if (inner) c2.r = -c2.r; // inner tangent
    pt d = c2.c-c1.c;
    double dr = c1.r-c2.r, d2 = norm(d), h2 = d2-dr*dr;
    if (d2 == 0 || h2 < 0) { assert(h2 != 0); return 0; } //
(identical)
    for (double s : {-1, 1}) {
        pt v = (d*dr + rot90ccw(d)*sqrt(h2)*s)/d2;
        out.pb({c1.c + v*c1.r, c2.c + v*c2.r});
    }
    return 1 + (h2 > 0); // if 1: circle are tangent
}
// circle targent passing through pt p
int tangent_through_pt(pt p, circle c, pair<pt, pt> &out) {
    double d = abs(p - c.c);
    if (d < c.r) return 0;
    pt base = c.c-p;
    double w = sqrt(norm(base) - c.r*c.r);
    pt a = {w, c.r}, b = {w, -c.r};
    pt s = p + base*a/norm(base)*w;
    pt t = p + base*b/norm(base)*w;
    out = {s, t};
    return 1 + (abs(c.c-p) == c.r);
}
```

§7.1.2 heron-formula.cpp

07_Geometria/Circulos_y_triangulos

```
using ld = long double;
ld triangle_area(ld a, ld b, ld c) {
    ld s = (a + b + c) / 2;
    return sqrtl(s * (s - a) * (s - b) * (s - c));
}
```

§7.1.3 triangles-and-circles.cpp

07_Geometria/Circulos_y_triangulos

```
// Requisito: Errores previos: if(a.b=0) asigna, perp no devuelve res
y se escribe l2 dos veces en circumCircle. Revisar fórmulas, tipos y
dependencias de rectas/puntos.
double rInCircle(double ab, double bc, double ca) {
    double s = (ab+bc+ca)/2;
    return sqrt(s*(s-ab)*(s-bc)*(s-ca));
}
double areaTri1(double a, double b, double c) {//heron
    double s = (a+b+c)/2;
    return sqrt(s*(s-a)*(s-b)*(s-c));
}
line perp(line a, point p) {//perpendicular
    line res;
    if(a.b=0)
        res.a = 0, res.b = 1, res.c = -p.y;
    else
        if(fabs(a.a)<EPS)
            res.a = 1, res.b = 0, res.c = -p.y;
        else
            res.a = -1.0/a.a, res.b = 1, res.c = -res.a*p.x-p.y;
}
bool circumCircle(point a, point b, point c, point &ctr, double &r)
```

```
{//circuncentro completo
    double area = areaTri(a,b,c);
    if(fabs(area)<EPS) return 0;
    line l1, l2;
    pointsToLine(a,b,l2);
    pointsToLine(a,c,l2);
    point p1 = point((a.x+b.x)/2.0,(a.y+b.y)/2.0), p2 =
point((a.x+c.x)/2.0,(a.y+c.y)/2.0);
    l1 = perp(l1,p1), l2 = perp(l2,p2);
    areIntersect(l1,l2,ctr);
    r = dist(a,b)*dist(b,c)*dist(a,c)/(4.0*areaTri(a,b,c));
    return true;
}
bool isInCircum(point a, point b, point c, point p) {//si esta dentro
del circulo circunscrito
    double r;
    point ctr;
    if(!circumCircle(a,b,c,ctr,r)) return false;
    return dist(ctr,p) <= r ;
}
bool inCircle(point a, point b, point c, point &ctr) {//incentro
    double r = rIncircle(a,b,c);
    if(r< EPS) return false;
    line l1,l2;
    point p1;
    double ratio = dist(a,b) / dist(a,c);
    p1 = translate(b, scale(tovec(b,c),ratio/(1+ratio)));
    pointsToLine(a,p1,l1);
    ratio = dist(b,a) / dist(b,c);
    p1 = translate(a, scale(tovec(a,c),ratio/(1+ratio)));
    pointsToLine(b,p1,l2);
    areIntersect(l1,l2,ctr);
    return true;
}
line toLinep(point a, point b, point c) {//para mediatriz
    line l;
    if(b.x == c.x)
        l.a = 0, l.b = 1 , l.c = -a.y;
    else if(b.y == c.y)
        l.a = 1, l.b = 0, l.c = -a.x;
    else
        l.a = 1/((b.y-a.y)/(b.x-a.x)), l.b = 1, l.c = -l.a*a.x-a.y;
    return l;
}
point circun(point a, point b, point c) {//circuncentro
    line l1, l2;
    l1 = toLinep(point((a.x+b.x)/2,(a.y+b.y)/2),a,b);
    l2 = toLinep(point((a.x+c.x)/2,(a.y+c.y)/2),a,c);
    areIntersect(l1,l2,a);
    return a;
}
bool circle2PtsRad(point a, point b, double r, point &c) {//dados 2
puntos y un radio
    double det = (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
    det = r * r / det - 0.25;
    if(det < 0.0) return false;
    det = sqrt(det);
    c.x = (a.x + b.x) * 0.5 + (b.y-a.y) * det;
    c.y = (a.y + b.y) * 0.5 + (a.x-b.x) * det;
    return true;
}
```

§7.2 Convex Hull

§7.2.1 convexhull.cpp

07_Geometria/Convex_Hull

```
struct pt {
    int x, y;
    pt(int a = 0, int b = 0) {x = a, y = b;}
    int operator == (pt const &o) const {
        return x == o.x && y == o.y;
    }
    pt operator - (pt const &o) {return {x - o.x, y - o.y};}
};
int dist(pt &a, pt &b) {
    int x = a.x - b.x, y = a.y - b.y;
    return x*x + y*y;
}
int orientation(pt &a, pt &b, pt &c) {
    int v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v<0) return -1;
    return v? 1: 0; // 0 colineal, 1 b esta a la derecha de a, c
}
int cw(pt &a, pt &b, pt &c, int colli = 0) {
    int o = orientation(a, b, c);
    return o<0 || (colli && !o);
}
int collinear(pt &a, pt &b, pt &c) {return !orientation(a, b, c);}
void convexhull(vector<pt> &v, int inclu) {
    pt p0 = *min_element(v.begin(), v.end(), [](pt a, pt b){
            return make_pair(a.y, a.x)<make_pair(b.y, b.x);
            });
    sort(v.begin(), v.end(), [&p0](pt &a, pt &b){
                int o = orientation(p0, a, b);
                if (!o) return dist(p0, a)<dist(p0, b);
                return o<0;
            });
    if (inclu) {
        int i = v.size()-1;
        while (i > -1 && collinear(p0, v[i], v.back())) i--;
        reverse(v.begin() + 1 + i, v.end());
    }
    vector<pt> st;
    for (int i= 0; i<v.size(); i++) {
        while (st.size()>1 && !cw(st[st.size()-2], st.back(), v[i],
inclu)) st.pop_back();
        st.push_back(v[i]);
    }
    if (!inclu && st.size() == 2 && st[0] == st[1]) st.pop_back();
    v = st;
    // devuelve los puntos sobre el mismo vector, en lado de reloj;
}
/*
    para calcular si 2 segmentos intersectan, hacer convex hull y
caso especial es puntos colineales
*/
```

§7.3 Delaunay y Voronoi

§7.3.1 stanford-delaunay.cpp

07_Geometria/Delaunay_y_Voronoi

```
#include <vector>
// Slow but simple Delaunay triangulation. Does not handle
// degenerate cases (from O'Rourke, Computational Geometry in C)
// Running time: O(n^4)
// INPUT:    x[] = x-coordinates
//           y[] = y-coordinates
// OUTPUT:   triples = a vector containing m triples of indices
//                     corresponding to triangle vertices
using namespace std;
typedef double T;
struct triple {
    int i, j, k;
    triple() {}
    triple(int i, int j, int k) : i(i), j(j), k(k) {}
};
vector<triple> delaunayTriangulation(vector<T>& x, vector<T>& y) {
    int n = x.size();
    vector<T> z(n);
    vector<triple> ret;
    for (int i = 0; i < n; i++)
        z[i] = x[i] * x[i] + y[i] * y[i];
    for (int i = 0; i < n-2; i++) {
        for (int j = i+1; j < n; j++) {
        for (int k = i+1; k < n; k++) {
            if (j == k) continue;
            double xn = (y[j]-y[i])*(z[k]-z[i]) - (y[k]-y[i])*(z[j]-
z[i]);
            double yn = (x[k]-x[i])*(z[j]-z[i]) - (x[j]-x[i])*(z[k]-
z[i]);
            double zn = (x[j]-x[i])*(y[k]-y[i]) - (x[k]-x[i])*(y[j]-
y[i]);
            bool flag = zn < 0;
            for (int m = 0; flag && m < n; m++)
            flag = flag && ((x[m]-x[i])*xn +
                    (y[m]-y[i])*yn +
                    (z[m]-z[i])*zn <= 0);
            if (flag) ret.push_back(triple(i, j, k));
        }
        }
    }
    return ret;
}
signed main()
{
    T xs[]={0, 0, 1, 0.9};
    T ys[]={0, 1, 0, 0.9};
    vector<T> x(&xs[0], &xs[4]), y(&ys[0], &ys[4]);
    vector<triple> tri = delaunayTriangulation(x, y);
    //expected: 0 1 3
    //          0 3 2
    int i;
    for(i = 0; i < tri.size(); i++)
        printf("%lld %lld %lld\n", tri[i].i, tri[i].j, tri[i].k);
    return 0;
}
```

§7.3.2 voronoy.cpp

07_Geometria/Delaunay_y_Voronoi

```
// Voronoi / Delaunay por barrido (Fortune), O(n log n). Falla si dos
puntos tienen la misma x:
// rotar la entrada 1 radián. Propiedades de Delaunay: el
circuncírculo de cada triángulo no
// contiene puntos de S; <= 3n-6 aristas; cada punto es adyacente a
su vecino más cercano; el
// MST euclidiano está contenido en sus aristas; maximiza el ángulo
mínimo. Vecino más cercano
// de q consultas: barrer el diagrama, O((n+q) log(n+q)) offline.
#define ll long long
#define pii pair<int, int>
using namespace std;
template<typename T>
using minpq = priority_queue<T, vector<T>, greater<T>>;
using ftype = long double;
const ftype EPS = 1e-12, INF = 1e100;
struct pt {
    ftype x, y;
    pt(ftype x = 0, ftype y = 0) : x(x), y(y) {}
    // vector addition, subtraction, scalar multiplication
    pt operator+(const pt &o) const {
        return pt(x + o.x, y + o.y);
    }
    pt operator-(const pt &o) const {
        return pt(x - o.x, y - o.y);
    }
    pt operator*(const ftype &f) const {
        return pt(x * f, y * f);
    }
    // rotate 90 degrees counter-clockwise
    pt rot() const {
```

```
        return pt(-y, x);
    }
    // dot and cross products
    ftype dot(const pt &o) const {
        return x * o.x + y * o.y;
    }
    ftype cross(const pt &o) const {
        return x * o.y - y * o.x;
    }
    // Length
    ftype len() const {
        return hypotl(x, y);
    }
    // compare points lexicographically
    bool operator<(const pt &o) const {
        return make_pair(x, y) < make_pair(o.x, o.y);
    }
};
// check if two vectors are collinear. It might make sense to use a
// different EPS here, especially if points have integer coordinates
bool collinear(pt a, pt b) {
    return abs(a.cross(b)) < EPS;
}
// intersection point of lines ab and cd. Precondition is that they
aren't collinear
pt lineline(pt a, pt b, pt c, pt d) {
    return a + (b - a) * ((c - a).cross(d - c) / (b - a).cross(d -
c));
}
// circumcircle of points a, b, c. Precondition is that abc is a non-
degenerate triangle.
pt circumcenter(pt a, pt b, pt c) {
    b = (a + b) * 0.5;
    c = (a + c) * 0.5;
    return lineline(b, b + (b - a).rot(), c, c + (c - a).rot());
}
// x coordinate of sweep-line
ftype sweepx;
// an arc on the beacah line is given implicitly by the focus p,
// the focus q of the following arc, and the position of the sweep-
line.
struct arc {
    mutable pt p, q;
    mutable int id = 0, i;
    arc(pt p, pt q, int i) : p(p), q(q), i(i) {}
    // get y coordinate of intersection with following arc.
    // don't question my magic formulas
    ftype gety(ftype x) const {
        if(q.y == INF) return INF;
        x += EPS;
        pt med = (p + q) * 0.5;
        pt dir = (p - med).rot();
        ftype D = (x - p.x) * (x - q.x);
        return med.y + ((med.x - x) * dir.x + sqrtl(D) * dir.len()) /
dir.y;
    }
    bool operator<(const ftype &y) const {
        return gety(sweepx) < y;
    }
    bool operator<(const arc &o) const {
        return gety(sweepx) < o.gety(sweepx);
    }
};
// the beach line will be stored as a multiset of arc objects
using beach = multiset<arc, less<>>;
// an event is given by
//     x: the time of the event
//     id: If >= 0, it's a point event for index id.
//         If < 0, it's an ID for a vertex event
//     it: if a vertex event, the iterator for the arc to be deleted
struct event {
    ftype x;
    int id;
    beach::iterator it;
    event(ftype x, int id, beach::iterator it) : x(x), id(id), it(it)
{}
    bool operator<(const event &e) const {
        return x > e.x;
    }
};
struct fortune {
    beach line; // self explanatory
    vector<pair<pt, int>> v; // (point, original index)
    priority_queue<event> Q; // priority queue of point and vertex
events
    vector<pii> edges; // delaunay edges
    vector<bool> valid; // valid[-id] == true if the vertex event
with corresponding id is valid
    int n, ti; // number of points, next available vertex ID
    fortune(vector<pt> p) {
        n = ((int) (p).size());
        v.resize(n);
        for(int i = (0); i < (n); i++) v[i] = {p[i], i};
        sort(all(v)); // sort points by coordinate, remember original
indices for the delaunay edges
    }
    // update the remove event for the arc at position it
    void upd(beach::iterator it) {
        if(it->i == -1) return; // doesn't correspond to a real point
        valid[-it->id] = false; // mark existing remove event as
invalid
        auto a = prev(it);
        if(collinear(it->q - it->p, a->p - it->p)) return; // doesn't
generate a vertex event
        it->id = --ti; // new vertex event ID
        valid.push_back(true); // label this ID true
        pt c = circumcenter(it->p, it->q, a->p);
        ftype x = c.x + (c - it->p).len();
        // event is generated at time x.
        // make sure it passes the sweep-line, and that the arc truly
shrinks to 0
        if(x > sweepx - EPS && a->gety(x) + EPS > it->gety(x)) {
            Q.push(event(x, it->id, it));
        }
    }
    // add Delaunay edge
    void add_edge(int i, int j) {
        if(i == -1 || j == -1) return;
        edges.push_back({v[i].second, v[j].second});
    }
    // handle a point event
    void add(int i) {
        pt p = v[i].first;
        // find arc to split
        auto c = line.lower_bound(p.y);
        // insert new arcs. passing the following iterator gives a
slight speed-up
        auto b = line.insert(c, arc(p, c->p, i));
        auto a = line.insert(b, arc(c->p, p, c->i));
        add_edge(i, c->i);
        upd(a); upd(b); upd(c);
    }
    // handle a vertex event
    void remove(beach::iterator it) {
        auto a = prev(it);
        auto b = next(it);
        line.erase(it);
        a->q = b->p;
        add_edge(a->i, b->i);
        upd(a); upd(b);
    }
    // X is a value exceeding all coordinates
    void solve(ftype X = 1e9) {
        // insert two points that will always be in the beach line,
        // to avoid handling edge cases of an arc being first or last
        X *= 3;
        line.insert(arc(pt(-X, -X), pt(-X, X), -1));
        line.insert(arc(pt(-X, X), pt(INF, INF), -1));
        // create all point events
        for(int i = (0); i < (n); i++) {
            Q.push(event(v[i].first.x, i, line.end()));
        }
        ti = 0;
        valid.assign(1, false);
        while(!Q.empty()) {
            event e = Q.top(); Q.pop();
            sweepx = e.x;
            if(e.id >= 0) {
                add(e.id);
            }else if(valid[-e.id]) {
                remove(e.it);
            }
        }
    }
};
```

§7.4 Geometría 3D

§7.4.1 a-geo3d-esencial.cpp

07_Geometria/Geometria_3D

```
// IPoint3 (long long, |coord| <= 1e9): predicados EXACTOS;
dot/norm2/triple en __int128, cross en ll.
// Point3 (long double): construcciones (proyecciones,
intersecciones, raíces); EPS3 según la escala.
// Pruebas: _pruebas/geometria-3d.cpp. Guía y tabla «¿Qué uso?»:
README.md.
using ll = long long;
using ld = long double;
using i128 = __int128;
const ld EPS3 = 1e-9;
const ld PI3 = acosl(-1.0L);
// Tipo "ancho" para productos: __int128 si T es entero, T si es
flotante.
template<class T> using Wide = conditional_t<is_floating_point_v<T>,
T, i128>;
// Uso: signo con tolerancia para flotantes y exacto para enteros
(incluye __int128).
// Retorna: -1, 0 o 1. Complejidad: O(1).
template<class T> int sgn3(T x) {
    if constexpr (is_floating_point_v<T>) return (x > EPS3) - (x < -
EPS3);
    else return (x > 0) - (x < 0);
}
// Uso: imprimir un __int128 (cout no lo soporta).
string toString128(i128 v) {
    if (v == 0) return "0";
    bool neg = v < 0; string s;
    while (v != 0) { int d = (int)(v % 10); s += char('0' + (d < 0 ?
-d : d)); v /= 10; }
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
// 1. PUNTO / VECTOR 3D
template<class T> struct P3 {
    T x = 0, y = 0, z = 0;
    P3() {}
    P3(T x_, T y_, T z_) : x(x_), y(y_), z(z_) {}
```

```
    template<class U> explicit P3(const P3<U>& o) : x(T(o.x)),
y(T(o.y)), z(T(o.z)) {}
    P3 operator+(const P3& o) const { return {x + o.x, y + o.y, z +
o.z}; }
    P3 operator-(const P3& o) const { return {x - o.x, y - o.y, z -
o.z}; }
    P3 operator-() const { return {-x, -y, -z}; }
    P3 operator*(T k) const { return {x * k, y * k, z * k}; }
    P3 operator/(T k) const { return {x / k, y / k, z / k}; }      //
entero: división truncada
    P3& operator+=(const P3& o) { return *this = *this + o; }
    P3& operator-=(const P3& o) { return *this = *this - o; }
    T& operator[](int i) { return i == 0 ? x : (i == 1 ? y : z); }
    T operator[](int i) const { return i == 0 ? x : (i == 1 ? y : z);
}
    bool operator==(const P3& o) const { return x == o.x && y == o.y
&& z == o.z; }
    bool operator!=(const P3& o) const { return !(*this == o); }
    bool operator<(const P3& o) const {
// lexicográfico (x,y,z)
        if (x != o.x) return x < o.x;
        if (y != o.y) return y < o.y;
        return z < o.z;
    }
};
using Point3 = P3<ld>;   // construcciones con raíces / divisiones
using IPoint3 = P3<ll>;  // predicados exactos
template<class T> ostream& operator<<(ostream& os, const P3<T>& p) {
    return os << "(" << p.x << ", " << p.y << ", " << p.z << ")";
}
template<class T> istream& operator>>(istream& is, P3<T>& p) { return
is >> p.x >> p.y >> p.z; }
// Uso: producto punto a·b. Retorna: exacto (__int128) con enteros.
O(1).
// Signo: >0 ángulo agudo, 0 perpendiculares, <0 obtuso.
template<class T> Wide<T> dot(const P3<T>& a, const P3<T>& b) {
    using W = Wide<T>;
    return W(a.x) * b.x + W(a.y) * b.y + W(a.z) * b.z;
}
// Uso: producto cruz a×b (perpendicular a ambos, |a×b| = área del
paralelogramo). O(1).
// Retorna: P3<T>. Con long long es exacto si |componentes| <= 2e9
(cada término <= 4e18).
template<class T> P3<T> cross(const P3<T>& a, const P3<T>& b) {
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y -
a.y * b.x};
}
// Uso: producto triple a·(b×c) = det[a;b;c] = volumen orientado del
paralelepípedo.
// Retorna: exacto con enteros (todo en __int128, |comp| <= 2e9 ->
|res| <= ~5e28). O(1).
// Ejemplo: triple({1,0,0},{0,1,0},{0,0,1}) = 1.
template<class T> Wide<T> triple(const P3<T>& a, const P3<T>& b,
const P3<T>& c) {
    using W = Wide<T>;
    return W(a.x) * (W(b.y) * c.z - W(b.z) * c.y)
         + W(a.y) * (W(b.z) * c.x - W(b.x) * c.z)
         + W(a.z) * (W(b.x) * c.y - W(b.y) * c.x);
}
// Uso: determinante 3x3 por filas. Igual a triple(fila0, fila1,
fila2).
template<class T> Wide<T> det3(const P3<T>& r0, const P3<T>& r1,
const P3<T>& r2) { return triple(r0, r1, r2); }
// Uso: norma cuadrada |a|^2 (exacta con enteros). Comparar
distancias SIN sqrt.
template<class T> Wide<T> norm2(const P3<T>& a) { return dot(a, a); }
// Uso: norma |a| (aproximada, long double).
template<class T> ld norm(const P3<T>& a) { return
sqrtl((ld)norm2(a)); }
template<class T> Wide<T> dist2(const P3<T>& a, const P3<T>& b) {
return norm2(a - b); }
template<class T> ld dist(const P3<T>& a, const P3<T>& b) { return
norm(a - b); }
// Uso: true si todas las componentes son 0 (exacto) o |c| <= EPS3
(flotante).
template<class T> bool isZero(const P3<T>& a) { return !sgn3(a.x) &&
!sgn3(a.y) && !sgn3(a.z); }
// Uso: vector unitario. Requiere a != 0 (si no, devuelve (0,0,0)).
inline Point3 unit(const Point3& a) { ld n = norm(a); return n < EPS3
? Point3() : a / n; }
// Uso: proyección del vector u sobre la dirección v (v != 0).
Retorna (u·v/|v|^2) v.
inline Point3 projVec(const Point3& u, const Point3& v) { return v *
(dot(u, v) / norm2(v)); }
// Uso: componente de u perpendicular a v.
inline Point3 rejectVec(const Point3& u, const Point3& v) { return u
- projVec(u, v); }
// Uso: un vector NO nulo perpendicular a v (v != 0). Exacto con
enteros. O(1).
// Hace v × (eje menos alineado con v): bien condicionado también con
flotantes.
// Ejemplo: anyPerp({0,0,5}) = {0,5,0}.
template<class T> P3<T> anyPerp(const P3<T>& v) {
    T ax = v.x < 0 ? -v.x : v.x, ay = v.y < 0 ? -v.y : v.y, az = v.z
< 0 ? -v.z : v.z;
    if (ax <= ay && ax <= az) return {0, v.z, -v.y};   // v × (1,0,0)
    if (ay <= az) return {-v.z, 0, v.x};               // v × (0,1,0)
    return {v.y, -v.x, 0};                             // v × (0,0,1)
}
// Uso: ángulo entre u y v en [0, pi]. Usa atan2(|u×v|, u·v): estable
cerca de 0 y pi
// (acos pierde precisión ahí y necesita recortar a [-1,1]). Vector
nulo -> 0.
template<class T> ld angle(const P3<T>& u, const P3<T>& v) {
    return atan2l(norm(cross(Point3(u), Point3(v))), (ld)dot(u, v));
}
// Uso: acos protegido (por si se necesita). Recorta a [-1,1] antes
de acos.
inline ld safeAcos(ld c) { return acosl(max((ld)-1, min((ld)1, c)));
}
inline ld safeSqrt(ld x) { return sqrtl(max((ld)0, x)); }
inline ld toRad(ld deg) { return deg * PI3 / 180; }
inline ld toDeg(ld rad) { return rad * 180 / PI3; }
// Uso: interpolación a + (b-a)t. t=0 -> a, t=1 -> b, t=0.5 -> punto
medio.
inline Point3 lerp(const Point3& a, const Point3& b, ld t) { return a
+ (b - a) * t; }
inline Point3 midpoint(const Point3& a, const Point3& b) { return (a
+ b) / 2; }
// Truco exacto: el punto medio de enteros puede no ser entero; usar
2*M = a+b.
template<class T> P3<T> midpoint2(const P3<T>& a, const P3<T>& b) {
return a + b; }
// ---------------- Predicados (exactos con IPoint3) ----------------
// Uso: orientación de d respecto al plano orientado abc.
// Retorna: +1 si d está del lado hacia el que apunta (b-a)×(c-a)
(desde d, a->b->c se ve
//   antihorario), -1 del otro lado, 0 si coplanares. Exacto con
enteros. O(1).
template<class T> int orient(const P3<T>& a, const P3<T>& b, const
P3<T>& c, const P3<T>& d) {
    return sgn3(triple(b - a, c - a, d - a));
}
template<class T> bool collinear(const P3<T>& a, const P3<T>& b,
const P3<T>& c) { return isZero(cross(b - a, c - a)); }
template<class T> bool coplanar(const P3<T>& a, const P3<T>& b, const
P3<T>& c, const P3<T>& d) { return orient(a, b, c, d) == 0; }
// Paralelos incluye antiparalelos; el vector nulo es paralelo a
todo.
template<class T> bool parallelVec(const P3<T>& u, const P3<T>& v) {
return isZero(cross(u, v)); }
template<class T> bool sameDirection(const P3<T>& u, const P3<T>& v)
{ return parallelVec(u, v) && sgn3(dot(u, v)) > 0; }
template<class T> bool perpendicularVec(const P3<T>& u, const P3<T>&
v) { return sgn3(dot(u, v)) == 0; }
// 2. DISTANCIAS DIRECTAS Y TRUCOS ARITMÉTICOS
// Uso: Manhattan |dx|+|dy|+|dz| y Chebyshev max(|dx|,|dy|,|dz|).
Exactas. O(1).
inline ll manhattan(const IPoint3& a, const IPoint3& b) { return
llabs(a.x - b.x) + llabs(a.y - b.y) + llabs(a.z - b.z); }
inline ll chebyshev(const IPoint3& a, const IPoint3& b) { return
max({llabs(a.x - b.x), llabs(a.y - b.y), llabs(a.z - b.z)}); }
// Uso: máxima distancia Manhattan entre dos puntos de pts. O(4n).
// Truco: |dx|+|dy|+|dz| = max sobre signos (s1,s2,s3) de
s1dx+s2dy+s3dz.
//   Basta fijar s1=+1 (4 combinaciones): para cada una, max(f) -
min(f).
inline ll maxManhattan(const vector<IPoint3>& pts) {
    ll best = 0;
    for (int m = 0; m < 4; m++) {
        ll sy = (m & 1) ? -1 : 1, sz = (m & 2) ? -1 : 1, mx =
LLONG_MIN, mn = LLONG_MAX;
        for (auto& p : pts) { ll f = p.x + sy * p.y + sz * p.z; mx =
max(mx, f); mn = min(mn, f); }
        if (!pts.empty()) best = max(best, mx - mn);
    }
    return best;
}
// Truco: Chebyshev en 3D NO se transforma en Manhattan como en 2D;
pero max Chebyshev
// = max por eje de (max coord - min coord). Distancia a planos
coordenados: |z| a XY, etc.
inline ll maxChebyshev(const vector<IPoint3>& pts) {
    if (pts.empty()) return 0;
    ll best = 0;
    for (int k = 0; k < 3; k++) {
        ll mx = LLONG_MIN, mn = LLONG_MAX;
        for (auto& p : pts) { mx = max(mx, p[k]); mn = min(mn, p[k]);
}
        best = max(best, mx - mn);
    }
    return best;
}
// Uso: cantidad de puntos enteros en el segmento cerrado AB =
gcd(|dx|,|dy|,|dz|) + 1.
// Ejemplo: (0,0,0)-(2,4,6) -> gcd=2 -> 3 puntos. Si A==B -> 1.
inline ll latticePointsSegment(const IPoint3& a, const IPoint3& b) {
    return gcd(gcd(llabs(b.x - a.x), llabs(b.y - a.y)), llabs(b.z -
a.z)) + 1;
}
// Uso: dirección primitiva (dividida por el MCD). v != 0.
//   asLine=true: además fija el signo (primera coordenada no nula >
0) para que v y -v
//   coincidan (rectas). asLine=false conserva el sentido
(rayos/direcciones).
inline IPoint3 normDir(IPoint3 v, bool asLine = true) {
    ll g = gcd(gcd(llabs(v.x), llabs(v.y)), llabs(v.z));
    if (g == 0) return v;
    v = v / g;
    if (asLine && (v.x < 0 || (v.x == 0 && (v.y < 0 || (v.y == 0 &&
v.z < 0))))) v = -v;
    return v;
}
// Uso: número de pares de vectores paralelos (misma recta) en vs.
O(n log n).
// Vectores nulos se ignoran.
inline ll countParallelPairs(const vector<IPoint3>& vecs) {
```

```
    map<IPoint3, ll> cnt; ll res = 0;
    for (auto v : vecs) if (!isZero(v)) res += cnt[normDir(v)]++;
    return res;
}
// Uso: máximo número de puntos sobre una misma recta. O(n^2 log n). Duplicados cuentan.
inline ll maxCollinear(const vector<IPoint3>& pts) {
    ll n = pts.size(), best = min<ll>(n, 1);
    for (ll i = 0; i < n; i++) {
        map<IPoint3, ll> cnt; ll same = 0, top = 0;
        for (ll j = i + 1; j < n; j++) {
            IPoint3 d = pts[j] - pts[i];
            if (isZero(d)) { same++; continue; }
            top = max(top, ++cnt[normDir(d)]);
        }
        best = max(best, 1 + same + top);
    }
    return best;
}
// Uso: cuántos puntos de pts están en el plano de a,b,c (no colineales). O(n). Exacto.
inline ll countOnPlane(const IPoint3& a, const IPoint3& b, const IPoint3& c, const vector<IPoint3>& pts) {
    ll r = 0;
    for (auto& p : pts) r += orient(a, b, c, p) == 0;
    return r;
}
// Uso: ¿p y q equidistan de a? Exacto, sin sqrt.
inline bool equidistant(const IPoint3& a, const IPoint3& p, const IPoint3& q) { return dist2(a, p) == dist2(a, q); }
// 3. ROTACIONES Y TRANSFORMACIONES
// --- Rotaciones de 90° exactas (sentido antihorario mirando desde +eje hacia el origen) ---
// Regla: rotar 90° sobre Z: (x,y,z) -> (-y, x, z). Sobre X: (x,y,z) -> (x,-z,y).
//        Sobre Y: (x,y,z) -> (z, y, -x). k veces = 90k grados (k puede ser negativo).
template<class T> P3<T> rotX90(P3<T> p, int k = 1) { k = ((k % 4) + 4) % 4; while (k--) p = {p.x, -p.z, p.y}; return p; }
template<class T> P3<T> rotY90(P3<T> p, int k = 1) { k = ((k % 4) + 4) % 4; while (k--) p = {p.z, p.y, -p.x}; return p; }
template<class T> P3<T> rotZ90(P3<T> p, int k = 1) { k = ((k % 4) + 4) % 4; while (k--) p = {-p.y, p.x, p.z}; return p; }
// Reflexiones respecto a planos coordenados (cambian la orientación: det = -1).
template<class T> P3<T> reflXY(const P3<T>& p) { return {p.x, p.y, -p.z}; }
template<class T> P3<T> reflXZ(const P3<T>& p) { return {p.x, -p.y, p.z}; }
template<class T> P3<T> reflYZ(const P3<T>& p) { return {-p.x, p.y, p.z}; }
// Uso: las 24 rotaciones propias del cubo como matrices de permutación con signo (det=+1).
//   Con includeReflections=true devuelve las 48 (incluye espejos, det=-1).
// Aplicación típica: comparar figuras de voxeles "salvo rotación": normalizar cada
//   orientación (trasladar al mínimo, ordenar) y quedarse con la menor.
using SignedPerm = array<array<ll, 3>, 3>;
inline vector<SignedPerm> cubeRotations(bool includeReflections = false) {
    vector<SignedPerm> res;
    array<ll, 3> perm = {0, 1, 2};
    do {
        for (int s = 0; s < 8; s++) {
            SignedPerm m{};
            for (int r = 0; r < 3; r++) m[r][perm[r]] = (s >> r & 1) ? -1 : 1;
            ll det = m[0][0] * (m[1][1] * m[2][2] - m[1][2] * m[2][1])
                   - m[0][1] * (m[1][0] * m[2][2] - m[1][2] * m[2][0])
                   + m[0][2] * (m[1][0] * m[2][1] - m[1][1] * m[2][0]);
            if (det == 1 || includeReflections) res.pb(m);
        }
    } while (next_permutation(perm.begin(), perm.end()));
    return res;
}
template<class T> P3<T> applyPerm(const SignedPerm& m, const P3<T>& p) {
    return {m[0][0] * p.x + m[0][1] * p.y + m[0][2] * p.z,
            m[1][0] * p.x + m[1][1] * p.y + m[1][2] * p.z,
            m[2][0] * p.x + m[2][1] * p.y + m[2][2] * p.z};
}
// --- Matriz 3x3 (long double) ---
struct Mat3 {
    ld a[3][3] = {};
    static Mat3 identity() { Mat3 m; for (int i = 0; i < 3; i++) m.a[i][i] = 1; return m; }
    Mat3 operator*(const Mat3& o) const {              // composición: (A*B)p = A(Bp)
        Mat3 r;
        for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) for (int k = 0; k < 3; k++) r.a[i][j] += a[i][k] * o.a[k][j];
        return r;
    }
    Point3 operator*(const Point3& p) const {
        return {a[0][0] * p.x + a[0][1] * p.y + a[0][2] * p.z,
                a[1][0] * p.x + a[1][1] * p.y + a[1][2] * p.z,
                a[2][0] * p.x + a[2][1] * p.y + a[2][2] * p.z};
    }
    Mat3 transpose() const { Mat3 r; for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) r.a[i][j] = a[j][i]; return r; }
    ld det() const {
        return a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1]) - a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0])
             + a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0]);
    }
};
// Uso: rotación de ángulo t (radianes, antihoraria vista desde +eje) sobre X, Y o Z.
// La inversa de una rotación es su transpuesta.
inline Mat3 rotXMat(ld t) { Mat3 m = Mat3::identity(); ld c = cosl(t), s = sinl(t); m.a[1][1] = c; m.a[1][2] = -s; m.a[2][1] = s; m.a[2][2] = c; return m; }
inline Mat3 rotYMat(ld t) { Mat3 m = Mat3::identity(); ld c = cosl(t), s = sinl(t); m.a[0][0] = c; m.a[0][2] = s; m.a[2][0] = -s; m.a[2][2] = c; return m; }
inline Mat3 rotZMat(ld t) { Mat3 m = Mat3::identity(); ld c = cosl(t), s = sinl(t); m.a[0][0] = c; m.a[0][1] = -s; m.a[1][0] = s; m.a[1][1] = c; return m; }
// Uso: matriz de Rodrigues: rotación de ángulo t alrededor del eje (dirección) axis != 0,
//   antihoraria vista desde la punta de axis. R = I cos t + (1-cos t) k k^T + sin t [k]x.
inline Mat3 rodrigues(Point3 axis, ld t) {
    Point3 k = unit(axis); ld c = cosl(t), s = sinl(t), C = 1 - c;
    Mat3 m;
    m.a[0][0] = c + k.x * k.x * C;       m.a[0][1] = k.x * k.y * C - k.z * s; m.a[0][2] = k.x * k.z * C + k.y * s;
    m.a[1][0] = k.y * k.x * C + k.z * s; m.a[1][1] = c + k.y * k.y * C;       m.a[1][2] = k.y * k.z * C - k.x * s;
    m.a[2][0] = k.z * k.x * C - k.y * s; m.a[2][1] = k.z * k.y * C + k.x * s; m.a[2][2] = c + k.z * k.z * C;
    return m;
}
// Uso: rotar p ángulo t alrededor de la recta que pasa por o con dirección axis.
inline Point3 rotateAroundLine(const Point3& p, const Point3& o, const Point3& axis, ld t) {
    return o + rodrigues(axis, t) * (p - o);
}
// Uso: rotación de Rodrigues aplicada directamente (sin matriz).
inline Point3 rotateRodrigues(const Point3& v, const Point3& axis, ld t) {
    Point3 k = unit(axis);
    return v * cosl(t) + cross(k, v) * sinl(t) + k * (dot(k, v) * (1 - cosl(t)));
}
// --- Transformación afín 4x4 (coordenadas homogéneas, último renglón 0 0 0 1) ---
struct Mat4 {
    ld a[4][4] = {};
    static Mat4 identity() { Mat4 m; for (int i = 0; i < 4; i++) m.a[i][i] = 1; return m; }
    static Mat4 translate(const Point3& t) { Mat4 m = identity(); m.a[0][3] = t.x; m.a[1][3] = t.y; m.a[2][3] = t.z; return m; }
    static Mat4 scale(ld sx, ld sy, ld sz) { Mat4 m = identity(); m.a[0][0] = sx; m.a[1][1] = sy; m.a[2][2] = sz; return m; }
    static Mat4 fromMat3(const Mat3& r) { Mat4 m = identity(); for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) m.a[i][j] = r.a[i][j]; return m; }
    // Composición: (A*B) aplica primero B y luego A.
    Mat4 operator*(const Mat4& o) const {
        Mat4 r;
        for (int i = 0; i < 4; i++) for (int j = 0; j < 4; j++) for (int k = 0; k < 4; k++) r.a[i][j] += a[i][k] * o.a[k][j];
        return r;
    }
    Point3 apply(const Point3& p) const {   // punto (w=1): se traslada
        return {a[0][0] * p.x + a[0][1] * p.y + a[0][2] * p.z + a[0][3],
                a[1][0] * p.x + a[1][1] * p.y + a[1][2] * p.z + a[1][3],
                a[2][0] * p.x + a[2][1] * p.y + a[2][2] * p.z + a[2][3]};
    }
    Point3 applyVec(const Point3& v) const { // vector (w=0): NO se traslada
        return {a[0][0] * v.x + a[0][1] * v.y + a[0][2] * v.z,
                a[1][0] * v.x + a[1][1] * v.y + a[1][2] * v.z,
                a[2][0] * v.x + a[2][1] * v.y + a[2][2] * v.z};
    }
};
// Uso: rotación alrededor de un punto c: T(c) * R * T(-c).
inline Mat4 rotateAroundPoint(const Point3& c, const Mat3& r) {
    return Mat4::translate(c) * Mat4::fromMat3(r) * Mat4::translate(-c);
}
// --- Base local ortonormal (cambio de base) ---
// Uso: Frame f(o, w) crea base {u,v,w} ortonormal derecha con w = unit(dirección dada).
//   toLocal: coordenadas globales -> locales. toGlobal: inversa.
// Aplicación: llevar un plano a z=0 (geometría 2D en el plano), o un eje a la dirección Z.
struct Frame {
    Point3 o, u, v, w;
    Frame() : o(), u(1, 0, 0), v(0, 1, 0), w(0, 0, 1) {}
    Frame(Point3 origin, Point3 dirW) : o(origin) {
        w = unit(dirW); u = unit(anyPerp(w)); v = cross(w, u);
    }
    Point3 toLocal(const Point3& p) const { Point3 d = p - o; return {dot(d, u), dot(d, v), dot(d, w)}; }
    Point3 toGlobal(const Point3& q) const { return o + u * q.x + v *
```

```
q.y + w * q.z; }
};
```

§7.4.2 b-geo3d-solidos.cpp

07_Geometria/Geometria_3D

```
// Cajas, prefix/diff 3D, voxeles, rectas, planos, triángulos,
tetraedros, esferas y mallas.
// Los estados son enums explícitos; "6V"/"2A" = magnitudes
multiplicadas (exactas).
#include "a-geo3d-esencial.cpp"
// 1. CAJAS ALINEADAS A LOS EJES (exacto, long long)
// Box3 cerrada [lo.x,hi.x] x [lo.y,hi.y] x [lo.z,hi.z] con lo <= hi
en cada eje.
struct Box3 { IPoint3 lo, hi; };
// Uso: caja desde dos vértices opuestos cualesquiera (ordena cada
eje).
inline Box3 makeBox(const IPoint3& a, const IPoint3& b) {
    return {{min(a.x, b.x), min(a.y, b.y), min(a.z, b.z)}, {max(a.x,
b.x), max(a.y, b.y), max(a.z, b.z)}};
}
// Uso: longitud de la intersección de [l1,r1] y [l2,r2]: max(0,
min(r1,r2) - max(l1,l2)).
// Truco base de TODO lo de cajas: los ejes son independientes.
inline ll overlapLen(ll l1, ll r1, ll l2, ll r2) { return max(0LL,
min(r1, r2) - max(l1, l2)); }
// Lados, volumen (ancho*alto*prof.), área superficial 2(ab+bc+ca),
diagonal^2 = a^2+b^2+c^2.
// boxVolume en long long: lados <= ~2e6 seguros; si no, usar
boxVolume128.
inline ll side(const Box3& b, int k) { return b.hi[k] - b.lo[k]; }
inline ll boxVolume(const Box3& b) { return side(b, 0) * side(b, 1) *
side(b, 2); }
inline i128 boxVolume128(const Box3& b) { return (i128)side(b, 0) *
side(b, 1) * side(b, 2); }
inline ll boxSurface(const Box3& b) { ll a = side(b, 0), c = side(b,
1), h = side(b, 2); return 2 * (a * c + c * h + h * a); }
inline ll boxDiag2(const Box3& b) { ll a = side(b, 0), c = side(b,
1), h = side(b, 2); return a * a + c * c + h * h; }
inline bool isCube(const Box3& b) { return side(b, 0) == side(b, 1)
&& side(b, 1) == side(b, 2); }
// Puntos enteros dentro de la caja cerrada (incluye borde) =
prod(lado+1).
// Cubitos unitarios dentro = volumen (si las coordenadas son
enteras).
inline ll latticePointsInBox(const Box3& b) { return (side(b, 0) + 1)
* (side(b, 1) + 1) * (side(b, 2) + 1); }
// Uso: volumen de la intersección = producto de las 3 longitudes de
solape. O(1).
inline ll interVolume(const Box3& a, const Box3& b) {
    ll r = 1;
    for (int k = 0; k < 3; k++) r *= overlapLen(a.lo[k], a.hi[k],
b.lo[k], b.hi[k]);
    return r;
}
// Uso: caja intersección si existe (puede ser degenerada:
cara/arista/vértice).
inline optional<Box3> interBox(const Box3& a, const Box3& b) {
    Box3 r;
    for (int k = 0; k < 3; k++) {
        r.lo[k] = max(a.lo[k], b.lo[k]); r.hi[k] = min(a.hi[k],
b.hi[k]);
        if (r.lo[k] > r.hi[k]) return nullopt;
    }
    return r;
}
// Uso: ¿volumen de intersección POSITIVO? Cada eje con solape
ESTRICTAMENTE > 0. (ABC361 B)
inline bool overlapPositive(const Box3& a, const Box3& b) {
    for (int k = 0; k < 3; k++) if (min(a.hi[k], b.hi[k]) -
max(a.lo[k], b.lo[k]) <= 0) return false;
    return true;
}
// Uso: ¿se tocan o intersecan (cerradas)? Cada eje con solape >= 0.
inline bool boxesTouchOrOverlap(const Box3& a, const Box3& b) {
    for (int k = 0; k < 3; k++) if (min(a.hi[k], b.hi[k]) -
max(a.lo[k], b.lo[k]) < 0) return false;
    return true;
}
// Uso: tipo de contacto entre dos cajas. Por eje: g = min(hi) -
max(lo).
//   algún g<0 -> SEPARADAS; nº de ejes con g==0: 0 VOLUMEN, 1 CARA,
2 ARISTA, 3 VÉRTICE.
// Supone cajas de volumen positivo (para CARA = área de contacto >
0). (ABC312 E)
enum BoxContact { BC_SEPARATED, BC_VERTEX, BC_EDGE, BC_FACE,
BC_VOLUME };
inline BoxContact boxContact(const Box3& a, const Box3& b) {
    int zeros = 0;
    for (int k = 0; k < 3; k++) {
        ll g = min(a.hi[k], b.hi[k]) - max(a.lo[k], b.lo[k]);
        if (g < 0) return BC_SEPARATED;
        zeros += g == 0;
    }
    return zeros == 0 ? BC_VOLUME : zeros == 1 ? BC_FACE : zeros == 2
? BC_EDGE : BC_VERTEX;
}
// Uso: área de contacto por cara (0 si el contacto no es de cara).
inline ll contactArea(const Box3& a, const Box3& b) {
    if (boxContact(a, b) != BC_FACE) return 0;
    ll r = 1;
    for (int k = 0; k < 3; k++) { ll g = min(a.hi[k], b.hi[k]) -
max(a.lo[k], b.lo[k]); if (g > 0) r *= g; }
    return r;
}
// Uso: ¿inner está contenida en outer (cerradas)?
inline bool containsBox(const Box3& outer, const Box3& inner) {
    for (int k = 0; k < 3; k++) if (inner.lo[k] < outer.lo[k] ||
inner.hi[k] > outer.hi[k]) return false;
    return true;
}
inline bool pointInBox(const Box3& b, const IPoint3& p) {
    for (int k = 0; k < 3; k++) if (p[k] < b.lo[k] || p[k] > b.hi[k])
return false;
    return true;
}
// Unión de dos cajas por inclusión-exclusión: |A|+|B|-|AnB|.
inline ll unionVolume2(const Box3& a, const Box3& b) { return
boxVolume(a) + boxVolume(b) - interVolume(a, b); }
// Intersección común de tres cajas (volumen).
inline ll interVolume3(const Box3& a, const Box3& b, const Box3& c) {
    auto ab = interBox(a, b);
    return ab ? interVolume(*ab, c) : 0;
}
// Uso: volumen cubierto por EXACTAMENTE 1, 2 y 3 de tres cajas.
Retorna {V1,V2,V3}. (ABC343 E)
//   S1 = Σ|Ai|, S2 = Σ|AinAj|, S3 = |A1nA2nA3|:
//   V3 = S3, V2 = S2 - 3 S3, V1 = S1 - 2 S2 + 3 S3.
inline array<ll, 3> exactlyCovered3(const Box3& a, const Box3& b,
const Box3& c) {
    ll s1 = boxVolume(a) + boxVolume(b) + boxVolume(c);
    ll s2 = interVolume(a, b) + interVolume(b, c) + interVolume(a,
c);
    ll s3 = interVolume3(a, b, c);
    return {s1 - 2 * s2 + 3 * s3, s2 - 3 * s3, s3};
}
// Uso: caja contenedora (bounding box) de puntos no vacíos. O(n).
inline Box3 boundingBox(const vector<IPoint3>& pts) {
    Box3 b{pts[0], pts[0]};
    for (auto& p : pts) for (int k = 0; k < 3; k++) { b.lo[k] =
min(b.lo[k], p[k]); b.hi[k] = max(b.hi[k], p[k]); }
    return b;
}
// ---------------- Prefix sums 3D y difference arrays 3D ----------
-----
// Pref3: suma de un bloque [x1..x2]x[y1..y2]x[z1..z2] (índices 0-
based inclusivos) en O(1).
//   Construcción O(X*Y*Z). Inclusión-exclusión con 8 esquinas.
(ABC366 D)
struct Pref3 {
    int X, Y, Z; vector<ll> s;   // s tiene (X+1)(Y+1)(Z+1) celdas,
borde 0
    int id(int x, int y, int z) const { return (x * (Y + 1) + y) * (Z
+ 1) + z; }
    // a(x,y,z) devuelve el valor de la celda 0-based.
    template<class Fn> Pref3(int X_, int Y_, int Z_, Fn a) : X(X_),
Y(Y_), Z(Z_), s((X_ + 1) * (Y_ + 1) * (Z_ + 1), 0) {
        for (int x = 1; x <= X; x++) for (int y = 1; y <= Y; y++) for
(int z = 1; z <= Z; z++)
            s[id(x, y, z)] = a(x - 1, y - 1, z - 1)
                + s[id(x - 1, y, z)] + s[id(x, y - 1, z)] + s[id(x,
y, z - 1)]
                - s[id(x - 1, y - 1, z)] - s[id(x - 1, y, z - 1)] -
s[id(x, y - 1, z - 1)]
                + s[id(x - 1, y - 1, z - 1)];
    }
    ll query(int x1, int x2, int y1, int y2, int z1, int z2) const {
        if (x1 > x2 || y1 > y2 || z1 > z2) return 0;
        x2++; y2++; z2++;
        return s[id(x2, y2, z2)] - s[id(x1, y2, z2)] - s[id(x2, y1,
z2)] - s[id(x2, y2, z1)]
             + s[id(x1, y1, z2)] + s[id(x1, y2, z1)] + s[id(x2, y1,
z1)] - s[id(x1, y1, z1)];
    }
};
// Diff3: sumar v a todo un bloque en O(1); build() reconstruye los
valores en O(XYZ).
struct Diff3 {
    int X, Y, Z; vector<ll> d;
    Diff3(int X_, int Y_, int Z_) : X(X_), Y(Y_), Z(Z_), d((X_ + 1) *
(Y_ + 1) * (Z_ + 1), 0) {}
    int id(int x, int y, int z) const { return (x * (Y + 1) + y) * (Z
+ 1) + z; }
    void add(int x1, int x2, int y1, int y2, int z1, int z2, ll v) {
// inclusivo, 0-based
        x2++; y2++; z2++;
        d[id(x1, y1, z1)] += v; d[id(x2, y1, z1)] -= v; d[id(x1, y2,
z1)] -= v; d[id(x1, y1, z2)] -= v;
        d[id(x2, y2, z1)] += v; d[id(x2, y1, z2)] += v; d[id(x1, y2,
z2)] += v; d[id(x2, y2, z2)] -= v;
    }
    vector<ll> build() const {   // valor de (x,y,z) en res[(x*Y +
y)*Z + z]
        vector<ll> a = d;
        for (int x = 0; x <= X; x++) for (int y = 0; y <= Y; y++) for
(int z = 1; z <= Z; z++) a[id(x, y, z)] += a[id(x, y, z - 1)];
        for (int x = 0; x <= X; x++) for (int y = 1; y <= Y; y++) for
(int z = 0; z <= Z; z++) a[id(x, y, z)] += a[id(x, y - 1, z)];
        for (int x = 1; x <= X; x++) for (int y = 0; y <= Y; y++) for
(int z = 0; z <= Z; z++) a[id(x, y, z)] += a[id(x - 1, y, z)];
        vector<ll> res(X * Y * Z);
        for (int x = 0; x < X; x++) for (int y = 0; y < Y; y++) for
(int z = 0; z < Z; z++) res[(x * Y + y) * Z + z] = a[id(x, y, z)];
        return res;
    }
};
// Uso: volumen cubierto por exactamente k cajas, para k = 0..n
(res[0] = volumen libre dentro
//   de la bounding box). Compresión de coordenadas + Diff3. O(n^3)
```

```
tiempo y memoria (n <= ~150).
//   unionVolume = suma de res[k] para k >= 1.
inline vector<ll> coverVolumes(const vector<Box3>& boxes) {
    int n = boxes.size();
    vector<ll> res(n + 1, 0);
    if (n == 0) return res;
    vector<ll> cs[3];
    for (auto& b : boxes) for (int k = 0; k < 3; k++) {
cs[k].pb(b.lo[k]); cs[k].pb(b.hi[k]); }
    for (int k = 0; k < 3; k++) { sort(cs[k].begin(), cs[k].end());
cs[k].erase(unique(cs[k].begin(), cs[k].end()), cs[k].end()); }
    int X = cs[0].size() - 1, Y = cs[1].size() - 1, Z = cs[2].size()
- 1;
    if (X <= 0 || Y <= 0 || Z <= 0) return res;   // todas las cajas
son planas
    auto idx = [&](int k, ll v) { return (int)
(lower_bound(cs[k].begin(), cs[k].end(), v) - cs[k].begin()); };
    Diff3 df(X, Y, Z);
    for (auto& b : boxes) {
        int x1 = idx(0, b.lo.x), x2 = idx(0, b.hi.x) - 1, y1 = idx(1,
b.lo.y), y2 = idx(1, b.hi.y) - 1;
        int z1 = idx(2, b.lo.z), z2 = idx(2, b.hi.z) - 1;
        if (x1 <= x2 && y1 <= y2 && z1 <= z2) df.add(x1, x2, y1, y2,
z1, z2, 1);
    }
    vector<ll> cnt = df.build();
    for (int x = 0; x < X; x++) for (int y = 0; y < Y; y++) for (int
z = 0; z < Z; z++)
        res[cnt[(x * Y + y) * Z + z]] += (cs[0][x + 1] - cs[0][x]) *
(cs[1][y + 1] - cs[1][y]) * (cs[2][z + 1] - cs[2][z]);
    return res;
}
inline ll unionVolume(const vector<Box3>& boxes) {
    auto r = coverVolumes(boxes); ll s = 0;
    for (size_t k = 1; k < r.size(); k++) s += r[k];
    return s;
}
// ---------------- Voxeles (cubitos unitarios) ----------------
// Uso: área superficial de una figura formada por cubitos unitarios
(celda (x,y,z) =
//   [x,x+1]x[y,y+1]x[z,z+1]). exteriorOnly=true descuenta cavidades
cerradas (flood fill
//   desde afuera con 6 vecinos). O(V) con V = volumen de la bounding
box + borde.
inline ll voxelSurface(const vector<IPoint3>& cubes, bool
exteriorOnly) {
    if (cubes.empty()) return 0;
    Box3 bb = boundingBox(cubes);
    ll X = side(bb, 0) + 3, Y = side(bb, 1) + 3, Z = side(bb, 2) + 3;
// borde de 1 celda
    auto id = [&](ll x, ll y, ll z) { return (x * Y + y) * Z + z; };
    vector<char> full(X * Y * Z, 0), outside(X * Y * Z, 0);
    for (auto& c : cubes) full[id(c.x - bb.lo.x + 1, c.y - bb.lo.y +
1, c.z - bb.lo.z + 1)] = 1;
    const int D[6][3] = {{1, 0, 0}, {-1, 0, 0}, {0, 1, 0}, {0, -1,
0}, {0, 0, 1}, {0, 0, -1}};
    if (exteriorOnly) {
        vector<array<ll, 3>> st = {{0, 0, 0}}; outside[0] = 1;
        while (!st.empty()) {
            auto [x, y, z] = st.back(); st.pop_back();
            for (auto& d : D) {
                ll nx = x + d[0], ny = y + d[1], nz = z + d[2];
                if (nx < 0 || ny < 0 || nz < 0 || nx >= X || ny >= Y
|| nz >= Z) continue;
                ll j = id(nx, ny, nz);
                if (!full[j] && !outside[j]) { outside[j] = 1;
st.pb({nx, ny, nz}); }
            }
        }
    }
    ll res = 0;
    for (ll x = 1; x + 1 < X; x++) for (ll y = 1; y + 1 < Y; y++) for
(ll z = 1; z + 1 < Z; z++) {
        if (!full[id(x, y, z)]) continue;
        for (auto& d : D) {
            ll j = id(x + d[0], y + d[1], z + d[2]);
            if (!full[j] && (!exteriorOnly || outside[j])) res++;
        }
    }
    return res;
}
// 2. RECTAS, RAYOS Y SEGMENTOS
// Line3: p + t d, con d != 0 (dirección nula = recta indefinida:
verificar antes).
struct Line3 {
    Point3 p, d;
    Line3() {}
    Line3(Point3 p_, Point3 d_) : p(p_), d(d_) {}
    Point3 at(ld t) const { return p + d * t; }
};
inline Line3 lineThrough(const Point3& a, const Point3& b) { return
Line3(a, b - a); }   // requiere a != b
// Uso: parámetro t del pie de la perpendicular de q sobre l.
Proyección = l.at(t).
inline ld paramOnLine(const Line3& l, const Point3& q) { return dot(q
- l.p, l.d) / norm2(l.d); }
inline Point3 projOnLine(const Line3& l, const Point3& q) { return
l.at(paramOnLine(l, q)); }
// Uso: distancia punto-recta = |d × (q-p)| / |d|.
inline ld distPointLine(const Line3& l, const Point3& q) { return
norm(cross(l.d, q - l.p)) / norm(l.d); }
inline bool pointOnLine(const Line3& l, const Point3& q) { return
distPointLine(l, q) <= EPS3; }
// Uso: punto del segmento [a,b] más cercano a q (a==b permitido).
O(1).
inline Point3 closestOnSegment(const Point3& a, const Point3& b,
const Point3& q) {
    ld L = norm2(b - a);
    if (L <= EPS3 * EPS3) return a;
    ld t = max((ld)0, min((ld)1, dot(q - a, b - a) / L));
    return a + (b - a) * t;
}
inline ld distPointSegment(const Point3& a, const Point3& b, const
Point3& q) { return dist(q, closestOnSegment(a, b, q)); }
// Exacto: ¿p está en el segmento cerrado [a,b]? (a==b permitido)
template<class T> bool onSegment(const P3<T>& a, const P3<T>& b,
const P3<T>& p) {
    return collinear(a, b, p) && sgn3(dot(p - a, p - b)) <= 0;
}
// Relación entre dos rectas en 3D. ¡No paralelas NO implica que se
corten! (alabeadas)
enum LineRel { LR_SKEW, LR_INTERSECT, LR_PARALLEL, LR_COINCIDENT };
// Versión EXACTA con rectas a1b1 y a2b2 enteras (a != b).
inline LineRel lineRelationExact(const IPoint3& a1, const IPoint3&
b1, const IPoint3& a2, const IPoint3& b2) {
    IPoint3 d1 = b1 - a1, d2 = b2 - a2;
    if (isZero(cross(d1, d2))) return collinear(a1, b1, a2) ?
LR_COINCIDENT : LR_PARALLEL;
    return orient(a1, b1, a2, b2) == 0 ? LR_INTERSECT : LR_SKEW;
}
// Uso: distancia entre rectas. Paralelas -> punto-recta. Si no: |
(p2-p1)·(d1×d2)| / |d1×d2|.
inline ld distLineLine(const Line3& l1, const Line3& l2) {
    Point3 n = cross(l1.d, l2.d);
    if (norm(n) <= EPS3 * norm(l1.d) * norm(l2.d)) return
distPointLine(l1, l2.p);
    return fabsl(dot(l2.p - l1.p, n)) / norm(n);
}
// Versión flotante de la relación (tolerancia en unidades de
distancia).
inline LineRel lineRelation(const Line3& l1, const Line3& l2) {
    if (norm(cross(unit(l1.d), unit(l2.d))) <= EPS3) return
distPointLine(l1, l2.p) <= EPS3 ? LR_COINCIDENT : LR_PARALLEL;
    return distLineLine(l1, l2) <= EPS3 ? LR_INTERSECT : LR_SKEW;
}
// Uso: puntos más cercanos c1 en l1 y c2 en l2 (perpendicular común
= segmento c1c2).
//   Requiere rectas NO paralelas. Si se cortan, c1 == c2 es la
intersección.
//   n = d1×d2, c1 = p1 + d1 ((p2-p1)·(d2×n))/(d1·(d2×n)), simétrico
para c2.
inline pair<Point3, Point3> closestPointsLines(const Line3& l1, const
Line3& l2) {
    Point3 n = cross(l1.d, l2.d), n1 = cross(l1.d, n), n2 =
cross(l2.d, n);
    Point3 c1 = l1.p + l1.d * (dot(l2.p - l1.p, n2) / dot(l1.d, n2));
    Point3 c2 = l2.p + l2.d * (dot(l1.p - l2.p, n1) / dot(l2.d, n1));
    return {c1, c2};
}
// Uso: intersección de dos rectas si es un único punto. Retorna
false si son alabeadas,
//   paralelas o coincidentes.
inline bool lineIntersection(const Line3& l1, const Line3& l2,
Point3& out) {
    if (lineRelation(l1, l2) != LR_INTERSECT) return false;
    out = closestPointsLines(l1, l2).first;
    return true;
}
// Uso: ángulo entre rectas en [0, pi/2] (no orientadas).
inline ld angleLines(const Point3& d1, const Point3& d2) { return
atan2l(norm(cross(d1, d2)), fabsl(dot(d1, d2))); }
// Uso: distancia entre segmentos [a,b] y [c,d] (cualquiera puede ser
degenerado). O(1).
//   La distancia es convexa en (s,t) ∈ [0,1]^2: mínimo interior
(perpendicular común con
//   parámetros válidos) o en el borde (extremo contra segmento).
inline ld distSegSeg(const Point3& a, const Point3& b, const Point3&
c, const Point3& d) {
    ld best = min({distPointSegment(c, d, a), distPointSegment(c, d,
b), distPointSegment(a, b, c), distPointSegment(a, b, d)});
    Point3 d1 = b - a, d2 = d - c, n = cross(d1, d2);
    if (norm(d1) > EPS3 && norm(d2) > EPS3 && norm(n) > EPS3 *
norm(d1) * norm(d2)) {
        Point3 n1 = cross(d1, n), n2 = cross(d2, n);
        ld s = dot(c - a, n2) / dot(d1, n2), t = dot(a - c, n1) /
dot(d2, n1);
        if (s >= 0 && s <= 1 && t >= 0 && t <= 1) best = min(best,
dist(a + d1 * s, c + d2 * t));
    }
    return best;
}
// Uso: ¿se cortan los segmentos cerrados [a,b] y [c,d]? EXACTO con
enteros
//   (|coord| <= 1e9). Maneja degenerados, colineales y alabeados.
O(1).
inline bool segSegIntersectExact(const IPoint3& a, const IPoint3& b,
const IPoint3& c, const IPoint3& d) {
    if (a == b) return onSegment(c, d, a);
    if (c == d) return onSegment(a, b, c);
    if (orient(a, b, c, d) != 0) return false;                 // no
coplanares
    IPoint3 n = cross(b - a, d - c);
    if (isZero(n)) {                                           //
paralelos
        if (!collinear(a, b, c)) return false;
        return onSegment(a, b, c) || onSegment(a, b, d) ||
onSegment(c, d, a) || onSegment(c, d, b);
```

```
    }
    // Coplanares no paralelos: lado 2D = componente k de la cruz (k = mayor |n_k|).
    int k = 0;
    for (int j = 1; j < 3; j++) if (llabs(n[j]) > llabs(n[k])) k = j;
    auto sd = [&](const IPoint3& o, const IPoint3& e, const IPoint3& q) { return sgn3(cross(e - o, q - o)[k]); };
    return sd(a, b, c) * sd(a, b, d) <= 0 && sd(c, d, a) * sd(c, d, b) <= 0;
}
// 3. PLANOS
// Plane (Long double): dot(n, x) = d con |n| = 1 (las fábricas normalizan).
//   Ecuación Ax+By+Cz+D=0 con (A,B,C) = n, D = -d. side(p) = distancia CON SIGNO
//   (>0 del lado al que apunta n).
struct Plane {
    Point3 n; ld d = 0;
    Plane() {}
    Plane(Point3 n_, ld d_) : n(n_), d(d_) { ld L = norm(n); n = n / L; d /= L; }   // n != 0
    static Plane fromPointNormal(const Point3& p, const Point3& nrm) { return Plane(nrm, dot(nrm, p)); }
    static Plane fromPoints(const Point3& a, const Point3& b, const Point3& c) {    // a,b,c no colineales
        Point3 nrm = cross(b - a, c - a); return Plane(nrm, dot(nrm, a));
    }
    static Plane fromCoeffs(ld A, ld B, ld C, ld D) { return Plane(Point3(A, B, C), -D); }
    array<ld, 4> coeffs() const { return {n.x, n.y, n.z, -d}; }
    ld side(const Point3& p) const { return dot(n, p) - d; }
    ld distTo(const Point3& p) const { return fabsl(side(p)); }
    int classify(const Point3& p) const { return sgn3(side(p)); }
// 1, 0, -1
    bool contains(const Point3& p) const { return classify(p) == 0; }
    Point3 proj(const Point3& p) const { return p - n * side(p); }
// proyección ortogonal
    Point3 reflect(const Point3& p) const { return p - n * (2 * side(p)); }
    Point3 anyPoint() const { return n * d; }
    // Base local: (u, v) en el plano y w = n. to2D da coordenadas 2D dentro del plano
    //   (preserva distancias, ángulos y áreas: se puede usar toda la geometría 2D).
    Frame frame() const { return Frame(anyPoint(), n); }
    pair<ld, ld> to2D(const Point3& p) const { Point3 q = frame().toLocal(p); return {q.x, q.y}; }
    Point3 from2D(ld u, ld v) const { return frame().toGlobal(Point3(u, v, 0)); }
};
// Mismo lado (estricto) / lados opuestos (estricto).
inline bool sameSide(const Plane& pl, const Point3& a, const Point3& b) { return pl.classify(a) * pl.classify(b) > 0; }
inline bool oppositeSides(const Plane& pl, const Point3& a, const Point3& b) { return pl.classify(a) * pl.classify(b) < 0; }
// IPlane EXACTO por tres puntos enteros: n = (b-a)×(c-a) (Long long), d = n·a (__int128).
//   side(p) exacto en __int128 (|coord| <= 1e9: |n| <= 8e18, |n·p| <= 2.4e28). Clasificar
//   miles de puntos contra el mismo plano sin recalcular la cruz.
struct IPlane {
    IPoint3 n; i128 d = 0;
    IPlane() {}
    IPlane(const IPoint3& a, const IPoint3& b, const IPoint3& c) : n(cross(b - a, c - a)), d(dot(n, a)) {}
    bool valid() const { return !isZero(n); }   // false si a,b,c colineales
    int classify(const IPoint3& p) const { return sgn3(dot(n, p) - d); }
};
// Uso: intersección recta-plano. IK_POINT con punto y parámetro t; IK_LINE si la recta está
//   contenida; IK_NONE si es paralela y fuera.
struct LPResult { InterKind kind; Point3 p; ld t; };
inline LPResult interLinePlane(const Line3& l, const Plane& pl) {
    ld den = dot(pl.n, l.d), num = pl.side(l.p);
    if (fabsl(den) <= EPS3 * norm(l.d)) return {fabsl(num) <= EPS3 ? IK_LINE : IK_NONE, Point3(), 0};
    ld t = -num / den;
    return {IK_POINT, l.at(t), t};
}
// Uso: intersección segmento [a,b]-plano. IK_POINT solo si el corte cae en el segmento.
inline LPResult interSegmentPlane(const Point3& a, const Point3& b, const Plane& pl) {
    if (norm(b - a) <= EPS3) return {pl.contains(a) ? IK_POINT : IK_NONE, a, 0};
    LPResult r = interLinePlane(lineThrough(a, b), pl);
    if (r.kind == IK_POINT && (r.t < -EPS3 || r.t > 1 + EPS3)) r.kind = IK_NONE;
    return r;
}
// Exacto: ¿el segmento cerrado [p,q] toca el plano abc? (a,b,c no colineales)
inline bool segmentTouchesPlaneExact(const IPoint3& a, const IPoint3& b, const IPoint3& c, const IPoint3& p, const IPoint3& q) {
    return orient(a, b, c, p) * orient(a, b, c, q) <= 0;
}
// Uso: intersección de dos planos. IK_LINE con la recta; IK_PLANE si coinciden;
//   IK_NONE si son paralelos distintos.
struct PPResult { InterKind kind; Line3 line; };
inline PPResult interPlanePlane(const Plane& p1, const Plane& p2) {
    Point3 dir = cross(p1.n, p2.n);
    if (norm(dir) <= EPS3) return {fabsl(p2.side(p1.anyPoint())) <= EPS3 ? IK_PLANE : IK_NONE, Line3()};
    ld a12 = dot(p1.n, p2.n), det = norm2(dir);           // |n1|=|n2|=1 => det = 1 - a12^2
    ld a1 = (p1.d - p2.d * a12) / det, be = (p2.d - p1.d * a12) / det;
    return {IK_LINE, Line3(p1.n * a1 + p2.n * be, dir)};
}
// Uso: intersección de tres planos. true y el punto si es único (det de normales != 0).
//   x = (d1 (n2×n3) + d2 (n3×n1) + d3 (n1×n2)) / (n1·(n2×n3)).
inline bool interThreePlanes(const Plane& a, const Plane& b, const Plane& c, Point3& out) {
    ld det = triple(a.n, b.n, c.n);
    if (fabsl(det) <= EPS3) return false;
    out = (cross(b.n, c.n) * a.d + cross(c.n, a.n) * b.d + cross(a.n, b.n) * c.d) / det;
    return true;
}
// Relación de planos: IK_PLANE coincidentes, IK_NONE paralelos, IK_LINE secantes.
inline InterKind planeRelation(const Plane& p1, const Plane& p2) { return interPlanePlane(p1, p2).kind; }
// Ángulo diedro (agudo) entre planos en [0, pi/2]; ángulo recta-plano en [0, pi/2].
inline ld anglePlanes(const Plane& p1, const Plane& p2) { return atan2l(norm(cross(p1.n, p2.n)), fabsl(dot(p1.n, p2.n))); }
inline ld angleLinePlane(const Point3& d, const Plane& pl) { return atan2l(fabsl(dot(d, pl.n)), norm(cross(d, pl.n))); }
// Proyección a planos coordenados: quitar el eje k (0=x,1=y,2=z). Preserva orientación 2D
//   si se usa el orden cíclico: quitar x -> (y,z), quitar y -> (z,x), quitar z -> (x,y).
template<class T> pair<T, T> dropAxis(const P3<T>& p, int k) {
    if (k == 0) return {p.y, p.z};
    if (k == 1) return {p.z, p.x};
    return {p.x, p.y};
}
// 4. TRIÁNGULOS
// Normal sin normalizar (|N| = 2·área). Exacta con enteros.
template<class T> P3<T> triNormal(const P3<T>& a, const P3<T>& b, const P3<T>& c) { return cross(b - a, c - a); }
template<class T> ld triArea(const P3<T>& a, const P3<T>& b, const P3<T>& c) { return norm(triNormal(a, b, c)) / 2; }
// (2·Área)^2 exacto en __int128: |coord| <= 9e8 (grado 4: 192·C^4 < 1.7e38).
inline i128 triArea2Squared(const IPoint3& a, const IPoint3& b, const IPoint3& c) { return norm2(triNormal(a, b, c)); }
template<class T> ld triPerimeter(const P3<T>& a, const P3<T>& b, const P3<T>& c) { return dist(a, b) + dist(b, c) + dist(c, a); }
inline Point3 triCentroid(const Point3& a, const Point3& b, const Point3& c) { return (a + b + c) / 3; }
template<class T> bool triDegenerate(const P3<T>& a, const P3<T>& b, const P3<T>& c) { return collinear(a, b, c); }
// Uso: coordenadas baricéntricas (wa, wb, wc) de la proyección de p sobre el plano abc.
//   p = wa a + wb b + wc c; dentro del triángulo <=> las tres >= 0. Triángulo no degenerado.
inline array<ld, 3> barycentric(const Point3& a, const Point3& b, const Point3& c, const Point3& p) {
    Point3 n = cross(b - a, c - a); ld n2 = norm2(n);
    ld wa = dot(cross(c - b, p - b), n) / n2, wb = dot(cross(a - c, p - c), n) / n2;
    return {wa, wb, 1 - wa - wb};
}
// Uso: ¿p está en el triángulo cerrado abc (en 3D)? EXACTO. Triángulo no degenerado.
//   Coplanar + los tres "lados" sin mezcla de signos (componente k dominante de la normal).
inline bool pointInTriangleExact(const IPoint3& a, const IPoint3& b, const IPoint3& c, const IPoint3& p) {
    if (orient(a, b, c, p) != 0) return false;
    IPoint3 n = triNormal(a, b, c);
    int k = 0;
    for (int j = 1; j < 3; j++) if (llabs(n[j]) > llabs(n[k])) k = j;
    int s1 = sgn3(cross(b - a, p - a)[k]), s2 = sgn3(cross(c - b, p - b)[k]), s3 = sgn3(cross(a - c, p - c)[k]);
    bool neg = s1 < 0 || s2 < 0 || s3 < 0, pos = s1 > 0 || s2 > 0 || s3 > 0;
    return !(neg && pos);
}
// Uso: punto del triángulo abc más cercano a p (Ericson, regiones de Voronoi). O(1).
//   Triángulo degenerado: se usa el mínimo sobre sus tres lados.
inline Point3 closestPointTriangle(const Point3& p, const Point3& a, const Point3& b, const Point3& c) {
    if (norm(cross(b - a, c - a)) <= EPS3) {
        Point3 best = closestOnSegment(a, b, p);
        for (Point3 q : {closestOnSegment(b, c, p), closestOnSegment(c, a, p)}) if (dist(q, p) < dist(best, p)) best = q;
        return best;
    }
    Point3 ab = b - a, ac = c - a, ap = p - a;
    ld d1 = dot(ab, ap), d2 = dot(ac, ap);
    if (d1 <= 0 && d2 <= 0) return a;
    Point3 bp = p - b; ld d3 = dot(ab, bp), d4 = dot(ac, bp);
    if (d3 >= 0 && d4 <= d3) return b;
    ld vc = d1 * d4 - d3 * d2;
    if (vc <= 0 && d1 >= 0 && d3 <= 0) return a + ab * (d1 / (d1 - d3));
    Point3 cp = p - c; ld d5 = dot(ab, cp), d6 = dot(ac, cp);
```

```
    if (d6 >= 0 && d5 <= d6) return c;
    ld vbb = d5 * d2 - d1 * d6;
    if (vbb <= 0 && d2 >= 0 && d6 <= 0) return a + ac * (d2 / (d2 -
d6));
    ld va = d3 * d6 - d5 * d4;
    if (va <= 0 && d4 - d3 >= 0 && d5 - d6 >= 0) return b + (c - b) *
((d4 - d3) / ((d4 - d3) + (d5 - d6)));
    ld den = 1 / (va + vbb + vc);
    return a + ab * (vbb * den) + ac * (vc * den);
}
inline ld distPointTriangle(const Point3& p, const Point3& a, const
Point3& b, const Point3& c) {
    return dist(p, closestPointTriangle(p, a, b, c));
}
// Uso: ¿el segmento cerrado [p,q] toca el triángulo cerrado abc?
EXACTO. Triángulo no
//   degenerado. Caso coplanar: extremo dentro o cruce con algún
lado.
inline bool segTriangleExact(const IPoint3& p, const IPoint3& q,
const IPoint3& a, const IPoint3& b, const IPoint3& c) {
    int o1 = orient(a, b, c, p), o2 = orient(a, b, c, q);
    if (o1 * o2 > 0) return false;
    if (o1 == 0 && o2 == 0)
        return pointInTriangleExact(a, b, c, p) ||
pointInTriangleExact(a, b, c, q) ||
               segSegIntersectExact(p, q, a, b) ||
segSegIntersectExact(p, q, b, c) || segSegIntersectExact(p, q, c, a);
    // La recta pq atraviesa el plano dentro del segmento: verificar
que pase por el triángulo.
    int s1 = orient(p, q, a, b), s2 = orient(p, q, b, c), s3 =
orient(p, q, c, a);
    bool neg = s1 < 0 || s2 < 0 || s3 < 0, pos = s1 > 0 || s2 > 0 ||
s3 > 0;
    return !(neg && pos);
}
// Uso: intersección rayo o + t·dir (t >= 0) con triángulo (Möller–
Trumbore). Retorna t.
//   Rayos paralelos al plano (incluido coplanar) -> nullopt.
Aproximado.
inline optional<ld> rayTriangle(const Point3& o, const Point3& dir,
const Point3& a, const Point3& b, const Point3& c) {
    Point3 e1 = b - a, e2 = c - a, h = cross(dir, e2);
    ld det = dot(e1, h);
    if (fabsl(det) <= EPS3 * norm(dir) * norm(e1) * norm(e2)) return
nullopt;
    ld inv = 1 / det; Point3 s = o - a;
    ld u = dot(s, h) * inv;
    if (u < -EPS3 || u > 1 + EPS3) return nullopt;
    Point3 q = cross(s, e1);
    ld v = dot(dir, q) * inv;
    if (v < -EPS3 || u + v > 1 + EPS3) return nullopt;
    ld t = dot(e2, q) * inv;
    if (t < -EPS3) return nullopt;
    return t;
}
// Uso: punto donde el segmento [p,q] corta al triángulo (si no es
coplanar). Aproximado.
//   Comprobar antes con segTriangleExact si las coordenadas son
enteras.
inline optional<Point3> segTrianglePoint(const Point3& p, const
Point3& q, const Point3& a, const Point3& b, const Point3& c) {
    auto t = rayTriangle(p, q - p, a, b, c);
    if (!t || *t > 1 + EPS3) return nullopt;
    return p + (q - p) * *t;
}
// 5. TETRAEDROS
// Uso: 6·V con signo = (B-A)·((C-A)×(D-A)). EXACTO con enteros. >0
si D está del lado de
//   la normal de ABC (ABC antihorario visto desde D). V = |6V| / 6.
template<class T> Wide<T> tetVolume6(const P3<T>& a, const P3<T>& b,
const P3<T>& c, const P3<T>& d) {
    return triple(b - a, c - a, d - a);
}
template<class T> ld tetVolume(const P3<T>& a, const P3<T>& b, const
P3<T>& c, const P3<T>& d) {
    ld v = (ld)tetVolume6(a, b, c, d); return fabsl(v) / 6;
}
template<class T> bool tetDegenerate(const P3<T>& a, const P3<T>& b,
const P3<T>& c, const P3<T>& d) { return coplanar(a, b, c, d); }
// Uso: ¿p dentro del tetraedro cerrado abcd (no degenerado)? EXACTO.
//   Para cada cara, p debe estar del mismo lado que el vértice
opuesto (o sobre la cara).
inline bool pointInTetraExact(const IPoint3& a, const IPoint3& b,
const IPoint3& c, const IPoint3& d, const IPoint3& p) {
    IPoint3 v[4] = {a, b, c, d};
    for (int i = 0; i < 4; i++) {
        IPoint3 f0 = v[(i + 1) % 4], f1 = v[(i + 2) % 4], f2 = v[(i +
3) % 4];
        int ref = orient(f0, f1, f2, v[i]), sp = orient(f0, f1, f2,
p);
        if (sp != 0 && sp != ref) return false;
    }
    return true;
}
// Altura desde d sobre la cara abc = 3V/Área(abc) = |6V| / |(b-
a)×(c-a)|.
template<class T> ld tetHeight(const P3<T>& a, const P3<T>& b, const
P3<T>& c, const P3<T>& d) {
    return fabsl((ld)tetVolume6(a, b, c, d)) / norm(triNormal(a, b,
c));
}
template<class T> ld tetSurface(const P3<T>& a, const P3<T>& b, const
P3<T>& c, const P3<T>& d) {
    return triArea(a, b, c) + triArea(a, b, d) + triArea(a, c, d) +
triArea(b, c, d);
}
inline Point3 tetCentroid(const Point3& a, const Point3& b, const
Point3& c, const Point3& d) { return (a + b + c + d) / 4; }
// 6. ESFERAS
struct Sphere { Point3 c; ld r = 0; };
inline ld sphereArea(ld r) { return 4 * PI3 * r * r; }
inline ld sphereVolume(ld r) { return 4 * PI3 * r * r * r / 3; }
// Casquete de altura h en esfera de radio R: área lateral 2πRh,
volumen πh²(3R-h)/3.
inline ld capArea(ld R, ld h) { return 2 * PI3 * R * h; }
inline ld capVolume(ld R, ld h) { return PI3 * h * h * (3 * R - h) /
3; }
// Cuerda a distancia dd del centro: 2 sqrt(r² - dd²).
inline ld chordLength(ld r, ld dd) { return 2 * safeSqrt(r * r - dd *
dd); }
// Uso: punto vs bola con centro y radio ENTEROS: -1 dentro, 0 sobre,
1 fuera. EXACTO.
inline int cmpPointSphereExact(const IPoint3& c, ll r, const IPoint3&
p) { return sgn3(dist2(c, p) - (i128)r * r); }
inline int cmpPointSphere(const Sphere& s, const Point3& p) { return
sgn3(dist(s.c, p) - s.r); }
inline ld distToSphereSurface(const Sphere& s, const Point3& p) {
return fabsl(dist(s.c, p) - s.r); }
inline ld distToBall(const Sphere& s, const Point3& p) { return
max((ld)0, dist(s.c, p) - s.r); }
// Uso: parámetros t (ordenados) donde l.at(t) toca la esfera: 0 (no
corta), 1 (tangente),
//   2 (secante). Rayo: quedarse con t >= 0. Segmento [a,b] con
lineThrough: 0 <= t <= 1.
inline vector<ld> lineSphere(const Line3& l, const Sphere& s) {
    ld t0 = paramOnLine(l, s.c);
    ld h2 = norm2(l.at(t0) - s.c), q = s.r * s.r - h2;
    if (q < -EPS3) return {};
    if (q <= EPS3) return {t0};
    ld dt = sqrtl(q) / norm(l.d);
    return {t0 - dt, t0 + dt};
}
inline vector<ld> raySphere(const Point3& o, const Point3& dir, const
Sphere& s) {
    vector<ld> r;
    for (ld t : lineSphere(Line3(o, dir), s)) if (t >= -EPS3)
r.pb(max((ld)0, t));
    return r;
}
inline vector<ld> segmentSphere(const Point3& a, const Point3& b,
const Sphere& s) {
    vector<ld> r;
    if (norm(b - a) <= EPS3) return r;
    for (ld t : lineSphere(lineThrough(a, b), s)) if (t >= -EPS3 && t
<= 1 + EPS3) r.pb(t);
    return r;
}
// Uso: relación esfera-esfera. Si hay contacto: circle (centro,
normal, radio);
//   tangencias -> radio 0 y center = punto de tangencia.
enum SphereRel { SR_SEPARATE, SR_EXT_TANGENT, SR_INTERSECT,
SR_INT_TANGENT, SR_CONTAINED, SR_SAME };
struct Circle3 { Point3 c, n; ld r = 0; };
inline SphereRel sphereSphere(const Sphere& s1, const Sphere& s2,
Circle3& circ) {
    ld dd = dist(s1.c, s2.c);
    if (dd <= EPS3 && fabsl(s1.r - s2.r) <= EPS3) return SR_SAME;
    if (dd > s1.r + s2.r + EPS3) return SR_SEPARATE;
    if (dd < fabsl(s1.r - s2.r) - EPS3) return SR_CONTAINED;
    Point3 u = unit(s2.c - s1.c);
    if (fabsl(dd - (s1.r + s2.r)) <= EPS3) { circ = {s1.c + u * s1.r,
u, 0}; return SR_EXT_TANGENT; }
    if (fabsl(dd - fabsl(s1.r - s2.r)) <= EPS3) {
        const Sphere& big = s1.r >= s2.r ? s1 : s2; Point3 dir = s1.r
>= s2.r ? u : -u;
        circ = {big.c + dir * big.r, u, 0}; return SR_INT_TANGENT;
    }
    ld a = (dd * dd + s1.r * s1.r - s2.r * s2.r) / (2 * dd);
    circ = {s1.c + u * a, u, safeSqrt(s1.r * s1.r - a * a)};
    return SR_INTERSECT;
}
// Volumen de la intersección de dos bolas (radios r1, r2, centros a
distancia dd) = 2 casquetes.
inline ld sphereIntersectionVolume(ld r1, ld r2, ld dd) {
    if (dd >= r1 + r2) return 0;
    if (dd <= fabsl(r1 - r2)) return sphereVolume(min(r1, r2));
    ld a = (dd * dd + r1 * r1 - r2 * r2) / (2 * dd);
    return capVolume(r1, r1 - a) + capVolume(r2, r2 - (dd - a));
}
// Distancia mínima entre las SUPERFICIES de dos esferas (0 si se
cortan).
inline ld sphereSurfaceGap(const Sphere& s1, const Sphere& s2) {
    ld dd = dist(s1.c, s2.c);
    if (dd >= s1.r + s2.r) return dd - s1.r - s2.r;
    if (dd <= fabsl(s1.r - s2.r)) return fabsl(s1.r - s2.r) - dd;
    return 0;
}
// Uso: intersección plano-esfera. Retorna IK_NONE, o IK_POINT
(tangente, radio 0) o
//   IK_LINE (círculo; se reutiliza el enum: "corte con dimensión
1").
inline InterKind planeSphere(const Plane& pl, const Sphere& s,
Circle3& circ) {
    ld h = pl.side(s.c);
    if (fabsl(h) > s.r + EPS3) return IK_NONE;
    circ = {pl.proj(s.c), pl.n, safeSqrt(s.r * s.r - h * h)};
    return fabsl(fabsl(h) - s.r) <= EPS3 ? IK_POINT : IK_LINE;
```

```
}
// Circuncentro de un triángulo en 3D (en su plano). Triángulo no degenerado.
inline Point3 circumcenter3(const Point3& a, const Point3& b, const Point3& c) {
    Point3 u = b - a, v = c - a, w = cross(u, v);
    return a + (cross(w, u) * norm2(v) + cross(v, w) * norm2(u)) / (2 * norm2(w));
}
// Esfera por 4 puntos no coplanares: centro = a + (|u|²(v×w) + |v|²(w×u) + |w|²(u×v)) / (2 u·(v×w)).
inline bool circumsphere(const Point3& a, const Point3& b, const Point3& c, const Point3& d, Sphere& out) {
    Point3 u = b - a, v = c - a, w = d - a;
    ld den = 2 * triple(u, v, w);
    if (fabsl(den) <= EPS3) return false;
    Point3 cen = a + (cross(v, w) * norm2(u) + cross(w, u) * norm2(v) + cross(u, v) * norm2(w)) / den;
    out = {cen, dist(cen, a)};
    return true;
}
// Minimum Enclosing Sphere (Welzl iterativo). O(n) esperado. Aproximado (long double).
//   Degenerados: 3 colineales -> esfera del par más lejano; 4 coplanares -> mejor esfera de
//   3 de ellos que contenga a los 4. Probado contra fuerza bruta en _pruebas.
inline Sphere sphereFrom2(const Point3& a, const Point3& b) { return {midpoint(a, b), dist(a, b) / 2}; }
inline Sphere sphereFrom3(const Point3& a, const Point3& b, const Point3& c) {
    if (norm(cross(b - a, c - a)) <= EPS3 * max((ld)1, max({dist2(a, b), dist2(b, c), dist2(a, c)}))) {
        Sphere s = sphereFrom2(a, b);
        for (auto t : {sphereFrom2(b, c), sphereFrom2(a, c)}) if (t.r > s.r) s = t;
        return s;
    }
    Point3 cen = circumcenter3(a, b, c);
    return {cen, dist(cen, a)};
}
inline Sphere minEnclosingSphere(vector<Point3> p) {
    if (p.empty()) return {};
    shuffle(p.begin(), p.end(), mt19937(712367));
    auto out = [](const Sphere& s, const Point3& q) { return dist(s.c, q) > s.r * (1 + 1e-12L) + EPS3; };
    auto from4 = [&](const Point3& a, const Point3& b, const Point3& c, const Point3& d) {
        Sphere s;
        if (circumsphere(a, b, c, d, s) && s.r < 1e18) return s;
        Sphere best{Point3(), 1e30L};
        Point3 q[4] = {a, b, c, d};
        for (int i = 0; i < 4; i++) {
            Sphere t = sphereFrom3(q[(i + 1) % 4], q[(i + 2) % 4], q[(i + 3) % 4]);
            if (!out(t, q[i]) && t.r < best.r) best = t;
        }
        return best;
    };
    Sphere s{p[0], 0};
    int n = p.size();
    for (int i = 1; i < n; i++) if (out(s, p[i])) {
        s = {p[i], 0};
        for (int j = 0; j < i; j++) if (out(s, p[j])) {
            s = sphereFrom2(p[i], p[j]);
            for (int k = 0; k < j; k++) if (out(s, p[k])) {
                s = sphereFrom3(p[i], p[j], p[k]);
                for (int l = 0; l < k; l++) if (out(s, p[l])) s = from4(p[i], p[j], p[k], p[l]);
            }
        }
    }
    return s;
}
// Geometría esférica: punto desde latitud/longitud (grados) y distancia por círculo máximo.
inline Point3 fromLatLon(ld latDeg, ld lonDeg, ld R) {
    ld la = toRad(latDeg), lo = toRad(lonDeg);
    return {R * cosl(la) * cosl(lo), R * cosl(la) * sinl(lo), R * sinl(la)};
}
// Distancia sobre la esfera de radio R entre dos puntos de su superficie (centro en el origen)
//   = R · ángulo central (atan2 estable).
inline ld greatCircleDist(const Point3& u, const Point3& v, ld R) {
return R * angle(u, v); }
// 7. POLIEDROS, PRISMAS Y MALLAS
inline ld prismVolume(ld baseArea, ld h) { return baseArea * h; }
// prisma / cilindro
inline ld pyramidVolume(ld baseArea, ld h) { return baseArea * h / 3; }    // pirámide / cono
template<class T> Wide<T> parallelepipedVolume(const P3<T>& a, const P3<T>& b, const P3<T>& c) {
    Wide<T> v = triple(a, b, c); return v < 0 ? -v : v;
}
// Malla triangular CERRADA y con orientación CONSISTENTE (todas las caras antihorarias vistas
//   desde fuera). 6V = Σ v0·(v1×v2) (tetraedros con el origen). Exacto con enteros.
//   Resultado < 0 => las caras están orientadas hacia adentro (invertir todas).
//   Si la malla no es cerrada el valor depende del origen y NO es un volumen.
using Face = array<int, 3>;
template<class T> Wide<T> meshVolume6(const vector<P3<T>>& v, const vector<Face>& f) {
    Wide<T> s = 0;
    for (auto& t : f) s += triple(v[t[0]], v[t[1]], v[t[2]]);
    return s;
}
template<class T> ld meshSurface(const vector<P3<T>>& v, const vector<Face>& f) {
    ld s = 0;
    for (auto& t : f) s += triArea(v[t[0]], v[t[1]], v[t[2]]);
    return s;
}
// Centroide del SÓLIDO (no de la superficie): Σ 6V_i (v0+v1+v2)/4 / Σ 6V_i. Malla cerrada y
//   orientada con volumen != 0.
template<class T> Point3 meshCentroid(const vector<P3<T>>& v, const vector<Face>& f) {
    Point3 acc; ld tot = 0;
    for (auto& t : f) {
        Point3 a(v[t[0]]), b(v[t[1]]), c(v[t[2]]);
        ld w = triple(a, b, c); tot += w; acc += (a + b + c) * (w / 4);
    }
    return acc / tot;
}
// ¿Cerrada y consistente? Cada arista dirigida (u,v) aparece una vez y (v,u) también.
inline bool meshClosedConsistent(const vector<Face>& f) {
    map<pair<int, int>, int> e;
    for (auto& t : f) for (int i = 0; i < 3; i++) if (++e[{t[i], t[(i + 1) % 3]}] > 1) return false;
    for (auto& [k, c] : e) if (!e.count({k.second, k.first})) return false;
    return true;
}
// Uso: orientar consistentemente una malla cerrada conexa (BFS sobre caras vecinas) y luego
//   hacia afuera (volumen positivo). Devuelve false si no es orientable / no es manifold.
template<class T> bool orientMesh(const vector<P3<T>>& v, vector<Face>& f) {
    int m = f.size();
    if (m == 0) return true;
    map<pair<int, int>, vector<int>> byEdge;
    for (int i = 0; i < m; i++) for (int j = 0; j < 3; j++) {
        int a = f[i][j], b = f[i][(j + 1) % 3];
        byEdge[{min(a, b), max(a, b)}].pb(i);
    }
    vector<int> seen(m, 0);
    auto hasDirected = [&](int i, int a, int b) {
        for (int j = 0; j < 3; j++) if (f[i][j] == a && f[i][(j + 1) % 3] == b) return true;
        return false;
    };
    for (int s = 0; s < m; s++) if (!seen[s]) {
        seen[s] = 1; vector<int> st = {s};
        while (!st.empty()) {
            int i = st.back(); st.pop_back();
            for (int j = 0; j < 3; j++) {
                int a = f[i][j], b = f[i][(j + 1) % 3];
                auto& lst = byEdge[{min(a, b), max(a, b)}];
                if (lst.size() != 2) return false;
                int o = lst[0] == i ? lst[1] : lst[0];
                bool bad = hasDirected(o, a, b);   // el vecino debe usar (b,a)
                if (!seen[o]) { if (bad) swap(f[o][1], f[o][2]); seen[o] = 1; st.pb(o); }
                else if (bad) return false;
            }
        }
    }
    if (meshVolume6(v, f) < 0) for (auto& t : f) swap(t[1], t[2]);
    return true;
}
// Polígono plano en 3D (vértices en orden): vector área (Newell) = Σ p_i × p_{i+1}.
//   |·|/2 = área, dirección = normal (regla de la mano derecha según el orden).
template<class T> P3<T> polygonVectorArea2(const vector<P3<T>>& poly) {
    P3<T> s;
    for (size_t i = 0; i < poly.size(); i++) s += cross(poly[i], poly[(i + 1) % poly.size()]);
    return s;
}
// Poliedro con caras poligonales PLANAS y CONVEXAS orientadas hacia afuera: triangulación en
//   abanico (v0, vi, vi+1) de cada cara. Retorna 6V exacto.
template<class T> Wide<T> polyhedronVolume6(const vector<P3<T>>& v, const vector<vector<int>>& faces) {
    Wide<T> s = 0;
    for (auto& f : faces) for (size_t i = 1; i + 1 < f.size(); i++) s += triple(v[f[0]], v[f[i]], v[f[i + 1]]);
    return s;
}
```

§7.4.3 c-geo3d-avanzado.cpp

07_Geometria/Geometria_3D

```
// Hull 3D exacto O(n^2), par más cercano, triángulo-triángulo, OBB (SAT), rayo-malla,
// tetraedro mínimo por un punto. Todo probado contra fuerza bruta.
#include "b-geo3d-solidos.cpp"
// 1. ENVOLVENTE CONVEXA 3D (incremental, exacta con enteros |coord| <= 1e9)
```

```cpp
// Uso: Hull3 h = convexHull3D(pts);
// Retorna: dim = -1 (vacío), 0 (todos iguales), 1 (colineales), 2 (coplanares: usar hull 2D
//   en Plane::to2D o dropAxis), 3 (sólido). Con dim == 3, faces = triángulos con índices
//   ORIGINALES de pts, orientados antihorario vistos desde fuera (normal hacia afuera).
// Caras coplanares salen trianguladas (usar countFacets para contar caras reales). Un punto
//   que cae sobre la envolvente ya construida no se agrega; uno agregado antes puede quedar
//   luego sobre una arista/cara y seguir en la triangulación (área y volumen no cambian).
//   Esquinas reales: hullExtremeVertices.
// Complejidad: O(n·F) = O(n^2) peor caso; n <= ~5000 cómodo.
struct Hull3 { int dim = -1; vector<Face> faces; };
inline Hull3 convexHull3D(const vector<IPoint3>& P) {
    Hull3 h;
    int n = P.size();
    if (n == 0) return h;
    vector<int> ord(n);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) { return P[a] < P[b]; });
    ord.erase(unique(ord.begin(), ord.end(), [&](int a, int b) { return P[a] == P[b]; }), ord.end());
    // Tetraedro inicial con vértices EXTREMOS (si no, un punto interior a una cara quedaría como
    //   vértice): i0 = mínimo y i1 = máximo lexicográfico; i2 = más lejano a la recta i0i1
    //   (empate: menor proyección sobre la recta); i3 = mayor |volumen| (empate: menor lexicográfico).
    int m = ord.size(), i0 = ord[0], i1 = ord[m - 1], i2 = -1, i3 = -1;
    h.dim = 0;
    if (m == 1) return h;
    h.dim = 1;
    IPoint3 dir = P[i1] - P[i0];
    unsigned __int128 bestD = 0; i128 bestDot = 0;
    for (int k : ord) {
        IPoint3 c = cross(P[k] - P[i0], dir);
        unsigned __int128 d = 0;
        for (int j = 0; j < 3; j++) d += (unsigned __int128)((i128)c[j] * c[j]);
        i128 dt = dot(P[k], dir);
        if (d > bestD || (d == bestD && d > 0 && dt < bestDot)) { bestD = d; bestDot = dt; i2 = k; }
    }
    if (i2 < 0) return h;
    h.dim = 2;
    i128 bestV = 0;
    for (int k : ord) {
        i128 v = triple(P[i1] - P[i0], P[i2] - P[i0], P[k] - P[i0]);
        if (v < 0) v = -v;
        if (v > bestV) { bestV = v; i3 = k; }
    }
    if (i3 < 0) return h;
    h.dim = 3;
    vector<Face> F;
    auto addOut = [&](int a, int b, int c, int inside) {   // orienta la cara hacia afuera
        if (orient(P[a], P[b], P[c], P[inside]) > 0) swap(b, c);
        F.pb({a, b, c});
    };
    addOut(i0, i1, i2, i3); addOut(i0, i1, i3, i2); addOut(i0, i2, i3, i1); addOut(i1, i2, i3, i0);
    unordered_set<ll> vis;
    for (int p : ord) {
        if (p == i0 || p == i1 || p == i2 || p == i3) continue;
        vis.clear();
        vector<char> isVis(F.size(), 0);
        bool any = false;
        for (size_t f = 0; f < F.size(); f++)
            if (orient(P[F[f][0]], P[F[f][1]], P[F[f][2]], P[p]) > 0) {
                isVis[f] = 1; any = true;
                for (int j = 0; j < 3; j++) vis.insert((ll)F[f][j] * n + F[f][(j + 1) % 3]);
            }
        if (!any) continue;                                  // dentro o sobre la envolvente
        vector<Face> G;
        for (size_t f = 0; f < F.size(); f++) {
            if (!isVis[f]) { G.pb(F[f]); continue; }
            for (int j = 0; j < 3; j++) {                    // arista del horizonte: (u,v) visible, (v,u) no
                int u = F[f][j], v = F[f][(j + 1) % 3];
                if (!vis.count((ll)v * n + u)) G.pb({u, v, p});
            }
        }
        F.swap(G);
    }
    h.faces = F;
    return h;
}
// Volumen x6 (exacto) y área de la envolvente (dim == 3).
inline i128 hullVolume6(const vector<IPoint3>& P, const Hull3& h) { return meshVolume6(P, h.faces); }
inline ld hullSurface(const vector<IPoint3>& P, const Hull3& h) { return meshSurface(P, h.faces); }
// Número de caras planas reales (une triángulos coplanares por su normal primitiva).
inline int countFacets(const vector<IPoint3>& P, const Hull3& h) {
    set<IPoint3> normals;
    for (auto& f : h.faces) normals.insert(normDir(triNormal(P[f[0]], P[f[1]], P[f[2]]), false));
    return normals.size();
}
// Índices de los vértices de la TRIANGULACIÓN (puede incluir puntos sobre aristas o caras).
inline vector<int> hullVertices(const Hull3& h) {
    vector<int> r;
    for (auto& f : h.faces) for (int x : f) r.pb(x);
    sort(r.begin(), r.end()); r.erase(unique(r.begin(), r.end()), r.end());
    return r;
}
// Índices de los vértices EXTREMOS (esquinas reales): les inciden >= 3 caras planas distintas.
inline vector<int> hullExtremeVertices(const vector<IPoint3>& P, const Hull3& h) {
    map<int, set<IPoint3>> inc;
    for (auto& f : h.faces) {
        IPoint3 nr = normDir(triNormal(P[f[0]], P[f[1]], P[f[2]]), false);
        for (int x : f) inc[x].insert(nr);
    }
    vector<int> r;
    for (auto& [v, s] : inc) if (s.size() >= 3) r.pb(v);
    return r;
}
// 2. PAR MÁS CERCANO EN 3D (rejilla aleatorizada, exacto)
// Uso: auto [i, j] = closestPair3D(pts, d2); d2 = distancia^2 mínima (exacta, __int128).
// Idea: con celdas de lado s >= d actual, un punto a distancia < d está en las 27 celdas
//   vecinas. Si d mejora, se reconstruye la rejilla (O(n) esperado en orden aleatorio).
// Requiere n >= 2. |coord| <= 1e9.
inline pair<int, int> closestPair3D(const vector<IPoint3>& P, i128& best) {
    int n = P.size();
    vector<int> o(n);
    iota(o.begin(), o.end(), 0);
    shuffle(o.begin(), o.end(), mt19937(20261105));
    pair<int, int> res = {o[0], o[1]};
    best = dist2(P[o[0]], P[o[1]]);
    auto fdiv = [](ll a, ll b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); };
    auto hsh = [](ll x, ll y, ll z) {
        unsigned long long h = (unsigned long long)x * 0x9E3779B97F4A7C15ULL;
        h ^= (unsigned long long)y * 0xC2B2AE3D27D4EB4FULL + (h << 6) + (h >> 2);
        h ^= (unsigned long long)z * 0x165667B19E3779F9ULL + (h << 6) + (h >> 2);
        return h;
    };
    ll s = 1;
    unordered_map<unsigned long long, vector<int>> grid;
    auto key = [&](const IPoint3& p) { return hsh(fdiv(p.x, s), fdiv(p.y, s), fdiv(p.z, s)); };
    auto rebuild = [&](int upto) {
        ll r = (ll)sqrtl((ld)best);
        while ((i128)r * r < best) r++;
        while (r > 1 && (i128)(r - 1) * (r - 1) >= best) r--;
        s = max(1LL, r);
        grid.clear();
        for (int k = 0; k <= upto; k++) grid[key(P[o[k]])].pb(o[k]);
    };
    if (best == 0) return res;
    rebuild(1);
    for (int i = 2; i < n; i++) {
        const IPoint3& p = P[o[i]];
        ll cx = fdiv(p.x, s), cy = fdiv(p.y, s), cz = fdiv(p.z, s);
        bool improved = false;
        for (ll dx = -1; dx <= 1; dx++) for (ll dy = -1; dy <= 1; dy++) for (ll dz = -1; dz <= 1; dz++) {
            auto it = grid.find(hsh(cx + dx, cy + dy, cz + dz));
            if (it == grid.end()) continue;
            for (int q : it->second) {
                i128 d = dist2(p, P[q]);
                if (d < best) { best = d; res = {q, o[i]}; improved = true; }
            }
        }
        if (best == 0) return res;
        if (improved) rebuild(i);
        else grid[key(p)].pb(o[i]);
    }
    return res;
}
// 3. INTERSECCIÓN TRIÁNGULO-TRIÁNGULO 3D (exacta, triángulos no degenerados)
// Idea: si se cortan, algún lado de uno toca al otro triángulo (no coplanares: los extremos
//   del segmento de corte están en bordes; coplanares: cruce de lados o contención).
// Complejidad: O(1) (6 pruebas segmento-triángulo). Contacto en un punto cuenta.
inline bool triTriIntersectExact(const array<IPoint3, 3>& A, const array<IPoint3, 3>& B) {
    for (int i = 0; i < 3; i++) {
        if (segTriangleExact(A[i], A[(i + 1) % 3], B[0], B[1], B[2])) return true;
        if (segTriangleExact(B[i], B[(i + 1) % 3], A[0], A[1], A[2])) return true;
```

```
    }
    return false;
}
// 4. CAJAS ORIENTADAS (OBB) — Teorema del eje separador
// OBB: centro c, ejes ortonormales u[0..2] (base derecha), semilados
e[0..2] >= 0.
// Dos convexos son disjuntos <=> existe un eje que separa sus
proyecciones. Para dos OBB
//   basta probar 15 ejes: 3 de A, 3 de B y 9 productos cruz. Tocarse
cuenta como intersección.
// Aproximado (long double). AbsR + EPS evita falsos "separados" con
aristas paralelas.
struct OBB { Point3 c; Point3 u[3]; ld e[3]; };
inline OBB obbFromBox(const Box3& b) {
    OBB o;
    o.c = (Point3(b.lo) + Point3(b.hi)) / 2;
    o.u[0] = {1, 0, 0}; o.u[1] = {0, 1, 0}; o.u[2] = {0, 0, 1};
    for (int k = 0; k < 3; k++) o.e[k] = (ld)(b.hi[k] - b.lo[k]) / 2;
    return o;
}
inline bool pointInOBB(const OBB& o, const Point3& p) {
    Point3 d = p - o.c;
    for (int k = 0; k < 3; k++) if (fabsl(dot(d, o.u[k])) > o.e[k] +
EPS3) return false;
    return true;
}
inline bool obbIntersect(const OBB& A, const OBB& B) {
    ld R[3][3], AR[3][3], T[3];
    for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) { R[i][j]
= dot(A.u[i], B.u[j]); AR[i][j] = fabsl(R[i][j]) + EPS3; }
    Point3 t = B.c - A.c;
    for (int i = 0; i < 3; i++) T[i] = dot(t, A.u[i]);
    for (int i = 0; i < 3; i++) {
        ld rb = B.e[0] * AR[i][0] + B.e[1] * AR[i][1] + B.e[2] *
AR[i][2];
        if (fabsl(T[i]) > A.e[i] + rb) return false;
    }
    for (int j = 0; j < 3; j++) {
        ld ra = A.e[0] * AR[0][j] + A.e[1] * AR[1][j] + A.e[2] *
AR[2][j];
        if (fabsl(T[0] * R[0][j] + T[1] * R[1][j] + T[2] * R[2][j]) >
ra + B.e[j]) return false;
    }
    for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) {      //
eje A.u[i] × B.u[j]
        int i1 = (i + 1) % 3, i2 = (i + 2) % 3, j1 = (j + 1) % 3, j2
= (j + 2) % 3;
        ld ra = A.e[i1] * AR[i2][j] + A.e[i2] * AR[i1][j];
        ld rb = B.e[j1] * AR[i][j2] + B.e[j2] * AR[i][j1];
        if (fabsl(T[i2] * R[i1][j] - T[i1] * R[i2][j]) > ra + rb)
return false;
    }
    return true;
}
// 5. RAYO CONTRA MALLA
// Uso: primer impacto del rayo o + t·dir (t >= 0) con una malla
triangular. O(F).
//   Retorna (t, índice de cara) o nullopt. Para muchas consultas:
BVH/rejilla (no incluido).
inline optional<pair<ld, int>> rayMesh(const Point3& o, const Point3&
dir, const vector<Point3>& v, const vector<Face>& f) {
    optional<pair<ld, int>> best;
    for (int i = 0; i < (int)f.size(); i++) {
        auto t = rayTriangle(o, dir, v[f[i][0]], v[f[i][1]], v[f[i]
[2]]);
        if (t && (!best || *t < best->first)) best = make_pair(*t,
i);
    }
    return best;
}
// 6. OPTIMIZACIÓN: tetraedro de volumen mínimo cortado por un plano
que pasa por P
// Ángulo triedro con vértice en el ORIGEN y aristas en direcciones
A, B, C; P interior.
// P = αA + βB + γC (α,β,γ > 0, por Cramer con productos triples). Un
plano que corta las
//   aristas en sA, tB, uC contiene a P <=> α/s + β/t + γ/u = 1. V =
s·t·u·|det(A,B,C)|/6.
//   Por AM-GM el mínimo es con α/s = β/t = γ/u = 1/3 (P = centroide
del triángulo de corte):
//   V_min = 27αβγ·|det|/6 = 4.5·αβγ·|det|. Si el vértice no es el
origen, trasladar antes.
inline ld minTetraThroughPoint(const Point3& A, const Point3& B,
const Point3& C, const Point3& P) {
    ld D = triple(A, B, C);
    ld al = triple(P, B, C) / D, be = triple(A, P, C) / D, ga =
triple(A, B, P) / D;
    return 4.5L * al * be * ga * fabsl(D);
}
```

§7.4.4 cheat-sheet-3d.md

07_Geometria/Geometria_3D

PARTE D — Cheat sheet 3D

Fórmulas directas, trucos aritméticos, reducciones a 2D/1D y riesgos numéricos. Las funciones citadas están en a-geo3d-esencial.cpp (A), b-geo3d-solidos.cpp (B) y c-geo3d-avanzado.cpp (C).

1. Vectores en una línea

Qué	Fórmula	Función
Punto	`a·b = ax bx + ay by + az bz`	`dot` (A)
Cruz	`a×b = (ay bz − az by, az bx − ax bz, ax by − ay bx)`	`cross` (A)
Triple	`[a,b,c] = a·(b×c) = det[a;b;c]`, cíclico: `[a,b,c]=[b,c,a]=−[b,a,c]`	`triple` (A)
Orientación	`sgn((B−A)·((C−A)×(D−A)))`; 0 ⇔ coplanares	`orient` (A)
Colineales	`(B−A)×(C−A) = 0`	`collinear` (A)
Paralelos / perpendiculares	`u×v = 0` / `u·v = 0`	`parallelVec`, `perpendicularVec` (A)
Ángulo	`atan2(|u×v|, u·v)` ∈ [0,π] (no acos)	`angle` (A)
Proyección de u sobre v	`(u·v / |v|²) v`	`projVec` (A)
Punto medio exacto	guardar `2M = A+B`	`midpoint2` (A)
Perpendicular a v	`v × (eje menos alineado con v)`	`anyPerp` (A)

Cuándo NO hace falta nada de esto. Si solo hay que comparar distancias, usa `dist2` y no hagas `sqrt`. Para "¿qué punto está más alto?" basta comparar la z. Si el enunciado da cajas alineadas a los ejes, cada eje se resuelve por separado como un intervalo. Ejemplo: "¿P está a distancia ≤ R de C?" con enteros se resuelve con `dist2(P,C) <= R*R`. El resultado es exacto, sin EPS.

2. Áreas y volúmenes

Objeto	Fórmula
Triángulo ABC	`A = |(B−A)×(C−A)| / 2` (en 3D no hay "área con signo" escalar: el signo es la dirección de la normal)
Tetraedro ABCD	`V = |(B−A)·((C−A)×(D−A))| / 6`; 6V con signo es exacto con enteros
Paralelepípedo (a,b,c)	`|[a,b,c]|`
Prisma / cilindro	`área base × altura`
Pirámide / cono	`área base × altura / 3`

Objeto	Fórmula
Caja	`V = a·b·c`, `S = 2(ab+bc+ca)`, `diagonal² = a²+b²+c²`
Esfera	`S = 4πr²`, `V = 4πr³/3`
Casquete (altura h)	`S = 2πRh` (lateral), `V = πh²(3R−h)/3`
Lente (dos bolas, d)	`a = (d²+r1²−r2²)/(2d)`, `V = cap(r1, r1−a) + cap(r2, r2−(d−a))`
Cuerda a distancia d	`2√(r²−d²)`
Malla cerrada orientada	`6V = Σ v0·(v1×v2)` (caras antihorarias vistas desde fuera)
Polígono plano en 3D	`2A·n̂ = Σ p_i × p_{i+1}` (Newell)
Altura de tetraedro	`h = |6V| / |(B−A)×(C−A)|`
Distancia en esfera	`R · ángulo central`; lat/long → `(R cos φ cos λ, R cos φ sin λ, R sin φ)`

El producto triple. `[a,b,c]` es el volumen con signo del paralelepípedo generado por a, b y c. Es positivo si (a,b,c) forma una terna derecha. En un tetraedro, el signo dice de qué lado del plano ABC queda D; el **volumen geométrico** es el valor absoluto. Para sumar volúmenes de una malla hay que usar el valor **con signo**: si cada cara se toma en valor absoluto, las partes que quedan fuera del sólido se suman en lugar de cancelarse.

3. Trucos aritméticos directos

Truco	Fórmula / idea	Ejemplo	Implementación	Prob típ
Intersección de intervalos	`max(0, min(r1,r2) − max(l1,l2))`	[0,5]∩[3,10] → 2	`overlapLen` (B)	ABC36
Volumen de intersección de cajas	producto de las 3 longitudes	ver abajo	`interVolume` (B)	ABC34
¿Volumen positivo?	**cada** eje con `min(r) − max(l) > 0` (estricto)	cajas tocándose → no	`overlapPositive` (B)	ABC36
Contacto cara/arista/vértice	por eje `g = min(hi) − max(lo)`; nº de ejes con g = 0: 1 cara, 2 arista, 3 vértice	cubos [0,1]³ y [1,2]×[0,1]² → cara	`boxContact`, `contactArea` (B)	ABC31
Unión de 2 cajas	`|A|+|B|−|A∩B|`		`unionVolume2` (B)	
Exactamente 1/2/3 de 3 cajas	`V3=S3`, `V2=S2−3S3`, `V1=S1−2S2+3S3`		`exactlyCovered3` (B)	ABC34
Unión de N cajas	compresión de coordenadas + diferencias 3D, O(N³)		`coverVolumes`, `unionVolume` (B)	
Distancia sin raíz	comparar `dist2`	`dist2 ≤ R²`	`dist2` (A)	casi to
Máx. Manhattan	`max` sobre signos (1, ±1, ±1) de `max f − min f`, `f = x ± y ± z`	O(4n)	`maxManhattan` (A)	
Máx. Chebyshev	máximo por eje de `max − min`		`maxChebyshev` (A)	
Puntos enteros en un segmento	`gcd(|dx|,|dy|,|dz|) + 1`	(0,0,0)–(2,4,6) → 3	`latticePointsSegment` (A)	
Dirección canónica	dividir por el MCD; para rectas, el primer componente no nulo > 0	(−2,4,−6) → (1,−2,3)	`normDir` (A)	conte paral
Pares paralelos	agrupar por `normDir`		`countParallelPairs` (A)	
Máx. colineales	por cada i, histograma de `normDir(pj−pi)`	O(n² log n)	`maxCollinear` (A)	

Truco	Fórmula / idea	Ejemplo	Implementación	Problema típico
Coplanares con un plano	`orient(a,b,c,p) == 0`		`countOnPlane`, `IPlane` (A/B)	
Puntos enteros en una caja	Π `(lado+1)`; cubitos = volumen		`latticePointsInBox` (B)	
Equidistantes	`dist2(a,p) == dist2(a,q)`		`equidistant` (A)	
Paridad en rejilla 3D	cada paso de 1 unidad cambia la paridad de `x+y+z` (coloreo de tablero 3D)		—	alcanzabilidad
Traslación	trasladar todo para fijar un punto en el origen: no cambia distancias, ángulos ni volúmenes		—	
Simetrías	cambiar signos o permutar coordenadas conserva las distancias (las 48 isometrías del cubo)		`cubeRotations(true)` (A)	figuras salvo simetría
Proyección	si la condición es por eje, resolver cada eje como un problema 1D		`dropAxis` (B)	

Cuándo multiplicar y cuándo comparar. Se multiplican las tres longitudes `overlapLen` cuando piden un **volumen**; el resultado ya vale 0 si algún eje no se solapa. Cuando piden un **sí/no** con volumen positivo, se compara que cada `min(r) − max(l)` sea **> 0**. Cuando solo importa que se toquen (contacto), basta **≥ 0**. Para contacto de área positiva, un eje debe dar exactamente 0 y los otros dos > 0.

Ejemplo (ABC343 E): los cubos $[0,7]^3$ y $[0,7]\times[6,13]\times[0,7]$ se solapan $7\cdot1\cdot7 = 49$ por inclusión-exclusión, sin construir ningún sólido.

4. Rotaciones y transformaciones

- **90° exactos (sin trigonometría):** sobre Z, `(x,y,z) → (−y, x, z)`. Sobre X, `(x,y,z) → (x, −z, y)`. Sobre Y, `(x,y,z) → (z, y, −x)`. Para 180° sobre Z se cambian los dos signos: `(−x, −y, z)`. 270° equivale a −90°. Son `rotX90/rotY90/rotZ90(p, k)` (A).
- **24 orientaciones del cubo:** son las permutaciones de ejes con cambios de signo de determinante +1 (`cubeRotations()`). Con reflexiones hay 48 (`cubeRotations(true)`). Para comparar figuras de voxeles "salvo rotación": por cada orientación se aplica, se traslada al mínimo, se ordena y se queda la menor como forma canónica.
- **Rotación ≠ reflexión.** Una reflexión invierte `orient` y el signo de los volúmenes (`det = −1`). Si el problema distingue piezas "en espejo" (quiralidad), usa solo las 24 rotaciones.
- **Ángulo arbitrario sobre un eje coordenado:** `rotXMat/rotYMat/rotZMat(t)` (antihorario mirando desde +eje). La composición es `(A*B)p = A(Bp)` y la inversa de una rotación es su transpuesta.
- **Eje arbitrario (Rodrigues):** $R = I\cos t + (1-\cos t)\,k\,k^T + \sin t\,[k]_\times$, con `rodrigues(axis, t)` y `rotateRodrigues(v, axis, t)`. Si el eje pasa por un punto o: `o + R(p−o)` (`rotateAroundLine`).
- **Afín 4×4:** `Mat4::translate`, `scale`, `fromMat3`; `apply` trata puntos (w=1) y `applyVec` vectores (w=0). Para rotar alrededor de un punto: `T(c)·R·T(−c)` (`rotateAroundPoint`).
- **Cambio de base:** `Frame(o, w)` construye una base ortonormal derecha {u,v,w}; `toLocal` y `toGlobal` son inversas. Con w = normal de un plano, las coordenadas (x,y) locales son geometría 2D dentro del plano.
- **Errores acumulados:** al encadenar muchas rotaciones flotantes conviene recalcular desde el ángulo total, o reortonormalizar. Con ángulos múltiplos de 90° usa siempre las versiones enteras.
- **Cuaterniones:** no se incluyen. En ICPC, Rodrigues más matrices cubren lo necesario, y los cuaterniones solo convienen cuando hay que interpolar orientaciones (SLERP), algo que casi no aparece.

5. Rectas, planos y distancias

Qué	Fórmula
Punto–recta	`|d × (Q−P)| / |d|`
Punto–plano (con signo)	`(n·Q − d) / |n|`
Recta–recta (no paralelas)	`|(P2−P1)·(d1×d2)| / |d1×d2|`
Clasificación de rectas	`d1×d2 = 0` → paralelas (coincidentes si además `d1×(P2−P1) = 0`); si no, `[P2−P1, d1, d2] = 0` → se cortan; si no, **alabeadas**
Perpendicular común	`n = d1×d2`, `c1 = P1 + d1 ((P2−P1)·(d2×n))/(d1·(d2×n))`
Recta ∩ plano	`t = −(n·P − d)/(n·dir)`; si `n·dir = 0`, paralela (o contenida)
Plano ∩ plano	dirección `n1×n2`; punto α `n1` + β `n2` resolviendo un sistema 2×2
Tres planos	`x = (d1(n2×n3) + d2(n3×n1) + d3(n1×n2)) / [n1,n2,n3]`
Reflexión en un plano	`P − 2 (n·P − d) n / |n|²`
Ángulo diedro	`atan2(|n1×n2|, |n1·n2|)` ∈ [0, π/2]
Recta–plano	`atan2(|d·n|, |d×n|)` ∈ [0, π/2]
Esfera por 4 puntos	`c = A + (|u|²(v×w) + |v|²(w×u) + |w|²(u×v)) / (2[u,v,w])`
Circuncentro 3D	`c = A + ((w×u)|v|² + (v×w)|u|²) / (2|w|²)`, con `w = u×v`

6. Geometría 3D que se reduce a 2D o 1D

Patrón	Reducción	Ejemplo sin cruz ni planos
Cajas alineadas que se cortan o tocan	3 problemas 1D de intervalos	ABC361 B: `min(r)−max(l) > 0` en x, y, z
Volumen = área 2D × altura	prisma de sección constante	ABC144 D: la botella inclinada se reduce a la sección a×b (rectángulo de lados a y b)
Sólido cortado por un plano paralelo a una cara	sección 2D	el volumen bajo z = h es el área de la base × h
Proyección a XY/XZ/YZ	`dropAxis(p, k)` quita una coordenada	"sombra" de puntos sobre el piso: geometría 2D en (x,y)
Plano general	`Plane::to2D/Frame`	puntos coplanares → envolvente 2D en coordenadas locales
Colisión de cajas	condición por eje	tres comparaciones en lugar de SAT
Rotación 3D sobre un eje coordenado	rotación 2D de las otras dos coordenadas	rotar sobre Z: (x,y) gira y z queda fija
Búsqueda espacial	ordenar por coordenada o usar una rejilla	par más cercano: rejilla de celdas de lado d
Consultas de suma en cubos	prefix sums 3D (8 esquinas)	ABC366 D (`Pref3`)
Muchas cajas pintadas	difference array 3D	`Diff3`
Celdas conectadas, cavidades	BFS/DFS en voxeles (6 vecinos)	`voxelSurface(…, true)`
Coordenadas enormes, pocas cajas	compresión de coordenadas y luego flood fill	`coverVolumes`
Conteos de regiones superpuestas	inclusión-exclusión	`exactlyCovered3`
Problema invariante por traslación	fijar un objeto en el origen	ABC343 E: primer cubo en (0,0,0)
Posiciones relativas	trabajar con diferencias `pj − pi`	conteo de colineales con `normDir`
"Vistas" de pilas de cubos	máximos por fila y por columna (2D)	WF 2017 Mission Improbable

7. Estabilidad numérica

Tamaño de los productos (C = máx |coordenada|; las diferencias llegan a 2C)

Cantidad	Grado	Cota	Tipo seguro
diferencias, Manhattan	1	2C	`long long`
componente de `cross`	2	$8C^2$	`long long` hasta C = 1e9 (8e18 < 9.22e18)
`dot`, `norm2`, `dist2`	2	$12C^2$	**`__int128`** si C > ~8.7e8 (la biblioteca ya devuelve `__int128`)
`triple`, `orient`, `6V`, `IPlane::classify`	3	$\sim 48C^3$	`__int128` hasta C ≈ 1e12
`norm2(cross)` = (2·área)², producto punto de dos normales	4	$192C^4$	`__int128` hasta C ≈ 9e8; `unsigned __int128` hasta 1e9
prueba "dentro de la esfera" por 4 puntos (determinante 5×5)	5	$\sim C^5$	**no alcanza** con C ≈ 1e8: hace falta `long double` (aproximado) o enteros grandes

Truco para evitar grado 4: para saber de qué lado está un punto **dentro de un plano** basta el signo de la componente dominante k de la cruz (`pointInTriangleExact`, `segSegIntersectExact`), que es de grado 2, en lugar de `dot(cross, normal)`.

Riesgos y cómo los maneja la biblioteca

Riesgo	Manejo
Overflow en cruz o triple	cruz en `11` (C ≤ 1e9); triple siempre en `__int128`
Coordenadas reales con pocas cifras decimales	escalar a enteros (×10^k) y usar los predicados exactos
Cancelación numérica	restar primero (trabajar con `B−A`, `C−A`) y luego multiplicar
División entre valores cercanos a 0	todas las funciones flotantes comparan el denominador con EPS y devuelven un estado (`IK_NONE`, `false`, `nullopt`)
Vector nulo / normal nula	`unit` devuelve (0,0,0); `IPlane::valid`; `triDegenerate`; `closestPointTriangle` cae a los lados
Planos degenerados	`Plane::fromPoints` requiere puntos no colineales: comprobar `collinear` antes
Rectas alabeadas	`lineRelation` nunca supone que dos rectas no paralelas se cortan
Puntos casi coplanares	con enteros, `orient` exacto; con flotantes, el resultado depende de EPS: escalar EPS al tamaño de los datos
Esferas tangentes	`sphereSphere` y `planeSphere` devuelven estados de tangencia con tolerancia
`sqrt` de negativo	`safeSqrt`; los discriminantes se recortan
`acos` fuera de [−1,1]	`angle` usa `atan2`; `safeAcos` recorta
Entero ↔ flotante	`Point3(ipoint)` es explícito; `long double` representa exactamente enteros hasta 2^{64}
Errores tras rotaciones	usar las rotaciones enteras de 90°; reortonormalizar si se encadenan muchas
Comparaciones flotantes	`sgn3` con EPS3 = 1e-9 absoluto: ajustarlo (relativo) si las coordenadas son ~1e6 o ~1e-3

Riesgo	Manejo
Orientación de superficies	`meshClosedConsistent`, `orientMesh`; volumen negativo → caras hacia adentro

Exacto vs aproximado

- **Exactos** (IPoint3 con `|coord| ≤ 1e9`): `orient`, `collinear`, `coplanar`, `parallelVec`, `onSegment`, `segSegIntersectExact`, `lineRelationExact`, `pointInTriangleExact`, `segTriangleExact`, `pointInTetraExact`, `tetVolume6`, `meshVolume6`, `IPlane`, `cmpPointSphereExact`, `convexHull3D`, `closestPair3D`, `triTriIntersectExact`, todo lo de `Box3`.
- **Aproximados** (`long double`, EPS3): proyecciones, intersecciones con coordenadas, distancias con raíz, ángulos, esferas, Rodrigues, OBB y la esfera mínima.

Regla: **decide con los predicados exactos y construye después** con flotantes. Por ejemplo, primero `segTriangleExact` y, solo si da `true`, `segTrianglePoint`.

§7.4.5 README.md

07_Geometria/Geometria_3D

Geometría 3D para ICPC (C++20)

Esta biblioteca está pensada para usarse como caja negra. Usa **predicados exactos con enteros** (`IPoint3`, `__int128`) para decidir y **`long double`** para construir puntos, distancias y ángulos. Se compila con GNU G++20 sobre la plantilla `00_Template/simple.cpp` (`int = long long`).

Parte	Archivo	Contenido
A — Esencial	a-geo3d-esencial.cpp	`P3<T>`, `Point3`/`IPoint3`, punto/cruz/triple, `orient`, colineal/coplanar, distancias, Manhattan/Chebyshev, MCD y direcciones, rotaciones de 90° y las 24/48 del cubo, `Mat3`, Rodrigues, `Mat4`, `Frame`
B — Sólidos	b-geo3d-solidos.cpp	`Box3` (intersección, contacto, inclusión-exclusión, unión de N), `Pref3`/`Diff3`, voxeles, `Line3`, segmentos, `Plane`/`IPlane`, triángulos, tetraedros, `Sphere`, esfera mínima, mallas
C — Avanzado	c-geo3d-avanzado.cpp	envolvente convexa 3D exacta, par más cercano 3D, triángulo-triángulo, OBB (SAT), rayo-malla, tetraedro mínimo por un punto
D — Cheat sheet	cheat-sheet-3d.md	fórmulas, trucos aritméticos, rotaciones, reducciones a 2D/1D, overflow y precisión
E — ¿Qué uso?	este archivo	tabla de situación → función
F — Problemas	problemas-3d.md	14 problemas con enlaces, análisis y orden de práctica
G — Priorización	este archivo	imprescindible / recomendable / avanzado
Pruebas	_pruebas/geometria-3d.cpp	136 325 comprobaciones contra fuerza bruta (resultado)

Uso

```
#include "c-geo3d-avanzado.cpp"   // incluye B, que incluye A, que incluye la plantilla
signed main() {
    IPoint3 A(0,0,0), B(4,0,0), C(0,4,0), D(0,0,4);
    cout << toString128(tetVolume6(A, B, C, D)) << endl;   // 64 (= 6V, exacto)
    cout << tetVolume(A, B, C, D) << endl;                 // 10.6667
}
```

- Para copiar a mano, conserva la parte A y agrega solo las secciones de B o C que necesites; cada sección depende de las anteriores. Las tres partes juntas compilan sin avisos con `-Wall -Wextra`.
- `IPoint3` acepta `|coordenada| ≤ 1e9`: la cruz se guarda en `long long`, y `dot`, `norm2` y `triple` devuelven `__int128`. Para imprimir un `__int128` usa `toString128`.
- `Point3(ip)` convierte de entero a flotante. `EPS3 = 1e-9` es absoluto: escálalo según el tamaño de los datos.
- Las funciones devuelven estados explícitos (`LineRel`, `InterKind`, `SphereRel`, `BoxContact`, `optional`, `bool`). Ninguna supone que existe una intersección única.
- Los nombres evitan las macros de la plantilla (`pi`, `F`, `S`, `all`, `vi`, `vb`, `vs`).

Verificación

```
g++ -std=gnu++20 -O2 -Wall -Wextra _pruebas/geometria-3d.cpp -o <tmp.exe>   (desde note_book)
geometria-3d: 136325 comprobaciones OK
```

Qué se compara contra fuerza bruta: voxeles para cajas (1, 2 y 3 cajas, unión de N, prefix y difference 3D); conteos directos para Manhattan, Chebyshev, MCD, colineales y paralelos; muestreo fino para la distancia entre segmentos; un oráculo racional exacto para segmento-triángulo; volúmenes parciales para "punto en tetraedro"; subconjuntos de 2, 3 y 4 puntos para la esfera mínima; todas las ternas $O(n^4)$ para el número de caras de la envolvente; $O(n^2)$ para el par más cercano; muestreo exacto para triángulo-triángulo y OBB; y minimización numérica para Gym 104248 G.

Casos fijos: cubo unitario, cubos iguales, disjuntos, con contacto por cara, por arista o por vértice, rectas alabeadas, paralelas y coincidentes, planos coincidentes y paralelos, tetraedro de volumen 0, cuatro puntos coplanares, cuatro rotaciones de 90°, vectores nulos, triples con coordenadas de 2e9, esferas tangentes (externa e interna), cáscara de voxeles con cavidad, envolventes de rejillas $(L+1)^3$ con duplicados y los ejemplos de ABC312 E, ABC343 E y ABC144 D.

Además se corrió un estrés aparte, no guardado en el repo: 5 926 envolventes con muchos coplanares sin ninguna discrepancia, y la envolvente más el par más cercano con 20 000 puntos en 41 ms.

¿Cuánta maquinaria necesito?

Ve de arriba hacia abajo y detente en el primer nivel que resuelva el problema:

1. **Una multiplicación:** volumen de una caja, de un prisma o el número de cubitos.
2. **Comparar intervalos:** cajas alineadas que se cortan, se tocan o se contienen (`overlapLen`).
3. **Una distancia:** usa `dist2` si solo hay que comparar, `dist` si piden el valor, y Manhattan o Chebyshev si el movimiento es por ejes.
4. **Proyectar a 2D o 1D:** un plano horizontal, condiciones por eje, una sección constante o rotaciones sobre un eje (ver el cheat sheet, §6).
5. **Producto punto o cruz:** ángulos, perpendicularidad, normales, áreas, punto–recta.
6. **Producto triple:** volúmenes, coplanaridad, lado de un plano, rectas alabeadas.
7. **Algoritmo avanzado:** envolvente 3D, par más cercano, esfera mínima, SAT, triángulo-triángulo.

PARTE E — ¿Qué uso?

Situación en el enunciado	Técnica / función
Cubos o cajas alineadas que se superponen	`overlapLen` por eje, multiplicar: `interVolume`
"¿Intersección de volumen positivo?"	`overlapPositive` (> 0 en los 3 ejes)
"¿Comparten una cara?" / tipo de contacto	`boxContact`, `contactArea`
Volumen cubierto por exactamente k de 3 cajas	`exactlyCovered3`
Unión de muchas cajas	`coverVolumes` / `unionVolume` (compresión + `Diff3`)
Sumas sobre subcubos	`Pref3`
Sumar a muchos subcubos y consultar al final	`Diff3`
Figura de cubitos: área, cavidades, componentes	`voxelSurface`, BFS de 6 vecinos
Muchas cajas con coordenadas ≤ 100	rejilla de ids de 100^3 (ABC312 E)
Distancia entre puntos / ¿dentro del radio?	`dist`, `dist2` ≤ R^2
Distancia máxima en Manhattan	`maxManhattan` (4 combinaciones de signos)
Puntos enteros sobre un segmento	`latticePointsSegment` (MCD + 1)
Contar rectas o direcciones paralelas	`normDir`, `countParallelPairs`, `maxCollinear`
Rotar 90° sobre Z	`(x,y,z) → (−y, x, z)`: `rotZ90`
Piezas iguales "salvo rotación"	`cubeRotations()` (24); con espejo, 48
Rotar un ángulo cualquiera alrededor de un eje	`rodrigues`, `rotateAroundLine`
Componer traslaciones y rotaciones	`Mat4`
Puntos en un plano horizontal	geometría 2D en (x,y): `dropAxis(p, 2)`
Puntos en un plano cualquiera	`Plane::to2D` / `Frame` + biblioteca 2D
Volumen de un tetraedro	`tetVolume6` (producto triple), `/6`
¿Cuatro puntos coplanares?	`orient(a,b,c,d) == 0`
¿De qué lado del plano está un punto?	`orient` (exacto), `IPlane::classify`, `Plane::side`
Rectas en 3D: ¿se cortan?	`lineRelationExact` / `lineRelation` (¡alabeadas!)
Distancia entre rectas alabeadas	`distLineLine`, `closestPointsLines`
Distancia entre segmentos	`distSegSeg`
¿Se cortan dos segmentos 3D?	`segSegIntersectExact`
Proyección o reflexión en un plano	`Plane::proj`, `Plane::reflect`
Recta o segmento contra plano	`interLinePlane`, `interSegmentPlane`
Corte de 2 o 3 planos	`interPlanePlane`, `interThreePlanes`
Punto en un triángulo 3D / baricéntricas	`pointInTriangleExact`, `barycentric`
Distancia punto–triángulo	`distPointTriangle`
Rayo o segmento contra triángulo	`rayTriangle` (Möller–Trumbore), `segTriangleExact`
¿Se cortan dos triángulos?	`triTriIntersectExact`
Punto en un tetraedro	`pointInTetraExact`
Esfera: punto dentro/fuera, corte con recta o rayo	`cmpPointSphereExact`, `lineSphere`, `raySphere`
Esfera contra esfera o plano	`sphereSphere`, `planeSphere`, `sphereIntersectionVolume`
Esfera por 4 puntos	`circumsphere`
Punto que minimiza la distancia máxima	`minEnclosingSphere`
Distancia sobre la Tierra	`fromLatLon`, `greatCircleDist`
Volumen o área de una malla cerrada	`meshVolume6`, `meshSurface`, `orientMesh`
Envolvente convexa: volumen, área, caras	`convexHull3D`, `hullVolume6`, `hullSurface`, `countFacets`
Par de puntos más cercano	`closestPair3D`
Cajas rotadas que colisionan	`obbIntersect` (SAT, 15 ejes)
Primer triángulo que toca un rayo	`rayMesh`
Plano por P que corta el volumen mínimo de un triedro	`minTetraThroughPoint`

PARTE G — Priorización ICPC

IMPRESCINDIBLE (llevar siempre)

- P3 con dot, cross, triple, norm2, dist2, orient, collinear, coplanar.
- Intervalos por eje: overlapLen, interVolume, overlapPositive, boxContact, inclusión-exclusión de 2 y 3 cajas.
- dist2 en lugar de dist, maxManhattan, latticePointsSegment, normDir.
- Rotaciones de 90° con cambios de signo y la regla "rotación ≠ reflexión".
- Volumen de tetraedro (tetVolume6) y su signo.
- Punto–recta, punto–segmento, punto–plano y proyección.
- Prefix sums 3D y BFS sobre voxeles.
- Reducir a 2D o 1D (cheat sheet §6) y la tabla de overflow (§7).

RECOMENDABLE (aparece con frecuencia en regionales)

- Line3: relación entre rectas (alabeadas), distLineLine, closestPointsLines, distSegSeg, segSegIntersectExact.
- Plane: intersecciones recta/segmento/2 planos/3 planos, reflexión, to2D.
- Triángulos: área, pointInTriangleExact, closestPointTriangle, rayTriangle, segTriangleExact.
- Esferas: lineSphere, sphereSphere, planeSphere, casquetes, círculo máximo.
- Diff3, coverVolumes (compresión de coordenadas), cubeRotations.
- rodrigues, Mat3, Frame.
- Mallas: meshVolume6, meshSurface.

AVANZADO / POCO FRECUENTE

- convexHull3D (aparece, pero pocas veces; $O(n^2)$ alcanza casi siempre).
- closestPair3D, minEnclosingSphere (alternativa aproximada: descenso hacia el punto más lejano).
- triTriIntersectExact, obbIntersect, rayMesh, orientMesh.
- Cilindro mínimo (Stars in a Can), MST cinético (Asteroid Rangers): se construyen a partir de las piezas anteriores.
- **No incluido a propósito:** envolvente 3D O(n log n), KD-tree 3D, Delaunay 3D, cuaterniones, booleanas de poliedros y unión de esferas. Su costo de implementación es alto y casi no aparecen; si hace falta, el KD-tree 2D del notebook se generaliza agregando la coordenada z.

§7.5 Grafos planares

§7.5.1 planar-graph-dual.cpp

07_Geometria/Grafos_planares

```
using ii = pair<int, int>;
vi g[tam];int n; // input graph (must be connected)
vi gd[tam];int nd; // output graph
vi nodes[tam]; // nodes delimiting region (in CW order)
map<ii,int> ps,es;
void get_dual(vector<pt> &p, vi &nods){ // p: points corresponding to
nodes
    ps.clear();es.clear();
    for(int x : nods){
        pt p1, p2;
        auto cmp = [&](int a, int b){
            p1 = p[a] - p[x], p2 = p[b] - p[x];
            a = p1.x > 0 && p1.y == 0 || p1.y > 0, b = p2.x > 0 &&
p2.y == 0 || p2.y > 0;
            if(a == b)
                return p1 % p2 > 0;
            return a < b;
        };
        sort(all(g[x]), cmp);
        for (int i=(0), nb_fin_1=(g[x].size()); i<nb_fin_1;
++i)ps[{x,g[x][i]}]=i;
    }
    nd=0;
    for(int xx : nods)for(auto yy:g[xx])if(!es.count({xx,yy})){
        int x=xx,y=yy;gd[nd].clear();nodes[nd].clear();
        while(!es.count({x,y})){
            es[{x,y}]=nd;nodes[nd].pb(y);
            int z=g[y][(ps[{y,x}]+1)%g[y].size()];x=y;y=z;
        }
        nd++;
    }
    for(auto p:es){
        pair<int,int> q={p.f.s,p.f.f};
        assert(es.count(q));
        if(es[q]!=p.s)gd[p.s].pb(es[q]);
    }
    for (int i=(0), nb_fin_3=(nd); i<nb_fin_3; ++i){
        sort(all(gd[i]));
        gd[i].erase(unique(all(gd[i])),gd[i].end());
    }
}
```

§7.6 Minkowski

§7.6.1 minkowski-sum-of-convex-polygons.cpp

07_Geometria/Minkowski

```
using ll = long long;
typedef vector<point> poly;
void norm(poly &pol) {
    int pos = 0;
    for (int i=(0), nb_fin_1=(pol.size()); i<nb_fin_1; ++i) {
        if(pol[i] < pol[pos])
            pos = i;
    }
    rotate(pol.begin(), pol.begin() + pos, pol.end());
}
poly minkos(poly &a, poly &b) {
    norm(a);
    norm(b);
    int posa = 0, posb = 0, ta = a.size(), tb = b.size();
    poly res;
    ll cro;
    while(posa < ta || posb < tb) {
        res.pb(a[(posa) % ta] + b[(posb) % tb]);
        cro = (a[(posa + 1) % ta] - a[posa % ta]) ^ (b[(posb + 1) %
tb] - b[posb % tb]);
        if(cro == 0)
            posa++, posb++;
        else if(cro < 0)
            posb++;
        else posa++;
    }
    return res;
}
```

§7.7 Polígonos

§7.7.1 centroide.cpp

07_Geometria/Poligonos

```
// calcula el centro de masa de un poligono antihorario
point cen(vector<point> p) {
    double x = 0, y = 0, area = 0, ax;
    int n = p.size()-1;
    for (int i=(0), nb_fin_1=(n); i<nb_fin_1; ++i) {
        ax = (p[i] ^ p[i+1]) / 2;
        area += ax;
        x += ax * (p[i].x + p[i+1].x) / 3;
        y += ax * (p[i].y + p[i+1].y) / 3;
    }
    return point(x / area, y / area);
}
```

§7.7.2 lattice-points.cpp

07_Geometria/Poligonos

```
using ll = long long;
//calculates number of integer points (x ; y) such for 0 ≤ x < n and
0 < y ≤ |k * x + b|
int count_lattices(double k, double b, long long n) {
    auto fk = floor(k);
    auto fb = floor(b);
    auto cnt = 0LL;
    if (k > 1 - EPS || b > 1 -EPS) {
        cnt += (fk * (n - 1) + 2 * fb) * n / 2;
        k -= fk;
        b -= fb;
    }
    auto t = k * n + b;
    auto ft = floor(t);
    if (ft > 1 - EPS) {
        cnt += count_lattices(1 / k, (t - floor(t)) / k, floor(t));
    }
    return cnt;
}
pair<double, double> kb(point a, point b)
{
    return make_pair((b.y - a.y) / (b.x - a.x), (a.y * b.x - a.x *
b.y) / (b.x - a.x));
}
// # of lattice points s.t. ax+by<=c, x,y>0 (a,b is positive integer)
ll f(ll a, ll b, ll c){
    if(c<=0) return 0;
    if(a<b) swap(a, b);
    ll m=c/a;
    if(a==b) return m*(m-1)/2;
    ll k=(a-1)/b, h=(c-a*m)/b;
    return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
}
// # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=Y (a,b is positive
integer)
ll g(ll a, ll b, ll c, ll X, ll Y){
    if(a*X+b*Y<=c) return X*Y;
    return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c-a*X-b*Y);
}
```

§7.7.3 point-in-convex-polygon.cpp

07_Geometria/Poligonos

```
using ll = long long;
// Check if a point is in, on, or out a convex Polygon
// in O(Log n)
ll IN = 0;
ll ON = 1;
ll OUT = 2;
vector<string> ANS = {"IN", "ON", "OUT"};
#define pt pair<ll,ll>
#define x first
#define y second
pt sub(pt a, pt b) { return {a.x - b.x, a.y - b.y}; }
ll cross(pt a, pt b) { return a.x*b.y - a.y*b.x; } // x = 180 -> sin
= 0
ll orient(pt a, pt b, pt c) { return cross(sub(b,a),sub(c,a)); }//
clockwise = -
// poly is in clock wise order
ll insidePoly(vector<pt> &poly, pt query) {
    ll n = poly.size();
    ll left = 1;
    ll right = n - 2;
    ll ans = -1;
    if (!(orient(poly[0], poly[1], query) <= 0
         && orient(poly[0], poly[n-1], query) >= 0)) {
        return OUT;
    }
    while (left <= right) {
        ll mid = (left + right) / 2;
        if (orient(poly[0], poly[mid], query) <= 0) {
            left = mid + 1;
            ans = mid;
```

```
        } else {
            right = mid - 1;
        }
    }
    left = ans;
    right = ans + 1;
    if (orient(poly[left], query, poly[right]) < 0) {
        return OUT;
    }
    if (orient(poly[left], poly[right], query) == 0
       || (left == 1 && orient(poly[0], poly[left], query) == 0)
       || (right == n-1 && orient(poly[0], poly[right], query) == 0))
{
        return ON;
    }
    return IN;
}
```

§7.7.4 point-in-general-polygon.cpp

07_Geometria/Poligonos

```
// Use insidepoly(poly, point)
// Returns if a point is inside=0, outside=1, onedge=2
// tested https://vjudge.net/solution/45869791/BIPDAUMWyupUW18AlWgd
// Seems to be O(n)??
int inf = 1 << 30;
int INSIDE = 0;
int OUTSIDE = 1;
int ONEDGE = 2;
int COLINEAR = 0;
int CW = 1;
int CCW = 2;
typedef long double ld;
struct point {
    ld x, y;
    point(ld xloc, ld yloc) : x(xloc), y(yloc) {}
    point() {}
    point& operator= (const point& other) {
        x = other.x, y = other.y;
        return *this;
    }
    int operator == (const point& other) const {
        return (abs(other.x - x) < .00001 && abs(other.y - y) <
.00001);
    }
    int operator != (const point& other) const {
        return !(abs(other.x - x) < .00001 && abs(other.y - y) <
.00001);
    }
    bool operator< (const point& other) const {
        return (x < other.x ? true : (x == other.x && y < other.y));
    }
};
struct vect { ld i, j; };
struct segment {
    point p1, p2;
    segment(point a, point b) : p1(a), p2(b) {}
    segment() {}
};
long double crossProduct(point A, point B, point C) {
    vect AB, AC;
    AB.i = B.x - A.x;
    AB.j = B.y - A.y;
    AC.i = C.x - A.x;
    AC.j = C.y - A.y;
    return (AB.i * AC.j - AB.j * AC.i);
}
int orientation(point p, point q, point r) {
    int val = static_cast<long long>(crossProduct(p, q, r));
    if(val == 0) {
        return COLINEAR;
    }
    return (val > 0) ? CW : CCW;
}
bool onSegment(point p, segment s) {
    return (p.x <= max(s.p1.x, s.p2.x) && p.x >= min(s.p1.x, s.p2.x)
&&
            p.y <= max(s.p1.y, s.p2.y) && p.y >= min(s.p1.y,
s.p2.y));
}
vector<point> intersect(segment s1, segment s2) {
    vector<point> res;
    point a = s1.p1, b = s1.p2, c = s2.p1, d = s2.p2;
    if(orientation(a, b, c) == 0 && orientation(a, b, d) == 0 &&
       orientation(c, d, a) == 0 && orientation(c, d, b) == 0) {
        point min_s1 = min(a, b), max_s1 = max(a, b);
        point min_s2 = min(c, d), max_s2 = max(c, d);
        if(min_s1 < min_s2) {
            if(max_s1 < min_s2) {
                return res;
            }
        }
        else if(min_s2 < min_s1 && max_s2 < min_s1) {
            return res;
        }
        point start = max(min_s1, min_s2), end = min(max_s1, max_s2);
        if(start == end) {
            res.push_back(start);
        }
        else {
            res.push_back(min(start, end));
            res.push_back(max(start, end));
        }
        return res;
    }
    ld x1 = (b.x - a.x);
    ld y1 = (b.y - a.y);
    ld x2 = (d.x - c.x);
    ld y2 = (d.y - c.y);
    ld u1 = (-y1 * (a.x - c.x) + x1 * (a.y - c.y)) / (-x2 * y1 + x1 *
y2);
    ld u2 = (x2 * (a.y - c.y) - y2 * (a.x - c.x)) / (-x2 * y1 + x1 *
y2);
    if(u1 >= 0 && u1 <= 1 && u2 >= 0 && u2 <= 1) {
        res.push_back(point((a.x + u2 * x1), (a.y + u2 * y1)));
    }
    return res;
}
int insidepoly(vector<point> poly, point p) {
    bool inside = false;
    point outside(inf, p.y);
    vector<point> intersection;
    for( uint32_t i = 0, j = poly.size()-1; i < poly.size(); i++, j =
i-1) {
        if(p == poly[i] || p == poly[j]) {
            return ONEDGE;
        }
        if(orientation(p, poly[i], poly[j]) == COLINEAR &&
onSegment(p, segment(poly[i], poly[j]))) {
            return ONEDGE;
        }
        intersection = intersect(segment(p, outside),
segment(poly[i], poly[j]));
        if(intersection.size() == 1) {
            if(poly[i] == intersection[0] && poly[j].y <= p.y) {
                continue;
            }
            if(poly[j] == intersection[0] && poly[i].y <= p.y) {
                continue;
            }
            inside = !inside;
        }
    }
    return inside ? INSIDE : OUTSIDE;
}
```

§7.7.5 polygon-diameter.cpp

07_Geometria/Poligonos

```
using ll = long long;
// Given a set of points, it returns
// the diameter (the biggest distance between 2 points)
// tested: https://open.kattis.com/submissions/13937489
const double eps = 1e-9;
int sign(double x) { return (x > eps) - (x < -eps); }
struct PT {
    double x, y;
    PT() { x = 0, y = 0; }
    PT(double x, double y) : x(x), y(y) {}
    PT operator - (const PT &a) const { return PT(x - a.x, y - a.y);
}
    bool operator < (PT a) const { return sign(a.x - x) == 0 ? y <
a.y : x < a.x; }
    bool operator == (PT a) const { return sign(a.x - x) == 0 &&
sign(a.y - y) == 0; }
};
inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.y; }
inline double dist2(PT a, PT b) { return dot(a - b, a - b); }
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a - b)); }
inline double cross(PT a, PT b) { return a.x * b.y - a.y * b.x; }
inline int orientation(PT a, PT b, PT c) { return sign(cross(b - a, c
- a)); }
double diameter(vector<PT> &p) {
    int n = (int)p.size();
    if (n == 1) return 0;
    if (n == 2) return dist(p[0], p[1]);
    double ans = 0;
    int i = 0, j = 1;
    while (i < n) {
        while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] - p[j]) >=
0) {
            ans = max(ans, dist2(p[i], p[j]));
            j = (j + 1) % n;
        }
        ans = max(ans, dist2(p[i], p[j]));
        i++;
    }
    return sqrt(ans);
}
vector<PT> convex_hull(vector<PT> &p) {
    if (p.size() <= 1) return p;
    vector<PT> v = p;
    sort(v.begin(), v.end());
    vector<PT> up, dn;
    for (auto& p : v) {
        while (up.size() > 1 && orientation(up[up.size() - 2],
up.back(), p) >= 0) {
            up.pop_back();
        }
        while (dn.size() > 1 && orientation(dn[dn.size() - 2],
dn.back(), p) <= 0) {
            dn.pop_back();
        }
        up.push_back(p);
        dn.push_back(p);
    }
    v = dn;
    if (v.size() > 1) v.pop_back();
    reverse(up.begin(), up.end());
    up.pop_back();
    for (auto& p : up) {
```

```
        v.push_back(p);
    }
    if (v.size() == 2 && v[0] == v[1]) v.pop_back();
    return v;
}
void test_case() {
    ll n;
    cin >>n;
    vector<PT> p(n);
    for (int i = 0;i<n;i++) cin >> p[i].x >> p[i].y;
    p = convex_hull(p);
    cout << fixed<<setprecision(10) << diameter(p) << "\n";
}
```

§7.7.6 polygon-extra.cpp

07_Geometria/Poligonos

```
#include "../Puntos_rectas_y_segmentos/point-extra.cpp"
#include "../Puntos_rectas_y_segmentos/segment-extra.cpp"
#include "../Puntos_rectas_y_segmentos/line-extra.cpp"
using ll = long long;
using pii = pair<int, int>;
enum {IN, OUT, ON};
struct polygon {
    vector<pt> p;
    polygon(int n) : p(n) {}
    int top = -1, bottom = -1;
    void delete_repetead() {
        vector<pt> aux;
        sort(p.begin(), p.end());
        for (pt &i : p)
            if (aux.empty() || aux.back() != i)
              aux.pb(i);
        p.swap(aux);
    }
    bool is_convex() {
        bool pos = 0, neg = 0;
        for (int i = 0, n = p.size(); i < n; i++) {
            int o = orient(p[i], p[(i+1)%n], p[(i+2)%n]);
            if (o > 0) pos = 1;
            if (o < 0) neg = 1;
        }
        return !(pos && neg);
    }
    lf area(bool s = false) { // better on clockwise order
        lf ans = 0;
        for (int i = 0, n = p.size(); i < n; i++)
            ans += cross(p[i], p[(i+1)%n]);
        ans /= 2;
        return s ? ans : abs(ans);
    }
    lf perimeter() {
        lf per = 0;
        for (int i = 0, n = p.size(); i < n; i++)
           per += abs(p[i] - p[(i+1)%n]);
        return per;
    }
    bool above(pt a, pt p) { return p.y >= a.y; }
    bool crosses_ray(pt a, pt p, pt q) { // pq crosses ray from a
        return (above(a, q)-above(a, p))*orient(a, p, q) > 0;
    }
    int in_polygon(pt a) {
        int crosses = 0;
        for (int i = 0, n = p.size(); i < n; i++) {
            if (on_segment(p[i], p[(i+1)%n], a)) return ON;
            crosses += crosses_ray(a, p[i], p[(i+1)%n]);
        }
        return (crosses&1 ? IN : OUT);
    }
    void normalize() { // polygon is CCW
        bottom = min_element(p.begin(), p.end()) - p.begin();
        vector<pt> tmp(p.begin()+bottom, p.end());
        tmp.insert(tmp.end(), p.begin(), p.begin()+bottom);
        p.swap(tmp);
        bottom = 0;
        top = max_element(p.begin(), p.end()) - p.begin();
    }
    int in_convex(pt a) {
        assert(bottom == 0 && top != -1);
        if (a < p[0] || a > p[top]) return OUT;
        T orientation = orient(p[0], p[top], a);
        if (orientation == 0) {
            if (a == p[0] || a == p[top]) return ON;
            return top == 1 || top + 1 == p.size() ? ON : IN;
        } else if (orientation < 0) {
            auto it = lower_bound(p.begin()+1, p.begin()+top, a);
            T d = orient(*prev(it), a, *it);
            return d < 0 ? IN : (d > 0 ? OUT: ON);
        } else {
            auto it = upper_bound(p.rbegin(), p.rend()-top-1, a);
            T d = orient(*it, a, it == p.rbegin() ? p[0] :
*prev(it));
            return d < 0 ? IN : (d > 0 ? OUT: ON);
        }
    }
    polygon cut(pt a, pt b) { // cuts polygon on line ab
        line l(a, b);
        polygon new_polygon(0);
        for (int i = 0, n = p.size(); i < n; ++i) {
            pt c = p[i], d = p[(i+1)%n];
            lf abc = cross(b-a, c-a), abd = cross(b-a, d-a);
            if (abc >= 0) new_polygon.p.pb(c);
            if (abc*abd < 0) {
              pt out; inter_ll(l, line(c, d), out);
              new_polygon.p.pb(out);
            }
        }
        return new_polygon;
    }
    void convex_hull() {
        sort(p.begin(), p.end());
        vector<pt> ch;
        ch.reserve(p.size()+1);
        for (int it = 0; it < 2; it++) {
            int start = ch.size();
            for (auto &a : p) {
                // if colineal are needed, use < and remove repeated
points
                while (ch.size() >= start+2 &&
orient(ch[ch.size()-2], ch.back(), a) <= 0)
                    ch.pop_back();
                ch.pb(a);
            }
            ch.pop_back();
            reverse(p.begin(), p.end());
        }
        if (ch.size() == 2 && ch[0] == ch[1]) ch.pop_back();
        // be careful with CH of size < 3
        p.swap(ch);
    }
    vector<pii> antipodal() {
        vector<pii> ans;
        int n = p.size();
        if (n == 2) ans.pb({0, 1});
        if (n < 3) return ans;
        auto nxt = [&](int x) { return (x+1 == n ? 0 : x+1); };
        auto area2 = [&](pt a, pt b, pt c) { return cross(b-a, c-a);
};
        int b0 = 0;
        while (abs(area2(p[n - 1], p[0], p[nxt(b0)])) > abs(area2(p[n
- 1], p[0], p[b0]))) ++b0;
        for (int b = b0, a = 0; b != 0 && a <= b0; ++a) {
            ans.pb({a, b});
            while (abs(area2(p[a], p[nxt(a)], p[nxt(b)])) >
abs(area2(p[a], p[nxt(a)], p[b]))) {
                b = nxt(b);
                if (a != b0 || b != 0) ans.pb({ a, b });
                else return ans;
            }
            if (abs(area2(p[a], p[nxt(a)], p[nxt(b)])) ==
abs(area2(p[a], p[nxt(a)], p[b]))) {
                if (a != b0 || b != n-1) ans.pb({ a, nxt(b) });
                else ans.pb({ nxt(a), b });
            }
        }
        return ans;
    }
    pt centroid() {
        pt c{0, 0};
        lf scale = 6. * area(true);
        for (int i = 0, n = p.size(); i < n; ++i) {
            int j = (i+1 == n ? 0 : i+1);
            c = c + (p[i] + p[j]) * cross(p[i], p[j]);
        }
        return c / scale;
    }
    ll pick() {
        ll boundary = 0;
        for (int i = 0, n = p.size(); i < n; i++) {
            int j = (i+1 == n ? 0 : i+1);
            boundary += __gcd((ll)abs(p[i].x - p[j].x),
(ll)abs(p[i].y - p[j].y));
        }
        return area() + 1 - boundary/2;
    }
    pt& operator [] (int i) { return p[i]; }
};
```

§7.8 Proximidad y KD Tree

§7.8.1 closest-pair-of-points.cpp

07_Geometria/Proximidad_y_KD_Tree

```
using ll = long long;
using ld = long double;
// It seems O(n log n), not sure but it worked for 50000
// This algorithms is not the best, TLE in CSES
// https://cses.fi/problemset/task/2194
#define x first
#define y second
long long dist2(pair<int, int> a, pair<int, int> b) {
    return 1LL * (a.x - b.x) * (a.x - b.x) + 1LL * (a.y - b.y) * (a.y
- b.y);
}
pair<int, int> closest_pair(vector<pair<int, int>> a) {
    int n = a.size();
    vector<pair<pair<int, int>, int>> p(n);
    for (int i = 0; i < n; i++) p[i] = {a[i], i};
    sort(p.begin(), p.end());
    int l = 0, r = 2;
    long long ans = dist2(p[0].x, p[1].x);
    pair<int, int> ret = {p[0].y, p[1].y};
    while (r < n) {
        while (l < r && 1LL * (p[r].x.x - p[l].x.x) * (p[r].x.x -
p[l].x.x) >= ans) l++;
        for (int i = l; i < r; i++) {
            long long nw = dist2(p[i].x, p[r].x);
            if (nw < ans) {
                ans = nw;
                ret = {p[i].y, p[r].y};
            }
```

```
        }
        r++;
    }
    return ret;
}
// Tested: https://vjudge.net/solution/52922194/ccPUXODAMWTzpzCEvXbV
void test_case() {
    ll n;
    cin >> n;
    vector<pair<int,int>> points(n);
    for (int i = 0;i<n;i++) cin >> points[i].x >> points[i].y;
    auto ans = closest_pair(points);
    cout << fixed << setprecision(6);
    if (ans.first > ans.second) swap(ans.first,ans.second);
    ld dist = sqrtl(dist2(points[ans.first],points[ans.second]));
    cout << ans.first << " " << ans.second << " " << dist << endl;
}
```

§7.8.2 kdtree.cpp

07_Geometria/Proximidad_y_KD_Tree

```
const int maxn=200005;
struct kdtree
{
    int xl,xr,yl,yr,zl,zr,max,flag; // flag=0:x axis  1:y 2:z
} tree[5000005];
int N,M,lastans,xq,yq;
int a[maxn],pre[maxn],nxt[maxn];
int x[maxn],y[maxn],z[maxn],wei[maxn];
int xc[maxn],yc[maxn],zc[maxn],wc[maxn],hash[maxn],biao[maxn];
bool cmp1(int a,int b)
{
    return x[a]<x[b];
}
bool cmp2(int a,int b)
{
    return y[a]<y[b];
}
bool cmp3(int a,int b)
{
    return z[a]<z[b];
}
void makekdtree(int node,int l,int r,int flag)
{
    if (l>r)
    {
        tree[node].max=-maxlongint;
        return;
    }
    int xl=maxlongint,xr=-maxlongint;
    int yl=maxlongint,yr=-maxlongint;
    int zl=maxlongint,zr=-maxlongint,maxc=-maxlongint;
    for (int i=l;i<=r;i++)
        xl=min(xl,x[i]),xr=max(xr,x[i]),
        yl=min(yl,y[i]),yr=max(yr,y[i]),
        zl=min(zl,z[i]),zr=max(zr,z[i]),
        maxc=max(maxc,wei[i]),
        xc[i]=x[i],yc[i]=y[i],zc[i]=z[i],wc[i]=wei[i],biao[i]=i;
    tree[node].flag=flag;
    tree[node].xl=xl,tree[node].xr=xr,tree[node].yl=yl;
    tree[node].yr=yr,tree[node].zl=zl,tree[node].zr=zr;
    tree[node].max=maxc;
    if (l==r) return;
    if (flag==0) sort(biao+l,biao+r+1,cmp1);
    if (flag==1) sort(biao+l,biao+r+1,cmp2);
    if (flag==2) sort(biao+l,biao+r+1,cmp3);
    for (int i=l;i<=r;i++)
        x[i]=xc[biao[i]],y[i]=yc[biao[i]],
        z[i]=zc[biao[i]],wei[i]=wc[biao[i]];
    makekdtree(node*2,l,(l+r)/2,(flag+1)%3);
    makekdtree(node*2+1,(l+r)/2+1,r,(flag+1)%3);
}
int getmax(int node,int xl,int xr,int yl,int yr,int zl,int zr)
{
    xl=max(xl,tree[node].xl);
    xr=min(xr,tree[node].xr);
    yl=max(yl,tree[node].yl);
    yr=min(yr,tree[node].yr);
    zl=max(zl,tree[node].zl);
    zr=min(zr,tree[node].zr);
    if (tree[node].max==-maxlongint) return 0;
    if ((xr<tree[node].xl)||(xl>tree[node].xr)) return 0;
    if ((yr<tree[node].yl)||(yl>tree[node].yr)) return 0;
    if ((zr<tree[node].zl)||(zl>tree[node].zr)) return 0;
    if ((tree[node].xl==xl)&&(tree[node].xr==xr)&&
        (tree[node].yl==yl)&&(tree[node].yr==yr)&&
        (tree[node].zl==zl)&&(tree[node].zr==zr))
    return tree[node].max;
    else
    return max(getmax(node*2,xl,xr,yl,yr,zl,zr),
               getmax(node*2+1,xl,xr,yl,yr,zl,zr));
}
signed main()
{
    // N 3D-rect with weights
    // find the maximum weight containing the given 3D-point
    return 0;
}
```

§7.8.3 nearest-neighbour.cpp

07_Geometria/Proximidad_y_KD_Tree

```
// Requisito de selección: El corte por coordenadas no distingue
puntos idénticos; resolver duplicados antes de recursión para evitar
una partición vacía.
using ll = long long;
// Closest Neighbor - O(n * log(n))
const ll N = 1e6+3, INF = 1e18;
ll n, cn[N], x[N], y[N]; // number of points, closes neighbor, x
coordinates, y coordinates
ll sqr(ll i) { return i*i; }
ll dist(int i, int j) { return sqr(x[i]-x[j]) + sqr(y[i]-y[j]); }
ll dist(int i) { return i == cn[i] ? INF : dist(i, cn[i]); }
bool cpx(int i, int j) { return x[i] < x[j] or (x[i] == x[j] and y[i]
< y[j]); }
bool cpy(int i, int j) { return y[i] < y[j] or (y[i] == y[j] and x[i]
< x[j]); }
ll calc(int i, ll x0) {
  ll dlt = dist(i) - sqr(x[i]-x0);
  return dlt >= 0 ? ceil(sqrt(dlt)) : -1;
}
void updt(int i, int j, ll x0, ll &dlt) {
  if (dist(i) > dist(i, j)) cn[i] = j, dlt = calc(i, x0);
}
void cmp(vi &u, vi &v, ll x0) {
  for(int a=0, b=0; a<u.size(); ++a) {
    ll i = u[a], dlt = calc(i, x0);
    while(b < v.size() and y[i] > y[v[b]]) b++;
    for(int j = b-1; j >= 0     and y[i] - dlt <= y[v[j]]; j--)
updt(i, v[j], x0, dlt);
    for(int j = b; j < v.size() and y[i] + dlt >= y[v[j]]; j++)
updt(i, v[j], x0, dlt);
  }
}
void slv(vi &ix, vi &iy) {
  int n = ix.size();
  if (n == 1) { cn[ix[0]] = ix[0]; return; }
  int m = ix[n/2];
  vi ix1, ix2, iy1, iy2;
  for(int i=0; i<n; ++i) {
    if (cpx(ix[i], m)) ix1.push_back(ix[i]);
    else ix2.push_back(ix[i]);
    if (cpx(iy[i], m)) iy1.push_back(iy[i]);
    else iy2.push_back(iy[i]);
  }
  slv(ix1, iy1);
  slv(ix2, iy2);
  cmp(iy1, iy2, x[m]);
  cmp(iy2, iy1, x[m]);
}
void slv(int n) {
  vi ix, iy;
  ix.resize(n);
  iy.resize(n);
  for(int i=0; i<n; ++i) ix[i] = iy[i] = i;
  sort(ix.begin(), ix.end(), cpx);
  sort(iy.begin(), iy.end(), cpy);
  slv(ix, iy);
}
```

§7.9 Puntos rectas y segmentos

§7.9.1 line-extra.cpp

07_Geometria/Puntos_rectas_y_segmentos

```
#include "point-extra.cpp"
struct line {
    pt v; T c; // v:direction c: pos in y axis
    line(pt v, T c) : v(v), c(c) {}
    line(T a, T b, T c) : v({b, -a}), c(c) {} // ax + by = c
    line(pt p, pt q) : v(q-p), c(cross(v, p)) {}
    T side(pt p) { return cross(v, p)-c; }
    lf dist(pt p) { return abs(side(p)) / abs(v); }
    lf sq_dist(pt p) { return side(p)*side(p) / (lf)norm(v); }
    line perp_through(pt p) { return {p, p + rot90ccw(v)}; } // line
perp to v passing through p
    bool cmp_proj(pt p, pt q) { return dot(v, p) < dot(v, q); } //
order for points over the line
    line translate(pt t) { return {v, c + cross(v, t)}; }
    line shift_left(double d) { return {v, c + d*abs(v)}; }
    pt proj(pt p) { return p - rot90ccw(v)*side(p)/norm(v); } // pt
proyected on the line
    pt refl(pt p) { return p - rot90ccw(v)*2*side(p)/norm(v); } // pt
reflected on the other side of the line
};
bool inter_ll(line l1, line l2, pt &out) {
    T d = cross(l1.v, l2.v);
    if (d == 0) return false;
    out = (l2.v*l1.c - l1.v*l2.c) / d; // floating points
    return true;
}
// bisector divides the angle in 2 equal angles
// interior line goes on the same direction as l1 and l2
line bisector(line l1, line l2, bool interior) {
    assert(cross(l1.v, l2.v) != 0); // l1 and l2 cannot be parallel!
    lf sign = interior ? 1 : -1;
    return {l2.v/abs(l2.v) + l1.v/abs(l1.v) * sign,
            l2.c/abs(l2.v) + l1.c/abs(l1.v) * sign};
}
```

§7.9.2 point-extra.cpp

07_Geometria/Puntos_rectas_y_segmentos

```
typedef double lf;
const lf eps = 1e-9;
const lf PI = acos(-1.0);
typedef double T;
struct pt {
    T x, y;
    pt operator + (pt p) { return {x+p.x, y+p.y}; }
    pt operator - (pt p) const { return {x-p.x, y-p.y}; }
    pt operator * (pt p) { return {x*p.x-y*p.y, x*p.y+y*p.x}; }
    pt operator * (T d) { return {x*d, y*d}; }
    pt operator / (lf d) { return {x/d, y/d}; } // only for floating
point
```

```
    bool operator == (pt b) { return x == b.x && y == b.y; }
    bool operator != (pt b) { return !(*this == b); }
    bool operator < (const pt &o) const { return y < o.y || (y == o.y
&& x < o.x); }
    bool operator > (const pt &o) const { return y > o.y || (y == o.y
&& x > o.x); }
};
int cmp(lf a, lf b) { return (a + eps < b ? -1 :(b + eps < a ? 1 :
0)); } // double comparator
T norm(pt a) { return a.x*a.x + a.y*a.y; }
lf abs(pt a) { return sqrt(norm(a)); }
lf arg(pt a) { return atan2(a.y, a.x); }
pt unit(pt a) { return a/abs(a); }
T dot(pt a, pt b) { return a.x*b.x + a.y*b.y; } // x = 90 -> cos = 0
T cross(pt a, pt b) { return a.x*b.y - a.y*b.x; } // x = 180 -> sin =
0
T orient(pt a, pt b, pt c) { return cross(b-a, c-a); } // clockwise =
-
pt rot(pt p, lf a) { return {p.x*cos(a) - p.y*sin(a), p.x*sin(a) +
p.y*cos(a)}; }
pt rotate_to_b(pt a, pt b, lf ang) { return rot(a-b, ang)+b; } //
rotate by ang center b
pt rot90ccw(pt p) { return {-p.y, p.x}; }
pt rot90cw(pt p) { return {p.y, -p.x}; }
pt translate(pt p, pt v) { return p+v; }
pt scale(pt p, double f, pt c) { return c + (p-c)*f; } // c-center
bool are_perp(pt v, pt w) { return dot(v, w) == 0; }
int sign(T x) { return (T(0) < x) - (x < T(0)); }
bool in_angle(pt a, pt b, pt c, pt x) { // x inside angle abc (center
in a)
    assert(orient(a, b, c) != 0);
    if (orient(a, b, c) < 0) swap(b, c);
    return orient(a, b, x) >= 0 && orient(a, c, x) <= 0;
}
// angle bwn 2 vectors [0, pi] -> [0, 180] and (-pi, 0) -> (180, 360)
lf angle_no_orientado(pt a, pt b) { return acos(max(-1.0, min(1.0,
dot(a, b)/abs(a)/abs(b)))); }
lf angle(pt a, pt b) { return atan2(cross(a, b), dot(a, b)); }
lf angle360(pt a, pt b) { // [0, 360)
    lf ang = angle(a, b); return (ang < 0 ? ang+2*PI : ang) *
360/(2*PI);
}
// returns vector to transform points
pt get_linear_transformation(pt p, pt q, pt r, pt fp, pt fq) {
    pt pq = q-p, num{cross(pq, fq-fp), dot(pq, fq-fp)};
    return fp + pt{cross(r-p, num), dot(r-p, num)} / norm(pq);
}
bool half(pt p) { // true if is in (0, 180] (line is x axis)
    assert(p.x != 0 || p.y != 0); // the argument of (0, 0) is
undefined
    return p.y > 0 || (p.y == 0 && p.x < 0);
}
bool half_from(pt p, pt v = {1, 0}) { // line is v (above v is true)
    return cross(v, p) < 0 || (cross(v, p) == 0 && dot(v, p) < 0);
}
bool polar_cmp(const pt &a, const pt &b) { // polar sort
    return make_tuple(half(a), 0) < make_tuple(half(b), cross(a, b));
    // return make_tuple(half(a), 0, sq(a)) < make_tuple(half(b),
cross(a, b), sq(b)); // further ones appear later
}
```

§7.9.3 radial-order-vasito.cpp

07_Geometria/Puntos_rectas_y_segmentos

```
#include "point-extra.cpp"
struct Cmp { // Usa el pt de point-extra.cpp
    pt r;
    Cmp(pt r):r(r){}
    int cuad(const pt &a)const {
        if(a.x>0&&a.y>=0)return 0;
        if(a.x<=0&&a.y>0)return 1;
        if(a.x<0&&a.y<=0)return 2;
        if(a.x>=0&&a.y<0)return 3;
        assert(a.x==0&&a.y==0);
        return -1;
    }
    bool cmp(const pt& p1, const pt& p2)const {
        int c1=cuad(p1),c2=cuad(p2);
        if(c1==c2)return p1.y*p2.x<p1.x*p2.y;
        return c1<c2;
    }
    bool operator()(const pt& p1, const pt& p2)const {
        return cmp(p1-r,p2-r);
    }
};
```

§7.9.4 segment-extra.cpp

07_Geometria/Puntos_rectas_y_segmentos

```
#include "point-extra.cpp"
bool in_disk(pt a, pt b, pt p) { // pt p inside ab disk
    return dot(a-p, b-p) <= 0;
}
bool on_segment(pt a, pt b, pt p) { // p on ab
    return orient(a, b, p) == 0 && in_disk(a, b, p);
}
// ab crossing cd
bool proper_inter(pt a, pt b, pt c, pt d, pt &out) {
    T oa = orient(c, d, a),
    ob = orient(c, d, b),
    oc = orient(a, b, c),
    od = orient(a, b, d);
    // Proper intersection exists iff opposite signs
    if (oa*ob < 0 && oc*od < 0) {
        out = (a*ob - b*oa) / (ob-oa);
        return true;
    }
    return false;
}
// intersection bwn segments
set<pt> inter_ss(pt a, pt b, pt c, pt d) {
    pt out;
    if (proper_inter(a, b, c, d, out)) return {out}; // if cross -> 1
    set<pt> s;
    if (on_segment(c, d, a)) s.insert(a); // a in cd
    if (on_segment(c, d, b)) s.insert(b); // b in cd
    if (on_segment(a, b, c)) s.insert(c); // c in ab
    if (on_segment(a, b, d)) s.insert(d); // d in ab
    return s; // 0, 2
}
lf pt_to_seg(pt a, pt b, pt p) { // p to ab
    if (a != b) {
        line l(a, b);
        if (l.cmp_proj(a, p) && l.cmp_proj(p, b)) // if closest to
projection = (a, p, b)
            return l.dist(p); // output distance to line
    }
    return min(abs(p-a), abs(p-b)); // otherwise distance to A or B
}
lf seg_to_seg(pt a, pt b, pt c, pt d) {
    pt dummy;
    if (proper_inter(a, b, c, d, dummy)) return 0; // ab intersects
cd
    return min({pt_to_seg(a, b, c), pt_to_seg(a, b, d), pt_to_seg(c,
d, a), pt_to_seg(c, d, b)}); // try the 4 pts
}
```

§7.9.5 segment-intersection.cpp

07_Geometria/Puntos_rectas_y_segmentos

```
using ld = long double;
// No the best algorithm, find a better one!!
// Given two segment, finds the intersection point.
// LINE if they are parallel and mulitple intersection??
// POINT with the intersection point
// NONE if not intersection
struct line {
    ld a, b; // first point
    ld x, y; // second point
    ld m()  { return (a - x)/(b - y); }
    bool horizontal() { return b == y; }
    bool vertical() { return a == x; }
    void intersects(line &o) {
        if (horizontal() && o.horizontal()) {
            if (y == o.y) cout << "LINE\n";
            else cout << "NONE\n";
            return;
        }
        if (vertical() && o.vertical()) {
            if (x == o.x) cout << "LINE\n";
            else cout << "NONE\n";
            return;
        }
        if (!horizontal() && !o.horizontal()) {
            ld ma = m();
            ld mb = o.m();
            if (ma == mb) {
                ld someY = (o.x - x)/ma + y;
                if (abs(someY - o.y) <= 0.000001) {
                    cout << "LINE\n";
                } else {
                    cout << "NONE\n";
                }
            } else {
                ld xx = (x*mb - o.x*ma + ma*mb*(o.y - y))/(mb - ma);
                ld yy = (xx - x)/ma + y;
                cout << "POINT " << fixed << setprecision(2) << xx <<
" " << yy << "\n";
            }
        } else {
            if (!horizontal()) {
                ld xx;
                if (x == a) {
                    xx = x;
                } else {
                    xx = (o.y - y)/m() + x;
                }
                ld yy = o.y;
                cout << "POINT "<< fixed << setprecision(2) << xx <<
" " << yy << "\n";
            } else {
                ld xx;
                if (x == a) {
                    xx = x;
                } else {
                    xx = (y - o.y)/o.m() + o.x;
                }
                ld yy = y;
                cout << "POINT "<< fixed << setprecision(2) << xx <<
" " << yy << "\n";
            }
        }
    }
};
void test_case() {
    line l[2];
    for (int i = 0; i < 2; i++) {
        ld x, y, a, b;
        cin >> x >> y >> a >> b;
        l[i].a = x;
        l[i].b = y;
        l[i].x = a;
        l[i].y = b;
```

```
    }
    l[0].intersects(l[1]);
}
```

§7.10 Semiplanos

§7.10.1 halfplanes-extra.cpp

07_Geometria/Semiplanos

```
#include "../Puntos_rectas_y_segmentos/point-extra.cpp"
struct halfplane {
    double angle;
    pt p, pq;
    halfplane() {}
    halfplane(pt a, pt b): p(a), pq(b - a) {
        angle = atan2(pq.y, pq.x);
    }
    bool operator < (halfplane b) const { return angle < b.angle; }
    bool out(pt q) { return cross(pq, (q-p)) < -eps; } // checks if p
is inside the half plane
};
const lf inf = 1e100;
// intersection pt of the lines of 2 halfplanes
pt inter(halfplane& h1, halfplane& h2) {
    if (abs(cross(unit(h1.pq), unit(h2.pq))) <= eps) return {inf,
inf};
    lf alpha = cross((h2.p - h1.p), h2.pq) / cross(h1.pq, h2.pq);
    return h1.p + (h1.pq * alpha);
}
// intersection of halfplanes
vector<pt> intersect(vector<halfplane>& b) {
    vector<pt> box = {{inf, inf}, {-inf, inf}, {-inf, -inf}, {inf, -
inf}};
    for (int i = 0; i < 4; i++) {
        b.pb({box[i], box[(i + 1) % 4]});
    }
    sort(b.begin(), b.end());
    int n = b.size(), q = 1, h = 0;
    vector<halfplane> c(n + 10);
    for (int i = 0; i < n; i++) {
        while (q < h && b[i].out(inter(c[h], c[h-1]))) h--;
        while (q < h && b[i].out(inter(c[q], c[q+1]))) q++;
        c[++h] = b[i];
        if (q < h && abs(cross(c[h].pq, c[h-1].pq)) < eps) {
            if (dot(c[h].pq, c[h-1].pq) <= 0) return {};
            h--;
            if (b[i].out(c[h].p)) c[h] = b[i];
        }
    }
    while (q < h-1 && c[q].out(inter(c[h], c[h-1]))) h--;
    while (q < h-1 && c[h].out(inter(c[q], c[q+1]))) q++;
    if (h - q <= 1) return {};
    c[h+1] = c[q];
    vector<pt> s;
    for (int i = q; i < h+1; i++) s.pb(inter(c[i], c[i+1]));
    return s;
}
```

§7.11 Teoría y fórmulas

§7.11.1 some-formulas.tex

07_Geometria/Teoria_y_formulas

Volume of Glass with Water (Volumen del Vaso)

Given:

- p is the height of the water
- r_1 is the big radius at the water's surface
- r_2 is the small radius of the base

The volume of the glass with water is given by:

$$\text{Volume} = p \cdot \pi \cdot \frac{\left(r_1^2 + r_2^2 + r_1 \cdot r_2\right)}{3}$$

Heron's Formula

Finding the area of a triangle by the length of its sides, also applicable for points using Euclidean distance. You can use the following code to get the area; if the square root is negative, then the triangle is not valid.

```
long double triangle_area(long double a, long double b, long double
c) {
    long double s = (a + b + c) / 2;
    return sqrtl(s * (s - a) * (s - b) * (s - c));
}
```

Sine and Cosine Laws

Let a, b, and c be the sides of the triangle, and A, B, and C the angles opposite to these sides, respectively.

The Sine Law:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

The Cosine Law:

$$c^2 = a^2 + b^2 - 2ab \cdot \cos C$$

§7.12 Tres dimensiones

§7.12.1 latlong.cpp

07_Geometria/Tres_dimensiones

```
#include <iostream>
#include <cmath>
/*
Converts from rectangular coordinates to latitude/longitude and vice
versa. Uses degrees (not radians).
*/
using namespace std;
struct ll
{
  double r, lat, lon;
};
struct rect
{
  double x, y, z;
};
ll convert(rect& P)
{
  ll Q;
  Q.r = sqrt(P.x*P.x+P.y*P.y+P.z*P.z);
  Q.lat = 180/M_PI*asin(P.z/Q.r);
  Q.lon = 180/M_PI*acos(P.x/sqrt(P.x*P.x+P.y*P.y));
  return Q;
}
rect convert(ll& Q)
{
  rect P;
  P.x = Q.r*cos(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
  P.y = Q.r*sin(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
  P.z = Q.r*sin(Q.lat*M_PI/180);
  return P;
}
signed main()
{
  rect A;
  ll B;
  A.x = -1.0; A.y = 2.0; A.z = -3.0;
  B = convert(A);
  cout << B.r << " " << B.lat << " " << B.lon << endl;
  A = convert(B);
  cout << A.x << " " << A.y << " " << A.z << endl;
}
```

§7.12.2 plane-vasito.cpp

07_Geometria/Tres_dimensiones

```
// Requisito: Necesita una versión 3D de pt con z y producto cruzado;
point.cpp del origen es 2D por defecto.
struct plane {
    pt a,n; // n: normal unit vector
    plane(pt a, pt b, pt c):a(a),n(((b-a)^(c-a)).unit()){}
    plane(){}
    bool has(pt p){return abs((p-a)*n)<=EPS;}
    double angle(plane w){return acos(n*w.n);}
    double dist(pt p){return abs((p-a)*n);}
    pt proj(pt p){inter(ln(p,p+n),p);return p;}
    bool inter(ln l, pt& r){
        double x=n*(l.p+l.pq-a),y=n*(l.p-a);
        if(abs(x-y)<=EPS)return false;
        r=(l.p*x-(l.p+l.pq)*y)/(x-y);
        return true;
    }
    bool inter(plane w, ln& r){
        pt nn=n^w.n;pt v=n^nn;double d=w.n*v;
        if(abs(d)<=EPS)return false;
        pt p=a+v*(w.n*(w.a-a)/d);
        r=ln(p,p+nn);
        return true;
    }
};
```

§8 Búsqueda y ordenamiento

§8.1 Búsqueda binaria paralela

§8.1.1 parallel-binary-search.cpp

...eda_y_ordenamiento/Busqueda_binaria_paralela

```
using vl = vector<long long>;
// Parallel Binary Search - O(nlog n * cost to update data structure
+ qlog n * cost for binary search condition)
struct Query { int i, ans; /*+ query related info*/ };
vector<Query> req;
void pbs(vector<Query>& qs, int l /* = min value*/, int r /* = max
value*/) {
  if (qs.empty()) return;
  if (l == r) {
    for (auto& q : qs) req[q.i].ans = l;
    return;
  }
  int mid = (l + r) / 2;
  // mid = (l + r + 1) / 2 if different from simple upper/lower bound
  for (int i = l; i <= mid; i++) {
    // add value to data structure
  }
  vector<Query> vl, vr;
  for (auto& q : qs) {
    if (/* cond */) vl.push_back(q);
    else vr.push_back(q);
  }
  pbs(vr, mid + 1, r);
  for (int i = l; i <= mid; i++) {
    // remove value from data structure
  }
  pbs(vl, l, mid);
}
```

§8.2 Búsqueda ternaria

§8.2.1 ternary-search.cpp

08_Busqueda_y_ordenamiento/Busqueda_ternaria

```
using ll = long long;
//Ternary Search - O(log(n))
//Max version, for minimum version just change signals
template<class Funcion>
```

```
int ternary_search(int l, int r, Funcion f){
    while(r - l > 3) {
        ll m1 = (l+r)/2;
        ll m2 = (l+r)/2 + 1;
        ll f1 = f(m1), f2 = f(m2);
        //if(f1 > f2) l = m1;
        if (f1 < f2) l = m1;
        else r = m2;
    }
    int ans = f(l);
    for(int i = l; i <= r; i++){
        ll tmp = f(i);
        //ans = min(ans, tmp);
        ans = max(ans, tmp);
    }
    return ans;
}
//Faster version - 300 iteratons up to 1e-6 precision
template<class Funcion>
double ternary_search(double l, double r, Funcion f, double
tolerancia = 1e-9){
    // for(int i = 0; i < No; i++){
    for (int iter = 0; iter < 300 && r - l > tolerancia; ++iter){
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        // if (f(m1) > f(m2))
        if (f(m1) < f(m2))
            l = m1;
        else
            r = m2;
    }
    return f(l);
}
```

§8.3 Divide and Conquer

§8.3.1 divide-and-conquer.cpp

08_Busqueda_y_ordenamiento/Divide_and_Conquer

```
using ll = long long;
vi dp(N+1, 0);
int bin(vi &v, int x) {
    int i = 0, j = v.size(), r = j;
    while (i<=j) {
        int mid = (i+j)>>1;
        if (v[mid]>=x)
            r = mid, j = mid-1;
        else i = mid+1;
    }
    return r;
}
void dc(vcor &v, int i, int j) {
    if(j==i)
        return;
    int mid = (i+j)>>1;
    dc(v, i, mid);
    dc(v, mid+1, j);
    vi aux;
    for (int k=(i), nb_fin_1=(mid+1); k<nb_fin_1; ++k)
        aux.pb(v[k].y);
    sort(aint(aux));
    for (int k=(mid+1), nb_fin_3=(j+1); k<nb_fin_3; ++k) {
        dp[k] = bin(aux, v[k].y);
    }
}
// use this option when the cut is not dividing the whole set:
// optimization from n ** 2 -> n * log n
void dc(vector<int> &v, int i, int j) {
    if (i > j) return;
    if (i == j) return;
    int ll = i, rr = j;
    while ( i <= j) {
        if (check(i,ll,rr)) {
            return dc(v,ll,i); // break instead of returning
        }
        if (check (j,ll,rr)) {
            return dc(v,j,rr);
        }
        i++, j--;
    }
    // check how can u combine everything
    return;
}
```

§8.4 Greedy y comparadores

§8.4.1 exchange-argument.cpp

...squeda_y_ordenamiento/Greedy_y_comparadores

```
// prove that the sequence AB provides a better answer than BA
// so you need to find a possible inequation: A < B
// and that its impossible: B < A
int cmp(array<int,2> &a, array<int,2> &b) {
    int ok = a[1] >= a[0];
    ok|= (b[1] >= b[0]? 2: 0);
    if (ok==3) return a[0] < b[0];
    else if (!ok) return a[1] > b[1];
    else return ok&1;
}
```

§8.5 Merge sort e inversiones

§8.5.1 merge-sort.cpp

08_Busqueda_y_ordenamiento/Merge_sort_e_inversiones

```
// Requisito: La línea ans.resize(v.size()) se ejecuta dentro de
solve(), antes de mergesort.
// Merge-sort with inversion count - O(nlog n)
int n, inv;
vector<int> v, ans;
void mergesort(int l, int r, vector<int> &v){
    if(l == r) return;
    int mid = (l+r)/2;
    mergesort(l, mid, v), mergesort(mid+1, r, v);
    int i = l, j = mid + 1, k = l;
    while(i <= mid or j <= r){
        if(i <= mid and (j > r or v[i] <= v[j])) ans[k++] = v[i++];
        else ans[k++] = v[j++], inv += j-k;
    }
    for(int i = l; i <= r; i++) v[i] = ans[i];
}
//in main
ans.resize(v.size());
```

§9 Strings

§9.1 Aho Corasick

§9.1.1 ahocorasik.cpp

08_Strings/Aho_Corasick

```
// El trie (o prefix tree) guarda un diccionario de
// strings como un arbol enraizado.
// Aho corasick permite encontrar las ocurrencias
// de todos los strings del trie en un
// string s O(n*sqrt(n))
const int alpha = 26;
const char L = 'a';
struct node {
    int next[alpha], end;
    // link -> back-edge, exit -> super link,
    // cnt -> saber para conteo de posibles
    // a lo mucho es: n*sqrt(n)
    int link = 0, exit = 0, cnt; // para aho corasick
    int& operator [] (int i) { return next[i]; }
};
vector<node> trie = {node()};
void add_str(string &s, int id = 1) {
    int u = 0;
    for (auto ch : s) {
        int c = ch-L;
        if (!trie[u][c]) {
            trie[u][c] = trie.size();
            trie.pb(node());
        }
        u = trie[u][c];
    }
    trie[u].end = id; // con id > 0
    // trie[u].cnt++; // para aho corasick
}
// aho corasick
void build_ac() {
    queue<int> q; q.push(0);
    while (q.size()) {
        int u = q.front(); q.pop();
        for (int c = 0; c < alpha; ++c) {
            int v = trie[u][c];
            if (!v) trie[u][c] = trie[trie[u].link][c];
            else q.push(v);
            if (!u || !v) continue;
            trie[v].link = trie[trie[u].link][c];
            trie[v].exit = trie[trie[v].link].end ?
              trie[v].link : trie[trie[v].link].exit;
            trie[v].cnt += trie[trie[v].link].cnt;
        }
    }
}
// cantidad de ocurrencias en s para cada patron
vector<int> cnt;
void run_ac(string &s) {
    int u = 0, sz = s.size();
    for (int i = 0; i < sz; ++i) {
        int c = s[i]-L;
        while (u && !trie[u][c]) u = trie[u].link;
        u = trie[u][c];
        int x = u;
        while (x) {
            int id = trie[x].end;
            if (id) cnt[id-1]++;
            x = trie[x].exit;
        }
    }
}
```

§9.1.2 dinamic-aho-corasick.cpp

08_Strings/Aho_Corasick

```
const int alpha = 26;
int node[20][tam][alpha];
struct ahoCorasick
{
    int root, nnode;
    int link[tam], cnt[tam], id;
    bool dead;
    void init(int idx)
    {
        dead = false; id = idx; root = 0; nnode = 0;
        mem(node[id][root], 0);
        for (int i=(0), nb_fin_1=(nnode + 1); i<nb_fin_1; ++i) cnt[i]
= 0;
    }
    void clear()
    {
        dead = true;
    }
```

```
    void insert(string &s, int val)
    {
        int len = s.size();
        int now = root;
        for (int i=(0), nb_fin_3=(len); i<nb_fin_3; ++i)
        {
            int nxt = s[i] - 'a';
            if(!node[id][now][nxt])
            {
                node[id][now][nxt] = ++nnode;
                mem(node[id][nnode],0);
                cnt[nnode] = 0;
            }
            now=node[id][now][nxt];
        }
        cnt[now] += val;
    }
    void insertdict(vector<string> &dict, vector<int> &vals)
    {
        for (int i=(0), nb_fin_5=(sz(dict)); i<nb_fin_5; ++i)
insert(dict[i], vals[i]);
    }
    void pushLinks()
    {
        queue<int> Q; link[root]=-1;
        Q.push(root);
        while(!Q.empty())
        {
            int u = Q.front(); Q.pop();
            for (int i=(0), nb_fin_7=(alpha); i<nb_fin_7; ++i)
            {
                if(!node[id][u][i]) continue;
                int v = node[id][u][i];
                int l = link[u];
                while(l != -1 && !node[id][l][i]) l = link[l];
                if(l!=-1) link[v] = node[id][l][i];
                else link[v] = 0;
                cnt[v] += cnt[link[v]];
                Q.push(v);
            }
        }
    }
    int query(string &p)
    {
        int u = root, ret = 0;
        for(char ch : p)
        {
            int nxt = ch - 'a';
            while(u != -1 && !node[id][u][nxt]) u = link[u];
            if(u == -1) u = 0;
            else u = node[id][u][nxt];
            ret += cnt[u];
        }
        return ret;
    }
} aho[20];
vector<string> dict[20];
vector<int> vals[20];
void add(string &s, int val)
{
    dict[0].pb(s);
    vals[0].pb(val);
    for (int i=(0), nb_fin_9=(20); i<nb_fin_9; ++i)
    {
        if(sz(dict[i]) > (1<<i))
        {
            for(auto it : dict[i]) dict[i + 1].pb(it);
            for(auto it : vals[i]) vals[i + 1].pb(it);
            dict[i].clear();
            vals[i].clear();
            aho[i].clear();
        }
        else
        {
            aho[i].init(i);
            aho[i].insertdict(dict[i], vals[i]);
            aho[i].pushLinks();
            break;
        }
    }
}
```

§9.2 Hashing

§9.2.1 fast-hashing.cpp

08_Strings/Hashing

```
const int N = 1e6 + 9; // Max size
int power(long long n, long long k, const int mod) {
  int ans = 1 % mod;
  n %= mod;
  if (n < 0) n += mod;
  while (k) {
    if (k & 1) ans = (long long) ans * n % mod;
    n = (long long) n * n % mod;
    k >>= 1;
  }
  return ans;
}
const int MOD1 = 127657753, MOD2 = 987654319;
const int p1 = 137, p2 = 277;
int ip1, ip2;
pair<int, int> pw[N], ipw[N];
void init() { // Call init() first!!!
  pw[0] =  {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i].first = 1LL * pw[i - 1].first * p1 % MOD1;
    pw[i].second = 1LL * pw[i - 1].second * p2 % MOD2;
  }
  ip1 = power(p1, MOD1 - 2, MOD1);
  ip2 = power(p2, MOD2 - 2, MOD2);
  ipw[0] =  {1, 1};
  for (int i = 1; i < N; i++) {
    ipw[i].first = 1LL * ipw[i - 1].first * ip1 % MOD1;
    ipw[i].second = 1LL * ipw[i - 1].second * ip2 % MOD2;
  }
}
struct Hashing {
  int n;
  string s; // 0 - indexed
  vector<pair<int, int>> hs; // 1 - indexed
  Hashing() {}
  Hashing(string _s) {
    n = _s.size();
    s = _s;
    hs.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pair<int, int> p;
      p.first = (hs[i].first + 1LL * pw[i].first * s[i] % MOD1) %
MOD1;
      p.second = (hs[i].second + 1LL * pw[i].second * s[i] % MOD2) %
MOD2;
      hs.push_back(p);
    }
  }
  pair<int, int> get_hash(int l, int r) { // 1-indexed
    pair<int, int> ans;
    ans.first = (hs[r].first - hs[l - 1].first + MOD1) * 1LL * ipw[l
- 1].first % MOD1;
    ans.second = (hs[r].second - hs[l - 1].second + MOD2) * 1LL *
ipw[l - 1].second % MOD2;
    return ans;
  }
  pair<int,int> get(int l, int r) { // 0-indexed
    return get_hash(l+1,r+1);
  }
  pair<int, int> get_hash() {
    return get_hash(1, n);
  }
};
```

§9.3 KMP

§9.3.1 kmp-automaton.cpp

08_Strings/KMP

```
// Very useful for some DP's with strings
// aut[i][j], you are in 'i' position, and choose character 'j', the
next position.
const int MAXN = 1e5 + 5, alpha = 26;
const char L = 'A';
int aut[MAXN][alpha]; // aut[i][j] = a donde vuelvo si estoy en i y
pongo una j
void build(string &s) {
    int lps = 0;
    aut[0][s[0]-L] = 1;
    int n = s.size();
    for (int i = 1; i < n+1; i++) {
        for (int j = 0; j < alpha; j++) aut[i][j] = aut[lps][j];
        if (i < n) {
            aut[i][s[i]-L] = i + 1;
            lps = aut[lps][s[i]-L];
        }
    }
}
```

§9.3.2 kmp-vasito.cpp

08_Strings/KMP

```
vector<int> kmppre(string& t){ // r[i]: Longest border of t[0,i)
    vector<int> r(t.size()+1);r[0]=-1;
    int j=-1;
    for (int i=(0), nb_fin_1=(t.size()); i<nb_fin_1; ++i){
        while(j>=0&&t[i]!=t[j])j=r[j];
        r[i+1]=++j;
    }
    return r;
}
void kmp(string& s, string& t){ // find t in s
    int j=0;vector<int> b=kmppre(t);
    for (int i=(0), nb_fin_3=(s.size()); i<nb_fin_3; ++i){
        while(j>=0&&s[i]!=t[j])j=b[j];
        if(++j==t.size())printf("Match at %lld\n",i-j+1),j=b[j];
    }
}
```

§9.4 Manacher

§9.4.1 manacher.cpp

08_Strings/Manacher

```
vi f(string &s) {
    int n = s.size();
    vi v(n);
    int l = 1, r = 1;
    for (int i=1 ; i<n;i ++) {
        v[i] = max(0ll, min(r-i, v[l+r-i]));
        while (s[i-v[i]] == s[i+v[i]]) v[i]++;
        if (i+v[i] > r) l = i-v[i], r = i+v[i];
    }
    return v;
}
vi manacher(string &s, int n) {
    string t = "";
```

```
    for (int i= 0; i<n; i++) {
        string aux = i?"$": "";
        aux.pb(s[i]);
        t += aux;
    }
    t = "#"+t+"@";
    vi v = f(t);
    int res = v[0];
    for (int i=0 ; i<v.size(); i++)
        res = max(res, v[i]);
    int l, r, sum;
    l = 1, r = 1, sum =0;
    for (int i =0 ; i<v.size(); i++)
    if (res==v[i]) {
        int lx = i-(res-1), rx = i+(res-1);
        int s = 1+((res-1)/2)*2; // string: aaa
        if (!(i&1)) s = (res/2)*2; // string aa
        if (sum < s) l = lx, r = rx, sum = s;
    }
    for (int i=l; i<=r; i++)
        if (t[i]!='$') cout<<t[i];
    cout<<endl;
    return v;
}
```

§9.5 Palindromic Tree

§9.5.1 palindromic-tree.cpp

08_Strings/Palindromic_Tree

```
const int alfa = 26;
const char L = 'a';
struct node {
  int next[alfa], link, len;
  int cnt;
  node(int x, int l = 0, int c = 1) {
    len = x, link = l, cnt = c;
    memset(next, 0, sizeof next);
  }
  int& operator [] (int i) { return next[i]; }
};
struct palindromic_tree {
  vector<node> tree;
  string s;
  int n;
  int last;
  palindromic_tree(string t = "") {
    n = last = 0;
    tree.pb(node(-1));
    tree.pb(node(0));
    for (auto &c: t)add_char(c);
  }
  int getlink(int p) {
    while (s[n - tree[p].len - 1] != s[n])
      p = tree[p].link;
    return p;
  }
  void add_char(char ch) {
    s.pb(ch);
    int p = getlink(last), c = ch - L;
    if (!tree[p][c]) {
      int link = getlink(tree[p].link);
      link = max<long long>(1, tree[link][c]);
      tree[p][c] = static_cast<long long>((tree).size());
      tree.pb(node(tree[p].len + 2, link, 0));
    }
    last = tree[p][c];
    tree[last].cnt++;
    n++;
  }
};
```

§9.6 Rotación mínima

§9.6.1 minimum-expression-extra.cpp

08_Strings/Rotacion_minima

```
// Dado un string s devuelve el indice donde comienza la rotación
lexicograficamente menor de s.
// O(n)
int minimum_expression(string s) {
    s = s+s; // si no se concatena devuelve el indice del sufijo
menor
    int len = s.size(), i = 0, j = 1, k = 0;
    while (i+k < len && j+k < len) {
        if (s[i+k] == s[j+k]) k++;
        else if (s[i+k] > s[j+k]) i = i+k+1, k = 0; // cambiar por <
para maximum
        else j = j+k+1, k = 0;
        if (i == j) j++;
    }
    return min(i, j);
}
```

§9.7 Suffix Array y LCP

§9.7.1 suffix-array-extra.cpp

08_Strings/Suffix_Array_y_LCP

```
using pii = pair<int, int>;
const int MAXL = 300;
struct suffixArray {
    string s;
    int n, MX;
    vector<int> ra, tra, sa, tsa, lcp;
    suffixArray(string &_s) {
        s = _s+"$";
        n = s.size();
        MX = max(MAXL, n)+2;
        ra = tra = sa = tsa = lcp = vector<int>(n);
        build();
    }
    void radix_sort(int k) {
        vector<int> cnt(MX, 0);
        for (int i = 0; i < n; i++)
            cnt[(i+k < n) ? ra[i+k]+1 : 1]++;
        for (int i = 1; i < MX; i++)
            cnt[i] += cnt[i-1];
        for (int i = 0; i < n; i++)
            tsa[cnt[(sa[i]+k < n) ? ra[sa[i]+k] : 0]++] = sa[i];
        sa = tsa;
    }
    void build() {
        for (int i = 0; i < n; i++)
            ra[i] = static_cast<unsigned char>(s[i]), sa[i] = i;
        for (int k = 1, r; k < n; k <<= 1) {
            radix_sort(k);
            radix_sort(0);
            tra[sa[0]] = r = 0;
            for (int i = 1; i < n; i++) {
                if (ra[sa[i]] != ra[sa[i-1]] || (sa[i]+k < n ?
ra[sa[i]+k] : -1) != (sa[i-1]+k < n ? ra[sa[i-1]+k] : -1)) ++r;
                tra[sa[i]] = r;
            }
            ra = tra;
            if (ra[sa[n-1]] == n-1) break;
        }
    }
    int& operator [] (int i) { return sa[i]; }
    void build_lcp() {
        lcp[0] = 0;
        for (int i = 0, k = 0; i < n; i++) {
            if (!ra[i]) continue;
            while (i+k < n && sa[ra[i]-1]+k < n && s[i+k] ==
s[sa[ra[i]-1]+k]) k++;
            lcp[ra[i]] = k;
            if (k) k--;
        }
    }
    // Longest Common Substring: construir el suffixArray s = s1 +
"#" + s2 + "$" y m = s2.size()
    pii lcs(int m) {
        int mx = -1, ind = -1;
        for (int i = 1; i < n; i++) {
            if (((sa[i] < n-m-1) != (sa[i-1] < n-m-1)) && mx <
lcp[i]) {
                mx = lcp[i]; ind = i;
            }
        }
        return {mx, ind};
    }
};
```

§9.8 Suffix Automaton

§9.8.1 suffix-automaton-extra.cpp

08_Strings/Suffix_Automaton

```
using ll = long long;
struct suffixAutomaton {
    struct node {
        int len, link; bool end;
        map<char, int> next;
        int fpos, cnt; ll out;
    };
    vector<node> sa;
    int last;
    ll substrs = 0;
    suffixAutomaton() {}
    suffixAutomaton(string &s) {
        sa.reserve(s.size()*2);
        last = add_node();
        sa[0].link = -1;
        for (char &c : s) add_char(c);
        for (int p = last; p; p = sa[p].link) sa[p].end = 1;
    }
    int add_node() { sa.pb({}); return sa.size()-1; }
    void add_char(char c) {
        int u = add_node(), p = last;
        sa[u].len = sa[last].len + 1;
        sa[u].fpos = sa[u].len - 1;
        while (p != -1 && !sa[p].next.count(c)) {
            sa[p].next[c] = u;
            substrs += p != 0 ? sa[p].len - sa[sa[p].link].len : 1;
            p = sa[p].link;
        }
        if (p != -1) {
            int q = sa[p].next[c];
            if (sa[p].len + 1 != sa[q].len) {
                int clone = add_node();
                sa[clone] = sa[q];
                sa[clone].len = sa[p].len + 1;
                sa[q].link = sa[u].link = clone;
                while (p != -1 && sa[p].next[c] == q) {
                    sa[p].next[c] = clone;
                    p = sa[p].link;
                }
            } else sa[u].link = q;
        }
        last = u;
    }
    void run(string &s) {
        int u = 0;
        for (int i = 0; i < s.size(); ++i) {
            while (u && !sa[u].next.count(s[i])) u = sa[u].link;
```

```
            if (sa[u].next.count(s[i])) u = sa[u].next[s[i]];
        }
    }
    int match_str(string &s) {
        int u = 0, n = s.size();
        for (int i = 0; i < n; ++i) {
            if (!sa[u].next.count(s[i])) return 0;
            u = sa[u].next[s[i]];
        }
        return count_occ(u);
    }
    int count_occ(int u) {
        if (sa[u].cnt != 0) return sa[u].cnt;
        sa[u].cnt = sa[u].end;
        for (auto &v : sa[u].next)
            sa[u].cnt += count_occ(v.second);
        return sa[u].cnt;
    }
    ll count_paths(int u) {
        if (sa[u].out != 0) return sa[u].out;
        for (auto &v : sa[u].next)
            sa[u].out += count_paths(v.second) + 1;
        return sa[u].out;
    }
    node& operator[](int i) { return sa[i]; }
};
```

§9.9 Suffix Tree

§9.9.1 suffix-tree-vasito.cpp

08_Strings/Suffix_Tree

```
struct SuffixTree {
    char s[MAXN];
    map<int,int> to[MAXN];
    int len[MAXN]={INF},fpos[MAXN],link[MAXN];
    int node,pos,sz=1,n=0;
    int make_node(int p, int l){
        fpos[sz]=p;len[sz]=l;return sz++;}
    void go_edge(){
        while(pos>len[to[node][s[n-pos]]]){
            node=to[node][s[n-pos]];
            pos-=len[node];
        }
    }
    void add(int c){
        s[n++]=c;pos++;
        int last=0;
        while(pos>0){
            go_edge();
            int edge=s[n-pos];
            int& v=to[node][edge];
            int t=s[fpos[v]+pos-1];
            if(v==0){
                v=make_node(n-pos,INF);
                link[last]=node;last=0;
            }
            else if(t==c){link[last]=node;return;}
            else {
                int u=make_node(fpos[v],pos-1);
                to[u][c]=make_node(n-1,INF);
                to[u][t]=v;
                fpos[v]+=pos-1;len[v]-=pos-1;
                v=u;link[last]=u;last=u;
            }
            if(node==0)pos--;
            else node=link[node];
        }
    }
};
```

§9.10 Trie

§9.10.1 trie.cpp

08_Strings/Trie

```
// Requisito de selección: Contiene dominios distintos (letras, bits
y comodines) con nombres repetidos y errores de índice/typos del
original; requiere separar namespaces, inicializar contadores y
completar go/ter.
const int MAXSIZE = 26;
struct node {
    int end;
    int grafo[MAXSIZE]
    node() {
        for (int i =0 ; i<MAXSIZE; i++) grafo[i] = 0;
    }
};
vector<node> v(1);
node ini;
void insert(string &s, int i = 0, int j = 0) {
    if (j>size(s)) {
        v[i].end = 1;
        return;
    }
    int to = s[j]-'a';
    if (!v[i].grafo[to]) v[i].grafo[to] = v.size(), v.pb(ini);
    insert(s, v[i].grafo[to], j+1);
}
void erase(string &s, int i = 0, int j = 0) {
    if (j>size(s)) {v[i].end = 0; return;}
    int to = s[j]-'a';
    if (!v[i].grafo[to]) return;
    return erase(s, v[i].grafo[to], j+1);
}
int get(string &s, int i = 0, int j = 0) {
    if (j>size(s)) return v[i].end;
    int to = s[j]-'a';
    if (v[i].grafo[to]) return get(s, v[i].grafo[to], j+1);
}
/*FOR BITS:*/
const int MAXSIZE = 2;
struct node {
    int c;
    int grafo[MAXSIZE];
    node() {grafo[0] = grafo[1] = 0;}
};
vector<node> v(1);
node ini;
void insert(int x, int i = 0, int j = 30) {
    v[i].c++;
    if (j==-1) return;
    int to = (x>>j)&1;
    if (!v[i].grafo[to])
        v[i].grafo[to] = v.size(), v.pb(ini);
    insert(x, v[i].grafo[to], j-1);
}
void erase(int x, int i = 0, int j = 30) {
    v[i].c--;
    if (j==-1) return;
    int to = (x>>j)&1;
    erase(x, v[i].grafo[to], j-1);
}
int get(int x, int i = 0, int j = 30) {
    // modify this function based on the problem.
    if (j==-1) return 0;
    int to = 1^((x>>j)&1);
    if (!v[i].grafo[to] || !v[v[i].grafo[to]].c) to ^= 1;
    if (!v[i].grafo[to] || !v[v[i].grafo[to]].c) return 0;
    return (to<<j) + get(x, v[i].grafo[to], j-1);
}
```

```
/* potente: */
struct node;
extern vector<node> trie;
struct node {
    int v[MAXSIZE];
    int end;
    node() {
        for (int i =0 ; i<MAXSIZE; i++) v[i] = 0; end =0 ;
    }
    int go(char c) {return (!v[c-'a']? -1: v[c-'a']);}
    void set(char c) {
        v[c-'a'] = (int)trie.size();
        trie.emplace_back(node());
    }
};
vector<node> trie(1);
void add(string &s, int j) {
    int i = 0;
    for (auto &c: s) {
        if (trie[i].go(c) == -1) trie[i].set(c);
        i = trie[i].go(c);
    }
    trie[i].ter = j;
}
vector<int> find (string &s) {
    vector<int> res;
    auto dfs = [&](int i, int j, auto &&dfs) {
        if (j == s.size()) {
            res.push_back(trie[i].ter);
            return ;
        }
        int go = trie[i].go(s[j]);
        if (go != -1) dfs(go, j+1, dfs);
        if (s[j] != '_') {
            go = trie[i].go();
            if (go != -1) dfs(go, j+1, dfs);
        }
    };
    dfs(0, 0, dfs);
    return res;
}
```

§9.11 Z Function

§9.11.1 z-function-vasito.cpp

08_Strings/Z_Function

```
vector<int> z_function(string& s){
    int l=0,r=0,n=s.size();
    vector<int> z(s.size(),0); // z[i] = max k: s[0,k) == s[i,i+k)
    for (int i=(1), nb_fin_1=(n); i<nb_fin_1; ++i){
        if(i<=r)z[i]=min(r-i+1,z[i-l]);
        while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
        if(i+z[i]-1>r)l=i,r=i+z[i]-1;
    }
    return z;
}
```

§10 Utilidades

§10.1 Aleatoriedad

§10.1.1 randoms.cpp

09_Utilidades/Aleatoriedad

```
using ll = long long;
// Get random numbers between [a, b]
mt19937
mt_rng(chrono::steady_clock::now().time_since_epoch().count());
// also for ll exists mt19937_64
ll randint(ll a, ll b) {
```

```
    return uniform_int_distribution<ll>(a, b)(mt_rng);
}
```

§10.2 Bits

§10.2.1 bit-tricks.cpp

09_Utilidades/Bits

```
y = x & (x-1)   // apagar el 1 más a la derecha
y = x & (-x)    // aislar el 1 más a la derecha (lowbit)
y = x | (x-1)   // propagar a la derecha el 1 más a la derecha
y = x | (x+1)   // encender el 0 más a la derecha
y = ~x & (x+1)  // aislar el 0 más a la derecha
// Builtins (sufijo ll para long long; x = 0 indefinido en clz/ctz):
__builtin_popcount(x)  // cantidad de unos      __builtin_parity(x)
// 1 si es impar
__builtin_clz(x)       // ceros a la izquierda  __builtin_ctz(x)
// ceros a la derecha
__builtin_ffs(x)       // 1 + índice del 1 más bajo (0 si x == 0)
for(int s=m;s;s=(s-1)&m)  // submáscaras no vacías de m, orden
decreciente
for(int s=0;s=s-m&m;)     // submáscaras no vacías de m, orden
creciente
```

§10.2.2 bits-manipulation-extra.cpp

09_Utilidades/Bits

```
* Operaciones a nivel de bits. Si n es ll usar 1ll<< en los
corrimientos.
x & 1           -> Verifica si x es impar
x & (1<<i)      -> Verifica si el i-esimo bit esta encendido
x = x | (1<<i)  -> Enciende el i-esimo bit
x = x & ~(1<<i) -> Apaga el i-esimo bit
x = x ^ (1<<i)  -> Invierte el i-esimo bit
x = ~x          -> Invierte todos los bits
x & -x          -> Devuelve el bit encendido mas a la derecha
(potencia de 2, no el indice)
~x & (x+1)      -> Devuelve el bit apagado mas a la derecha (potencia
de 2, no el indice)
x = x | (x+1)   -> Enciende el bit apagado mas a la derecha
x = x & (x-1)   -> Apaga el bit encendido mas a la derecha
x = x & ~y      -> Apaga en x los bits encendidos de y
* Funciones del compilador gcc. Si n es ll agregar el sufijo ll, por
ej: __builtin_clzll(n).
__builtin_clz(x)      -> Cantidad de bits apagados por la izquierda
__builtin_ctz(x)      -> Cantidad de bits apagados por la derecha.
Indice del bit encendido mas a la derecha
__builtin_popcount(x) -> Cantida de bits encendidos
* Logaritmo en base 2 (entero). Indice del bit encendido mas a la
izquierda. Si x es ll usar 63 y clzll(x).
// O(1)
int lg2(const int &x) { return 31-__builtin_clz(x); }
* Itera, con indices, los bits encendidos de una mascara.
// O(#bits_encendidos)
for (int x = mask; x; x &= x-1) {
    int i = __builtin_ctz(x);
}
* Itera todas las submascaras de una mascara. (Iterar todas las
submascaras de todas las mascaras es O(3^n)).
// O(2^(#bits_encendidos))
for (int sub = mask; sub; sub = (sub-1)&mask) {
}
```

§10.2.3 optimizing-pragmas-for-bitset.cpp

09_Utilidades/Bits

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
// popcnt is The difference to TLE and AC in
https://cses.fi/problemset/task/2137/
```

§10.3 CPP

§10.3.1 bultin-functions-out-dated.cpp

09_Utilidades/CPP

```
# Sum the values of a iterable
# Very important to put 0ll to avoid overflows
accumulate(v.begin(),v.end(),0ll)/n;
```

§10.3.2 cpp-stuff.cpp

09_Utilidades/CPP

```
const double DINF=numeric_limits<double>::infinity();
// set/map de doubles con tolerancia (builtins y submáscaras: ver
Bits/bit-tricks.cpp)
struct comp {
    bool operator()(const double& a, const double& b) const {
        return a+EPS<b;}
};
set<double,comp> w; // o map<double,int,comp>
```

§10.3.3 exponential-notation.cpp

09_Utilidades/CPP

```
using ll = long long;
// O(n) convert numbers to Exponential Notation
// (e.g 0102.150 -> 1.0215E2)
// only float numbers > 0
string exponential_notation(string s) {
    int firstPos = find_if(all(s), [&](char c) {
        return c != '0' && c != '.';
    }) - s.begin();
    int dotPos = find(all(s), '.') - s.begin();
    ll base = dotPos - (firstPos+(firstPos <= dotPos));
    s.erase(dotPos, 1);
    for (int i = 0; i < 2; i++) { //erase traveling zeros
        while (s.back() == '0') s.pop_back();
        reverse(all(s));
    }
    if (s.size() > 1) s.insert(1, ".");
    if (base != 0) s+= "E" + to_string(base);
    return s;
}
```

§10.4 Entrada salida

§10.4.1 io-int128.cpp

09_Utilidades/Entrada_salida

```
// Requisito: Requiere compilador GNU con enteros de 128 bits.
// Lectura y escritura con cin/cout, compatibles con fast.
__int128 read() {
    string s; cin >> s;
    bool negative = !s.empty() && s[0] == '-';
    __uint128_t x = 0;
    for (int i = (!s.empty() && (s[0] == '-' || s[0] == '+')); i <
(int)s.size(); ++i)
        x = x * 10 + (s[i] - '0');
    return negative && x ? -static_cast<__int128>(x - 1) - 1 :
static_cast<__int128>(x);
}
void print_iostream(__int128 x) {
    __uint128_t magnitude = x < 0 ? __uint128_t(0) - __uint128_t(x) :
__uint128_t(x);
    string digits;
    do { digits.pb(char('0' + magnitude % 10)); magnitude /= 10; }
while (magnitude);
    if (x < 0) cout << '-';
    reverse(all(digits)); cout << digits;
}
```

§10.4.2 python-fast-in.py

09_Utilidades/Entrada_salida

```
import sys
inputs = sys.stdin.read().splitlines()
con = 1
ln = 0
while True:
    a, b = map(int, inputs[ln].split())
    ln += 1
    # a = int(input())
    # b = int(input())
    if a == 0:
        break
    su = 0
    for i in range (a):
        su += int(inputs[ln])
        ln += 1
    print(f"Bill #{con} costs {su}: each friend should pay {su //
b}")
    print()
    con += 1
```

§10.4.3 scanf-and-printf-extra.cpp

09_Utilidades/Entrada_salida

```
// Requisito: En simple.cpp int significa long long: usar %lld para
esos valores. Los formatos listados describen los tipos nativos.
* Lectura segun el tipo de dato (Se usan las mismas para imprimir):
scanf("%d", &value);   // int
scanf("%ld", &value);  // long y long int
scanf("%c", &value);   // char
scanf("%f", &value);   // float
scanf("%lf", &value);  // double
scanf("%s", &value);   // char*
scanf("%lld", &value); // long long int
scanf("%x", &value);   // int hexadecimal
scanf("%o", &value);   // int octal
* Impresion de punto flotante con d decimales, ejemplo 6 decimales:
printf("%.6lf", value);
```

§10.4.4 split-string-extra.cpp

09_Utilidades/Entrada_salida

```
// Divide el string s por cada espacio ' ' y devuelve un vector<> con
los substrings resultantes.
// Para dividir el string por un caracter especifico, agregar el
parametro c y cambiar el while.
vector<string> split(const string &s/*, char c*/) {
    vector<string> v;
    stringstream ss(s);
    string sub;
    while (ss >> sub) v.pb(sub);
    // while (getline(ss, sub, c)) v.pb(sub);
    return v;
}
```

§10.4.5 string-streams.cpp

09_Utilidades/Entrada_salida

```
// Requisito: Ejemplo de lectura: el bloque pertenece a solve().
using ll = long long;
using vl = vector<long long>;
// For some complex reading of input
// st is the same as a cin, but you pass the string
string line;
getline(cin, line);
stringstream st(line);
vl in;
ll x;
while (st >> x) {
    in.pb(x);
}
```

§10.5 Fechas

§10.5.1 dates.cpp

09_Utilidades/Fechas

```
int dateToInt(int y, int m, int d){
    return 1461*(y+4800+(m-14)/12)/4+367*(m-2-(m-14)/12*12)/12-
        3*((y+4900+(m-14)/12)/100)/4+d-32075;
}
```

```cpp
void intToDate(int jd, int& y, int& m, int& d){
    int x,n,i,j;x=jd+68569;
    n=4*x/146097;x-=(146097*n+3)/4;
    i=(4000*(x+1))/1461001;x-=1461*i/4-31;
    j=80*x/2447;d=x-2447*j/80;
    x=j/11;m=j+2-12*x;y=100*(n-49)+i+x;
}
int DayOfWeek(int d, int m, int y){   //starting on Sunday
    static int ttt[]={0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
    y-=m<3;
    return (y+y/4-y/100+y/400+ttt[m-1]+d)%7;
}
```

§10.6 Interactivos

§10.6.1 main.cpp

09_Utilidades/Interactivos

```cpp
using namespace std;
signed main() {
    ios_base::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    vector<int> mapa(static_cast<long long>(1e6), 0);
    int res= 1, n;
    cin>>n;
    for (int s = 0; s<n; s++) {
        int x; cin>>x;
        for (int i = 1; i*i<=x; i++)
        if (!(x%i)) {
            mapa[i]++;
            if (mapa[i]>1) res = max(res, i);
            if (i*i!=x) {
                mapa[x/i]++;
                if (mapa[x/i]>1)
                    res = max(res, x/i);
            }
        }
    }
    cout<<res<<'\n';
    return 0;
}
```

§10.7 Interactivos › pruebas

§10.7.1 interactive-tester-vasito.py

09_Utilidades/Interactivos/pruebas

```python
#Easier method with bash commands:
#mkfifo fifo
#(./solution < fifo) | (./interactor > fifo)
# tester for cf 101021A (guess a number, queries: is it >=k?)
import random
import subprocess as sp
seed = random.randint(0, sys.maxint);random.seed(seed)
n=random.randint(1,1000000)
try:
    p=sp.Popen(['./a.out'],stdin=sp.PIPE,stdout=sp.PIPE)
    it=0
    s=p.stdout.readline()
    while it<25 and s and s[0]!='!':
        k=int(s)
        assert k>=1 and k<=1000000
        if n>=k: p.stdin.write('>=\n')
        else: p.stdin.write('<\n')
        s=p.stdout.readline()
        it+=1
    assert s and s[0]=='!'
    k=int(s.split()[1])
    assert k==n
except:
    print 'failed with seed %s' % seed
    raise
```

§10.8 Python

§10.8.1 decimal-precisition-python.py

09_Utilidades/Python

```python
"""
For problems that needs more decimal precisition
You can try this code but in C++,
it will not pass in this problem
https://cses.fi/problemset/task/1728/
"""
from decimal import *
getcontext().prec = 200 #The decimal precisition
n = int(input())
nums = [int(x) for x in input().split(" ")]
ans = Decimal(0)
for i in range(0,len(nums)):
    for j in range(i+1,len(nums)):
        for k in range(1,nums[j]+1):
            ans += max(0,nums[i]-k)/Decimal(nums[i]*nums[j])
            # Also for reduce getcontext().prec = 100
            # You can sum integer part, then apply the decimal
division
print("{:.6f}".format(ans)) #The rounding half even to six decimals
```

§10.8.2 python.py

09_Utilidades/Python

```python
# reopen
import sys
sys.stdout = open('out','w')
sys.stdin  = open('in' ,'r')
//Dummy example
R = lambda: map(int, input().split())
n, k = R(),
v, t = [], [0]*n
for p, c, i in sorted(zip(R(), R(), range(n))):
    t[i] = sum(v)+c
    v += [c]
    v = sorted(v)[::-1]
    if len(v) > k:
        v.pop()
print(' '.join(map(str, t)))
```

§10.9 Rendimiento

§10.9.1 pragmas.cpp

09_Utilidades/Rendimiento

```cpp
//#pragma GCC target("popcnt")
//It's worth noting that after adding  __builtin_popcount() is
replaced to corresponding machine instruction (look at the
difference). In my test this maked x2 speed up. bitset::count() use
__builtin_popcount() call in implementation, so it's also affected by
this.
#pragma GCC target ("avx2")
#pragma GCC optimization ("O3")
#pragma GCC optimization ("unroll-loops")
#pragma GCC target("popcnt")
#pragma GCC target("avx,avx2,sse3,ssse3,sse4.1,sse4.2,tune=native")
#pragma GCC optimize(3)
#pragma GCC optimize("O3")
#pragma GCC optimize("inline")
#pragma GCC optimize("-fgcse")
#pragma GCC optimize("-fgcse-lm")
#pragma GCC optimize("-fipa-sra")
#pragma GCC optimize("-ftree-pre")
#pragma GCC optimize("-ftree-vrp")
#pragma GCC optimize("-fpeephole2")
#pragma GCC optimize("-fsched-spec")
#pragma GCC optimize("-falign-jumps")
#pragma GCC optimize("-falign-loops")
#pragma GCC optimize("-falign-labels")
#pragma GCC optimize("-fdevirtualize")
#pragma GCC optimize("-fcaller-saves")
#pragma GCC optimize("-fcrossjumping")
#pragma GCC optimize("-fthread-jumps")
#pragma GCC optimize("-freorder-blocks")
#pragma GCC optimize("-fschedule-insns")
#pragma GCC optimize("inline-functions")
#pragma GCC optimize("-ftree-tail-merge")
#pragma GCC optimize("-fschedule-insns2")
#pragma GCC optimize("-fstrict-aliasing")
#pragma GCC optimize("-falign-functions")
#pragma GCC optimize("-fcse-follow-jumps")
#pragma GCC optimize("-fsched-interblock")
#pragma GCC optimize("-fpartial-inlining")
#pragma GCC optimize("no-stack-protector")
#pragma GCC optimize("-freorder-functions")
#pragma GCC optimize("-findirect-inlining")
#pragma GCC optimize("-fhoist-adjacent-loads")
#pragma GCC optimize("-frerun-cse-after-loop")
#pragma GCC optimize("inline-small-functions")
#pragma GCC optimize("-finline-small-functions")
#pragma GCC optimize("-ftree-switch-conversion")
#pragma GCC optimize("-foptimize-sibling-calls")
#pragma GCC optimize("-fexpensive-optimizations")
#pragma GCC optimize("inline-functions-called-once")
#pragma GCC optimize("-fdelete-null-pointer-checks")
```

§11 Python

§11.1 enteros_y_bits.py

10_Python

```python
# ENTEROS Y BITS: int no desborda (sin __int128 ni bigint).
Resultados en comentarios.
import math
from fractions import Fraction
from decimal import Decimal, ROUND_HALF_UP
# --- log2 exacto (int(math.log(1000, 10)) da 2: nunca usar floats
para esto) ---
# n.bit_length() = floor(log2 n) + 1  ->  floor(log2 n) =
n.bit_length() - 1  (== __lg)
def ceil_log2(n): return (n - 1).bit_length()          # menor k con
2^k >= n
def next_pow2(n): return 1 << (n - 1).bit_length()     # tamaño de
segment tree iterativo
def is_pow2(x): return x > 0 and x & (x - 1) == 0
# --- bits --- popcount: bin(m).count("1") o m.bit_count() (3.10+)
# lowbit: m & -m   ctz: (m & -m).bit_length() - 1   bit i: m >> i & 1
# Prioridad: en Python & | ^ van ANTES que == (en C++ al revés): x &
1 == 1 es (x & 1) == 1
def bits(mask):                            # índices de los bits
encendidos
    while mask:
        low = mask & -mask
        yield low.bit_length() - 1
        mask ^= low
def submasks(m):                           # submáscaras no vacías de m
(todas: O(3^n))
    s = m
    while s:
        yield s
        s = (s - 1) & m
MASK64 = (1 << 64) - 1                     # (-1) & MASK64 = 2^64 - 1
(ver como unsigned)
def to_signed64(x):                        # simular overflow de long
long
    x &= MASK64
    return x - (1 << 64) if x >> 63 else x
# --- bases --- int("ff", 16) = 255   int("z", 36) = 35   int("
42\n") = 42
# format(5, "08b") = "00000101"   f"{255:x}" = "ff"   bin(10)[2:] =
```

```
"1010"
DIG = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
def to_base(n, b):                          # entero -> cadena en base b (2..36)
    if n == 0: return "0"
    neg, n, s = n < 0, abs(n), []
    while n:
        n, r = divmod(n, b)
        s.append(DIG[r])
    return "-" * neg + "".join(reversed(s))
# --- división (DISTINTA a C++) --- -7 // 2 = -4, -7 % 2 = 1 (C++: -3, -1); con m > 0, a % m >= 0
# "/" SIEMPRE da float: (10**17 + 1) / 1 pierde el +1. Con enteros usar "//".
def ceil_div(a, b): return -(-a // b)
def div_trunc(a, b):                        # división truncada como en C++
    q = abs(a) // abs(b)
    return q if (a < 0) == (b < 0) else -q
# --- raíces exactas --- int(math.sqrt(10**18 - 1)) = 10**9 (MAL); math.isqrt da el piso exacto
def is_square(n): r = math.isqrt(n); return r * r == n
def iroot(n, k):                            # piso de la raíz k-ésima (n >= 0)
    lo, hi = 0, 1 << (n.bit_length() // k + 1)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid ** k <= n: lo = mid
        else: hi = mid - 1
    return lo
# --- modular --- pow(a, e, m); inverso: pow(a, -1, m) (3.8+, ValueError si gcd != 1)
MOD = 10**9 + 7
def build_fact(n, mod=MOD):                 # factoriales e inversos para nCr en O(1)
    fact = [1] * (n + 1)
    for i in range(1, n + 1): fact[i] = fact[i - 1] * i % mod
    inv = [1] * (n + 1)
    inv[n] = pow(fact[n], mod - 2, mod)
    for i in range(n, 0, -1): inv[i - 1] = inv[i] * i % mod
    return fact, inv
def ext_gcd(a, b):                          # (g, x, y) con a*x + b*y = g
    if b == 0: return a, 1, 0
    g, x, y = ext_gcd(b, a % b)
    return g, y, x - (a // b) * y
def crt(r1, m1, r2, m2):                    # x = r1 (m1), x = r2 (m2) -> (x, lcm) o None
    g, p, _ = ext_gcd(m1, m2)
    if (r2 - r1) % g: return None
    l = m1 // g * m2
    return (r1 + (r2 - r1) // g * p % (m2 // g) * m1) % l, l
# --- math --- gcd/lcm con varios argumentos (3.9+); comb, perm, factorial, prod: exactos
def sieve(n):                               # criba con slicing (mucho más rápida que doble for)
    is_p = bytearray([1]) * (n + 1)
    is_p[0:2] = b"\x00\x00"
    for i in range(2, math.isqrt(n) + 1):
        if is_p[i]: is_p[i * i::i] = bytes(len(range(i * i, n + 1, i)))
    return is_p
# str(10**5000) da ValueError en 3.11+: sys.set_int_max_str_digits(0)
# --- exactos y floats ---
# Fraction("0.1") + Fraction("0.2") == Fraction("0.3"); Fraction(0.1) arrastra el error del float
# Fraction(math.pi).limit_denominator(1000) = 355/113
# round(2.5) = 2 (al par); escolar: Decimal("2.5").quantize(Decimal("1"), rounding=ROUND_HALF_UP)
# Imprimir floats SIEMPRE con formato: f"{x:.6f}" (str(0.00001) = "1e-05")
```

§11.2 estructuras.py

10_Python

```
# ESTRUCTURAS DE PYTHON (y su equivalente en C++). Resultados en comentarios.
import random
from collections import defaultdict, Counter, deque
from heapq import heappush, heappop, heapify, nlargest
from bisect import bisect_left, bisect_right, insort
from itertools import accumulate, permutations, combinations, product, groupby
from functools import lru_cache, cmp_to_key
from operator import itemgetter
# --- tuplas (pair): se comparan lexicográficamente y son hashables (claves de dict/set) ---
pts = [(1, 2), (3, 2), (0, 5)]
pts.sort(key=lambda t: (-t[1], t[0]))    # y desc, luego x asc; también key=itemgetter(1, 0)
first, *mid, last = [1, 2, 3, 4]         # mid = [2, 3]
M = [[1, 2, 3], [4, 5, 6]]
T = [list(r) for r in zip(*M)]           # transpuesta; rotar 90° horario: zip(*M[::-1])
# --- listas (vector) ---
good = [[0] * 3 for _ in range(2)]       # [[0]*3]*2 comparte la MISMA fila (error clásico)
v = [1, 2, 3, 4, 5]                      # v[::-1], v[1:3], v[-2:], copia: v[:]
pre = list(accumulate(v, initial=0))     # suma de v[l:r] = pre[r] - pre[l]   (3.8+)
uniq = list(dict.fromkeys([3, 1, 3, 2])) # [3, 1, 2]: sin repetidos y sin reordenar
xs = sorted(set(v)); idx = {x: i for i, x in enumerate(xs)}   # compresión de coordenadas
# max([], default=0); argmin: min(range(len(a)), key=a.__getitem__); print(*v)
# --- dict / defaultdict / Counter (map) ---
g = defaultdict(list)                    # OJO: leer g[k] CREA la clave; consultar con "k in g"
cnt = Counter("abracadabra")             # cnt["z"] = 0 sin crearla; cnt.most_common(2)
# Counter: anagramas c1 == c2; resta c1 - c2; intersección c1 & c2 (mínimos)
RND = random.getrandbits(60)             # anti-hack: guardar claves como k ^ RND
# --- set --- A | B, A & B, A - B, A ^ B, A <= B; frozenset es hashable
# No hay set ordenado: lista ordenada + bisect/insort, heap con borrado perezoso o Fenwick.
# --- deque: colas (list.pop(0) es O(n)) ---
def window_max(a, k):                    # máximo de cada ventana de tamaño k, O(n)
    dq, res = deque(), []
    for i, x in enumerate(a):
        while dq and a[dq[-1]] <= x: dq.pop()
        dq.append(i)
        if dq[0] <= i - k: dq.popleft()
        if i >= k - 1: res.append(a[dq[0]])
    return res
# --- heapq: heap de MÍNIMOS (max-heap: guardar -x); h[0] = mínimo; nlargest(k, a) ---
def dijkstra(adj, s):                    # adj[u] = [(v, w)]
    dist = [float("inf")] * len(adj)
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heappop(pq)
        if d > dist[u]: continue         # entrada vieja (borrado perezoso)
        for v2, w in adj[u]:
            if d + w < dist[v2]:
                dist[v2] = d + w
                heappush(pq, (d + w, v2))
    return dist
# --- bisect: lower_bound = bisect_left, upper_bound = bisect_right ---
# cuántos en [l, r]: bisect_right(s, r) - bisect_left(s, l); insort(s, x) inserta ordenado
def first_true(lo, hi, ok):              # menor x en [lo, hi) con ok(x); hi si ninguno
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo
# --- itertools: fuerza bruta --- permutations(a, k), combinations(a, k), product([0, 1], repeat=n)
rle = [(ch, len(list(gr))) for ch, gr in groupby("aaabcc")]   # run-length: [(a,3),(b,1),(c,2)]
# --- functools ---
@lru_cache(maxsize=None)                 # DP top-down; fib.cache_clear() entre casos
def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
def largest_number(nums):                # comparador custom con cmp_to_key
    cmp = lambda a, b: (a + b < b + a) - (a + b > b + a)
    return str(int("".join(sorted(map(str, nums), key=cmp_to_key(cmp)))))
# --- strings: inmutables --- "  a  b ".split() = ["a", "b"]; s.find("z") = -1
# ord("a") = 97, chr(99) = "c"; editar: t = list(s) ... "".join(t)
# Construir cadenas largas con lista + "".join(): s += ... en un loop es lento.
```

§11.3 problemas_python_vs_cpp.py

10_Python

```
# PROBLEMAS QUE SON MÁS FÁCILES EN PYTHON QUE EN C++
# Cada función recibe la entrada completa como texto y devuelve la salida esperada.
# Ejecutar: python problemas_python_vs_cpp.py  -> prueba los ejemplos.
# Para enviar: la función con  print(f(sys.stdin.read()))
import sys, math
from fractions import Fraction
if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)
# 1) Codeforces 1606B "Update Files"  (n, k <= 10^18)
# Un archivo está en 1 computadora; hay n computadoras y k cables. Cada hora, cada
# computadora con el archivo puede copiarlo a otra por un cable. Horas mínimas.
# Idea: mientras haya menos de k copias, se duplican; luego crecen k por hora.
# Las horas duplicando son el menor h con 2^h >= min(n, k) = (min(n, k) - 1).bit_length().
# En C++ con log2 (double) y n ~ 10^18 el redondeo da respuestas erróneas; aquí es exacto.
def update_files(inp):
    data = inp.split()
    res = []
    for i in range(int(data[0])):
        n, k = int(data[1 + 2 * i]), int(data[2 + 2 * i])
        h = (min(n, k) - 1).bit_length()
        cur = 1 << h
        h += max(0, -(-(n - cur) // k))  # techo de (n - cur) / k
        res.append(h)
    return "\n".join(map(str, res))
assert update_files("4\n8 3\n6 6\n7 1\n1 1\n") == "4\n3\n6\n0"
assert update_files("1\n100000000000000000 1000000000000000000\n")
```

```
== "60"
# 2) CodeChef FCTRL2 "Small factorials": t casos, 1 <= n <= 100, imprimir n!
# 100! tiene 158 dígitos: en C++ hay que programar multiplicación de bigint con arreglos.
def small_factorials(inp):
    nums = list(map(int, inp.split()))
    return "\n".join(str(math.factorial(n)) for n in nums[1:])
assert small_factorials("4\n1\n2\n5\n3\n") == "1\n2\n120\n6"
# 3) SPOJ JULKA: Klaudia y Natalia tienen T manzanas juntas y Klaudia tiene D más.
# T y D con hasta 100 dígitos (10 casos). Imprimir lo de cada una.
# Klaudia = (T + D) / 2, Natalia = (T - D) / 2. En C++: suma, resta y división de bigint.
def julka(inp):
    data = inp.split()
    res = []
    for i in range(0, len(data), 2):
        total, more = int(data[i]), int(data[i + 1])
        res.append(str((total + more) // 2))
        res.append(str((total - more) // 2))
    return "\n".join(res)
assert julka("10\n2\n") == "6\n4"
assert julka(f"{10**100}\n2\n") == f"{5 * 10**99 + 1}\n{5 * 10**99 - 1}"
# 4) CSES 1745 "Money Sums": n <= 100 monedas de valor <= 1000. Imprimir cuántas sumas
# distintas se pueden formar y cuáles, en orden.
# Un int de Python funciona como bitset de tamaño ilimitado: el bit s indica si la suma s
# es posible y "reach << x" suma x a todas a la vez. En C++ necesitas bitset<MAXS>
# con el tamaño fijo en compilación; aquí no se fija nada.
def money_sums(inp):
    data = list(map(int, inp.split()))
    coins = data[1:1 + data[0]]
    reach = 1                           # solo la suma 0
    for x in coins:
        reach |= reach << x
    bits = bin(reach)[:1:-1]            # bits[s] == "1" <=> suma s posible (LSB primero)
    sums = [s for s in range(1, len(bits)) if bits[s] == "1"]
    return f"{len(sums)}\n{' '.join(map(str, sums))}"
assert money_sums("4\n4 2 5 2\n") == "9\n2 4 5 6 7 8 9 11 13"
# 5) Suma armónica exacta: dado n (<= 1000), imprimir 1 + 1/2 + ... + 1/n como p/q
# irreducible. El denominador llega a cientos de dígitos: en C++ desborda long long
# desde n ~ 40. Con Fraction se suma exacto y siempre queda reducida.
def harmonic(inp):
    n = int(inp)
    h = sum(Fraction(1, i) for i in range(1, n + 1))
    return f"{h.numerator}/{h.denominator}"
assert harmonic("1") == "1/1" and harmonic("3") == "11/6" and
harmonic("4") == "25/12"
assert len(harmonic("1000")) > 300     # p/q con cientos de dígitos
# 6) Ordenar números de hasta 10^6 dígitos (sin ceros a la izquierda) de menor a mayor.
# Convertir a int sería lento (y pasa el límite de 4300 dígitos). Ordenar por la TUPLA
# (longitud, cadena): más dígitos = más grande; con igual longitud, gana el orden lexicográfico.
def sort_big(inp):
    data = inp.split()
    nums = data[1:1 + int(data[0])]
    nums.sort(key=lambda s: (len(s), s))
    return "\n".join(nums)
assert sort_big("5\n3141592653589793238462643383279 5\n1\n3\n10\n3\n") == \
    "1\n3\n3\n10\n3141592653589793238462643383279 5"
# 7) Producto de dos números de 10^4 dígitos, o a^b mod m con a de 1000 dígitos:
#    print(int(x) * int(y))   /   print(pow(int(a), b, m))
#    En C++: FFT/Karatsuba o leer a como cadena y reducir dígito a dígito.
assert int("9" * 50) * int("9" * 50) == 10**100 - 2 * 10**50 + 1
assert pow(int("12345678901234567890" * 50), 10**18, 10**9 + 7) == \
    pow(int("12345678901234567890" * 50) % (10**9 + 7), 10**18, 10**9 + 7)
print("problemas_python_vs_cpp.py: OK")
```

§11.4 template_completo.py

10_Python

```
# Template completo: imports típicos + debug + límites. Para concursos.
# Local:  python sol.py -d < in.txt      (PowerShell: Get-Content in.txt | python sol.py -d)
# Juez:   se ejecuta sin "-d" -> dbg() no imprime nada.
import sys, os, math, random
from collections import defaultdict, Counter, deque
from heapq import heappush, heappop, heapify
from bisect import bisect_left, bisect_right, insort
from itertools import accumulate, permutations, combinations, product, groupby
from functools import lru_cache, reduce, cmp_to_key
LOCAL = "-d" in sys.argv
if LOCAL and os.path.exists("in.txt"):
    sys.stdin = open("in.txt")              # redirigir ANTES de "input = sys.stdin.readline"
input = sys.stdin.readline
if hasattr(sys, "set_int_max_str_digits"):  # Python 3.11+: int(str)/str(int) fallan con
    sys.set_int_max_str_digits(0)           # más de 4300 dígitos; 0 = sin límite
def dbg(*args):                             # va a stderr: el juez no lo lee
    if LOCAL:
        print(*args, file=sys.stderr)
def I():   return int(input())                    # un entero
def MI():  return map(int, input().split())       # varios enteros en una línea
def LI():  return list(map(int, input().split())) # lista de enteros
def S():   return input().strip()                 # una cadena (readline deja el '\n')
def LS():  return input().split()                 # lista de palabras
INF = float("inf")                          # o 1 << 62 si quieres que todo siga siendo int
MOD = 10**9 + 7
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))
out = []
def solve():
    n = I()
    a = LI()
    dbg(f"{n=} {a=}")                       # imprime "n=3 a=[1, 2, 3]"
    out.append(str(sum(a)))
def main():                                 # código dentro de función: variables locales,
    for _ in range(I()):                    # más rápido que a nivel global
        solve()
    sys.stdout.write("\n".join(out) + "\n")
main()
# --- Si hay recursión profunda (DFS de 10^5 niveles) en CPython, reemplaza main() por:
# import threading
# sys.setrecursionlimit(1 << 25)
# threading.stack_size(1 << 27)             # 128 MB de pila; bajar a 1 << 26 si da MLE
# threading.Thread(target=main).start()
# En PyPy esto no sirve: usar DFS iterativo o @bootstrap (ver rendimiento.py).
```

§11.5 template.py

10_Python

```
import sys
input = sys.stdin.readline
def I():   return int(input())                    # un entero
def MI():  return map(int, input().split())       # varios enteros en una línea
def LI():  return list(map(int, input().split())) # lista de enteros
def S():   return input().strip()                 # una cadena
out = []
def solve():
    n, k = MI()
    if n == 0:
        out.append("0")
        return                          # termina solo este caso
    out.append(str(n - (1 << k)))
for _ in range(I()):
    solve()
sys.stdout.write("\n".join(out) + "\n")
```